

# 让AI在企业落地

人人都要知道的方法论

刘老师 著



# 目 录

|                                     |    |
|-------------------------------------|----|
| 第 1 章 95% 的失败率背后 .....              | 1  |
| 1.1 一个刺痛的开场：技术成功，无人买单 .....         | 1  |
| 1.2 从 Palantir 到 FDE 岗位的爆发 .....    | 3  |
| 1.3 企业 AI 落地的五大路障 .....             | 5  |
| 1.4 FDE 的定义与三层特质 .....              | 10 |
| 1.5 一个项目的静默死亡 .....                 | 13 |
| 章末工具箱 .....                         | 15 |
| 第 2 章 你不是在卖模型，是在交付结果 .....          | 18 |
| 2.1 FDE 与算法工程师、销售工程师、交付顾问的区别 .....  | 18 |
| 2.2 结果 $\neq$ 交付物，结果 = 客户行为改变 ..... | 20 |
| 2.3 FDE 的一个完整项目周期 .....             | 23 |
| 2.4 FDE 的全套能力模型 .....               | 24 |
| 章末工具箱 .....                         | 27 |
| 第 3 章 进场：先画出五张地图 .....              | 28 |
| 3.1 灯塔客户战略：先做标杆，不做炮灰 .....          | 29 |
| 3.2 行胜于言：少问“你想要什么”，多看“你现在怎么干” ..... | 32 |
| 3.3 进场尽调：五张地图 .....                 | 34 |
| 3.4 采购、法务、安全审查的通关策略 .....           | 41 |
| 3.5 一次进场尽调，是怎么避开一个大坑的 .....         | 43 |
| 章末工具箱 .....                         | 45 |

|                                    |     |
|------------------------------------|-----|
| 第 4 章 找到正确的问题：PSF 与 MVD .....      | 49  |
| 4.1 概念验证坟墓 .....                   | 50  |
| 4.2 先有对的问题，才有对的方案 .....            | 52  |
| 4.3 三关检验 .....                     | 56  |
| 4.4 MVD：不是缩水版，是尖刀 .....            | 61  |
| 4.5 砍掉八成需求，反而活了 .....              | 68  |
| 章末工具箱 .....                        | 71  |
| 第 5 章 从能跑到用户真用：激活部署 .....          | 74  |
| 5.1 企业部署的“首日魔咒” .....              | 74  |
| 5.2 降低使用门槛：寄生于他已经在用的东西 .....       | 77  |
| 5.3 评估体系驱动质量：上线后才是开发的开始 .....      | 81  |
| 5.4 集成大战：绕开、硬啃、还是妥协 .....          | 85  |
| 5.5 一次真实的三周激活复盘 .....              | 88  |
| 5.6 一次集成大战的完整复盘 .....              | 91  |
| 章末工具箱 .....                        | 93  |
| 第 6 章 守住续约与扩大收入 .....              | 96  |
| 6.1 客户为什么流失：五类原因 .....             | 97  |
| 6.2 守住续约：用健康分在流失前预警 .....          | 102 |
| 6.3 扩大收入：从守到攻 .....                | 107 |
| 6.4 案例：Anthropic 与 FIS 的扩张路径 ..... | 113 |
| 6.5 案例：一个差点流失的客户，是怎么救回来的 .....     | 115 |

|                                   |     |
|-----------------------------------|-----|
| 6.6 扩大收入的另一面：别让客户觉得被绑定 .....      | 118 |
| 章末工具箱 .....                       | 119 |
| 第 7 章 规模化复制：从定制项目到平台产品 .....      | 121 |
| 7.1 定制化陷阱与三级复制杠杆 .....            | 122 |
| 7.2 产品化四问：这个能力，值不值得做成产品？ .....    | 127 |
| 7.3 从定制到平台：用数据验证，用复盘沉淀 .....      | 130 |
| 7.4 生态借势与组织政治 .....               | 134 |
| 7.5 一个组件，是怎么复制到五个客户的 .....        | 136 |
| 章末工具箱 .....                       | 138 |
| 第 8 章 技术选型与本体论 .....              | 140 |
| 8.1 RAG、Agent、评测的落地要点 .....       | 141 |
| 8.2 Palantir 本体论 (Ontology) ..... | 151 |
| 8.3 案例：一个医院的动词建模实战 .....          | 155 |
| 章末工具箱 .....                       | 158 |
| 第 9 章 底线与道路 .....                 | 163 |
| 9.1 六条职业道德底线 .....                | 163 |
| 9.2 FDE 的职业成长与出路 .....            | 171 |
| 9.3 选择雇主与组织的重要性 .....             | 177 |
| 章末工具箱 .....                       | 179 |
| 附录 A FDE 常用指标清单 .....             | 181 |
| A.0 怎么用这份清单 .....                 | 181 |

|                                  |     |
|----------------------------------|-----|
| A. 1 时间类指标 .....                 | 181 |
| A. 2 转化与商业化指标 .....              | 182 |
| A. 3 经营与复制指标 .....               | 184 |
| A. 4 质量类指标（针对 AI 输出本身） .....     | 185 |
| A. 5 一份拿来就能用的指标看板 .....          | 186 |
| 附录 B 全流程交付 SOP 模板合集（逐章全录版） ..... | 187 |
| B. 0 怎么用这本附录 .....               | 187 |
| B. 1 模板总表（T01 - T14） .....       | 187 |
| B. 2 逐章模板全录 .....                | 188 |
| B. 3 T14 SOW（工作说明书）模板 .....      | 202 |
| B. 4 一份项目的“模板使用顺序” .....         | 204 |
| 附录 C 全书案例与项目索引 .....             | 205 |
| C. 0 说明 .....                    | 205 |
| C. 1 案例总表 .....                  | 205 |
| C. 2 按章节定位案例（快速跳转） .....         | 207 |
| C. 3 关键案例速查（写书用） .....           | 208 |
| C. 4 素材缺口说明（诚实交代） .....          | 208 |

# 序言

写完前两本之后，我收到最多的不是夸，是问题。

很多读者照着第一本《人人都能做智能体》，真把智能体做出来了——能跑，能答，demo 能放给领导看，能引来一阵掌声。然后呢？就停在掌声里了。有人告诉我，他们的智能体上线那天特别风光，两周后打开后台，最后一条真实记录，还停在上线的测试数据上。

也有人读完第二本《拆解大模型》，参数、注意力、token 讲得头头是道，理论一点不怵。可真要自己上手，给业务搭一个能用的东西，还是卡在一句话上："我知道原理，但我不知道它该吃什么样的数据、交给谁用。"

我这才慢慢想明白一件事：大家缺的，不是"怎么把智能体做出来"，也不是"怎么理解大模型"，而是中间那段——**怎么让一个"能跑的智能体"，变成一个"真干活的智能体"**。

这段路不长，但坑特别多。

数据是脏的，接口是黑盒，业务嘴上说的需求和实际的不是一回事；方案做出来了，真正负责用的人不买账；系统上线了，一个月没人碰，比从没上线更让人难受。这些，demo 里看不见，原理里推不出来，书里更没人愿意写。它们只会在真实项目里，一个一个，撞到你脸上。

这本书，就是把路上这些坑，和绕开坑的笨办法，都给你摊开。

我给自己定了三条规矩：

第一，**不讲概念，讲动作**。每一章末尾都放一张能直接照做的清单，别指望靠悟。

第二，**不吹成功，讲失败**。我写过不少翻车的项目，翻车的细节，往往比成功的故事更值钱。

第三，**不追标题好看，追那件事"真的成了"**。衡量的标准不是"上线了没有"，而是"有没有人真的开始用了"，并且用起来确实比以前省事。

所以这三本，其实是三步：第一本，让你**能做出来**；第二本，让你能**看得懂**；这一本，让你能**交付得成**——让 AI 真正开始干活，替你自己、也替你的客户，把某一件具体的、麻烦的、以前靠人反复磨的事，稳稳当地做好。

别把这本书当成技术的续篇。它更像一本工地上的操作手册。你手里的智能体，不是用来发朋友圈的，是用来去现场干活的。

打开它之前，先想清楚一件事：你手头最想让 AI 帮手干下去的那件麻烦事，到底是什么？

刘老师

2026 年 10 月

公众号 AI 建造



扫码添加作者微信



微信扫码加入星球

扫码订阅知识库



易经 AI 传道

# 第 1 章 95% 的失败率背后

企业 AI 失败的真正原因，不是模型不够好，而是没有人对"落地"这件事负责。

## 1.1 一个刺痛的开场：技术成功，无人买单

先讲一个我亲眼见过、也听过不下十个版本的故事。

一支团队，接了一个企业的 AI 项目。技术难度不低，但他们做得不错。模型调得漂亮，RAG 的召回率压到了能吹牛的水平，压测数据拿出来，外行看了会以为这是哪家独角兽的团队。演示那天，老板当场拍了板，说"这就是我们要的东西"。

上线那天，锣鼓喧天。技术负责人发了朋友圈，配图是系统的截图，配文是"从 0 到 1，正式上线"。底下几十个赞，评论里清一色是"牛逼""厉害"。当晚还请了顿饭，喝到半夜，人人都觉得这事成了。

然后，就没有然后了。

第一个月，日活跃用户一只手数得过来。你问起来，客户那边的对接人说"员工还不习惯，正在推广"。第二个月，还是那几个数。第三个月，连对接人都不回消息了，那个系统安静地躺在服务器上，像个没人认领的孤儿，没人再打开，也没人记得它当初承载了多少期待。

再后来，IT 部门做服务器清理，把它归档了。连个告别仪式都没有。项目正式死亡。

团队冤不冤？冤。

他们没偷懒。代码写得干净，模型也能跑，接口也没断。甚至可以说，从纯粹的技术标准来看，这是个"成功"的项目——它跑起来了，它没崩，它的每个指标都是绿的。

但商业上，它死了。账面上，它一毛钱续费都没有，一分钱价值都没留下来。

这就是企业 AI 领域最残酷、也最普遍的一个现象：**技术上成功，商业上无人买单。**

我在这个圈子里泡了这些年，见过太多这种死法。死的时候往往悄无声息，没有事故，没有争吵，没有谁站出来说一声"我搞砸了"。它就是那么安安静静地，被人忘掉了。

很多人会把这种现象归结为"客户不懂技术""市场还没成熟""老板叶公好龙"。这些解释都有道理——我甚至自己都这么说过，说过不止一次。但它们都没说到根子上。说客户不懂技术，可那些做出成绩的项目，客户也没多懂；说市场不成熟，可钱明明在往外花；说老板叶公好龙，可老板是真的拍了板、真金白银砸了预算进来的。

根子是什么？是整个过程中，**没有一个人，真正对"让 AI 在客户那里落地、用起来、产生价值"这件事负责。**

研发负责把模型做出来，交付完就撤，撤的时候还觉得"我的部分已经做完了"。销售负责把单子谈下来，签完提成到手，转头就去追下一个单。客户那边更是一锅粥——业务部门当甩手掌柜，觉得"这是 IT 的事"；IT 部门当接盘侠，觉得"这是业务的事"；而真正该用这个系统的那些一线员工，从头到尾没被问过一句"你们到底要它干嘛"。

于是，"落地"这件事，就成了一个真空地带。它不在任何人的 KPI 里，不属于任何人的职责范围。你问研发，研发说归销售；你问销售，销售说归客户；你问客户，客户说归供应商。一圈问下来，没人管。

而真空，恰恰是项目死掉的地方。一个东西没了主人，就像一条没了阀门的管道，水永远到不了该去的地方。

不过我得先说清楚一件事，免得你误会这本书全是讲"技术成功但没人用"这一种死法。

实际上，还有另一种死法，死得更早、更痛。它甚至都走不到"上线"那一步——技术还没来得及出场，项目就已经在数据的泥潭里咽了气。

我见过的那样一个项目：一家做家电代工的工厂，老板在饭局上听了 AI 的故事，热血上头，签了个四十万的合同，说要"全面智能化"。团队五六个人信心满满地进场，准备大干一场。结果两个月过去，他们连一个模型的边都没摸着——每天从早到晚，就是在洗一张张对不上账的表。

这个项目我留着后面慢慢讲。眼下你只需要记住：不管是"技术成功了没人买单"，还是"技术压根没机会成功"，它们背后站着的，是同一个东西——**没有一个角色，从头到尾对"落地"这件事负过责。**

这本书要讲的，就是这个真空地带，以及一个专门为填这个真空而生的岗位——**FDE, Forward Deployed Engineer, 前线部署工程师。**

在往下讲之前，我先说一句大实话：如果你是奔着听一套"AI 万能方法论"来的，这本书会让你失望。因为我要讲的，绝大多数时候跟模型无关。它跟数据有关，跟流程有关，跟人有关，跟那些最土、最不性感、最没人愿意碰的事情有关。

而恰恰是这些没人愿意碰的事，决定了那 95% 的项目，是死是活。

## 1.2 从 Palantir 到 FDE 岗位的爆发

FDE 这个词，不是中国人发明的，也不是这几年才有的。

它最早出自 Palantir，一家做大数据分析的美国公司，成立于 2003 年。这家公司最早的班底里，有一帮做支付反欺诈出身的工程师——当年 PayPal 上骗子满天飞，就是这帮人靠算法和数据，硬把欺诈率压了下去。后来他们看中了另一块更硬的骨头：政府和大企业的数据。

Palantir 的客户是谁？CIA、FBI、美国军方、各类情报机构，再往后才是摩根大通、空客这类巨型企业。这些地方有一个共同点：数据量大得吓人，杂得离谱，敏感得要命，而且每一家的需求，长得完全不一样。政府那套系统，和企业那套系统，差别比火星和金星还大。

当时 Palantir 碰到的难题，和今天所有做企业 AI 的人碰到的一模一样：**软件做得再好，拿到客户那里就是跑不起来，因为客户的数据、流程、组织，全都和你的假设对不上。**

你照着教科书做的产品，到了客户的机房，发现人家内部的数据格式是你这辈子没见过的；你预设的接口，对方根本没有；你所谓的"标准流程"，在人家那里是靠几个老员工口口相传维系的。

换了大多数软件公司，这时候就该打退堂鼓了：客户环境太烂，不是我们的问题。但 Palantir 做了一个决定，后来被证明是它商业模式里最关键的一步——**派工程师直接住进客户现场。**

注意，这些工程师不是去做简单的技术支持，也不是去做售前演示忽悠人。他们长期驻扎在客户的办公室里，和客户的一线员工一起吃盒饭、一起加班，理解客户的真实业务到底是怎么跑的，帮客户把数据接进来、把流程理顺、把系统调通，直到客户真的用起来、并且用它解决了真问题。

这些人，Palantir 称之为 Forward Deployed Engineer——被"向前部署"到客户前线的工程师。

"向前部署"这个词，军队味道很重。说白了就是：别的公司把人放在后方研发部，写代码；Palantir 把人放到前线，直接面对客户那片混乱的战场。产品是死的，人是活的，客户环境千奇百怪，那就让最懂技术的人亲自去播平它。

Palantir 正是靠着这一支前线部队，才能在极其难啃的政府和大企业客户里，一次次把看起来不可能做成的项目做成，最终成长为一家千亿美金市值的公司。它上市的时候，华尔街一边看不懂它的商业模式，一边又舍不得它的股价——因为没人说得清，这家公司为什么能成，能成的秘密，恰恰藏在那群穿着衬衫蹲在客户机房里的人身上。

后来，FDE 这个模式被 OpenAI、Anthropic，以及一大批 AI 创业公司纷纷抄走。大家发现，光有强大的模型还不够。模型再强，也得有人把它翻译成客户听得懂、用得上、肯掏钱的东西。你必须有一群人，愿意走到客户身边，把模型的潜力翻译成客户的现实。

而真正把"FDE"推到台前的，是近几年的一份报告。

2025 年前后，MIT 有个叫 NANDA 的实验室，发布了一份关于 AI 时代职业变化的报告。报告里提到，随着大模型能力越来越强，企业里最稀缺的，不再是"会写代码的人"，也不是"会调模型的人"，而是一种能把 AI 能力真正带进企业业务、并把它做成可验证结果的人——这类岗位，正是 FDE。

我手头没有那份报告能给你精确到个位数的招聘数据，但你打开任何一家招聘平台，都能看到类似的趋势：FDE 相关的岗位需求，在一年左右的时间里涨了好几倍，跑得远比传统的软件工程师、算法工程师岗位快。猎头的话更难听一点，他们说，"现在缺的不只是会用 AI 的人，缺的是能带着 AI 进场子、把事儿办成的人，这种人，简历上根本招不到。"

为什么偏偏是这个岗位爆发了？

因为所有人都看到了同一个冰山下的事实：**当模型的能力不再稀缺，落地就变成了新的稀缺品。**

这话这两年听得多了，都快成顺口溜了。但顺口溜之所以流传，恰恰因为它戳中了真相。三年前，一个能写代码的人很值钱，因为模型还不行，得靠人一行行敲；三年后，模型自己就能写代码，写代码这件事本身，突然变得不那么值钱了。可企业要的从来不是"一段代码"，而是"一件办成了的事"。这件事，模型替不了你，平台替不了你，只有那个肯走上前线、一头扎进客户现实的人，才办得了。

不过，讲到这我得提个醒，别被那些招聘广告带偏。岗位爆发是真的，但乱象也是真的。这套路上，我看见过太多人，就是把原来的售前、实施、驻场外包，换了个 FDE 的新名字，就敢挂牌上岗。关于这个，我在书的后半部分会单独拆开讲——什么叫真 FDE，什么叫"FDE cosplay"。

眼下你先记住这一节的结论：FDE 不是新瓶装旧酒，它是一个被"模型不值钱了、但落地值钱了"这个时代变化逼出来的、有明确职责边界的岗位。它到底该干什么，我们先把企业 AI 为什么这么难落地这件事掰开，你就清楚了。

### 1.3 企业 AI 落地的五大路障

要理解 FDE 为什么稀缺，得先理解一件事——企业 AI 落地，到底难在哪。

坊间流传着一个说法，企业 AI 项目的失败率，高达 95%。这个数字到底准不准，我没法给你一个权威出处。但我可以说的是：在我接触的这个圈子里，项目"做了但没真正用起来"的比例，确实是压倒性的高。高到你随便拉一个做过企业 AI 的人出来，都能给你倒出一堆烂尾项目的名字。

而且注意，这 95% 里，绝大多数项目用的模型，都是没问题的。今天的模型已经强到让三年前的人目瞪口呆，写代码、读文档、做问答，样样都能来。可项目还是死。

问题出在，从"模型能做什么"到"客户真的得到价值"之间，横着五道几乎人人都会踩进去的路障。这五道路障，没有一道是模型的错，但每一道都能随随便便要了项目的命。

我一道一道说。

#### 路障一：数据是脏的

这是最基础、也最被低估的一关。

很多企业，你一问"数据情况怎么样"，对方拍着胸脯说"我们数据都有，信息化很早就做了"。话说到这份上，你多半就信了——毕竟人家上了系统，人家有数据库，人家还给你看了几张报表。

等真进了场，你才会发现，所谓"都有"，是个天大的误会。

我见过一家制造企业的一线车间。那些所谓的经营数据，到今天还只记在一线工人手写的纸质单据上——皱巴巴的纸，上头是圆珠笔写的数字，有的纸边还卷着油污。你要问这些数据进没进系统？没进，一张都没进。数据的第一落点，是工人的手。

纸之外，还有更绝的。有些关乎良品率的关键工艺参数，压根不在纸上，也不在系统里，只在老师傅的脑子里。你拿着问卷去问，老师傅手一摆："这个啊，我手上有数。"温度调到多少、压力给多大、什么时候该敲一下，全凭他几十年的手感。这些参数，没有一个是数字，没有一条进了电脑。

再往上，就是系统层。企业倒是真上过系统，但那是十几年前买的一套破烂 ERP 和 CRM，连一个最基础的 API 接口都调不出来。你想拿它的数据，只能靠人工一条条往外导，导出来的格式还对不上。

更离谱的在后面。你想找当年的开发商要接口文档、要源码，一查，那家给企业做管理软件的本地小外包公司，五年前就已经倒闭清算了。源码丢了，文档没了，数据库还加着密——钥匙在谁手里，没人知道。

这一串下来，你手里所谓的数据，四散在纸上、脑子里、老系统里、还有跟着倒闭公司一起消失的硬盘里。

所以我说，企业 AI 不是输在"模型不会算"，而是输在"算之前，先把账本翻出来"这一步，就已经比想象中难上一百倍。

而且数据的脏，还不只是"找不到"和"对不齐"这一种。还有一种更阴的脏，叫"看着对，其实是错的"。

我碰到过一件真事。给一家企业做知识库，上线测试，AI 回答错了一个产品问题。我们第一反应是模型胡说了，第二反应是召回错了，第三反应是扫描件解析错了。一路查下去，技术链路全对。最后才发现，错的是很多年前打印出来的一张产品设计文档——上头有一处印错了。全公司几个老员工都知道这里印错了，平时在脑子里自动改过来，从来不提。可这个"正确的改法"，从头到尾没进过任何系统。于是 AI 忠实地读了那份错误资料，非常准确地，给出了一个错误答案。

**AI 可以非常准确地处理一份错误知识。** 这就是脏数据最狠的地方：它不是明摆着烂给你看，它把错误包装得跟真理一模一样。你用干净数据做出再漂亮的演示，到了生产环境，全是这种货色等着你。

## 路障二：流程是断的

就算你千辛万苦把数据拿到了，第二个拦路虎，是流程。

AI 要产生价值，必须嵌进企业的业务流程里——这是句正确的废话，但真正下过场的人，才知道这句话的分量。

因为很多企业的流程，本身就是混乱的、口头的、靠人肉维系的。

一个审批，要过七个人，每个人都能在任何一环把球踢给下一个人；一个数据，要嘴上问三次才能到位，问到最后一个人，他告诉你"其实这个数我上个月就发过一版，但他没看"。一个本该两分钟走完的流程，在真实世界里可能要漂两天。

这里面有个很残酷的两难。

你做的 AI，如果绕开这些真实流程，自己另起一套——那它就是个玩具，是个没人会去看的漂亮界面，因为它解决不了任何真实的业务问题。

你如果硬要嵌进去，又会撞上流程里的各种既得利益和历史包袱：这个环节的人不愿意交权，那个环节的规则是二十年前定的没人敢动，还有一堆"我们一直是这么干的"。

我还见过一种特别典型的断法：流程在"交接"的地方断掉。上游部门的数据出来了，下游部门不用，或者用的时候自己再抄一遍、改一遍，于是同一条数据，在系统里是三个版本。你问谁手里的版本是准的，三个人各执一词，都觉得自己手上的没问题。

这种断裂的流程里，AI 就像一个被请来修水管的水管工，结果发现整栋楼的管线图是乱的、阀门口径还不一样，他手里那个最先进的工具，一时间不知道往哪儿使。

## 路障三：组织是复杂的

比流程更麻烦的，是组织。

企业里，任何一件事的成功，都不只取决于技术，更取决于"人"：谁是这件事的受益者，谁是潜在的抵制者，谁拿着决策权，谁只是嘴上说说。

这三类人的分布，往往和你想的完全相反。

一个提高效率的 AI 系统，可能让某个部门的人显得"多余"，于是他们就成了隐形的阻力——但阻力从来不明着来。项目评审会上，他们点头；私下里，他们卡数据、拖审批、把问题包装成"安全性还不确定"，让事情永远推进不下去。

一个打通数据的项目，可能动了某个部门的信息垄断权。那个部门的存在感,有一半来自"只有我手里有完整数据"，你把数据打通了，他们的护城河就没了。于是审批就永远卡在"再研究研究"。

我见过最典型的一幕，是一个项目的关键接口，卡在某个负责人手里三个月。你跟他谈技术，他说再评估；你跟他谈安全，他说预算不够；你跟他谈预算，他说再开一次会。三个月，会开了七八轮，接口一个没开。最后你才打听出来，项目落地之后，他那套靠信息不透明维持的权威，就要土崩瓦解。他凭什么给你开？

组织层面的暗流，比任何技术 bug 都更能杀死一个项目。技术 bug 你能看到、能定位、能修；组织里的暗流，你看不到，它只是让所有事情都"刚好"推不动。

#### 路障四：系统是老的

大部分企业，尤其是制造业、传统行业，系统是十几年前甚至更久远的老古董。

没有 API，没有文档，有的甚至找不着当年的开发商。你要让 AI 和这些老系统集成，很多时候不是写代码的问题，是考古学的问题——你得先搞清楚，这个系统当年是谁建的、数据存在哪、格式是什么。

这活儿有多荒诞，我给你还原一个场景。

你到了客户那，问 IT："这套 ERP 的接口文档在哪？"对方沉默两秒，说："这套系统是十四年前买的，当时的项目经理早离职了。"你再问："那找开发商要？"对方更沉默了："开发商.....五年前就倒闭了，我们自己也联系不上。"

于是你的任务列表里，多了一条：到市场上找这家已经消失的公司的源码。找不到源码，就去找当年的离职员工，约出来吃饭，一条条问这个系统的表结构到底是什么样的。问完回来，数据还是加密的，你连它怎么解都不知道。

这跟技术能力没关系。任你模型调得多溜，面对一个源码丢失、数据库加密、开发商倒闭的老系统，你也只能干瞪眼。

说白了，老系统这一关，考验的不是你会不会写代码，而是你有没有那个耐心和路子，把这些"历史遗留的屎山"一点点摸清楚。而这，恰恰是大多数人最不愿意碰的活。

## 路障五：没人定义成功

最后一道路障，也是最致命的：绝大多数项目，从头到尾，没有一个人定义过"什么叫做成了"。

是省了钱？少了人？决策变快了？错误率低了？没人说清楚。

于是到最后验收的时候，交付方说"我按需求开发完了"，客户说"可是业务没变"。双方都觉得自己冤，也都说得对——因为他们从头到尾，说的就不是同一件事。

交付方嘴里的"完成"，是代码写完了、功能上线了、验收单签了。客户心里想的"成功"，是我的人真的少加班了、钱真的没再漏了、决策真的变快了。这两件事，中间隔着一条鸿沟，而从来没人在项目开始的第一天，把这条鸿沟摆到台面上。

我见过太多的项目，就是死在这个"没人定义成功"上。上线了，热闹了，没人用了，然后不了了之。你去复盘，问任何一方"当时你们说好的成功标准是什么"，没有一个人答得上来。

这五道路障——数据、流程、组织、系统、成功标准——没有一道是纯技术问题，但每一道都横在"模型"和"价值"之间。

有意思的是，这五道里的任何一道，都"够不上"让任何一个传统岗位来负责。研发不管数据干不干净，他只管模型能不能跑；销售不管流程断不断，他只管单子签没签；客户内部的IT不管组织复不复杂，他只管系统稳不稳。结果就是，五道路障，恰好全部落在那片没人负责的真空里。

而 FDE，就是那个专门用来跨越这五道路障的角色。这也正是那句话的由来：**模型的能力已经不再稀缺，稀缺的是把模型落地的人。**

你可能会问：凭什么一个人，能同时对付数据、流程、组织、系统、成功标准这五样东西？他得是个什么样的人？

这就要说到 FDE 到底是个什么玩意儿了。

## 1.4 FDE 的定义与三层特质

那么，FDE 到底是什么？

我不想给你一个学院派的、正确的废话定义。那种定义正确到挑不出毛病，但你拿去对照自己，发现屁用没有。我想给一个可操作的、你能拿去对照自己的定义：

**FDE 是一个愿意住在客户现场、把"模型能做什么"一路负责到"客户真的得到什么"的工程师。**

这句话看着简单，拆开看，里面藏着 FDE 的三层特质。这三层，缺一层，都不是 FDE。

### 第一层：技术通才

FDE 说到底，得先是一个工程师，能动手。这一点没得商量。一个连代码都看不明白、连部署都不会的人，走到客户现场，连对话都插不进去。

但他不需要是某个领域最顶尖的专家。你不需要是论文作者，不需要精通某一个模型的所有细节。你需要的是另一种东西——一种"通才"的能力：模型调得动，数据管道接得上，API 写得出来，部署搞得定，遇到没见过的老系统也能想办法啃下来。

重要的不是深度，而是广度，以及那种"遇到任何技术问题都不慌、总能找到路"的动手能力。

我给你两个对照，你就明白差别了。

一种是只擅长一件事的人。他可能是 RAG 的高手，聊起向量检索、召回策略、重排序如数家珍。可一到客户现场，发现这家厂的数据全是纸质单据、系统是个没有 API 的老古董，他就歇菜了——这活儿离他擅长的那个点太远，他不知道从哪下手，只能尴尬地站在那。

另一种是通才。他未必能把你问到向量检索的最前沿，但面对一堆烂数据、一个死掉的老系统，他的第一反应不是"这不是我的专业"，而是"行，我看看能从哪里撬开一条缝"。缺接口，他自己写一个；缺数据，他去车间蹲着看工人怎么填单子；缺文档，他厚着脸皮去约十几年前的离职员工吃饭。这种人，才扛得住真实项目里的鸡零狗碎。

企业现场的问题，从来不会贴心地长成你熟悉的样子。它永远是乱的、跨界的、半吊子的。技术通才，就是那个能把乱成一团的东西，一条条理顺、找到路的人。

## 第二层：业务翻译能力

这是 FDE 和普通工程师最本质的区别，也是最值钱的一层。

普通工程师，客户说"我要一个系统"，他就去写一个系统。需求文档拿来，照着做，做完交付，收工。

FDE 不能这样。FDE 要做的，是先搞懂客户嘴里那个含糊的需求，背后真正卡住的业务是什么，然后用技术语言，把它翻译成一个能落地的方案。

这两个动作，一个都不能少。

先听懂业务。客户嘴上说的，和真正要的，往往差着十万八千里。有家车企找我聊，开口就是"我们要做数字人、无人直播、短视频脚本"。听着像要采购一套虚拟主持人的设备。可你往下问几层，才知道他们真正要的，根本不是一个会动的数字人，而是一条内容生产到数据复盘的全链路——总部要看到所有直播间和短视频的数据，经销商账号要纳入监测，脚本要按车型和热点自动生成，投放结果要回流指导下一轮选题。

客户说的需求是"数字人"，真正要的东西是"一条链路"。你如果照着他第一句话去做，做完一定没人用。

所以 FDE 要同时会说两种语言。一种是企业内部的语言：业务、流程、指标、痛点、谁拍板谁受益谁抵制。一种是技术的语言：模型、接口、数据、系统。他站在两种语言中间，当那个翻译。

这个"翻译"能力，才是 FDE 真正值钱的地方。因为模型大家都会调，现在调模型的门槛越来越低，连不会写代码的人都能搓出一个像样的智能体。但能把"老板想要数字化"翻译成"先解决采购那张每周要核对两天的对账单"的人，太少了。能把"客户要个知识库"翻译成"我们要解决的是客服每单多花三分钟找答案"的人，也太少了。

翻译对了，方案才落得下去。翻译错了，技术再牛也是白搭。

## 第三层：主人翁意识

这是最玄、但也最要命的一层。

传统意义的分工里，研发做完自己的模块就完事，销售签完单就完事，谁也不对最终结果负全责。项目死了，研发说"我代码没问题"，销售说"我单签下来了"，客户的业务说"我们不会用"，IT 说"我们只负责维护"。

你看，没人说谎，但项目就是死了。

FDE 不一样。FDE 以"客户是否真正用起来、是否真正得到价值"作为自己工作的成败标准。

项目上线后没人用？那不是"客户的问题"，是 FDE 的问题。客户用了一个月不了了之？那不是"交付完成了"，那是 FDE 没做完。系统跑得好但业务没变？那不是"成功上线"，那是没交付。

这种"对最终结果负责到底"的主人翁意识，是 FDE 区别于一切"换名字的外包""高级售前""驻场实施"的核心。

我见过反例，一抓一大把。有的"实施顾问"，把系统装上、跑一遍演示、培训完拍屁股走人，一个月后系统没人用，他说"我的交付已经完成了"。有的"售前专家"，方案讲得天花乱坠，签完单人就不见了，问他落没落地，他说"那是交付团队的事"。这些都不是 FDE，哪怕他们把名片上的头衔换成了 FDE。

我也见过正面。那种真把项目当自己事的人，客户一句"最近用得少了"，他第二天就打车过去，蹲在工位边上看员工到底卡在哪。这种人不多，但每一个都值钱。

主人翁意识这层，说穿了不是什么玄学，就是一个朴素的问题：**项目死了，你会不会觉得是自己的责任？**会，那你离 FDE 就近了；不会，那你再懂技术，也只是个高级外包。

三层特质，串起来是一个完整的逻辑：

技术通才，让他能动手；业务翻译能力，让他能听懂；主人翁意识，让他能负责到底。

能动手、能听懂、能负责——三者缺一不可，合在一起，才构成了 FDE。缺了动手，他是纸上谈兵；缺了翻译，他是无头苍蝇敲键盘；缺了负责，他做得再漂亮，也只是个过客。

## 1.5 一个项目的静默死亡

光讲概念，还是有点空。我用一个完整的时间线，看看一个项目是怎么从"技术成功"走向"静默死亡"的。你可以把它当成一个典型标本，反复对照自己手里的项目。

那一整个过程的每一步，分开看都没什么大错，可合在一起，就是一个必然的死局。

**立项期。**客户的高管听了一场 AI 分享，回来拍板："我们要全面拥抱 AI。"这句话从高管嘴里出来的时候，是热血沸腾的；等它顺着层层汇报传到执行层那里，已经变成了一个模糊的、宏大的、谁也没弄明白的愿景。所有人都在点头，没有人知道到底要做成什么。但 deadline 有了，预算也有了，合同也签了。

有些项目，还没有开始，方向就已经错了。老板要的是"全面智能化"这个想象，可没人把它翻译成具体到能动手的问题。这个病根，在立项那天就埋下了。

**开发期。**团队进场，开始做。因为没有一个人真正沉到业务里去看"客户到底在哪个环节最疼"，大家只能凭经验猜，把需求往"通用平台"上靠。做出来的东西，功能很全，看板很多，演示很炫，但每一项都悬浮在真实业务之上，像一件华丽但没人能穿的衣服。

当然，也有连开发期都熬不过去、死得更早的。这就是我开头提过的那家代工厂。

那家工厂的老板，是在一个饭局上被朋友安利了 AI 之后，才下定决心要"全面智能化"的。朋友正手舞足蹈地讲 AI 有多神：文档自己写，代码自己跑，报表自己拉。老板听着，眼睛就亮了。他的需求翻译成成人话，是这么个意思：把人事的底层数据、财务的流水，全公司各个环节打通，上面搭一层智能体和数据看板，让信息自己流转起来。

听着像智能制造的宣传片。合同签了，四十万，不算大，但足够一个刚拉起来的新团队冲进去。

进场的是五六个人，踌躇满志。终于接了个"智能制造"的大单，终于能甩开那些小打小闹，干点真东西了。每个人心里，大概都预演过模型跑通那一刻、看板亮起来那一刻。

然后事情开始不对劲。

第一个星期，他们想找数据，找到的是车间里那些皱巴巴的纸质单据。经营数据记在工人手上，一张都没进系统。

第二个星期，他们想找工艺参数，找到了几个老师傅。那些关乎良品率的关键参数，全在老师傅的脑子里，凭手感调机器，几十年的经验，没有一个数字进了电脑。

第三个星期，他们想接系统。企业倒是上过系统，可那是十几年前买的破烂 ERP 和 CRM，连一个最基础的 API 都调不出来。想找当年的开发商要接口文档，一查，那家本地小外包公司五年前就倒闭清算了，源码丢了，数据库还加着密。

到了第四十五天，他们在做什么？

在洗数据。

不是调模型，不是搭 Agent，不是画看板。是每天从早到晚，把一张张对不上账的表、一个个口径不清的字段，一点点往一块拼。六周过去了，连一个最简单的模型调优、一个具体的业务 workflow，都还没开始做。

更麻烦的是账。按他们当时的消耗速度估算，光是把这堆陈年烂账理清楚，人力物力的成本，已经快要超过四十万的订单总额了。换句话说，活还没开干，钱已经烧穿了合同价。

这时候，团队被架在火上烤。退，前期的工资差旅全打水漂，手里还捏着写死阶段节点和验收标准的合同，单方面违约的代价承受不起；进，就是个无底洞，越干亏损越多，后面不知道还要填多少人口进去。

这个项目最后怎么样了，不重要。重要的是，你可能已经看出来——它和前面那个"上线后没人用"的项目，是两种不同的死法：一个死在开发期，连演示都没走到；一个死在部署期，演示大获成功之后。但你把镜头拉远一点，会发现它们的死因，其实是同一个。

**演示期。**回到那条"技术成功"的时间线。演示很成功。会议室里，投屏上的系统行云流水，高管频频点头，销售趁热打铁，签了字。这是整个项目里最高光的时刻，也是麻痹性最强的时刻。

因为演示有个天生的作弊点：它演示的不是真实业务，而是一个被人为擦干净了的小舞台。数据是挑过的，流程是走平的，意外是被屏蔽的。演示越漂亮，越没人去问那个要命的问题——这东西，明天塞进真实的、脏乱的、有人扯皮的环境里，还能不能跑？

**部署期。**问题开始显现。数据接不上，因为没人提前摸清数据在哪——这时候再回头补课，成本和立项时比已经翻了几倍。员工不用，因为系统不在他们熟悉的界面上，要他们为了

一个新东西改掉用了十年的工作习惯，凭什么？权限卡住，因为负责审批的那个部门，恰恰是这个系统动了它奶酪的人。

部署期才是真正的考试。前面积累的所有侥幸、所有回避，到了这一步，会一个接一个地爆掉。而这时候，往往已经没有一个人愿意站出来，把这个烂摊子捡起来负责到底。

**静默期。**没有争吵，没有大事故。系统只是慢慢地、安静地，没有人再打开。三个月后，IT 把它归档，像收走一件过季的衣服。项目结束。

技术指标上，它几乎无可挑剔——跑得稳，没崩过，功能也都在。可商业结果上，它等于零。

你看，整个过程中，没有谁是“坏”的。研发努力了，销售努力了，客户也不是故意捣乱。代工厂那五六个人，甚至比谁都勤奋——谁能在工厂里跟烂账死磕四十五天还不走？可勤奋救不了一个没人负责的项目。

所有的努力，最终都消散在了那个真空地带里：没有人在立项时把需求翻译成具体问题，没有人在开发时沉到业务里，没有人在部署时对“用起来”负责，没有人在最后对一个明明白白的成功标准负责到底。

这，就是 FDE 存在的意义。

它不是多出来的一个普通岗位，而是那个负责把“技术成功”翻译成“商业成功”、把“模型能力”翻译成“客户价值”的关键角色。往下读，你会看到它是怎么样一步步，把上面那个必死的时间线，从立项的第一天开始，改写成一个个活下来的故事的。

## 章末工具箱

### FDE 岗位能力自评表

对照下面每一项，给自己打 0-5 分。分数不代表水平高低，只代表你在这一项上的“被验证程度”——你做过什么，才算真的拥有它。别客气，也别吹牛，这张表只有你自己看。

#### 技术通才

[ ] 能在一个不熟悉的技术栈里，快速跑通一个最小可用的东西（0-5）

[ ] 遇到没文档的老系统，能靠翻代码、问人、试错把接口摸出来

[ ] 能独立完成从数据接入到部署上线的一条完整链路

[ ] 对 RAG、Agent、工具调用等概念，不止"听过"，真实做过

[ ] 面对烂数据（纸质、对不上账、口径不一），有动手清洗的路子，而不是只会抱怨

### **业务翻译能力**

[ ] 能把客户一句含糊的需求，反推成具体要解决的业务问题

[ ] 能说清"客户以前怎么做、现在怎么做"之间的差别

[ ] 能和业务负责人用业务语言对话，而不是一上来谈技术方案

[ ] 能听懂、也能表达"这个方案能为客户省多少钱 / 赚多少钱"

[ ] 客户说一个炫酷的词（数字人、中台、智能体平台），你能往下追出他真正的痛点

### **主人翁意识**

[ ] 项目上线后没人用，你会觉得"这是我的责任"，而不是"客户的问题"

[ ] 你衡量自己成败的标准，是客户是否真正用起来、产生了价值

[ ] 你愿意为最终结果负责到底，而不是做完自己的模块就撤

[ ] 遇到阻力时，你倾向于去现场解决，而不是发个邮件了事

[ ] 系统静默死亡了，你会睡不着，而不是觉得"至少交付完成了"

### **怎么用这张表**

如果"技术通才"高、后两项低，你目前是个不错的工程师，距离 FDE 还差"翻译"和"负责"两步。

如果"业务翻译"和"主人翁意识"都高、技术偏弱，你其实已经具备了 FDE 最稀缺的内核，技术可以补。

三项都不高也没关系——这本书从下一章开始，会把每一项拆成你能练的动作。

## 本章要点回顾

企业 AI 项目的头号死因，不是模型不行，而是"落地"这件事没人负责，落进了真空地带。

技术成功、商业失败，是这一行最普遍的结局；另一种死法是技术根本没机会上场，卡死在数据这一关。

FDE 出自 Palantir，是把工程师"向前部署"到客户现场、一路负责到客户真正用起来的角色。

岗位爆发的根源：模型能力不再稀缺，落地成了新的稀缺品。

五大路障：数据脏、流程断、组织复杂、系统老、没人定义成功——没有一道是技术问题，但每一道都能杀死项目。

FDE 三层特质：技术通才（能动手）、业务翻译（能听懂）、主人翁意识（能负责到底）。缺一不可。

## 常见误区速查

**把"没人用"赖给客户：**"客户不懂技术""员工不配合"——这是最省事的甩锅。真相是没人对落地负责。→ 项目死了，先问自己负没负过责。

**以为技术做好就成功：**模型跑通、指标全绿，不等于项目成功。商业上没人买单，技术再漂亮也是零。→ 结果不是"能跑"，是"有人用、有结果"。

**把岗位爆发当成随便进的快钱：**FDE 需求涨了，但不等于换个名字就能当 FDE。售前、实施、驻场外包换名片，还是外包。→ 先搞清楚这个岗位的责任边界。

**低估数据这一关：**"客户说数据都有"就信了。真进场才发现数据在纸上、脑子里、老系统里、倒闭公司的硬盘里。→ 进场前先摸清数据地图。

**立项时不做翻译：**把老板的"全面智能化"当成需求，直接开工。做着做着发现，没人知道做成了什么。→ 立项第一天，先把愿景翻译成一个具体问题。

## 本章行动清单

读完这一章，先做一件事：翻出你最近经手过的、或者你听说过的、一个"上线后没人用"的项目，用书里那套镜头扫一遍——

它是死在五道路障里的哪一道（还是好几道一起）；

那个项目里，有没有一个人从头到尾对"落地"负责；

如果当时有一个合格的 FDE，他最该在哪个节点、做什么不一样的动作。

这道题没有标准答案，但你每扫一遍，就离"下次轮到你的时候不重蹈覆辙"近一点。下一章，我会把 FDE 和你熟悉的那些岗位——算法工程师、销售工程师、交付顾问——一个个摊开比，讲清楚它到底差在哪，以及"结果"这两个字，对 FDE 意味着什么。

<div style="page-break-after: always"></div>

## 第 2 章 你不是在卖模型，是在交付结果

FDE 的产出，不是代码，不是模型，而是能被客户使用、并且愿意续费的结果。

在上一章，我们说清楚了 FDE 为什么会存在。这一章，我们要把这个岗位的边界画清楚——它到底和那些你熟悉的岗位差在哪，以及"结果"这两个字，为什么是 FDE 全部工作的锚。

### 2.1 FDE 与算法工程师、销售工程师、交付顾问的区别

很多人第一次听到 FDE，第一反应是："这不就是驻场开发吗？""这不就是售前吗？""这不就是实施顾问吗？"

不是。虽然它们长得有点像、活儿有时候也重叠，但内核完全不同。我们先看一张表。

| 岗位         | 核心产出            | 关心的成功标准             | 在项目里的位置                |
|------------|-----------------|---------------------|------------------------|
| 算法工程师      | 模型、算法           | 模型的指标（准确率、召回率、推理速度） | 造武器的人                  |
| 销售工程师 / 售前 | 演示、方案、合同        | 能不能拿下单子             | 卖武器的人                  |
| 交付顾问 / 实施  | 一个按需求上线的系统      | 按 SOW 交付、验收通过       | 运送和安装武器的人              |
| FDE        | 客户真正用起来、产生价值的结果 | 客户有没有因此变好、有没有续费     | 手把手教客户用好武器、并确保它发挥了作用的人 |

看懂这张表，你就明白了 FDE 和其余三个岗位的本质差异。但光看表还是抽象。我给你讲一种我见过太多次的死法。

有一类企业 AI 项目，复盘的时候特别憋屈。因为你查来查去，会发现一个吊诡的事实：项目里的每一个人，都没做错自己的事。

**算法工程师，对模型负责。**他的工作是让模型在跑分上变强。至于模型到了客户手里有没有用，那不是他的职责范围。这没有贬义——分工本该如此。但这也意味着，模型强不等于项目成。

我见过一个做客服问答的，模型跑分一骑绝尘，准确率、召回率、推理速度，验收那天指标全绿。上线三个月，日活是零。你说他做错了吗？没有。他连客户现场都没去过两次，他只认指标。没人告诉他，客服组长根本不想让一线“偷懒”——使用数据一旦冲上去，那个组的 KPI 就要被重新盘一遍。这一层，跟他隔着一整个部门；跟他的代码，隔得更远。

**销售工程师（售前），对签单负责。**他的工作是让客户相信“这玩意儿能解决你的问题”，然后签约。问题在于，签单之后发生什么，往往不在他的考核里。于是“过度承诺”成了一种行业常态：销售画了个月亮，交付端只能交出个灯泡。

他也没做错。他在方案里写了“三个月上线、全流程自动化、能顶几个客服”，这套话术不砸下去，单子就拿不下来。至于客户的数据里有一半是纸质单据、那套老系统连个接口都开不出来，他在签字的那个下午，是真不知道。你也不能要求他知道——都知道了，这单还怎么签？

**交付顾问（实施），对合同负责。**他的工作是照着 SOW（工作说明书）把系统做出来、验收通过。SOW 上写什么，他就做什么，做完拿钱走人。至于系统上线后用户用不用、业务变没变，同样不在 SOW 里。

他更冤。系统照合同装好了，验收单白纸黑字签了字，尾款也到账了，每一项需求都打上了勾。你没在合同里写“上线后日活要到多少”，现在反过来怪他没让用户用起来，这跟怪一个送外卖的没把饭喂进人嘴里，有什么区别？

而 FDE，**对结果负责。**他的成功标准只有一个：客户有没有因为这套 AI，真的变好了——省了钱、增了单、提了效，并且愿意为此继续付费。

注意，上面这个死掉的项目里，四个人物有三个在场，缺席的恰恰是第四个。算法、销售、交付，三段各自管各自，中间那条缝，没人补。日复一日，缝就裂成了沟，项目掉进去，连个响都没有。

所以 FDE 不是什么比另外三个"高级"的岗位。它是一个专门用来填这条缝的角色。一个企业 AI 项目里，算法、销售、交付各司其职又互相脱节时，FDE 是那个把这三段接起来、并且一路扛到"客户真正用起来"的人。

再说深一层，免得误会。有人会问，FDE 跟"驻场外包"是不是一回事？毕竟都待在客户现场。

不是。这俩看着像，内核正好相反。

驻场外包，卖的是时间。人在你公司坐着，你叫我干啥我干啥，按人头、按月结。FDE 不卖时间，卖的是"把问题拆清以后，企业不再需要人肉去扛"。

我见过一个场景，最能说明这中间的差别。同一个教育转化的烂摊子，转化率低得吓人。外包的思路是：补话术、多追单、让机器人更像真人——全是加量的活，人越堆越多。FDE 的思路是另一套：先别急着改话术，先看漏斗到底漏在哪。结果发现，用户上完第一节课后，从来没有被推进到"我的问题已经被识别了、我需要一个下一步的方案"这个状态。缺的压根不是话术，是一个状态跃迁。

同样坐在客户现场，一个在卖时间，一个在卖判断。

说到根上，客户缺的，从来不是一把更锋利的锤子。他们缺的是一个能走进车间办公室、告诉他们这把锤子该往哪颗钉子上砸的人。这个人，就是 FDE。

## 2.2 结果 $\neq$ 交付物，结果 = 客户行为改变

"对结果负责"，这句话听着像正确的废话。要让它有用，得先把"结果"定义清楚。

FDE 语境下的"结果"，有一个特别容易被误解的地方。

很多人以为，结果就是交付物——我交付了一个系统、一份报告、一个模型，这就是结果。

不对。

**交付物是手段，不是结果。结果，是客户的行为发生了改变。**

这一句得拆开揉碎，因为它太重了。它几乎是后面全书所有判断的那个原点。

具体说：

不是"我交付了一个问答系统"，而是"客服从每天翻两小时文档，变成了秒回客户"。

不是"我交付了一个对账工具"，而是"采购助理从每周花两天对账，变成了十分钟搞定"。

不是"我交付了一个报表看板"，而是"老板从月底才知道经营状况，变成了每天睁眼就能看到"。

看出区别了吗？交付物是"你做了什么"，结果是"客户因此变成了什么样"。

为了说透，我再给你看一个更极端的例子。它来自一个教育转化的项目，数据我做过脱敏。

那套系统的漏斗烂得吓人：近三十天进了 4090 条线索，真正走到支付的只有 92 个，粗略一算，支付率 2.25%。再往流转里看，更难看：D1 到课 1346 人，交作业的只剩 322；D2 到课 1006 人，完课 506；最后进入支付页的，50 人。每一层都在漏。

现在，如果你带着"交付物"的脑子去干活，你会交付一个更会聊天的 AI 客服。把话术写顺一点、多追几次单、让机器人更像真人一点。这些都不算错，但一条都没踩在刀刃上——你把话术写成了一朵花，也改变不了漏斗的形状。

真正的病灶，藏在细节里：用户上完第一节课之后，从来没有被推进到"我的问题已经被识别了，我需要一个下一步的方案"这个状态。这一跳不发生，后面所有的话术，都是对着空气广播。

所以这个项目真正该交付的结果，不是一个"AI 客服"这个物件，而是那个状态跃迁——让 D1 的作业变成一次启动诊断，让 D2 的人工接管，从"噪音"变成一串被识别出来的成交信号。

同一个项目，"交付物"视角看到的是机器人会不会聊天，"结果"视角看到的是用户在漏斗第几层掉了队、为什么掉队。这俩隔着的，是一整个 FDE 的价值观。

再看一个海外的老例子，Palantir 的。他们帮一家企业把 SAP、Salesforce、合同管理、订舱这些老旧系统接起来，做了好几个月。上线之后，客户每年多收回几千万美元，问询量还降

了差不多一成。你去问那个客户，你花的钱买的是啥？他不会说"我买了 Ontology"，他会说"我买回了那些本来会漏走的钱"。Palantir 的人自己说得也直白：Ontology 这个词，本身一分钱不值。

还有一拨人，最鲜活。那些靠做企业代运营、获客、出海陪跑起家的团队，转行来干 FDE，活得比不少正儿八经有技术团队的公司还滋润。他们跟老客户开口，不讲模型，不讲架构，就一句："我能帮你省两个人、提效三成。"单子，就签了。

你看，这几个例子，说的其实是同一句话：**客户掏钱的动机，从来不是"你造了个多厉害的东西"，而是"我这边的一個具体麻烦，没了"。**

为什么要把这条线画得这么死？因为它最难，也最容易糊弄。

交付物，是你这一侧的。你写代码、你出报告、你上线系统，指针握在你自己手里，多累多难你都能控制。行为改变，发生在客户那一侧——在别人的工位上、别人的流程里、别人的 KPI 下面。你控制不了它，只能撬动它。

这也是为什么 FDE 常常得待在现场。你坐在自己办公室，眼睛只能盯着自己的那堆交付物，越盯越觉得自己干得漂亮。只有待在客户现场，你才看得见那个客服是不是真的不再翻文档了、那个采购是不是真的不用熬夜对账了。

一旦你接受了这条线，很多选择会自动变清晰：你会开始问"客户用了吗"，而不是"我写完了吗"；你会开始看"业务指标变了没"，而不是"验收有没有通过"；你会开始在意"三个月后客户还在用吗"，而不是"尾款收没收到"。

反过来，不接受这条线的人，会在一件又一件"交付物满分、行为零分"的项目里，累到怀疑人生。做出来的东西又全又漂亮，客户就是不买单。原因只有一个：他从头到尾交付给人家的，都只是手段。

记住这句话，它会贯穿全书：**FDE 卖的不是代码，也不是模型，而是"客户行为的改变"。**

换句更狠的大白话——客户不是为 AI 付钱。客户在为少漏的线索、少丢的订单、少干的返工、少扯的皮、以及不再死绑在某一个老员工身上而付钱。想通这一层，你才算摸到了 FDE 的门。

## 2.3 FDE 的一个完整项目周期

既然 FDE 从头到尾对结果负责，你就得先对"全程"有张地图。不然容易一头扎进某个环节，忘了自己到底要走到哪。

我每次带新人进场，干的第一件事，是在白板上画六个箭头。画完告诉他：这，就是你接下来的全部工作。

一个完整的 FDE 项目周期，六个阶段，一条不能断的链：

**进场**——走进客户现场，搞清楚这家公司到底在做什么、疼在哪、谁能拍板、钱从哪出。这一步，决定你后面 90% 的胜率。进错了门、看错了人、摸错了底，后面每一步，都在还这笔债。

**定义**——从客户那句模糊的需求里，找到一个真正值得做、也做得动的问题，把它写成一句清楚的话。不做这一步，你后面干得越卖力，错得越离谱。

**部署**——用最小可行部署（MVD）切入，把第一刀真实地切进客户的业务里。记住是"切进去"，不是"摆上去"。这一刀要见血，要真的有人能用、用什么结果。

**激活**——系统上线，不算完。让用户真正用起来、习惯用、离不开，才是这一关。上线那天再热闹，一周后没人打开，等于零。

**续费**——守住已经证明的价值，让客户愿意继续掏钱；再从这个价值点，往更多场景扩张。做不成这一步，前面所有的努力，都是一场无偿的技术表演。

**复制**——把这次踩过的坑、沉淀下来的方法和组件，搬到下一个客户、下一个项目去。做不到这一步，你就是在用命换钱，永远原地起跑。

这六个阶段，就是这本书的主线。第二章之后，第三章讲"进场"，第四章讲"定义"，第五章讲"激活"，第六章讲"续费"，第七章讲"复制"。而"部署"，被拆进了第四章的 MVD 里。

这张地图上，还有三个地方，特别容易读错。我一个个说。

第一个，它不是一条直线，是一个环。很多项目做到第五步发现续不上费，你以为问题出在续费，其实是第三步切的那一刀，从一开始就不是客户最疼的地方。这时候你得老老实实回到第一、二步去。走不通就掉头，这不丢人；明知走不通还死扛，才丢人。

第二个，这六个阶段，拆开看，每一个单独拎出来，都是一门能养活一家小公司的专业活。进场是咨询和销售的事，定义是产品经理的事，部署是工程的事，激活是运营的事。在大多数公司里，这六段是六拨人在干。FDE 的特别之处在于，他要从头站到尾。他不一定亲自写每一段代码，但他得对整条链负责任。

第三个，也是最容易被忽略的：真正的功夫，一半花在动手之前，一半花在动手之后。进场和定义这两步，一行代码没写，却决定了你有没有资格写；激活和续费这两步，代码早就写完了，却决定了你写的东西算不算数。夹在中间的"部署"，反而是最不神秘的一步。

说到这，有人会问：这么长的一条链，一个人扛得动吗？扛不动的时候多了去了。所以才要 MVD，才要缩范围。你不是要一口气把六段都推到满分，你是要保证这条链，在"最小"的状态下，从头通到尾。链通不顺，你砍掉哪一段，前面做的都是零。

这也顺带回答了为什么 FDE 这个岗位难做：它要求的能力跨度，比任何一个单一技术岗位都大。工程师只管一段，FDE 要盯着六段，还得能一眼看出哪一段在漏水。

这两年 Agent 火起来，不少人松了口气，觉得 AI 能把 FDE 的活接过去一大半。我不这么看。

Agent 确实让 FDE 的手变多了，可它没有替你长出客户现场的眼睛，也没有替你承担签字的那个责任。六段链里最重的两段——进场看人，结果担责——恰恰是 Agent 帮不上的。工具越强，越要有人盯着"这条链到底有没有真的通"。那个人，还是 FDE。

## 2.4 FDE 的全套能力模型

地图有了，下一个问题更实在：站在这条链上的人，到底得长什么样？

我把 FDE 的能力成长，归纳成五个阶段。这五个阶段是递进的，但不是说必须 100% 打通上一段才能进下一段——它们更像五个逐渐打开的能力层。你打开到第几层，就能接多大、多复杂的活。

### 阶段一：技术基础

这是入场券。你得能动手：写代码、调模型 API、搭 RAG、用 Agent、处理数据。这一层，说难也难，说简单也简单——它是最"可学"的一层，网上课程一大堆，认真练上几个月就能入门。

但记住：技术基础只是门票，不是全部。我见过不少简历，"精通 RAG""熟练 Agent"写得满满当当，真扔进项目里，连客户的一张 Excel 都解不利索。问题不在他技术差，在于他以为技术基础就是 FDE 的全部。这一层，只是让你有资格站在门口。

## 阶段二：企业门槛

这是从"会写 demo"到"能进企业"的分水岭。

企业的环境，和本地开发完全是两个世界。遗留系统、老数据库、安全合规、私有化部署、权限体系.....这一层要练的，是"在脏、乱、旧的真实企业环境里，把东西跑起来"的能力。

我给你看一个反面，记得死死的。

一支五六个人的团队进场，接了个四十万的智能制造单子，本以为是去调模型、搭 Agent 的。进了场才发现：一线车间的数据记在纸质单据上，良品率的参数装在老师傅的脑子里，企业倒是有 ERP，却是十几年前买的，连个 API 都调不出来；当年开发这套系统的外包公司，五年前就倒闭了，源码丢了，数据库还加着密。

这支团队技术差吗？不差。他们死在阶段二的门槛上——只会写 demo，没练过跟脏旧世界打交道。六周过去，还卡在洗数据，模型一个没调，钱倒先烧穿了合同价。

你能不能接上一个没有文档的老系统？能不能在一个不许联网的环境里部署？能不能让方案活着穿过客户的安全审查？这三问答不上来，你就还是个票友，不是个能进场的 FDE。

## 阶段三：运营成熟度

东西上线了，考验才真正开始。

这一层练的是：系统稳不稳定、坏了怎么快速恢复、异常怎么监控、出了问题怎么排查。用一句更狠的话说——**上线不是结束，上线才是真正开发的开始**。能把一个系统在真实业务里稳定运营下来，你才算过了"可靠"这一关。

为什么这一层要单独成段？因为上线之后冒出来的问题，你在开发机上，永远想象不到。

我做过一个知识库。上线测试，AI 回答错了一个产品问题。我们第一反应是模型胡说，查了，不是；又查知识库召回，也不是；再查扫描件解析，都对。一路追下去，技术链路全是对的。真正错的，是很多年前打印出来的一份产品设计文档，上面印错了一个数字。老员工都

知道这处错，平时在脑子里自动掰回来，可这个"正确的改法"，从来没进过任何一个系统。于是，AI 忠实读走了那份错资料，非常准确地，给了一个错误答案。

这种坑，只有上线后才冒头。运营成熟度，练的就是你撞上这种坑时的那份定力：不慌，能追，追得到根。再高一层的，还会更进一步——为失败而设计：上游模型挂了就自动降级，再挂就切缓存，用户的体验，不中断。

## 阶段四：软技能与商业

到这一层，你已经不只是在"做"了，更是在"谈"和"算"。

软技能，是听懂潜台词、处理冲突、建立信任。商业能力，是算得清一笔账：这个项目值多少、省多少、赚多少、验收按什么标准算。

这一层最难学，因为没有教科书，只能靠一个个真实项目、一次次真实的磕碰去磨。但它常常是决定性的——前面三层练得再硬，这一层不开窍，你也只配给人打下手。

说个特别有说服力的事实。这几年，真正靠企业 AI 服务活得滋润的那批人，多数不是算法工程师，也不是架构师，而是过去做企业代运营、获客、出海陪跑的。他们没有正规的技术团队，这反而成了优势——天然避开了深度定制的坑。他们靠 *vibe coding* 自己搓轻量智能体，但真正让单子签下来的，是那句"我能帮你省两个人、提效三成"，以及跟老客户攒了多年的信任。

这批人告诉我们一件事：解决 80% 到 90% 的场景，拼的是业务理解的深度，远没到拼高深技术。业务理解、算账、谈单，这些恰恰是阶段四的东西。这也印证了那句老话——**工程能力是 FDE 的门槛，客户与商业能力决定 FDE 的上限。**

## 阶段五：对外证明

最高的一层，是"可传承"和"可复制"。

你做的项目，换一个人，能不能接手？你沉淀的方法，能不能复制到下一个客户？你积累下来的东西，能不能变成组件、变成产品、变成团队的资产？

到了这一层，你就不再是一个"接活的人"，而是一个"能放大价值的人"。

那批代运营出身的人，为什么能活得久？除了懂业务，还有个关键：他们手里攥着多年积累的存量客户，深度绑定了 WorkBuddy、阿里、360 这些平台生态，厂商还会给他们导流。他们把一个一个定制项目，滚成了可以持续放大的一盘生意。这就是阶段五的样子。

五个阶段，对应的正是本书的脉络：技术基础，是你翻开这本书就已经在打的地基；企业门槛和运营成熟度，贯穿在进场、部署、激活这些章节里；软技能与商业，是续费和复制这两章的核心；而对外证明，就是第七章规模化复制的终极目标。

最后补一句很重的话，把它放在这一章的末尾：在中国，FDE 最危险的结局，不是找不到活，而是兴冲冲冲进去，把自己干成了一个懂点 AI 的驻场外包。

这五个阶段，就是防止你滑向那个结局的护栏。每往上走一层，你就离"卖时间"远一点，离"卖结果"近一点。

## 章末工具箱

FDE 能力模型五阶段对照表

| 阶段        | 要练的能力               | 最典型的标志           | 对应本书章节  |
|-----------|---------------------|------------------|---------|
| 1. 技术基础   | 编码、模型 API、RAG、Agent | 能独立做出一个能跑的 demo  | 全书前提    |
| 2. 企业门槛   | 遗留系统、接口、安全合规、私有化    | 能在真实企业环境里把东西跑起来  | 第 3、4 章 |
| 3. 运营成熟度  | 稳定性、监控、故障恢复、热修复     | 上线后系统能稳定运营       | 第 5 章   |
| 4. 软技能与商业 | 客户沟通、信任、ROI、计价      | 能算清账、谈得下、续得上     | 第 6 章   |
| 5. 对外证明   | 可传承、组件化、产品化、复制      | 项目能沉淀成资产、复制到多个客户 | 第 7 章   |

### 用这张表给自己定位

如果你在 1、2 层之间：先把企业门槛补上，别再满足于"跑通 demo"。去找一个脏、乱、旧的真实环境练手，而不是再做一只漂亮的玩具。

如果你卡在 3 层：说明你缺的不是技术，是对"上线之后"的关注——往下读第 5 章。

如果你 1-4 层都不错、唯独 5 层乏力：你的天花板在"复制"，第 7 章是给你的。

如果四层都很强，却总觉得自己在当高档外包：翻回这一章 2.1 结尾那段。你多半不是能力不够，是没拿到"定义"和"沉淀"的那点权力。

## 交付物 vs 结果：一句话自查

每当你做完一件事、准备汇报之前，先问自己一句：

**这个月，客户的哪一个具体动作，变了？**

答得上来，还能用一个数字撑住（"客服找资料从两小时到秒回""漏单从上月 17 单掉到 3 单"），你交付的是结果。

答不上来，只说得出一堆"我建了什么""我上线了什么""我优化了什么"，那你交付的，多半还只是交付物。

## 四个岗位：一句话记住边界

算法工程师：对模型负责，指标绿是他的全部。

销售工程师：对签单负责，拿到单是他的全部。

交付顾问：对合同负责，验收通过是他的全部。

FDE：对结果负责，客户变好、愿意续费，是他的全部——也是这四个人里，唯一一个事情做砸了之后，除了他自己，没人能替他兜底的角色。

从下一章开始，我们正式进入 FDE 的工作现场。第一件事，是"进场"——在你写下一行代码之前，先学会看清一个企业。

<div style="page-break-after: always"></div>

## 第 3 章 进场：先画出五张地图

进场前 90% 的胜率，取决于你有没有画清数据、流程、组织、系统、政治这五张地图。

很多 FDE 栽跟头，不是栽在技术不够，而是栽在"进错了门"——要么进了个根本不值得服务的客户，要么进去之后两眼一抹黑，被客户的真实状况打了个措手不及。

这一章，讲的是你在写下第一行代码之前，该做的所有事：挑对客户，看对人，摸清底。

有人会问：书不是讲"怎么交付"吗，怎么一上来先讲"怎么进场"？因为问题不是凭空找到的。前一章说要对问题做三关检验，但问题本身，长在客户现场的泥地里——长在车间的纸单据上，长在老师傅的脑子搭着的那点经验里，长在两个部门积了三年怨气的那张办公桌两侧。你不进场，就只能在会议室里，听一套被人精心粉刷过的说辞。

所以这一章，是我整套方法论里，唯一一门"不看代码"的功夫。但它比代码要命。

我见过太多人，技术手册背得滚瓜烂熟，进场第一周就被现实教做人。差别不在代码，就在这件事上：你有没有在动手之前，先把这家公司看得明明白白。

### 3.1 灯塔客户战略：先做标杆，不做炮灰

先讲一件我忘不掉的糗事。

有次一个需求，我按真实交付周期和人力成本，报了十万。调研要几天，开发要几周，上线陪跑又要几周，人力加差旅再加交付风险，十万真的只是贴着成本报的，没黑他。

客户当面没说什么，我当是成了。

第二天，他甩过来一张截图，问我："为什么网上有人只报两千？"

我点开一看，就是个卖模板的。要么是压根没听清需求就甩了个地板价，要么是准备用免费的东西拼一拼。两份报价单，交付范围、验收标准、质量保障，完全是两个世界的东西。但对甲方来说，上面写的都是四个字："AI 解决方案"。他说不出一句"这俩到底差在哪"，他只知道一个十万，一个两千，差了五十倍。

这单最后没成。被那个无底线兜底的小作坊，用成本价甚至亏本价抢走了。

我不怪客户。换我是他，看到一个能便宜五十倍的选择摆在那儿，我也会心动。我怪的是自己——把力气花在了一个错误的战场上。

互联网是个比价修罗场。公域询价这个基本盘，本质上不可靠。你在那儿接单，等于把自己放进了最低价拍卖会：比谁的报价更低，比谁更能扛价、更能亏。你不是在交付价值，你是在跟一群不打算赚钱的人赛跑，这场赛跑，你从起跑线上就输了。

那怎么办？答案是：别在公域抢单，在线下挣信任。

这，就是灯塔客户战略的起点。

刚入行的人有个通病：有单就接，来者不拒。这事能理解，没项目的时候，谁都得先活下来。但你要是把"有单就接"当成长期战略，很快就会掉进泥潭——被一堆小单、烂单、伪需求单拖死。累死累活一年，口碑没攒下，钱也没赚到，人就废了。

真正聪明的做法，是集中资源，先拿下一个能当"招牌"的标杆客户。

什么是灯塔客户？不是"最大"的那个，也不是"给钱最多"的那个，而是"做成之后，能照亮你后面一整片市场"的那个。

它通常有几个特征。

第一，问题典型。它的痛点，是这个行业里千千万万家公司共有的痛点。你解决它一家，等于手里攥了一张能复制到同行的通行证。反过来，那种只有他家才有的、怪到找不到第二家的需求，做得再漂亮，也只能卖一回。

第二，愿意配合。客户不是把你当"乙方"使唤，是真的想一起把事做成。这一点极其关键。不配合的客户，再有钱，也会在数据和流程上把你卡死，你连门都进不去几次。

第三，有决策权的人直接参与。拍板的人愿意出来露个面、聊两句，比你在下面绕三个月都管用。

第四，结果可展示。做出来的效果，能变成案例、变成标杆、变成你下一单里讲故事的素材。灯塔客户给你最值钱的东西，不是那一单的钱，是"可以拿去给别人看"的那份底气。

同时，你要学会筛掉两类客户。

一类是伪需求客户。嘴上说着要 AI，其实就是赶个时髦，或者写进 KPI 给上面看。需求是想象的，预算是没有的，痛苦是不疼的。你陪他聊三个月，方案改了八版，最后一问预算，他说"我们先探讨探讨"。

另一类是无决策权客户。跟你聊得火热的那个人的签不了字、批不了预算。你陪他开十次会，不如找到真正拍板的人开一次。判断标准很残酷：聊了三个会，还没见到出钱的那个人的面，这单基本就是虚的。

这两类客户，识别起来有个土办法，各问一句。

问伪需求客户："这个项目要是做成，您打算看哪个数发生变化？"他要答不上来，或者答的都是"感觉会更好""提升竞争力"这种虚词，说明他自己都不清楚要什么。

问无决策权客户："这个预算，最后要谁点头？"他要是支支吾吾，说"得看领导""我提上去再议"，你就要警惕了——他可能只是个传话的。

各问一句，一分钟，就能帮你把八成的水客户挡在门外。

怎么筛？有个很土但管用的判断：在签单之前，先把"谁出钱、谁拍板、谁受益、谁抵制"这四件事问清楚。问不清，这个客户就不值得你花大力气。

这里再回到那件"十万对两千"的糗事。那单为什么黄？表面看是价格，根子上是我把客户放进了比价的语境里。真正的优质客户，很少是"全网比价"比出来的。他们靠的是信任——朋友的推荐、老客户的转介绍、你过去攒下的口碑。这些信任，只能在线下、在一次次的物理接触里长出来，长得很慢，但一旦长出来，是抢不走的。

所以，与其把精力耗在公域抢单，不如把手上每一个线下的信任关系做深。网上发内容、做发声，只是引子，是让人认识你的入口；真正的成交，永远发生在你坐在他办公室、喝着他茶、看他员工怎么干活的那一刻。

这，也是灯塔客户战略的底层逻辑：先用一个真客户、一个真结果，证明你靠谱；后面的客户，才会因为"靠谱"这两个字，主动找上门来。灯塔不是终点，灯塔是撬动后面一整片市场的第一根杠杆。

我知道这么说，有点站着说话不腰疼——新团队，哪来的老客户、哪来的转介绍？那我说个最土的起步法：从身边最信得过你的那一两个熟人做起，把第一个灯塔做成，哪怕它小，哪怕它不起眼。

我见过活得最滋润的那批做企业 AI 的，多数根本不是算法出身，是过去做企业代运营、做出海陪跑的。他们没什么像样的技术团队，但他们手里，攒了一堆多年的存量客户和信任关系。回头跟老客户说一句"我能帮你省两个人、提效三成"，单子就签了。为什么他们单子好签？因为信任已经在线下攒了十年，比价这个环节，压根就没机会出现。

所以灯塔客户战略，对新手真正的含义是：别指望一上来就钓条大鱼，先咬住一个信你、配合你、问题又典型的小客户，把它做成第一个招牌。招牌立起来，后面的事，才轮得到你挑。

为了不把日子过成炮灰，我习惯把遇到的客户，粗粗分成四档。

第一档，灯塔。痛点典型、愿意配合、决策权的人在，做成了能复制、能当招牌。这种，抢着做，让利也做。

第二档，炮灰。公域比价来的、预算没影的、需求听上去很大其实不疼的。这种，能不做就不做。

第三档，金矿。有钱、有预算，但问题太特殊，做成一单就一单，复制不动。这种，可以做，但别指望它给你攒口碑，就把它当现金牛。

第四档，毒药。伪需求加无决策权，再加回款烂。这种，白纸黑字也别签，签了就是给自己埋雷。

四档不是给人贴标签，是提醒你：每一次签单，都是在给后面的自己选路。选灯塔，你越走越宽；选毒药，你越陷越深。

### 3.2 行胜于言：少问"你想要什么"，多看"你现在怎么干"

去年有段时间，我常蹲在一家客户的采购部。就坐在最边上那个文员的工位斜后方，一张小板凳，一坐一上午。

那姑娘一上午干了什么，我记得清清楚楚。打开供应商发来的 Excel，打开财务系统，两台显示器来回看，一条条对账。对上一笔，键盘敲一下；对不上的，截图发微信问人。中间接了三个电话，全是催单的。快到中午，她起身去了趟茶水间，回来接着对。

我没问她一句"你想要啥系统"。我就是看。看了两个上午，我比客户自己的领导，还清楚她的活儿卡在哪。

找到客户之后，很多人的第一反应是：坐下来，问需求。

"您想要什么样的系统？""您理想中的效果是什么样的？""您对 AI 有什么期待？"

这么问，大概率会得到一个让你误入歧途的答案。

因为客户嘴里说出来的需求，往往不是他真正的需求。他不是骗你，是他自己也没想清楚，或者他只会用"我想要个 AI 系统""把数据打通""全面智能化"这种空话来表达。这些话，听着挺大，实际上什么都没说。

更有效的做法，是行胜于言：少问他要什么，多去看他现在是怎么干的。

这背后的道理一句话：需求会撒谎，行为不会。

我见过太多次了。老板在会上一本正经地说"我们最大的痛点是数据孤岛"，可你走到楼下，看到的是采购助理每周花两天，把供应商的对账单一条条誊进 Excel，眼睛都快贴到屏幕上了。那个正在流血的伤口就在那，可没人提起它，因为它太不起眼了，老板早就习惯了它在那儿慢慢流血。

你怎么去观察？三个层次。

第一层，看他的人在做什么。到现场去，坐到那个干活的员工旁边，看他一上午到底在做什么动作。他是在几十个窗口之间来回切换？是在手工把 A 表的数据往 B 表里誊？是在等某个永远慢半拍的审批？还是在反复给一个三天两头出错的系统擦屁股？这些真实的动作，才暴露了真正的痛点。

这里有个窍门：别坐得太显眼，别一上来就提问。你就安安静静在旁边看一上午，拿个本子记。人一旦知道你在观察，多少会装一下。你要看的，是他没察觉你在看时，那个最自然、最原始的动作。

第二层，看他的事在流。跟踪一条业务线，从起点走到终点。一个订单从进来，到最后关单，中间经过了谁的审批、绕了几个弯、卡在哪一步、在谁的桌上躺了几天。你会发现，真正拖慢业务的那个环节，往往不是老板嘴里说的那个。

第三层，看他的钱在走。哪个环节在漏钱？哪个环节在重复花钱？哪个环节因为慢、因为错，造成了额外损失？钱，是商业语言里最诚实的信号。一个流程慢，你可以说是"体验不好"；但一个流程漏钱、重复花钱，那是能写到合同里的硬指标。

看钱有个笨办法，就是找财务要三张表：哪类支出在涨、哪类回款在拖、哪个环节老是在赔。别小看这三张表，它们往往比任何需求访谈都更接近真相——一个公司嘴上说"我们要效率"，账上写的却是"我们在漏钱"，你听前者还是信后者？

我自己有个习惯：凡是进场，先跟财务聊半小时。不是问账，是问"你们最头疼的三件事"。财务看问题的角度，和业务、和技术，完全不一样。他们说出来的那个"最烧钱又最没辙"的环节，十有八九，就是这单 AI 最该去啃的那块骨头。

这里有个反例，第 4 章会从头到尾细讲：那家代工厂。团队进场第一天，坐在会议室里听老板讲"全面 AI 化"的愿景，听了半天，听得热血沸腾。假如他们当时没在会议室坐那么久，

而是早一天走到车间里，亲眼看看那张每周要手工核对两天的采购对账单，看看那些还压在纸面上的经营单据，结局可能完全不同。

所以，记住这一句：新手听客户说什么，老手看客户怎么做。

而 FDE，必须是那个愿意把手插进客户真实工作流里、沾一手泥的人。

贴身跟踪这事，说得轻巧，做起来有个度。你跟得太近，人家紧张，动作变形，你看到的就是一场表演；跟得太松，人家根本不带你，你只配看到他们想让你看的那部分。我摸索出来的分寸是：先花半天，让人习惯你这个人杵在那儿，等他开始当你不存在了，那些真实的抱怨、真实的绕路、真实的重复劳动，才会自己冒出来。

还有，多听抱怨，尤其是一线员工对现有系统的抱怨。他们骂得越狠的地方，八成就是你将来最该下手的地方。

追问也有门道。别问“你觉得哪里可以改进”，这种问题太虚，答不出东西。问具体的：“你刚才这个动作，为什么要先导出去、再导回来？”“这张表，是谁传给你的，你拿到之后还要改什么？”顺着具体动作往下问，你才能挖到那个他都已经麻木了的真痛。你不沾泥，就永远只配在会议室里，听一版又一版被美化过的故事。

### 3.3 进场尽调：五张地图

行胜于言，看的是“点”。但只看点，容易一叶障目。你还需要一个全局的框架，把这家公司彻底摸清。

这个框架，就是五张地图：数据地图、流程地图、组织地图、系统地图、政治地图。

这五张图不是我想出来的花活。它是我把这些年撞过的坑归了归类，归出来的。凡是企业 AI 落地的项目，踩的坑翻来覆去就那么五类：跟老系统集成、安全合规、部署可靠性、商业价值证明、知识传承。这五类障碍，往前倒推一步，就是你进场时该摸清的五样东西——我把它们画成五张地图。

这五类障碍往前倒推一步，其实就对上了五张地图：跟老系统集成，对应系统地图；安全合规，对应数据地图里“谁有权、能不能上网”那一半；部署可靠性，对应流程地图——你的东西得真嵌进流程转起来；商业价值证明，对应组织地图，谁受益、谁买单；知识传承，对应政治地图里那些隐性的人情与暗流。

进场之前，这五张地图你画得越清楚，后面踩坑的概率就越低。画不清楚，你不是在进场，是在进坑。下面一张一张说。

## 数据地图：数据在哪、谁有权、质量如何

数据，是 AI 的粮食。没有数据，模型再强也是饿着肚子。

画数据地图，就搞清三件事：数据在哪，数据归谁管，数据质量如何。

先说"在哪"。

这一问，常常在不经意间，就给整个项目判了刑。

我讲一个真实的例子，这个故事后一章还会从别的角度讲，但这里我要从"数据地图"的角度讲。国内某顶级家电品牌的一家代工厂，要搞 AI 转型，合同四十万。团队五六个人进场，满心期待要大干一场。结果第一个星期，他们想找数据，找到的是车间里那些皱巴巴的纸质单据——有些经营数据，到今天还只记在工人手写的单子上，录入系统？不存在的。

第二个星期，他们想找工艺参数，找到了几个老师傅。那些关乎良品率的关键参数，只存在于老师的脑子里，人家凭手感调机器，几十年的经验，没有一个数字进过电脑。

数据在纸单据上、在人脑子里。这就是"数据在哪"最要命的一种答案：它根本不在一个能被 AI 读到的形态里。你连粮食都没有，还谈什么做饭。

我把企业的数据形态，粗略分成四档，一档比一档难啃。

第一档，系统里的结构化数据。有库、有表、有接口，这是最好下嘴的，三周能见效果。

第二档，散落在 Excel、Word、扫描件里。要做抽取、解析、清洗，工期翻倍起。

第三档，纸质单据和台账。得先找人录入、做 OCR、做结构化，这是一门独立的体力活。

第四档，人脑里的经验。像代工厂那个凭手感调机器的老师傅，参数只在脑子里，你得把人请来、把经验倒出来。这一档最难，也最贵，因为它不是数据工程，是知识工程。

进场第一天，你就该把客户的每一类关键数据，归到上面某一档里。归完你自然就知道：这单子是顺手能接，还是要先脱层皮。

再说"归谁管"。

很多时候，数据拿不拿得到，不取决于技术，而取决于那个攥着数据钥匙的人，愿不愿意松手。你可能技术上一小时就能调通接口，但那个管数据的部门，能让你走流程走上一个月。这一项，往下会跟“组织地图”“政治地图”缠在一起，先记着。

关于“谁有权”，我记得早期有个教训。客户口径说“数据都可以给”，我信了，方案按全量数据设计。真到要数据那天，IT 说“安全条线还没批”，安全说“要走合规流程”，合规说“等业务负责人签字”，业务负责人又说要等上面的批示。一圈踢皮球，踢了一个月。

从那以后，我把“谁有权”这条，拆成两问来问：一问“名义上谁批准”，一问“实际上卡在谁”。两问的答案常常不是同一个人。你要提前把“实际卡点”那个人找出来，而不是等进去之后，被一张张表踢来踢去。

最后说“质量如何”。

字段全不全？口径统不统一？要不要清洗？清洗的成本有多大？这一项，直接关系到你后面的工作量和报价。很多项目看着单子漂亮，一做起来，才发现光清数据就要吃掉一半工期。你进场前不把这一项问清，报价就等着往里贴钱。

画完数据地图，你就能提前回答那个最关键的问题：这个项目，是“有数据可用”的活，还是一场先要翻旧账的浩劫？

那家代工厂，如果进场第一天，先花半天把数据地图画出来，答案会刺眼得很清楚：它是一场浩劫。数据不在库里，在纸上、在人脑里。可惜他们是一头扎进去，到了第四十五天，还在洗数据，连界面都没给客户看到。这一刀，本来是可以提前躲开的。

所以数据地图怎么问出来？三个问题，一句句问，别怕问得土。

第一句：“这份数据，现在存在哪？”让他指给你看——是系统、是文件、是抽屉里的纸，还是得去问某个老师傅。指得出来，才算数。

第二句：“我要用这份数据，得找谁签个字？”把权限那条路提前走一遍，看它卡在哪个环节。

第三句：“这数据，准吗？”问这句的时候，盯着那个管数据的人的眼睛。他要是犹豫半秒，说“有份数据是线下对的、没进系统”，你就抓到了一条大鱼。

数据地图画到这一步，你对项目就有底了：它是三周能见效果的活，还是三个月都未必出得了第一版的活。

## 流程地图：业务怎么流、瓶颈在哪

数据是静的，流程是动的。AI 要产生价值，得嵌进流程里。一个躺在数据库里的模型，不改变任何人的任何动作，就一文不值。

画流程地图，就是沿着业务线，把关键流程从头到尾捋一遍：谁做什么、输出什么、交给谁、卡在哪。

重点找两类东西。

一类是重复动作。每天、每周都在做、且可以标准化的动作。对账、誊数、催单、汇总报表、回同样的问题。这些是 AI 最好下手的地方——重复，意味着有规则可循；有规则，才谈得上自动化。

一类是瓶颈环节。整条流程里最慢、最容易出错、最依赖某个人的一环。全公司等一个人的签字，一个环节错了后面全部返工，一个关键岗位的人请假整个业务停摆。这些是价值最大的地方，因为撬动它们，撬动的是全局。

这里要特别提一个坑：很多流程，是"名义流程"和"实际流程"两张皮。

名义上，数据该走系统、该过审批、该留痕。实际上，大家靠嘴、靠微信、靠私下变通，系统是个摆设。你照着名义流程画一张漂亮的流程图，交上去，客户还很满意——可那是张废图。

怎么挖出实际流程？还是老办法，贴身跟。跟一个真实的单子，从头走到尾，看它真正的每一步落在谁手上、以什么方式交接。你会发现，真正的流程，和你听到的，经常是两码事。

把流程地图画实，有个笨办法比什么方法论都好使：挑一个真实的单子，从头跟到尾。跟的时候，脑子里不停念三个问题——这一步谁做？做多久？做完交给谁？等这个单子走到头，把每一步、每一个交接、每一段等待时间标出来，瓶颈自己就浮出来了。

我见过最典型的，是一个审批要过五道，其中三道是"看一眼就签"的橡皮图章。全标出来之后，客户自己都愣了：原来我们一半的时间，耗在三道没用的签字上。

画完一张流程地图，我通常会逼自己交出三样东西。

一样是现状流程——它到底在怎么跑，不是它该怎样跑。

一样是瓶颈清单——哪几步最慢、最易错、最吃人工，按耽误的钱排个序。

一样是重复动作清单——哪些环节天天在重复、且能标准化。这个清单，就是你后面 MVD 的候选池。第 4 章要讲的最小可行部署，第一刀往哪切，答案一半藏在这张清单里。

三样东西交齐，流程这张图才算画完，而不是画了个框框就收工。

## 组织地图：谁拍板、谁受益、谁抵制

做企业项目，本质上是和组织里的人打交道。技术反而是次要的。

画组织地图，要回答几个问题：这件事，谁有最终决策权？做成了，谁受益、谁升职？做砸了，谁背锅？谁可能因为这件事丢饭碗、丢权威，从而成为一个隐形的抵制者？

其中，“找出抵制者”这件事，价值之大，怎么强调都不过分。

一个提高效率的 AI 系统，看起来是“纯粹的进步”，好像人人该拍手叫好。但落到具体的人头上，它可能意味着某个部门的人要被裁，某个中层靠信息不透明维持的权威被打破，某个老员工引以为傲的“不可替代性”被稀释。

这些人的反对，不会写在脸上。他们会藏在每一次“再研究研究”里，藏在每一次审批的无故拖延里，藏在“这个需求还不明确，我们回去再议”里。你看不出他在反对，但项目就是走不动。

到这里，组织地图已经隐隐指向了旁边那张更黑的图——政治地图。这个先按下，下一节再说。

画组织地图有个捷径：从“谁最紧张”入手。你进场谈这个项目，去看每个在场的人的反应。谁眼睛亮了、谁一直抢话、谁从头到尾不吭声、谁在你说到某个环节时脸色变了一下。这些反应，比墙上的组织结构图诚实得多。

还有个土办法：请客户吃饭，别只请领导，也请一线骨干。饭桌上，酒过三巡，谁跟谁不对付、谁的口碑怎么样、这公司真说了算的是谁，几句话就套出来了。办公室里的正襟危坐，套不出这些。

组织地图里还藏着一个新手常混的概念：决策链和汇报线，是两回事。

汇报线上，谁是经理、谁是总监、谁向谁汇报，一清二楚。但真要拍板一个项目，走的往往是另一条看不见的决策链——谁跟老板走得近，谁在关键议题上说一不二，谁的意见哪怕是随口一提，别人也得接住。

你对着汇报线做工作，方向全是错的。权限表上写着 A 负责这个项目，可你每次找 A，他都要“回去请示一下”。请示谁？请示的才是真正的决策链。

把决策链和汇报线都标出来，组织这张图才真的有用。否则你手里的，只是一张墙上的旧架构图。

### 系统地图：遗留系统、接口、技术债

企业不是白纸。你进场看到的那家公司，上面密密麻麻，全是积攒了十几年的系统。

画系统地图，就是摸清这套 IT 家底：现在在跑的有哪些系统？用的什么技术？哪些有接口，哪些是黑盒？当年的开发商还在不在？文档还在不在？

这听着是技术人员该干的活，好像可以扔给后端同事。但它的商业意义巨大——系统的现状，直接决定了你方案的成本和可行性。

还是那家代工厂。

他们第三个星期想接系统，发现企业是上过系统的，但那是十几年前买的破烂 ERP 和 CRM，连一个最基础的 API 接口都调不出来。想找当年的开发商要接口文档，一打听，那家本地小外包公司，五年前就已经倒闭清算了。源码丢了，数据库还加着密。

一个连 API 都调不出的老系统，光是要接上它，就可能吃掉你一半的预算。你以为你在做 AI，其实你在做软件考古——把一具十几年前的尸体，挖出来接上电。

我把系统按接口情况分三种颜色记：白的，有标准接口、有文档、有活人在维护，算你的朋友；灰的，接口能调但没文档，得靠人肉试，算半个朋友；黑的，就是代工厂那种——开发商倒闭、源码丢失、数据库加密，这种一碰就是无底洞，能绕就绕。

进场时把要碰的系统挨个标上颜色，白的放心用，灰的谨慎用，黑的，直接把它算进风险清单，报给客户看。不是所有黑盒，都值得你冲进去赌命。

系统地图的价值，就是让你在进场前就看清：这单子的真实成本里，有多少是 AI，有多少是考古。

砍系统地图时，有个问题必须问死：那些老系统，除了当年写它的人，现在还有没有人能维护？很多企业的 IT 家底，看着一屋子系统，实际能维护的人，要么离职了，要么退休了，要么就是那个快退休、谁都不敢让他走的"唯一懂的人"。

这种系统，别指望它配合你。能绕着走就绕着走；绕不开的，就把"集成它"这一件事，当成一个独立的小项目单独报价。你把考古的钱算进 AI 的报价里，账就乱了，客户也会觉得你在敲竹杠。

## 政治地图：部门利益、预算归属、隐性阻力

这是最微妙、也最容易被新手忽略的一张图。

任何一件事，只要涉及多个部门，就一定有政治。预算从哪个部门出？谁和谁不对付？谁在给谁使绊子？这个项目，是不是某个领导拿来表演政绩的？

政治地图画不出来，你甚至可能从一开始就站错了队——对着 A 部门大谈特谈，结果最后拍板、出钱的却是 B 部门，而 B 部门恰好是 A 部门的死对头。你越努力，越把两边都得罪。

这条线，我放到 3.5 的那个完整案例里细讲。这里先记住一句：一张政治地图，画的是人心，不是架构图。架构图上没有的东西，往往才是真正卡死项目的东西。

政治地图怎么画？没有标准答案，但有几个必问的问题：这个项目，是谁最先提的？他提这事，是为了解决真问题，还是为了在老板面前立一个"数字化转型"的功？预算挂在哪个部门的头顶？那个部门图的是什么？要动的，是不是某个人的奶酪？

问这些，得找对时机、放低姿态。都是台面下的事，没人会在会议室里跟你讲。私下的一支烟工夫，往往比开三次会都有用。

五张地图，说穿了，都是同一件事：在动手之前，先看清这家公司作为一个真实生态系统的全貌。

数据是它的土壤，流程是它的血管，组织是它的骨架，系统是它的旧家具，政治是它的暗流。

你把这五样看透了，才谈得上"进场"；看不透，你进的就不是"场"，而是"坑"。

### 3.4 采购、法务、安全审查的通关策略

画完了五张地图，你以为就能开干了？还差最后一关——商务流程。

大企业和国企的采购、法务、安全审查，是出了名的磨人。很多项目，技术上都谈成了，最后却死在采购流程、死在合同条款、死在安全审查上。你交付团队在前面冲锋陷阵，后腰一整个流程卡住，前面全白干。

这三关，光有技术是过不去的，得有策略。一关一关说。

#### 采购关

核心是提前摸清客户的采购规则：多少钱以下的单子走什么流程？要不要招投标？要不要三家比价？回款账期是多久？发票怎么开？谁签字？

最忌讳的是，技术都做了一半，才发现采购流程根本没走通，或者这个客户的回款账期，长得能拖死你的现金流。

有个动作很有用：进场第一天，就去找采购的人，把“这单要怎么走”问清楚。很多技术团队一辈子不跟采购说话，等要付款了才开始补流程，一补三个月。

顺便说一句，采购关和 3.1 的灯塔战略是通的。你在公域接的比价单，天然就会被拖进“三家比价、价低者得”的流程里，从签单那一刻就输了。而靠信任进来的单子，往往能谈成更合理的商务条款。选客户这件事，决定着你在采购关的起点有多难。

这话延伸一句：别把公域线索一棍子打死，它只能当入口。网上发内容、写文章，是为了让客户在找你之前，就已经知道你是干什么的、你会怎么干。等他带着这点认知来找你，比价的冲动就小了一半。发声在网上，成交在线下——这句，就是采购关最好的护身符。

采购关还有几个高频雷，我一次说清。第一个，预算和立项。很多客户跟你聊得火热，其实内部连个立项都没有，钱根本没影。第二个，三家比价。公域来的单子这关几乎躲不掉；信任单有时能走单一来源采购，省你半条命。第三个，回款账期和质保金。有些合同，尾款压你一年，还要再压一笔质保金。账期这一项，谈合同的时候必须问死，别等做完了才发现是“做完一年后付三成”。

## 法务关

法务卡点，往往集中在几个老问题上：数据归属、知识产权、保密条款、违约责任、源代码要不要交。

这里要给技术人提个醒：合同是你唯一能拿来保护自己的东西。

别因为"关系好""都是朋友"，就不好意思谈条款。真出问题的时候，"朋友"这两个字帮不了你，"条款"才行。老板今天跟你说"咱哥俩这关系还签什么"，明天项目扯皮了，他翻脸比翻合同还快。

特别留意两条。一条是验收标准要写死，写清楚"什么算做完"，别用"达到预期效果"这种能扯一年的词。另一条是违约责任要对称，别只有罚你的、没有罚他的。

法务这关，除了验收标准和违约责任，还有一条最容易踩：源代码和知识产权的归属。有些甲方默认"我花了钱，源代码就是我的"，你不白纸黑字写清，后面扯皮能扯到天荒地老。哪些该交、哪些该留，签之前都写明白。

## 安全审查关

企业，尤其是金融、医疗、国企，安全审查是一票否决的。

你可能要面对：私有化部署、数据不出内网、权限审计、漏洞扫描、渗透测试、等保要求。这些要求，会反过来决定你的技术方案——有些看起来炫酷的技术，就是因为过不了安全审查，直接被毙。

所以，安全审查不该是最后才想起来应付的"麻烦"，而应该是方案设计时就考虑在内的约束。你方案里写"数据上云、用大厂 API"，到了安全关里一句"数据必须私有化"，整个方案就得推翻重来。

一个很实用的技巧：进场第一步，主动问客户要一份"合规与安全要求清单"。与其做到一半被审查卡住，不如一开始就把规则问清楚。规则越早拿到，你的技术路径就越清晰，返工就越少。

安全关里，私有化和数据出网是两条硬线。金融、医疗、涉密单位，数据基本别想出内网。你越早拿到他们的安全要求清单，越能早点把方案里那些"要上云"的部分改掉，省得到验收前最后一脚被毙。很多团队栽在安全关，不是方案不行，是到最后才发现，自家方案里藏着一个过不了审的隐性依赖。

三关讲完了，最后说一句总的想法：商务这三关，不能串行等。

新手常见的时间表是——技术先谈，谈成了再走采购，走完采购再过法务，过完法务再碰安全。一关一关排队，等到最后一关，半年过去了。

老手的做法是，进场第一天就三线并行：让老板去推内部立项，让法务先把合同模板丢过来，让安全先把检查清单发过来。技术方案往后的每一天，都照着这三份文件的约束来调。

省下来的，不是流程，是那一遍又一遍推翻重来的返工。

### 3.5 一次进场尽调，是怎么避开一个大坑的

前面讲的是地图本身，这一节讲一个完整的故事，说明画地图到底怎么救命。

一支小团队，接了一个中型制造企业的 AI 项目。明面上，需求很常规：给销售部门做一个"客户流失预警"系统。

如果直接开干，他们会怎么做？拉客户的 CRM 数据、训一个流失预警模型、做个看板给销售总监看。技术路径清清楚楚，闭着眼都能走。

但他们留了个心眼，进场时，顺手画了一下组织和政治地图。这一画，画出了一个重大问题。

真正推动这个项目的，是销售总监。但负责 CRM 数据、手握数据权限的，是 IT 部门的老大。而这两个人，私下里关系相当僵。

根子在上一个信息化项目。那个项目 IT 部门主导，销售总监处处不配合，最后项目黄了，两边梁子就结下了。这口气，一直没散。

更要命的是这里面还藏着一层。这个"流失预警系统"一旦做成，等于变相在说"销售团队丢客户，是因为跟进不力"。而销售总监真正想要的，其实不是预警——他想借这个系统，向老板证明：客户流失是产品质量问题，不是销售的问题。他想把锅，甩给生产部门。

你看这局棋：明面上是一个普普通通的 AI 项目，底下盘着的是两段积怨、一口锅、和一个想甩锅的人。

如果团队没画这张图，一头扎进去，结果几乎是注定的：数据拿不到（IT 卡着不让碰），需求对不上（销售总监要的根本不是预警，是甩锅），最后项目做不成还是小事，一圈人被得罪了个遍，这个中型企业圈子也就那么点大，口碑先黄了。

画了地图之后，团队做了两件事。

一是不再把这个项目定位成"销售部门的预警工具"，而是回头和几方沟通，拉上能拍板的老板，一起把目标重新定义成"全链条的客户健康度"——让它不再是谁手里的一件政治武器，而是一件全公司都能用的活。

二是提前预判到数据这一关会卡在 IT 那里，于是第一步没有去要全量数据，而是先拿一个最小切片验证价值，用那一点点看得见的成果，去换 IT 和老板的信任，再逐步扩大数据范围。

结果，项目虽然做慢了半拍，但绕开了那个几乎必踩的政治地雷，最终活了。

这个故事，你可能觉得有点戏剧化，好像每一步都是设计好的。但我想说，它的每一环，都是我在真实项目里反复撞见的东西：明面需求和真实需求错位；数据权限卡在一个积怨已久的敌对部门手里；一个好端端的项目，被人当成政治工具在用。

画不画那张地图，决定了你是提前拆雷，还是闭着眼踩上去。

这个故事还有一层，值得单独说。那个团队后来复盘，说他们最庆幸的，不是在画地图的当下发现了问题，而是"进场就画地图"这个动作本身，被练成了一种习惯。第一张政治地图替他们省了一条命，往后他们做别的项目，第一件事永远是先画地图——哪怕客户再急，也要挤出半天，把五张图各过一遍。

急，是最容易让人跳过这一步的理由。客户说"这项目十万火急，下周就要方案"，你一急，进场就开干，干到一半再回头补调查，补出来的全是马后炮。

我的经验正好反过来：越急的单子，越要先把地图画了。因为急的项目，往往藏着最多的雷。

多说一句怎么认隐藏的抵制者。他们从来不说"我反对"，他们用三种更隐蔽的招。

第一种，拖。永远在"再研究研究"，一拖就是几周，拖到你精疲力尽。

第二种，改口。今天说这样，明天说那样，需求像烙饼一样翻来翻去，让你永远做不完。

第三种，孤立。把项目圈起来，不让关键的人、关键的数据跟你接触，让你在一个真空中瞎忙。

这三种人，靠问是问不出来的，得靠那张组织地图和政治地图，提前把他们标出来。标出来了，你才知道下一步该去争取谁、绕开谁、或者干脆把谁拉进项目里，用利益把他变成合作者。

最后再补一句：五张地图不是画完就锁死的死图。项目推进的每一天，地图都在变——今天还配合你的部门，明天可能因为人事变动翻了脸；今天还堵着你的那个人，明天可能就调走了。所以地图要时不时更新，别拿进场那天的那一版，去指导三个月后的决策。

## 章末工具箱

### 工具一：进场尽调问卷（五张地图版）

以下问题，进场前逐条过。答不上来的，先别签合同，先回去摸底，摸不清楚就再摸。宁可在签单前多花三天，别在进场后填三个月的坑。

#### 数据地图

[ ] 关键数据存在哪？是数据库、Excel、纸质单据，还是只在某个人脑子里？

[ ] 数据的形态，AI 能不能直接读？要不要先做录入、识别、结构化？

[ ] 谁拥有数据权限？拿到数据要过谁那一关？那关好不好过？

[ ] 数据质量如何？字段全吗？口径统一吗？

[ ] 要不要清洗？清洗的工作量和成本，估过吗？

[ ] 有没有“纸面上对、实际上在人脑子里改”的隐性数据？

#### 流程地图

[ ] 核心业务线从起点到终点，经过哪些步骤、哪些人？

[ ] 从头到尾走一遍，最快多久？最慢多久？差异在哪？

[ ] 哪些环节是高度重复、可标准化的？

[ ] 最慢、最易出错、最依赖某个人的环节，是哪一个？

[ ] 名义流程和实际流程，是一套还是两套？

[ ] 有没有"嘴上走系统、实际靠微信"的环节？

### 组织地图

[ ] 这件事，谁最终拍板？谁出钱？

[ ] 做成了谁受益、谁升职？做砸了谁背锅？

[ ] 谁可能因为这件事丢饭碗、丢权威，从而成为抵制者？

[ ] 一线的真实用户是谁？他们现在是怎么干活的？

### 系统地图

[ ] 现在在跑的有哪些系统？用的什么技术栈？

[ ] 哪些系统有接口、哪些是黑盒？接口要接，找谁开？

[ ] 关键系统的文档还在吗？当年的开发商还在吗？

[ ] 有多少技术债？接一个老系统，大概要花多少成本？

### 政治地图

[ ] 预算从哪个部门出？这个部门图的是什么？

[ ] 涉及部门之间有没有过节？谁和谁不对付？

[ ] 这个项目，是否被某个人当作政绩或政治工具在用？

[ ] 有没有谁，表面支持、暗地里会拖后腿？

## 工具二：灯塔客户四问自查

签单前，把下面四件事问清楚。问不清的，这个客户就不值得你花大力气。

谁出钱？——钱从哪个预算走，账期多久。

谁拍板？——真正有决策权的那个人，我见过了吗。

谁受益？——做成了，谁得利、谁升职，他愿不愿意推。

谁抵制？——做成了，谁失权、谁丢饭碗，他会怎么拦。

### 工具三：五张地图一张表速查

| 地图   | 回答的问题          | 画不出来会怎么样          |
|------|----------------|-------------------|
| 数据地图 | 数据在哪、归谁、质量如何   | 一头扎进数据治理无底洞，洗到超预算 |
| 流程地图 | 业务怎么流、瓶颈在哪     | AI 落不进真实工作流，做了没人用 |
| 组织地图 | 谁拍板、谁受益、谁抵制    | 找不到真甲方，被隐形抵制者拖死   |
| 系统地图 | 遗留系统、接口、技术债    | 低估集成成本，一半预算拿去修旧账  |
| 政治地图 | 部门利益、预算归属、隐性阻力 | 站错队，把做成的事做成政治牺牲品  |

### 工具四：五张地图的绘制顺序

五张图不用同时开画，有个省力的顺序。

先画数据地图和系统地图，这两张偏硬，靠查、靠访、靠要清单就能画，最能快速暴露"这单子干不干得动"。

再画流程地图，靠现场跟单，把业务线捋出来，顺便把一线的真实声音听全了。

最后画组织地图和政治地图，这两张最软，靠吃饭、靠私下聊、靠观察与会反应，急不得，得让信息慢慢浮上来。

顺序不是死规定。但记住一条：硬的地图先画，软的地图后画。别一上来就问人家部门恩怨，人家还跟你不熟，什么都不会说。

### 本章要点回顾

灯塔客户战略：不接比价单，线下挣信任，用一个标杆照亮一片市场。

筛客户，先问清四件事：谁出钱、谁拍板、谁受益、谁抵制。

需求会撒谎，行为不会。少问"你想要什么"，多看"你现在怎么干"。

进场先画五张地图：数据、流程、组织、系统、政治。

数据地图防的是"数据治理无底洞"，系统地图防的是"软件考古成本"。

政治地图最容易被忽略，也最要命——它画的是人心，不是架构图。

采购、法务、安全三关，规则要进场第一天就问清，别做到一半回来补流程。

## 常见误区速查

**有单就接：**来者不拒，被伪需求单和无决策权客户拖到废。→ 先筛，再进。

**在公域比价：**一个十万的报价，被一个两千的地板价打垮。→ 线下建信任，别在修罗场赛跑。

**只问"你想要什么"：**客户说"数据孤岛"，真相是采购助理每周手薅两天账。→ 去看他怎么干。

**只看名义流程：**画了张漂亮的流程图，实际全在微信里跑。→ 贴身跟一个真单子。

**把地图当形式：**五张图一画就扔一边。→ 地图的价值，是用它改你接下来的每一步。

**商务流程留到最后：**技术做完了才开始补采购、谈安全。→ 进场第一天先把规则问清。

## 本章行动清单

读完这一章，先做三件事：

第一件，翻出你现在手里正在谈的客户，用"灯塔客户四问"过一遍。重点看第三问"谁受益"和第四问"谁抵制"——这两条答不上来，你先别急着推进。

第二件，约这个客户里一个一线员工，不是领导，就一线干活的。坐他旁边，安安静静看他一上午。把看到的真实动作，跟客户在会上的说法，放在一起比一比，看差多少。

第三件，用"五张地图的问卷"，从头到尾填一遍。哪张图上全是问号，哪里就是你将来最大的坑。把最黑的那张图，标出来，接下来先啃它。

这三个动作，加起来可能花掉你三四天。但比起进场后，在泥地里填三个月的坑，这点时间是笔划算得不能再划算的买卖。

画完五张地图，你就看清了这家公司。接下来，是全书最核心的一步——从这堆庞杂的现实里，找到那个真正值得解决、也做得动的问题。下一章，我们讲 PSF 与 MVD。

<div style="page-break-after: always"></div>

## 第 4 章 找到正确的问题：PSF 与 MVD

杀死项目的，从来不是技术。是从一开始就做错了问题。

那家代工厂的老板，是在一个饭局上被朋友安利了 AI 之后，才下定决心要"全面智能化"的。

饭桌上聊天的套路，大家都很熟。有人刚用了豆包，又试了 Codex 和 Claude Code，回来跟你讲 AI 有多神：文档自己写，代码自己跑，连报表都能自己拉。老板听着听着，眼睛就亮了。

他的需求，翻译成成人话大概是这么个意思：把人事的底层数据、财务的流水，全公司各个环节打通，上面搭一层智能体和数据看板，让信息自己流转起来。

听起来像智能制造的宣传片。合同签了，总金额四十万，不算大，但足够一个刚拉起来的新团队冲进去。

进场的是五六个人。他们当时的状态，应该是踌躇满志的——终于接了一个"智能制造"的大单，终于能甩开那些 winform 小票友，干点真东西了。每个人心里，大概都预演过一遍：模型跑通的那一刻、看板上第一条曲线跳起来的那一刻、跟老板汇报成果时那个"值了"的表情。

然后事情开始不对劲。

第一个星期，他们想找数据。找到的是车间里那些皱巴巴的纸质单据。有些所谓的经营数据，到今天还只记在一线工人手写的单子上，录入系统？不存在的。

第二个星期，他们想找工艺参数。找到了几个老师傅。那些关乎良品率的关键参数，只存在于老师的脑子里，人家凭手感调机器，几十年的经验，没有一个数字进了电脑。

第三个星期，他们想接系统。企业倒是上过系统，但那是十几年前买的破烂 ERP 和 CRM，连一个最基础的 API 接口都调不出来。想找当年的开发商要接口文档，发现那家本地小外包公司五年前就已经倒闭清算了，源码丢了，数据库还加着密。

到了第四十五天，他们在做什么？

在洗数据。

不是调模型，不是搭 Agent，不是画看板。是每天从早到晚，把一张张对不上账的表、一个个口径不清的字段，一点点往一块拼。六周过去了，连一个最简单的模型调优、一个具体的业务 workflow，都还没开始做。

更麻烦的是账。

按他们当时的消耗速度估算，光是把这堆陈年烂账理清楚，人力物力的成本就已经快要超过四十万的订单总额了。换句话说，活还没开干，钱已经烧穿了合同价。

这时候，团队被架在火上烤。

退，前期的工资差旅全打水漂，手里还捏着写死阶段节点和验收标准的合同，单方面违约的代价根本承受不起。进，就是个无底洞，越干亏损越多，后面不知道还要填多少人日进去。

这个项目最后怎么样，不重要。重要的是，它为什么会走到这一步。

这五六个人，技术差吗？未必。他们大概率能把 RAG 搭起来，能把 Agent 跑通，原理讲得比谁都清楚。他们甚至不缺勤奋——谁能在工厂里跟烂账死磕四十五天还不走，谁就不是懒人。

他们死在了一个更靠前的地方：**根本没搞清楚，这家公司的问题到底是什么。**

不是“数据没打通”这个问题没价值，而是这个问题，压根不是他们一个五六个人、刚拉起来两个月的团队，该去碰的活。问题选错了，后面所有的努力、加班、死磕，都只是在错误的方向上，把力气耗干。

这就是这一章要讲的事：怎么在一开始，就把问题选对。

## 4.1 概念验证坟墓

在讲怎么选对问题之前，得先承认一个扎心的事实：很多企业 AI 项目，死的时候，尸检报告上都写着同一行字——

**PoC 做得挺好。**

这是最要命的地方。PoC 不是失败的原因。PoC 漂漂亮亮地活下来，然后项目整体死掉，这个反差才是问题。

我去过太多这样的会议室了。花一两天，拿几份文档做个问答，用模型生成一份报告，接几个工具跑一段自动化，半天就能在旁边录个演示视频。投屏一开，演示跑得行云流水，会议室里掌声响了，老板点头了，大家站起来拍了个合照。散会之后，那个系统，再也没人打开过第二次。

演示越漂亮，落地越容易死。这个看似违背直觉的现象背后，有几个几乎必然的原因。

### **第一，演示用的是干净数据，生产是脏数据。**

PoC 里喂进去的，是整理好的、格式标准的、没有缺失的样本。文档是挑过的，字段是对齐的，接口是能通的。演示环境里没有意外，所以你跑得很顺。

但真实环境里的数据长什么样？我在这行的这些年，看到的实际状况，荒诞得像魔幻现实。

有些经营数据记在皱巴巴的纸质单据上；有些关键参数只存在于老师傅的脑子里；有些企业上了系统，却是十几年前买的破烂 ERP，连 API 都调不出；更绝的是，当年给他们开发软件的本地小公司可能早就倒闭清算了，你连找个人买接口文档都找不到。

演示阶段回避掉的这些数据问题，会在上线第一天，原封不动地，一个一个找回来。

### **第二，演示解决的是“能不能跑”，生产要的是“会不会有人用”。**

一个能回答问题的 Demo，和一个真的嵌进员工日常工作流、每天被几百上千次调用的系统之间，隔着一条很宽的沟。

沟的这一边是工程：代码、模型、接口、部署。沟的那一边是习惯、权限、责任、绩效考核——全是人和组织的事，跟技术半毛钱关系没有。

你可以让 AI 回答得再漂亮，只要它没有进入真实工作流，就没有产生任何业务结果。你可以让系统把接口调得再勤，只要权限、审批和失败恢复没做，企业也不敢真的让它去执行。

### **第三，也是最根本的——从头到尾，没有一个人定义过“什么算成功”。**

PoC 跑通了，大家都很高兴。但这个"跑通"，到底意味着什么？是省了多少钱，还是少了多少人，还是决策变快了？没有人把这个结果钉下来。

于是到最后验收的时候，交付方说"我按需求开发完了"，客户说"可是业务没变"。双方说的都对，也都错——因为他们从头到尾，说的就不是同一件事。

除了这三个技术性的原因，还有一个更隐蔽的组织原因，藏在 PoC 的起点。

在很多公司里，卖东西的人和干活的人，是两拨人。销售为了拿下单子，会往高了承诺：能一次性打通、能三个月上线、能替代一半人工。等签完单，交付团队进场一看，承诺的是星空，现实是泥地。销售已经在数提成，交付只能硬着头皮收拾残局。

这种"承诺与交付的脱节"，再加上前面三个原因，共同把"概念验证"一步步做成了"概念验证坟墓"。

概念验证这个概念本身没有错。错的是，我们把 PoC 当成了终点，而不是一个用来验证"问题到底选得对不对"的小实验。

PoC 的正确用法，应该是小成本、快速度地去检验：这个问题是不是真问题、值不值得投入、客户到底会不会买单。它是一张便宜的试纸，不是一个交付物。可在大多数项目里，它被做成了后者——于是"概念验证"这个词，慢慢就变成了"概念验证坟墓"的前半句。

那么，怎样才能不把 PoC 做成坟墓？答案是，在动手之前，先把问题本身过一遍。这就是接下来的内容。

## 4.2 先有对的问题，才有对的方案

在前面，我们看到的代工厂，是"问题选错"的极端案例。但在更日常的场景里，"选错问题"往往没那么戏剧化，它表现得无声无息。

它可能是：客户给你一个词，你自己脑补了一个需求；你兴冲冲做了三个月，回头发现客户要的和做的，根本不是一回事。也可能是：你解决了 A，客户却一直念叨 B，最后项目验收，两边都不满意。

选问题这件事，难就难在——它看起来太简单了，简单到大多数人根本不会停下来认真做。客户说了要什么，开工就是了，谁还会去怀疑"这个需求本身对不对"？

可恰恰是这个"想当然"，埋下了绝大多数企业 AI 项目翻车的根子。

这一节，我要把"先选对问题"这件事，掰开揉碎讲清楚。

硅谷创投圈有个用了很多年的老词，叫 **Product-Market Fit**，产品市场匹配。意思是，产品要找到和市场的契合点，否则做得再好也没人买。

FDE 干活之前，其实要过的是更前面的一关：**Problem-Solution Fit**，**问题-方案匹配**。

这两个词，顺序不能乱。

PMF 默认你已经有了一个产品，然后去市场里找它的买家。这是个"从产品出发"的思路——产品是既定的，市场是待定的。

PSF 反过来。它要求你先把手里那套方案放一边，先把"客户当下最该被解决的那个问题"找出来，再让方案去配它。这是个"从问题出发"的思路——问题是既定的，方案是待定的。

区别看着细微，实际是天壤之别。

因为大多数人干活，都是反着来的。手上会什么，就拿什么去找问题。

会 RAG 的，看什么都像知识库项目；会 Agent 的，看什么都像自动化流程；会做数字人的，看什么都像直播间缺个人；会调模型 API 的，看什么都是"再训一个模型"。这跟手里攥着锤子、看什么都像钉子的区别，实在不大。

我见过太多项目，出发点不是"客户最大的麻烦是什么"，而是"我最想展示的技术是什么"。技术是主语，问题是宾语。客户只是在为你的技术买单，而你并不真的在乎他的业务有没有变好——这种项目，哪怕交付了，也是一锤子买卖。

问题选错的代价，远比方案做错的代价大。

方案做错了，改就是了，顶多返工、加班、重写。损失是有限的。

问题选错了，你所有的努力都砸在一个根本不该投入的地方。就像开头那家代工厂的团队，花六周跟数据死磕，磕到最后才想明白，这件事压根不是他们一个五六个人的新团队该碰的活——这种错，不是返工能救回来的，是整条命都搭进去了。

而且，问题选错这件事，往往不是一开始就显出来的。它藏在"客户说得很有道理""需求听起来很宏大""这是个大单"这些美好的表象下面，等你进场两周、三周、六周，才慢慢露出獠牙。等你意识到错了，沉没成本已经高到你抽身都抽不动。

但是，光说"先有问题，后有方案"，还是太虚。真要想对问题，你得先学会看清楚——一个问题，其实有好几层。

## 问题的三个层级

我把一个问题，拆成三层来看。

最上面一层，是**表层诉求**。就是客户嘴上说的那句话："我要做个知识库""我要打通数据""我要个数字人"。这些话，用词漂亮，但基本等于没说——它没告诉你任何关于"为什么"的信息。

中间一层，是**直接障碍**。就是真正卡住业务不动的那个环节。客服找不到答案、采购对账花两天、线索跟进不过来。到了这一层，才开始有血有肉。

最底下一层，是**根本原因**。就是那个直接障碍为什么会存在。客服找不到答案，是因为知识散在几十份文档里、没有一个统一的口径；采购对账花两天，是因为上游的数据从来就没对齐过。

FDE 要做的，就是逼着自己，从第一层，一路下钻到第三层。

为什么必须下钻到根因？

因为停在表层，你会接一个"全面 AI 化"的巨无霸项目；停在直接障碍，你会做一个解决表面症状的工具；只有摸到根因，你才知道，这个问题的解法到底是一个系统、一次流程重设计，还是压根不该用 AI 来解决。

这里有个反直觉的地方，值得单独拎出来讲：**有些问题的根因，根本不需要 AI。**

一个流程慢，可能只是审批链条设计得太烂，改个流程比训个模型有效得多；一个数据乱，可能只是没人定过字段标准，制定个规范比搭数据中台便宜得多。FDE 的诚实，恰恰在于敢告诉客户"你这个不用 AI"，而不是硬塞一个模型进去、赚一笔不义之财。

怎么下钻？用"五个为什么"这类追问，或者更直接的——不断问"然后呢""为什么"。你可以把它当成一个剥洋葱的过程：每剥开一层，问一句"这一层的真正原因是什么"，直到你剥到一个能算出账、能落地的核心。

开头那家代工厂，如果当时停下来，从"打通全公司数据"往下剥——数据为什么不打通？因为多个系统不互通。系统为什么不互通？因为老系统没有接口。为什么没有接口？因为十几年前的开发商倒闭了。剥到这里就会明白，这已经变成一个软件考古学问题，而不是一个 AI 问题。

到这里，你可能已经感觉到了：选对问题，不是靠灵感，而是靠一套可以重复的、往深处挖的动作。

再举一个更完整、也更好照着做下去的下钻，让你看清这条剥洋葱的路，具体是怎么走的。

假设客户开口说："我们想做一个内部知识库。"

第一问，为什么想要知识库？因为客服回客户问题太慢，客人等得不耐烦。

第二问，客服为什么慢？因为他们每接一个冷门问题，都要去几十份文档里翻，一份份打开、一个个搜，有的答案还散在几个不同部门的文件夹里，版本都不一样。

第三问，为什么答案要散在这么多地方？因为公司从来没有一个统一存放"标准答案"的地方，知识的归属，是一笔糊涂账。

第四问，为什么没人去建一个统一的地方？因为没有部门对"答案谁说了算"这件事负责。销售一套说法，售后一套说法，谁也拍不了板。

第五问，为什么谁也拍不了板？因为拍板，意味着将来答案错了，要有人站出来担责，而这家公司组织里，恰恰没有这个责任机制。

剥到第五层，你才看得清，这个"知识库需求"的根子，根本不在"缺一套系统"，而在"缺一个敢对答案负责的人，和一套能给答案纠错的机制"。这个机制立不起来，你哪怕把全世界最好的 RAG 搭起来，喂进去的，依旧是一堆自相矛盾的文档，AI 再准，答出来的照样是错的。

五问走完，问题的形状就彻底变了——从"做一套知识库系统"，变成了"先把某一个高频问题的标准答案和负责人定下来"。前者是个无底洞，后者是个能下手的小切口。

这就是下钻的价值。它不是为了让问题显得深刻，而是为了把那个模糊的大需求，一路逼成一个"有责任主体、有明确口径、能算出账"的具体问题。带着这样的问题去过三关，才不会浪费哪怕一天。

那么，怎么判断一个已经挖出来的问题，值不值得做？我用的是三关。

### 4.3 三关检验

一个需求拿到手，别急着动手，先过三关。任何一关过不去，这事儿就不该现在做。

三关的顺序也有讲究：先确认它疼，再确认它值，最后确认它做得出来。很多人跳过了前两关，直接冲第三关（"技术上能不能实现"），结果就一次次地回到那家代工厂的泥潭里。

下面一关一关说。

#### 第一关：痛点检验——这是真痛，还是痒

很多老板嘴里说的"问题"，其实连他自己都描述不清，只是听了个风口词就兴奋了。

所以第一关要判断的，不是"这事儿听起来重不重要"，而是"它现在正在让谁付出什么样的代价"。

真痛点有一个很硬的信号：**客户已经在为这个问题付出真实的成本了**。不是"将来可能会"，是"现在正在"。

怎么识别这个信号？看三样东西。

第一，看有没有具体的、正在发生的损耗。客服每天要在几十份文件里翻答案，翻到客户不耐烦；一个数据流程，要人手工整理两个小时；一个订单环节，漏一单就损失几百块。这些是正在流血的伤口，不是体检报告上"建议随访"的潜在风险。

第二，看损耗能不能被量化。省了多少时间、少了多少人工、降低了多少错误率，哪怕是个大概的数，也比"提升了效率"这种空话强一百倍。能量化，说明客户是认真的；量化不了，说明他自己都没搞明白问题在哪。

第三，看这个问题，是不是卡住了更重要的东西。比如一个排期环节慢，导致整个交付后延，连带着下游一起等。这种"瓶颈型"问题，解决的是这一处，撬动的是全局，价值往往被低估。

反过来，那些“我们想全面 AI 化”“把数据都打通”“做一个智能体平台”的宏大愿望，多数时候连痒点都算不上，只是老板的一次想象。

想象可以很大，但它不疼。

我习惯用一句很土的话来做最后一道过滤：**如果这个问题一直不解决，到年底，会有人因此被扣钱、多加班、或者丢客户吗？** 答案模棱两可的，多半是痒点，先放一放。

这里要特别警惕一种情况：老板说要解决 A，但你一深挖，发现真正疼的是 B。比如老板说“我们想做知识库，把老员工的经验都放进去”，背后真正的问题，可能是客服每天在几十份文件里找不到答案。前者是个愿望，后者才是个痛点。FDE 的价值，恰恰在于从那个模糊的愿望里，把真正的痛点挖出来。

那么，具体怎么问，才能把真痛点挖出来？我有四个很土、但管用的问法。

**第一个问法：“这件事，现在是谁在做？他每周要花多长时间？”** 把问题落回到一个具体的人和具体的耗时上。如果客户答不上来，说明这事压根没人真的在做，也就谈不上痛点。

**第二个问法：“如果这个月不解决，下个月会发生什么？”** 逼客户把“重要”翻译成“后果”。没有后果的问题，都是伪问题。

**第三个问法：“这件事如果做好了，谁最开心？谁可能不开心？”** 前半问题帮你找到付费动机，后半问题帮你提前看到阻力和政治风险。有些问题的解法，卡就卡在某个部门的人会因为因此丢饭碗。

**第四个问法，是给销售场景的：“如果要为这件事付钱，会从哪个部门的预算出？”** 钱从哪出，基本决定了谁是这个项目真正的甲方。谈需求时对着 A 讲，付款时却要找 B，这就是没搞清预算来源的代价。

这四问，本质上在做一件事：把一个漂浮在空中的“需求”，按到地上，变成一个具体的人、一个具体的代价、一个具体的预算。

## **第二关：经济性检验——有没有人愿意为它付钱**

这一关，纯技术出身的人最容易栽。

技术人评估一件事，习惯看两个维度："能不能做出来"，以及"做出来漂不漂亮"。但企业买不买账，看的是另一个东西：**这件事到底值多少钱，值不值得拿出来，单独花钱解决。**

这两套标准，经常是脱节的。技术上很难的东西，商业上可能一分钱不值；技术上很土的东西，商业上可能是个金矿。

为什么会这么拧？因为技术和商业，衡量的坐标系完全不同。技术衡量的是"难度"和"优雅"，商业衡量的是"省了多少钱"和"赚了多少钱"。一个技术人引以为傲的精妙架构，在客户眼里可能只是一堆他看不懂、也用不上的代码。

这里有个我在做项目时反复撞见的残酷现实。假设你接了个三十万的 AI 定制单，签合同签得顺顺利利。等你带着人进场，调研一拆，才发现：要把合同里承诺的功能跑通，得先把这家企业十年攒下的脏数据洗干净、把接口一个个打通，而这项前置的数据治理工程，光人力成本就要一百万。

这时候你怎么办？

你只能硬着头皮去找老板，说："老板，为了交付这三十万的项目，您得先额外批一百万预算，让我们先把您公司过去十年的陈年烂账梳理干净。"

没有老板会认这笔账。

他心里的第一反应是：我花三十万买高科技，一个月都认；你现在连个界面都没让我看到，就管我要一百万去扫垃圾，你是不是在诈骗？

这一刻，预期的崩塌是双向的。他觉得你无能、在敲竹杠，你觉得他不可理喻、不尊重客观规律。双方都委屈，双方都没错——错的是谁？错的是那个在签合同之前，没有算过这笔经济账的人。

所以经济性检验，要提前问三句：

第一句，客户愿不愿意为这个问题，单独掏钱？注意是"单独"。"全面 AI 化"这种打包的需求，客户肯掏钱，不代表里面的每一个子问题，他都愿意单独买单。

第二句，解决问题省下的钱，或者赚到的钱，够不够覆盖投入？这是一个 ROI 的粗算。别太精确，但一定要有。省的钱盖不住花的钱，这活儿做了就是亏的。

第三句，如果现在不做 AI，这个问题的现状还能撑多久？它是在持续流血，还是已经适应了、不疼了？很多问题之所以"值得做"，是因为它有紧迫性；一旦客户已经习惯了那个低效的流程，反而不急着改了。

算不清账的问题，做得再漂亮，最后也收不回钱。这不是耸人听闻，这是我在无数项目里，看着同行一个个栽进去之后，得出的血泪结论。

说到 ROI 的粗算，很多人会被"算不清"吓退，觉得这活儿是财务干的。其实不用那么复杂，一个野路子的三段估算就够用了：

第一段，算"省"。这个动作现在要几个人、每天花几小时，折成人力成本是多少。有了 AI 之后，能省到多少。差额，就是省下来的钱。

第二段，算"多赚"。解决这个问题之后，会不会带来额外的单子、更高的转化、更低的流失。哪怕只估一个非常保守的比例，也比没有强。

第三段，算"投入"。包括你的开发费、客户要买的数据和算力、改流程的组织成本。这一项最容易被人为压低，尤其要小心前置的数据治理成本——就是代工厂那种坑。

三段一减，正的就是值得做，负的就是别碰。算不到小数点后两位没关系，算错方向，才是真正致命的。

顺带说一句，经济性检验的最终形态，其实是"按结果收费"——不是收一个开发费就完事，而是跟客户约定好，按"省了多少钱、增了多少钱"来分成。这是后话，会在讲"扩大收入"的那一章展开。但它的种子，其实在这一关就埋下了：一个连账都算不清的问题，永远走不到按结果收费那一步。

### 第三关：可行性检验——在约束里，做得出来吗

前两关都过了——问题是真的、痛苦的，也值得付费。最后一关，卡在"能不能做出来"。

注意我的用词。这里说的可行性，不是"理论上能不能实现"，而是"在客户给你的约束里，能不能交付"。

这两个是完完全全的两码事。

理论上能不能实现，是技术问题，答案多半是"能"。但交付要面对的是一堆现实约束，我数一下，至少有五个：

**数据**——数据到底拿不拿得到？是结构化还是纸质？有没有人愿意配合提供？

**接口**——老系统的接口开不开得了？开接口要找谁、走什么流程、卡多久？

**人**——业务负责人配不配合？线上一线用户愿不愿意用？有没有一个能拍板、也肯背锅的人？

**时间**——客户要的是一个月上线，还是给你六个月？时间一压缩，很多方案直接就废了。

**权限与合规**——安全审查过不过得去？数据能不能出局域网？私有化部署的要求达不达到？

这五个约束，任何一个卡住，项目就动不了。

更隐蔽的问题在于依赖链。很多方案，拆开看每一步都可行，拼在一起就不行了——因为中间的依赖链太长，只要一个环节断了，整条链就停摆。

我举个具体的链。数据依赖接口，接口依赖权限，权限依赖某个部门的负责人点头，而这个负责人，可能恰恰是这件事最大的反对者——因为项目落地之后，他那套靠信息不透明维持的权威，会土崩瓦解。你跟他谈技术，他跟你谈安全；你跟他谈安全，他跟你谈预算；你跟他谈预算，他说再研究研究。链就这么断在“人”这一环上，跟你的代码水平，一点关系没有。

新手最容易问的是“这个技术能不能做”，老手第一个问的是“客户到底能不能给我需要的那些条件”。

数据在谁手里、谁有权限开、哪个业务负责人肯为结果负责——这些才是决定项目能不能落地的东西，比模型本身的难度更要命。一个连“谁负责验收”都答不上来的项目，技术方案写得再漂亮，也是空中楼阁。

三关过完，如果都稳当，你才进入下一个问题：怎么开始做。

如果第一关就没过，那根本不是东西好坏的问题，是你选错了题。如果第二关没过，题是对的，但账算不过来，做了也收不回钱。如果第三关没过，题对、账也对，但你还没资格碰——别硬上。

三关的顺序不能乱。先确认它疼，再确认它值，最后确认它做得来。

## 顺手写下来：一段合格的问题陈述

三关都过了，在动手之前，我还习惯做一件小事：把问题写成一句清楚的话。

大多数人，哪怕心里已经想明白了，也懒得写下来。结果是，一进到执行阶段，思路又开始发散，做着做着，又回到了那个模糊的大需求上。

写下来的好处，是逼你把"选对了的问题"，钉成一个具体的、可回看的靶子。

一段合格的问题陈述，通常包含三个零件：

谁（在某一个具体场景里），正在被什么（具体的麻烦），困扰到什么程度（可量化的代价）。

比如说，不是"采购流程效率低"，而是"采购助理每周要花两天手工核对供应商对账单，平均每个月还会出现两四处对错，导致付款延误"。

看出差别了吗？前者是个标签，后者是个靶子。

标签是可以被糊弄过去的，靶子是必须被命中的。你带着"采购效率低"进场，做半年也不知道到底解决没解决；你带着"每周两天的对账、每月两四处对错"进场，做没做成，采购助理第二周就能告诉你。

这句话，还会成为你后面每一步的锚。MVD 的定义单要围着它写，验收标准要围着它定，最后向上汇报、向客户证明价值，也是拿着它说话。

所以，别小看这一句。它是"选对问题"和"做对东西"之间，常常被省略、却最不该省略的那座桥。

## 4.4 MVD：不是缩水版，是尖刀

三关过了，问题选对了，接下来是"怎么开始做"。

这里我要引出一个词，它可能比 PoC 和 MVP 都更贴合 FDE 的场景：**MVD, Minimum Viable Deployment, 最小可行部署**。

它跟 MVP 到底有什么区别？

MVP 是"最小可行产品", 重心落在"产品"两个字上。做出来的东西, 可以是一个原型、一个 Demo, 哪怕它只是个半成品。它验证的是"这个产品设想, 有没有人买单", 跑通一个演示就算达标。

而 MVD 的重心, 落在"部署"两个字上。做出来的东西, 必须真的被人用起来, 嵌进真实工作流里, 哪怕范围小到只有一条线。它验证的不是"想得好不好", 而是"这刀下去, 业务到底有没有被切开"。

用一句话说清两者的区别: **MVP 可以是个漂亮的演示, MVD 必须是一把能切进真实业务的尖刀。**

演示可以是残缺的, 但尖刀必须是完整的、能见血的。这就是为什么, 同样是"最小可行", MVP 做出个能跑的原型就交差, MVD 非要逼你把一条从入口到结果输出的完整链路跑通。

拿"做客服知识库"这件事举例, 两者的差别一目了然。

MVP 的做法: 搭一个问答界面, 喂进去二十份文档, 问"退款政策是什么", 能答出来, 录个视频发给老板看。一个下午就能搞定, 演示也很漂亮。

MVD 的做法: 找到客服真正最常被问的那一类问题(比如"物流到哪了"), 把这条线从入口(客户提问)到结果(客户真的得到答案、不用再转人工)完整跑通, 让真实客服在真实工作里先试用一周。

前者验证的是"模型能不能回答"; 后者验证的是"这个问题值不值得被 AI 解决、以及解决后客户还用不用人工"。

看起来前者更"快", 但前者那个快, 是虚假的快——因为你不投入真实工作流, 就永远不知道真正的阻力在哪里。

在讲军规之前, 我还想把这个词里的"部署"二字说透, 因为太多人忽略了它。

"部署", 英文是 Deployment, 在软件行业里, 它原本是个再普通不过的词: 把代码发布到生产环境, 让它跑起来。

但在 FDE 的场景里, "部署"的含义要重得多。它不只是"代码上线了", 而是"这件事真的进入了企业的日常工作, 并且有人在用、在用出结果"。

这中间隔着多少东西？

要有人真的愿意打开它——这需要降低使用门槛，需要寄生在用户熟悉的界面上。要有人敢让它去执行——这需要权限、审批、失败恢复。要有人肯为它的结果负责——这需要明确验收标准、明确出错谁兜底。

代码上线，一天的事；让系统被用起来、用出价值，是场长跑。MVP 关心的，是前者；MVD 关心的，是从前者到后者的那条完整的路。

这也是为什么，我把这一章的核心，从"怎么做 PoC"，硬生生扭到了"怎么做 MVD"。因为企业要的，从来不是一个能演示的东西，而是一条能真正切开业务的、完整的小径。

MVD 有三条军规。这是我在踩了无数次坑之后总结出来的，一条都不能省。

### **军规一：用真实数据，别用合成数据糊弄**

这是第一条，也是最容易被破的一条。

做一个 Demo，很多人图省事，用的是生成的数据、脱敏到面目全非的样本、或者手工捏的几个"理想案例"。演示效果很好，因为演示环境里没有意外，一切都按你预设的来。

但真实业务里，全是意外。

乱序的 webhook，说断就断的网络，对不上账的字段，格式千奇百怪的扫描件，还有那份印错了一个数字、全公司只有几个老员工知道"这里要手动改一下"的产品文档。

这里有个真实的遭遇，我一直记着。

有次给客户做知识库，上线测试的时候，AI 回答错了一个产品问题。我们第一时间怀疑是模型胡说，又怀疑是知识库召回错了，再怀疑是扫描件解析错了，一路追下去，发现技术链路全部正确。

真正错的，是很多年前打印出来的一张产品设计文档——上面有一处印错了。公司的老员工都知道这处错，平时在脑子里自动改过来，所以从没出过事。但这个"正确的改法"，从来没有进入任何线上系统。

于是 AI 忠实读取了那份错误资料，非常准确地，给出了一个错误答案。

这件事的教训很深：**AI 可以非常准确地处理一份错误知识**。你用合成数据做出来的东西，之所以演示好看，是因为你把现实里这一万个"脏东西"全部屏蔽掉了。等它上线第一天，这些脏东西会一起扑上来，你的系统毫无防备。

再往深一层想，合成数据的问题，不只是"不真实"，它还制造了一种虚假的安全感。演示成功了一次，团队就以为"系统已经能用了"，于是放松了对真实数据的警惕。等到生产环境里第一个异常怼到脸上，才慌了神。

那么，真实数据到底会以哪几种方式，给你一个下马威？我见到的，大致有三种。

第一种，是**缺失**。你以为有的字段，其实大半是空的；你以为完整的记录，其实断头断尾。拿到的表，一查，一行对的都没有。

第二种，是**口径不一**。同一个字段，不同部门填的含义不一样，名字一样、意思天差地别。你以为在比苹果和苹果，其实是在比苹果和橘子。

第三种，是**隐含的错误真值**。就像那份印错的文档——数据本身"看起来"是对的，但它的真值，早就被人在线下改成了一个和系统里不一样的东西。这种最阴险，因为它不显山不露水，却能让你的系统"非常准确地犯错"。

提前知道有这三种脏，不是为了吓自己，而是为了在进场的第一天，就带着拆弹的谨慎去看数据，而不是带着抽样演示的轻松去看数据。

MVD 的军规一，就是逼你从第一天起，就面对真实数据、真实冲突、真实例外。疼，但疼得值。因为只有真实数据，才能暴露真正的问题、真正的边界、真正该收口的节点。

## 军规二：缩小范围，而不是缩小价值

很多人一听"最小"，第一反应就是把功能砍掉几个，觉得自己挺懂"MVP 思维"。

不对。

MVD 缩的是**范围**，不是**价值**。范围和价值，是两个经常被混为一谈、实则截然不同的东西。

范围，指的是你这一版要覆盖多少个环节、接多少个系统、服务多少类用户。价值，指的是你这一版真正解决的那个问题，对客户来说重不重要。

正确的缩法，是把范围切到极窄——就解决一个部门里，一天要重复几十次的一个小动作。但这个被解决的动作，价值必须是完整的、明显的。

用户要能一眼看到变化：以前要两小时，现在十分钟；以前靠人一条条核对，现在自动；以前三天两头漏，现在不漏。

错误的缩法，是范围没怎么缩，价值先没了。做一个包罗万象、但每个功能都只做一半、哪个都不能真正省事的半成品。这种"看起来什么都沾一点，其实什么都不顶用"的东西，比一个窄但完整的切片，差远了。

判断你缩得对不对，有个很简单标准：

做完之后，客户能不能用一句话，说清楚"以前怎么做，现在怎么做"。

能说清楚，说明你把一个完整的价值切片做出来了。说不清楚，多半是你把范围和价值一起缩没了，做出了一堆"半拉子"。

说个具体的缩法，还是回到代工厂那个"打通全公司数据"的巨无霸。

假设你进场后，不是一头扎进去，而是先花一天，列一张清单——列的是"这家公司现在每天在重复、且能算出成本的重复劳动"。你会发现清单里躺着几十条，比如：

采购助理每周要花两天，手工核对供应商送来的对账单和财务流水；

质检员每天要把产线报上来的数据，一条条誊进 Excel；

仓库月底盘点的差异，要三个人对三天。

这时候，你就有了几十个候选项。接下来不是全做，而是用三关，一个小切口一个小切口地过，挑出那个最疼、最值、又最做得动的一个。

采购对账单这件事，就特别合适：它疼（每周两天、还会对错），它值（省下的人工是真金白银），它做得动（就几张表、一个明确的比对规则、范围极窄）。

于是你的 MVD，收敛成了这一句："把采购每周手工核对两天的对账单，变成系统自动比对、只把差异推给人复核。"

范围有多小？它不碰人事、不碰财务、不碰车间、不碰任何跨部门协同，就解决采购这一个动作。

价值有多完整？完整到采购助理第一天就能用上，第二周就能看到自己省了多少时间。

这就是"缩范围不缩价值"的具象化。你做的，是从几十个问题里，挑出那一个，然后把它做透，而不是把几十个问题，每个都做一半。

这里还有个很多人忽略的点：缩小范围，不只是为了降低开发成本，更是为了降低"第一次相信"的风险。客户第一次跟一个新团队合作，他不敢押太大的注。你给他一个窄但完整的东西，让他亲眼看到"这事确实变了"，他才有底气陪你往下一步走。反过来，你一上来承诺一个横跨三部门的巨无霸，他信你的成本，会高到他自己都不敢签。

### 军规三：定死截止时间，用时间倒逼取舍

第三条军规，是给范围上一把锁。

我在前面反复提过一个看似极端的做法：把第一次交付，压到八小时左右。

八小时当然不是什么神奇的魔法数字，也不是用来考核团队速度的 KPI。它的作用，是人为制造一个足够强的约束——时间一短，你就没办法把所有需求都塞进第一版，于是被迫去回答那个最该回答、却总被回避的问题：

**客户此刻，最该被解决的那一个麻烦，到底是什么？**

为什么一定要这个死线？

因为如果没有一个死线，范围就一定会膨胀。这是人性里的必然，跟团队自律不自律关系不大。

膨胀，通常来自两个方向。

客户这边的膨胀是这样的：他看到一个效果，很自然地说"这个也顺便做了吧""那个能不能一起加上"。于是小项目又慢慢长回大项目。

团队自己的膨胀是这样的：忍不住"再加一个功能，显得更专业""再优化一下，显得更厉害"。加着加着，离那个最初的价值越来越远。

需求像雪球，越滚越大，最后又滚回了那个无底洞。而你回头看，会发现最初那个小小的、干净的价值切片，已经被埋在最底下，找都找不到了。

定死截止时间，就是把"还能加什么"这个问题，硬生生地，换成"怎样最快产生完整价值"。

做不出来怎么办？做不出来，往往不是技术不行，而是问题还没切到足够小。这时候该做的，不是加班，不是硬扛，而是继续缩小范围——把那个"最该被解决的一个麻烦"，切得再小一点，小到能在限定的时间里做完整。

这里有个很微妙的心理，值得说道说道。很多人下意识地认为，"范围大"等于"价值高"，"范围小"等于"价值低"。所以让他们缩范围，他们会本能地抗拒，觉得这是在贬低自己。

恰恰相反。范围小、价值完整的东西，它的价值是被放大、被聚焦的；范围大、价值摊薄的东西，它的价值反而是被稀释的。一把刀，削铁如泥，靠的是刀刃那一线的锋利，不是刀背的厚度。

关于"定死截止时间"，很多人对"时间"还有一种顽固的误解，我再多说一句。

第一种误解：时间松一点，质量就会更好。事实往往相反——时间一松，范围就先膨胀，最后做出来的东西更多，但纪念碑一样摆着没人用。时间是范围的闸门，闸门一开，水就漫进来了。

第二种误解：定死线是一种不近人情的压迫。其实恰恰相反，死线是对交付方的保护。它给你一个正当的、不用硬扛的理由：不是我不做，是这一版做不完了，我们放下一轮。没有死线，你连拒绝客户加需求的底气都没有。

第三种误解：时间是固定资源，只能抠团队。其实时间逼出的，从来不是加班，而是取舍。取舍的对象也不是时间本身，而是范围——把范围切小，你的时间立刻就"变多"了。

理解了这三层，你就明白，所谓"定死截止时间"，本质不是管理时间，而是管理范围。

三条军规连起来，逻辑是一贯的：

先承认世界是脏的（用真实数据），再把范围切窄到只留价值（缩范围不缩价值），最后用时间锁死它（定死截止）。

这三条同时成立，你做出的，才是一个能活着进生产的 MVD，而不是又一个活不过上线当天的 PoC。

最后再补一个提醒，因为太多人在这里翻车：他们以为自己做出了 MVD，其实做出来的，只是换了个名字的别的东西。我见过最典型的，是三种"伪 MVD"。

第一种，叫"面条 MVD"。范围是切窄了，可价值也一起被切没了。为了追求"最小"，把一个本来能解决问题的小切片，硬生生砍成了一个不痛不痒的小功能——解决了一个根本没人不在乎的小问题。这种"最小"是最偷懒的缩法，它缩的是价值，不是范围。

第二种，叫"演示 MVD"。嘴上说是最小可行部署，骨子里还是个 demo。用干净数据、放大的效果，跑个漂亮演示给老板看，没有真进生产，没被真实数据冲刷过，也没有一个人在真实工作流里真正用过。等它"上线"那天，就是它死的那天——因为它从头到尾，没接触过真实世界里那一个个脏东西。

第三种，叫"拼盘 MVD"。把几个半成品凑到一起，看起来范围挺小，其实每个都没做完。A 做了一半，B 通了半个，C 只有界面没有后端。单看每一个都不大，合起来，却是一个"啥都沾一点、啥都不顶用"的东西。

怎么一眼区分真 MVD 和伪 MVD？就一句话：真 MVD，一定有一个真实的人，在他真实的工作里，用这个东西完成了一件真实的事，并且那件事真的成了。三个"真实"，缺一个，你做出的就不是 MVD，而是又一个自欺欺人的半成品。

带着这条尺子，回头看下面 4.5 那个砍掉八成需求的项目，你就明白它凭什么活了下来——因为它把这三个"真实"，一个不落地占全了。

## 4.5 砍掉八成需求，反而活了

讲到这，光讲道理还不够。我用一个更具体的例子，说清楚"选对问题 + 缩小范围"，落到真实项目里，到底是什么样。

它是一个教育转化的场景，数据来自一组真实的项目材料，我做了一些脱敏和提炼。

先看一组数字。

近三十天，新增线索 4090 个。有支付时间的，92 个。粗略一算，支付率 2.25%。

再往下看流转：

D1 到课，1346 人；

D1 交作业，只有 322 人；

D2 到课，1006 人；

D2 完课，506 人；

最后进入支付页的，50 人。

漏斗很清晰，也很有问题：从 4090 个线索，一路漏到 50 个进支付页，每一层都在疯狂流失。

现在，如果你是那个被请来解决"转化太低"的人，你会怎么做？

大部分人，会把它当成一个"AI 回复"项目来干。优化方向大概是：让 AI 话术更顺一点，多追几次单，把 D3 到 D6 的追单写得更狠一点，让机器人更像真人一点。

这些努力，不是完全没用。但它们全都不是主刀口。你就算把话术写成了一朵花，也改变不了漏斗的形态——因为根上的问题，根本不在这。

真正的病灶在哪？

在 D1 到课之后，用户没有被推进到"我的问题已经被识别了，我需要下一步方案"这个状态。

你看这个细节：D1 到课 1346 人，可 D1 交作业的只有 322 人，掉了一大半。为什么？因为那个"作业"，从头到尾就只是个作业。它没有成为一次启动诊断——没有去搞清楚：这个用户每天能投入多久？有没有预算压力？是宝妈、学生、还是上班族？他卡的是剪辑、流量、时间、信任、家人反对，还是"怕学不会"？

这些信息，一旦被结构化，后面的销售就不再是广播式的群发，而是分层触达——每个人收到的，是针对他自己卡点的那一句话。反过来，这些信息不收集，你说得再漂亮，也是对着空气广播。

再看 D2 的人工接管率这一项。

很多团队一看到"人工接管率高"，第一反应是"要让 AI 多挡一点，好减轻人工压力"。

但这事不能那么快下结论。

因为 D2 的高人工接管，恰恰可能是离成交最近的那座金矿。用户在这个节点，问的是分期、是付款路径、是权益确认、是"我到底适不适合"。这些哪里是噪音？这是正儿八经的成交信号。

你把所有问题都交给 AI 自动回复，看似高效，其实可能把最值钱的那批客户，亲手送走了。

那么，从诊断到动作，具体怎么落？我把它拆成三步。

第一步，改作业为诊断。把 D1 那个"交个作业"的动作，换成一份简短的诊断问卷——不是查户口，是几个能快速定位用户卡点的选择题：你每天能投入多长时间？之前有没有做过类似的事情？现在最担心的是什么？答案不需要多，够打标签就行。

第二步，沉淀成标签。把诊断结果，随手写回用户的那条线索里：时间投入、预算敏感度、卡点类型、购买阻力。一个用户从"匿名流量"，变成"带标签的活人"。

第三步，按标签分层触达。卡时间的人，给他强调"碎片时间也能学"；卡预算的人，给他分期方案；怕学不会的，给他看同类型学员的出单记录。话术没变多，只是从"对着一万人广播"，变成了"对着一百个人说不同的话"。

你看，三步里面，没有一步是"让 AI 说得更漂亮"。它们全都是在做同一件事：把那个一直没被识别出来的状态，识别出来，然后让下一步的动作有据可依。

所以这个项目最后定的方向，不是"AI 全自动成交"这种听起来性感、实则危险的东西，而是：

入口问题，AI 处理；

常规 FAQ，AI 处理；

而"能力焦虑、家庭阻力、价格犹豫、权益确认"这类问题，快速打上标签，交给人去收口。

你看，这中间砍掉了多少东西？

"AI 全自动成交"的大愿景，砍了；无差别的广播追单，砍了；全线自动化的执念，也砍了。

留下来的，是一个很窄、但每一环都精准踩在成交链上的东西。

这不是缩水。这是把力气，从"做得全"，挪到了"做得准"上。看上去做了更少的事，实际上每一件事，都做在了更有价值的地方。

这个例子和代工厂的例子，恰好是一正一反。

代工厂失败，是因为问题选得太大、太虚，一头扎进了"打通全公司数据"的泥潭；教育转化成功，是因为把"转化太低"这个大问题，一路剥到了"D1 作业没有变成启动诊断"这个可以一击命中的小切口。

同样面对一个宏大的诉求，有人被它带着走，有人把它切开。差别，就在那一套"三关检验 + MVD"的动作上。

回到开头那家代工厂。

如果他们进场的第一天，不是一头扎进"打通全公司数据"这个巨大的问题，而是先找到一个最疼、最值、也做得动的小切口——比如，先解决"采购那张每周要人工核对两天的对账单，能不能自动拉出来"——结局大概率会完全不同。

那个对账单，是真的疼（采购每周花两天），真的值（省下来的人工是真的钱），也做得动（数据就一两张表，范围足够窄）。用 MVD 的逻辑，先把它切出来，跑通，让老板亲眼看到"这事确实变了"，后面再谈更大的，才有底气。

问题选对之前，所有的努力，都只是看起来在前进。

选对问题、用 MVD 切出第一刀，这只是开了一个头。接下来你要面对的，是更难的一关：把做出来的东西，真正激活——让用户愿意用、习惯用、离不开。

那是一个全新的战场。在那里，技术退居其次，用户行为的改变才是唯一的胜负手。我们在下一章再聊。

## 章末工具箱

### 工具一：三关检验清单

拿到需求后，逐条过一遍。任何一条回答不出来，先回去问，别急着动手。

#### 痛点检验

- [ ] 这个问题"现在"正在让谁付出什么样的真实代价？
- [ ] 这个代价能不能被量化（时间、人工、错误率、钱）？
- [ ] 如果不解决，到年底会有人因此被扣钱 / 多加班 / 丢客户吗？
- [ ] 客户说的是具体现象，还是一场宏大想象？

### 经济性检验

- [ ] 客户愿意为这个问题单独掏钱吗（不是打包买单）？
- [ ] 解决问题省下的钱或赚到的钱，够不够覆盖投入？
- [ ] 前置成本（数据治理、接口打通、采买）是不是已经超过合同额本身？
- [ ] 如果不做 AI，这个问题还能撑多久、代价多大？

### 可行性检验

- [ ] 需要的数据真的拿得到吗？在谁手里？谁有权限？
- [ ] 业务负责人愿意为结果负责吗？
- [ ] 依赖链有多长？最关键的那一环，卡在谁身上？
- [ ] 在客户给定的时间、安全、权限约束里，做得出来吗？

## 工具二：MVD 定义单

| 项目        | 怎么填                   |
|-----------|-----------------------|
| 要解决的那一个价值 | 用一句话说清"以前怎么做，现在怎么做"   |
| 明确不做的范围   | 哪些环节、系统、用户，这次坚决不碰     |
| 真实数据来源    | 数据在哪、谁提供、质量如何、有没有隐藏的坑 |
| 截止时间      | 定死一个时间，用时间倒逼取舍        |
| 完成标志      | 什么结果，算"这一刀真的切进去了"     |

## 工具三：缩小范围的三个提问

当范围膨胀、不知道砍哪里的时候，逐个问自己：

砍掉它，客户还能不能完整解决那个核心麻烦？ 能，就砍。

留下它，是为了客户的价值，还是为了我们自己的面子？为了面子，就砍。

它能不能放到下一轮再做，而不会让这一轮的价值打折？能，就放到下一轮。

三个问题的答案，会帮你把那个越滚越大的雪球，重新压回一把尖刀。

## 本章要点回顾

项目死的真正原因，多半不是技术，是问题一开始就选错了。

PoC 坟墓的三个推手：干净数据、没人定义成功、销售与交付脱节。

一个问题有三层：表层诉求、直接障碍、根本原因，要一路下钻到根因。

三关检验：先疼（痛点）、再值（经济性）、后做得动（可行性），顺序不能乱。

MVD 三条军规：真实数据、缩范围不缩价值、定死截止时间。

好的 FDE，敢告诉客户“你这个不用 AI”，而不是硬塞一个模型进去。

## 常见误区速查

**把 PoC 当交付物：**以为演示通过就万事大吉，结果上线无人用。→ PoC 是试纸，不是产品。

**从技术出发找问题：**先有锤子再找钉子，会 RAG 看啥都像知识库。→ 从问题出发，让方案去配问题。

**只问“能不能做”，不问“值不值得”：**技术上可行，商业上血亏，忙活一年收不回钱。→ 三关顺序不能乱。

**把范围大当成价值大：**做了个巨无霸半成品，不如一个窄而完整的切片。→ 缩范围，不缩价值。

**怕得罪客户，不敢缩小范围：**客户要啥给啥，越给越多，最后一起完蛋。→ 用一条死线，正当地说“下一轮再做”。

## 本章行动清单

读完这一章，别急着翻过去，先做三件事：

第一件，翻出你最近正在谈、或者正在做的一个需求，用"三关检验清单"过一遍。重点看第二关——经济性。我敢打赌，大多数坑都出在这一关，因为技术人对它最不敏感。

第二件，把这个需求，用"谁、被什么、困扰到什么程度"的句式，写成一个句子。写不出来的，说明你还没把问题想清楚，先别动代码。

第三件，看看你现在手里这个 MVD 的"范围"，是不是还能再切小一半。记住：切小的是范围，不是价值。如果切小之后，客户依然能用一句话说清"以前怎么做、现在怎么做"，那这一刀就切对了。

这三个动作，加起来花不了你二十分钟，但能帮你躲开我这几年踩过的大部分坑。

<div style="page-break-after: always"></div>

## 第 5 章 从能跑到用户真用：激活部署

上线不是终点。让用户真正用起来、习惯用、离不开，才是。

一个系统，跑起来了，躺在服务器上。这不叫成功。

真正难的，是下一件事：让现实里那些每天忙得脚不沾地的员工，愿意放下手头的活儿，打开你这个新东西，用一次，再用一次，直到离不开它。

这一步，叫激活。它是企业 AI 落地里，死掉项目最多的一关。

### 5.1 企业部署的"首日魔咒"

先描述一个画面，你可能见过，也可能马上会见到。

项目上线那天，是很热闹的。项目经理拉了个全员大群，发了一段"经过三个月奋战，XX 系统今日正式上线"的通知，@ 了所有人。IT 提前一晚把系统在各个工位上装好了。运气好的话，还约了个培训，会议室里坐满，讲师讲了一个小时，大家点头。

第一天，数据动了。看板跳了。后台的调用数字，蹭蹭往上走，有人截图发进群里，配一句"起飞了"。那一刻，所有人都觉得，成了。

一周之后，日活掉到个位数。

一个月之后，系统还躺在服务器上，但已经没人记得它还在。偶尔有新人入职，IT 照着流程给他开个账号，他登录一次，收藏夹里就多了一个永远不会再点开的网址。

我把这个现象叫作“首日魔咒”：**上线当天是巅峰，此后再无来者。**

热闹散得这么快，是谁的问题？

常见的甩锅对象有三个。第一个是用户懒——“这些员工太不思进取，好工具摆在那儿都不知道用”。第二个是领导不重视——“老板只是上线那天露了个脸，后面就不管了”。第三个是培训不到位——“就讲了一个小时，谁学得会”。

这三条，都对一点。但都没说到根子上。

先说培训。培训解决得了“会不会用”，解决不了“为什么要用”。一场培训下来，员工学会了点哪个按钮，可走出会议室、回到工位，那些“学会”的操作用不上三天就还给老师了，因为他工作里没有那股劲儿。会，不等于用；用，得先有理由。

再说领导。领导重视，顶多把系统变成一种“政治正确”——开会提两句，考核里加一条“使用率”。但这种重视是外压的，外压能逼人点开，逼不出依赖。被逼着用的系统，领导一松口，立马反弹回原样。

这三样，都不是药。真正的病根，在下面这句话里。

根子在于一个很多人不愿意承认的事实：**用户没有任何理由，为了“你上线了系统”，而改变自己的行为。**

你把视角切到那个员工身上看看。他昨天还在用 Excel 记台账，用得好好的，熟。今天你塞给他一个新系统，界面是新的，按钮是新的，逻辑是新的。他要用，得先学；学的过程中，一定会有卡壳；卡壳了还得来问你。更麻烦的是，他拿不准这玩意儿算出来的东西对不对，万一错了，谁背锅？是他，还是那个系统？

这么一算，用你的系统，对他而言，净收益是负的。

学习成本、适应成本、犯错风险，全堆在他一个人头上。而收益呢？省的那点事，是模糊的，是“以后会体现出来的”。一边是眼前实实在在的麻烦，一边是虚无缥缈的好处，换谁，都选继续用 Excel。

所以，他不用，不是他懒，是他聪明。

想通了这一层，你就明白了一个残酷的前提：**你不能指望用户主动拥抱你的系统。**你要做的，是反过来——把"使用"这件事，变成一件几乎零成本、又明显划算的事。

这就要求两个东西同时到位。

第一，把门槛降到最低，低到他对"切换"几乎没有感觉。不能在让他用之前，先扔给他一堆要学、要配置、要判断的东西。

第二，让他第一时间尝到甜头。注意，是"第一"时间，不是"慢慢体会"。用户对一件新东西的判断，往往在第一次接触的前几分钟里就完成了。心理学上有个说法，叫峰终定律——人对一段体验的印象，主要由最爽或最糟的那个瞬间、以及结尾的瞬间决定。套到这里就是：他第一次用你的系统，那几分钟是爽的、是"卧槽这么省事"，后面他就愿意再来；那几分钟是懵的、是卡的，你后面做再多，也难把他拉回来。

关于"第一时间的甜头"，我后面会反复讲。这里先把贯穿全章的判断刻下来：

**别指望用户主动拥抱你的系统，你要做的是，让"使用"变成一件几乎零成本、又明显划算的事。**

再看一眼"首日魔咒"那条曲线，你会注意到一个细节：上线当天的流量，绝大部分是好奇心，不是需求。那波峰值，是"大家来看看这新东西是啥"。看完了，没有留下来的理由，自然就散了。

我自己第一次撞上魔咒，是在一个内部工具上。上线那天我在群里发了欢迎语，还配了表情包，自认为挺体贴。头两天有同事私信我说"这玩意儿挺好使"。可我忘了一件事：我把入口放在了一个要手动点开、还要登录三层的网页里。从第三天起，没人再点。等我第七天打开后台，看见日活那根线笔直掉到零，才明白——热闹是我自己炒起来的，冷场也是我自己埋的。那一回我学到的，就是后来常跟人念叨的这句：入口的位置，决定了一半的激活命运。

顺带说一句，别把"上线当天的热闹"当成指标。我那会儿看后台数据，上线头三天，日活曲线漂亮得像心电图，全是"点进去看看"的好奇心。到第七天，曲线坠崖，最后落在几张真的每天都要用的人脸上。这中间掉的，不是用户，是那些本来就留不住的人。你留不住他们，不是因为你不够热情，是因为他们进来逛了一圈，没发现这儿有什么非用不可的东西。

激活难，就难在这儿。它考验的，全是你上线之前埋下的功夫。热闹散场之后，还留下来继续用的那个数字，才是你真实交付的成绩单。

所以，破解魔咒，不靠上线那天的那场热闹，而靠你提前埋好的两件事：门槛，和甜头。热闹是别人给的，门槛和甜头，是你设计出来的。

门槛怎么降，甜头怎么给，下面就一节一节拆开说。

## 5.2 降低使用门槛：寄生于他已经在用的东西

激活的第一要务，是降低切换成本。

先给你一个特别管用的判断标准。同样一个“让员工用 AI”的活儿，一个做法是搭个漂亮的内网平台，让人专门登录进去用；另一个做法，是在企微里塞个机器人，一句话吊起来就能干活。三个月后看日活，哪个高，闭着眼睛都能答。人不会为了你的平台改变习惯，但会为了一个“顺手就够到”的入口，多用一次。

一个很反直觉、但极其有效的原则是：**不要逼用户来你的系统，而是让你的能力，出现在他已经在用的地方。**

用户的时间都花在哪？在企业微信、在钉钉、在微信、在飞书、在 OA、在 Excel、在邮件。这些，才是他们每天都在用的界面。你去看看一个普通员工的桌面，十几个窗口常年开着，没有一个是为你这个新系统专门留的位置。

所以，一个聪明的部署，第一刀往往不做一个独立的“平台”，而是把 AI 的能力，寄生进这些已有的入口里。

为什么第一刀不做独立平台？因为独立平台对你来说是“产品”，对用户来说是“另一个要学的软件”。你每让用户多学一个软件，就是在和他的惰性掰手腕，而这场掰手腕，你历史上一场也没赢过。反过来，寄生在已有入口，等于绕开了这场掰手腕——他没有“接受一个新东西”的心理成本，只有“咦，原来的地方多了个顺手功能”的小惊喜。

### 寄生：把能力塞进他已经在用的界面

具体来说，有几种寄生方式。

**钉进聊天工具。**用户在企业微信里，一句话就把 AI 调出来了，不用另开一个网页、不用记一个网址、不用多装一个软件。入口成本为零。这事为什么管用？因为聊天工具是他一天里打开次数最多的窗口，他人蹲在哪，你的能力就出现在哪。

**嵌入 OA 流程。** 用户本来就要走审批，你在审批的某个环节，自动帮他把数据算好、把草稿写好，他只需要点个确认。他没有"多用一个系统"的感觉，他只是发现：诶，今天这个审批怎么变快了，草稿都给我拟好了。

我还原一个场景。某公司的报销审批，原来要申请人先填一张表，再自己去财务系统里查这个月还剩多少预算，再把发票编号一条条核对，最后提交。一圈下来，二十分钟，还常出错。我们把 AI 嵌进去之后，申请人还是打开原来那张报销单，但预算额度是自动带出来的，发票编号是自动校验的，就连"这笔钱合不合规"，AI 也在提交前先给了一遍提示。他干的事一点没变，时间从二十分钟掉到三分钟。

注意，他没有"去用 AI"。AI 是在他原来那条路上，把坑填平了。

**挂在 Excel 或现成系统上。** 用户每天都要打开那个表，你就在那个表旁边加一个按钮，点一下，脏活累活自动干了。他不觉得"我在用 AI"，他只觉得"我那个记台账的活儿，突然轻松了"。

这三种方式的共同点，是同一个：**不让用户来找能力，而是让能力走到用户正在干活的地方。**

寄生的本质，是尊重一个现实：**用户不会为你的系统改变习惯，但你可以顺着他的习惯，把价值塞进去。**

当然，寄生也不是万能的。有些场景你寄不进去——客户压根没有统一入口，或者那个入口僵到你没法动。这时候别硬凹，退一步，做一个"入口极轻"的小插件或快捷方式，能少登录一步是一步。寄生的精神，不是"必须用别人的界面"，而是"离用户越近越好，离新平台越远越好"。

举个最朴素的例子。客户的目标如果是"采购对账快一点"，你最好的落地方式，绝不是做一个独立的"采购对账 AI 平台"，让采购员每天专门登录进去用。而是把 AI 挂在采购员已经在用的那个流程上——他打开原来的对账单，系统已经自动把差异标红了，他只需要扫一眼那几行红字，点确认。平台？他压根不需要知道有这回事。

## **默认值：什么都不做，就是对的**

寄生解决的是"入口"问题。紧接着是"上手"问题：用户进来了，站在一个功能面前，不知道该怎么用，怎么办？

答案是**默认值设计**。

意思是，让你的系统“什么都不做，就是正确的结果”。用户不需要配置、不需要选择、不需要判断，打开就能用；甚至不打开，系统也替他该干的干了，只把例外推给他。

为什么要这样？因为人的天性，是尽量不做决策。你给用户十个设置项，他一个都不会碰；你给他一个“开箱即用、默默干活”的默认结果，他反而会依赖上它。

举一个极端的例子。如果你做一个周报生成器，最好的形态不是让用户“录入信息、点击生成、选择模板、导出”，而是每天后台默默记录他的工作数据，到了周五，自动在他邮箱里躺着一条“您的周报草稿已生成，点此确认”。用户连想都不用想，确认一下就行了。

数一数这条线：录入、生成、选模板、导出，每一个都是决策点，每一个都会筛掉一批人。你把这些决策点全部拿掉，只留下一个“确认”，用户就没理由不用了。

默认值设计的内核，就一句话：**你替用户把该做的决定都做了，只把“真正需要他拍板”的那个例外，留给他。**

再补一个例子。你做的是“异常预警”：系统每天扫一遍数据，只有发现对不上的地方，才推一条消息给负责人，剩下九百九十九条正常记录，一声不吭。这就是默认值设计的最高形态——正常情况零打扰，例外才来敲门。用户不需要盯着系统，系统只在他必须出手的那一刻，才让他出手。

反过来，如果你做的预警，是让用户自己去设置“哪些字段要对账、阈值设多少、多久扫一次”，等于把本该你扛的判断，原封不动扔回给了他。他大概率不设，或者设错，然后回头怪你的系统不准。

## 减少认知负担：别让用户做选择，更别让他学新概念

这是第三个，也是我见过最容易被忽略的一个。

很多团队做系统，有个执念：功能要全，参数要多，最好让用户“可配置、可扩展”。他们觉得自己做的是个通用平台。但他们忘了，坐在屏幕前的那个用户，压根不想配置、不想扩展，他只想把今天那点活儿干完，早点下班。

**功能的堆砌，对项目经理来说是成就感，对用户来说是认知负担。**每多一个按钮，就多一分“这个是个啥、我点了会不会出事”的犹豫。犹豫一多，人就不来了。

减少认知负担，落到操作上，就三句话：一次只看一件事，一次只做一个决定，尽量不引入新概念。

"不引入新概念"这条，我想多啰嗦两句。太多 AI 系统的失败，是败在"教育用户"上——动不动让用户理解什么是 RAG、什么是知识库、什么是 Agent、什么是提示词。用户不是来学这些的，他是来解决"客户问退货政策，我翻半天文档"的。

这里有个我印象很深的例子，来自一个 AI 产业园的三周实施。

他们给一批不太懂技术的业务员工做知识库。按"正统"思路，要上 RAG——先向量化，再镶检索，再调 embedding，架势拉满。但这个团队没这么干。他们用了个"带索引的 Grep"，说白了，就是调优过的全文检索。

为什么？因为对那批用户来说，他们要的就是"我扔一个词进去，几秒内把对的内容给我搜出来"。Grep 这一步，检索快、结果对、够用，三个标准全占。而 RAG 那套，是个好技术，但对这批人来说，是认知负担，是"还得解释为什么有时候召不准"的额外麻烦。

这个 Grep 有多好用？据实施者原话，是"猛得飞起"，连图片抓取的适配率都做到了 100%。

这件事给我的教训很实在：**技术选型不是越新越高大上越好，而是越贴合用户当下需求越好**。用户不需要你炫技，他需要你把他的活儿接过去。能用检索解决的事，别硬上向量库，给自己找事，也给用户添乱。

再配一个更贴近"甜头"的例子，我在别处讲过很多次：一个人工整理两小时的流程，你用 AI 压到十分钟。这中间，你没让用户学任何新东西，他干的事、看的界面，和原来一模一样，只是慢的那部分被替掉了。他做完第一单，低头看了看表，还剩一个多小时——这个瞬间，就是他心里的那声"卧槽"。

这就是我在别处反复提的那个道理：第一次交付，要制造的是"瞬间的正反馈"，不是"系统的全貌"。全貌可以后讲，甚至永远不主动讲；那声"卧槽"，才是用户回来用第二次的全部理由。

还有一个减少认知负担的细节，朴素但致命：名字别起玄了。给用户的按钮、菜单、提示词，用他工作里的原话，别用你的行业黑话。他脑子里装的是"对账""退货""排班"，你屏幕上跳出来"语义检索""向量召回"，他扭头就走。你把黑话当专业，其实是在他和你之间砌墙。

减少认知负担，还有一个时间维度上的讲究：复杂度要一点点加，别一次性全倒给用户。健身圈有个词叫"渐进超负荷"——今天先让他举五公斤，等他适应了，再加到十公斤。放到这里一个意思：你今天一股脑把最复杂的界面、全部的功能、一堆配置项摊开给他，他当场就被劝退了；你今天只给一个简单、能用的，等他主动问"还能不能再做点别的"，你再亮下一招。需求是他自己张口要的，认知负担就成了他自己愿意扛的。

还有一句最狠的：宁缺毋滥。与其给用户十个用不上的功能，不如把一个功能做到他离不开。配置项能删就删，按钮能合并就合并，菜单能砍一层就砍一层。每删掉一个用户用不上的东西，你就替他省下一次犹豫；省到后面，他点开来，眼里只剩一件事可做。这时候，激活就成了顺理成章。

把这三招摆在一起看，其实是一条线上的三步：先让能力出现在他眼皮底下（寄生），再让他不用做任何决定就能用（默认值），最后让整个过程不费脑子（减负）。三步走下去，"用你"就从一件"要学"的事，变成了一件"顺手"的事。人都是顺着阻力最小的方向活的，你要做的，是让你的系统，正好躺在他阻力最小的那条路上。

记住这一节收尾那句老话：用门槛每降低一分，激活的难度就下降十分。

### 5.3 评估体系驱动质量：上线后才是开发的开始

把门槛降下来了，用户也愿意用了。但下一个问题，几乎立刻就会接上来：系统的输出，经常是错的、是烂的，怎么办？

我先给你画一个我值班看见过的画面。上线第二周的某个周二下午，我在客户的工位旁，看一个客服大姐问系统一个退货政策问题。系统张口就答，答错了，还答得特别自信，一副不容置疑的架势。大姐抬头看了我一眼，没说话，默默切回她的老办法——一份贴满便利贴的纸质手册。那一刻我心里清楚：这个用户，大概率不会再打开了。

错不可怕，错得自信才可怕。真实用户，从来不会给你第二次"解释"的机会。

别觉得这是小概率。真实用户一进来，你的系统就要第一次面对真实世界的脏数据、真实场景的刁钻问题。它一定会犯错，只是错在哪、错多少，你未必想得到。

很多团队在这里犯一个错误：以为"上线"就是终点，上线之后就开始庆祝、收尾、准备下一个项目。

大错特错。**上线，恰恰是真正开发的开始。**因为只有真实用户开始用，你才会第一次看到：原来真实数据里的坑这么多、原来用户的真实需求和我猜的差这么多、原来系统会在这里出错。

上线前，你对系统的所有判断，都是"我以为"；上线后，用户用脚投票出来的数据，才是"事实上"。

这一步该做的，是搭一套**评估体系**，让质量在真实使用中不断被校准。

怎么搭？三步。先给个总纲：**建指标，用指标定位劣质输出，快速修复。**

## 第一步：定义指标

不要用"体验怎么样""好用不好用"这种虚话，要落到可量化的东西上。

比如：回答的准确率、被用户采纳的比例、单次处理节省的时间、被用户标记为"答非所问"的次数、用户反复追问同一个问题的频率。

这里有个讲究：指标要能**自动采集**，而不是靠人去填问卷。用户不会认真填问卷，但用户的行为不会撒谎——他采纳了没有、他追问了没有、他静悄悄关掉了页面没有，这些痕迹，后台都记得清清楚楚。你要做的，是把这些痕迹捞出来，变成数字。

我把常用的几个指标列个单子，供你挑：一次解决率（用户问一次就翻篇，不用追问第二次）、人工转接率（多少问题最后还是扔给了人）、采纳率（AI 给的答案，用户采没采纳）、被否定率（用户点了个"不对"）、追问频率（同一个问题被反复问）、任务完成时长（把"以前两小时，现在十分钟"变成能追踪的数字）。你不用全上，挑两三个，先跑起来，比搭一屋子好看没用的图表强。

## 第二步：用指标定位劣质

指标的意义，不在于好看，而在于帮你找到"哪些输出是烂的"。

很多人搭了看板，就满足于"数字在涨"。这是把指标当成了装饰品。指标真正的用法，是当警报器：哪个问题被反复追问、哪些回答被反复否定、哪类输入的采纳率掉下去了，指标要第一时间喊出来。

真实使用中的数据，会源源不断地暴露这些问题。你要做的，是盯住指标，第一时间发现它，而不是等客户打电话来骂你。

### 第三步：快速修复

发现烂输出之后，立刻修。改提示词、补知识、加规则、调整流程——越快越好。

注意"越快"两个字。质量问题，暴露得越早，代价越小；拖得越久，用户的心凉得越透。一个用户连问三次都得到错误答案，你第四天才修好，他早就认定"这玩意儿不行"，你再想把他拉回来，难了。

这套"上指标、找劣质、快修复"的循环，我把它叫作**热修复文化**。

它的日常，没有你想象的那么宏大，就是些琐碎的、停不下来的活：周一上午看上周的被否定率排名，把最烂的十条捞出来；周二改提示词，把被追问最多的那批答案补进知识库；周三上改完的版本，周四再回来看，被否定率掉没掉下去。一周一周这么滚。没有这层日复一日的滚，指标就是墙上的一幅画，好看，顶不了用。

它的核心逻辑，是承认一个事实：AI 系统在上线前的质量，永远是"拍脑袋"的质量；只有上线后，在真实数据里滚一圈，它才会慢慢变好。所以你要做的，不是追求"上线那天就完美"，而是建立一个能持续变好的机制。

有一个真实的教训，我一直记着，值得反复讲。

2023 年，我给一家企业做知识库。熟人客户，之前没正式合作过，但彼此认识。上线测试的时候，AI 回答错了一个产品问题。

团队的追查，是按部就班来的，一共四步。

第一步，怀疑模型胡说。是不是模型在编？查下来，不是，模型是照着检索结果说的。

第二步，怀疑知识库召回错了。是不是该召回的没召回、召回了错的？查下来，也没有，召回的内容和问题是吻合的。

第三步，怀疑扫描件解析错了。是不是文档转文字的时候出了岔子？查下来，还是对，解析出来的字，和原件一致。

到这一步，技术链条上能查的几乎都查了，每一步都是正确的。那错的到底在哪？

真正错的，是很多年前打印出来的一张产品设计文档——上面有一处印错了。准确地说，印错了一个数字。

而这家公司的老员工，都知道这一处印错。他们平时用的时候，脑子里会自动把这个数改过来，所以这么多年从没出过事。但问题是，这个"正确的改法"，从来没被写进任何一个线上系统，它只活在老员工的脑子里。

于是，AI 忠实地读取了那份错误资料，非常准确地，给出了一个非常准确的错误答案。

金句在这：**AI 可以非常准确地处理一份错误知识。**

这句话，值得贴在每一个做企业 AI 的人脑门上。它讲了两件事。

第一，别一出问题就怪模型。模型很多时候是"忠实"的，忠实到可怕——你喂它什么，它老实给你什么。错在模型的时候，反而往往不是最麻烦的；错在数据、错在知识本身的时候，才最隐蔽、最要命。

第二，真实使用中暴露的问题，往往在你上线前的任何测试里都测不出来。你在测试环境里能测出什么？测出模型照稿念。但你测不出"这张稿子本身就是错的，而全公司只有三个老员工知道它错了"。这些东西，藏在真实知识、真实业务细节里，只有等用户真正用起来，才会浮出水面。

讲完这个案例，我再补两个关于"评估"的实务提醒。

第一个提醒：准确率这个指标，会骗人。你如果只统计"回答正确率"，会发现数字贼好看，九十多分。但用户真正在乎的，往往不是"总体对了多少"，而是"那关键的一次，错了"。一个采购员一个月问系统一百个问题，九十九个对，就那一个关于合同金额的错了，让他赔了一笔——你在他心里就是零分。所以评估里，一定要单独拎出一类指标："关键问题的准确率"，或者更狠一点，"错误造成的代价"。别让平均分，把致命伤给平均没了。

第二个提醒：得有人在线上，负责"纠正"。那个知识库的故事里，最扎心的一点是，错误的知识在系统里躺了很多年，一直没人去改——因为从来没规定过"谁负责让知识变对"。评估体系要是只有"发现"、没有"纠正的人和流程"，就只是个报警器，问题一遍遍报，没人修。所以搭指标的时候，顺手把这题答了：这个错，发现了，谁来改？他有没有权限改？改完多久生效？答不上来，你的热修复就只是热，修不成。

所以，别怕上线后出问题。怕的是，你没有一套机制，去发现并修掉这些问题。

热修复文化，说到底，是给"犯错"提前留好了位置。它不追求上线当天零失误，它保证的是：不管哪里错了，我都能第一时间知道，并且尽快修掉。换句话说，热修复要的从来不是"不犯错"，而是"错被打捞上来的速度，永远快于错误扩散的速度"。

评估背后还藏着一个隐患，叫质量漂移。你的系统今天答得准，不代表三个月后还准——它的知识在老化、数据在变、业务流程在改，而它自己不会主动告诉你。所以评估体系必须"常亮"，而不是上线那周跑一次就收工。你今天放过的一个小错，会顺着时间慢慢长大，最后变成续约谈判桌上，客户嘴里那句"东西越来越不好用了"。防漂移最好的办法，是让指标天天自己跑，人只看异常。

这套评估体系跑顺了，还会送你一个副产品：续约谈判桌上的底气。明年跟客户谈续约，光说"我们这一年干得不错"，没用；你把采纳率、被否定率、一次解决率这几个数往桌上一摆，客户自己就看明白了。热修复文化，修的是质量，攒的是证据。

## 5.4 集成大战：绕开、硬啃、还是妥协

激活的路上，还有一座绕不开的大山——集成。

我先用一个画面开场。项目做到一半，我在客户的机房门口蹲了一下午，看着那台跑了十几年的老服务器，风扇嗡嗡转，机身上一层灰，接口文档是一沓不知道哪年的纸，纸角都卷了。那一刻我明白，接下来这段日子，跟调模型没关系了，全是跟旧世界打交道的手艺。

你的 AI 系统要落地，就必然要和客户现有的东西打交道：遗留系统、权限体系、数据管道、老数据库。这个过程，往往是项目里最脏、最费劲、最磨人的部分。

我把它叫作"集成大战"。打这一仗，最关键的不是技术，而是**策略**——什么时候该绕，什么时候该啃，什么时候该妥协。

### 什么时候绕

如果某个系统的集成成本，高到不成比例，而它对当前这一版的价值贡献又很小，那就绕。

比如，你要做的核心价值是"客服秒回"，结果发现客服系统是个十年老古董，没有 API。这时候，你没必要一上来就去啃它——先做一个独立的小入口，或者寄身在聊天工具上，把核心价值先跑出来，用成果去证明"这事值得做"，回头再谈集成。

绕，不是逃避，而是用最小代价先拿到阶段性胜利，为后面的硬仗争取资源和信任。

反过来，最怕的是根本没盘过，一脚踩进去才发现：客户那个老系统连最基础的 API 都调不出来，当年那家本地开发小公司五年前就倒闭了，源码丢了、数据库还加着密。这种坑，等你想绕的时候，人已经在泥里了。所以"绕"这个决定，做得越早越值钱，前提是你进场前就画过系统地图——第三章那张。

很多新手觉得"绕"是怂、是能力不够。错了。绕，是一种资源调度能力——你把力气用在该用的地方，而不是和一辆老古董死磕三个月，最后什么都交付不出来。

## 什么时候啃

如果某个集成，是你价值链条上必不可少的一环——不接它，你的系统就是断的，价值就无法完整交付——那就只能硬啃。

打几个比方，你就懂"必不可少"是什么标准。你要做实时对账，就得接客户的财务系统；你要做员工数据的 Agent，就得接它的 HR 老库；你要让 AI 直接下订单，就得接它的订单系统。这种时候，没有"绕"这个选项，因为绕开了，你的价值就没了。

硬啃的时候，有几个实用的办法：翻源码、看数据库结构（如果权限允许）、抓着客户的老员工问、做一层"适配层"把新老系统隔开、或者干脆重新抓取数据。

啃这个东西，靠的是耐心和"总能找到路"的信念。它不性感，但它是真功夫。

落到工具上，现在有些现成路子能用：比如通过 MCP 之类的协议去连 SAP、Oracle 这种企业级老系统，省得手写一整套对接。但工具只是省了"写代码"这一步，真正难的从来不是连上，而是把两边的业务语义对齐——你这边叫"客户"，他那边可能叫"往来单位"，还拆成了三张表。这一步，工具代替不了，只能靠人去磨。

## 什么时候妥协

有些集成，客户自己内部都搞不定——比如另外一个系统的接口，掌握在一个死活不肯配合的部门手里。这时候，纯技术手段已经没用了，你需要的是**组织层面的妥协和借力**。

要么，搬出能拍板的人，从上往下压；要么，调整方案，让那个卡住的系统先"旁路"掉，等政治问题解决了再接。

这里有个心法：妥协不是认输，是把“现在必须拿下的战场”和“可以以后再打的战场”分开。你现在跟一个对抗部门死磕，赢了也花了三个月，核心价值一分没推进；不如先把能绕的旁路了，把核心价值交付出来，让对方自己看到“这玩意儿没碍着我也挺好用”，立场自己就软了。人先看到好处，比你先讲道理，好使一百倍。

这，就回到了第三章画“政治地图”的价值：你提前知道哪一环会卡在谁身上，就能提前想好是绕、是啃、是妥协，而不是临到跟前了才手忙脚乱。

除了系统本身，集成的战场上还有两场看不见的暗战，很多人栽在里面还浑然不觉。

一场是权限。你的系统要干活，得跟客户现有的账号、角色、审批线对上。可客户的权限体系，常常和他自己口述的不一样：纸面上是分级分权的，实际里是“一个万能账号，全部门共用”；你要接单点登录，发现他那边压根没有统一的身份源，每个系统一套自己注册的账号。这时候硬啃 SSO，能啃出人命。更务实的做法，往往是先说明白“我们先用谁的身份、谁在什么条件下能做什么”，把权限范围钉死成一个双方都认的清单，再谈技术上怎么接。权限这事，先把“边界”谈清楚，比先写一堆对接代码，重要得多。

另一场是数据管道。两个系统要过数据，中间那条管道，就是你新架起来的命脉。它最怕的，不是传输慢，是“对不上”——这边说转了十笔，那边只收到九笔，那不见的一笔，查起来能查三天。所以只要涉及过数据，“对账”这个动作就一定要有：两边各记一笔，定时比对，对不上的自动报警。别等财务月底发现账不平，再回头找你。

### 比“接不接得上”更要命的，是“扛不扛得住事故”

上面讲的，是“把系统接上”这一层。我还得再补一层，它比接上更关键：**接上之后，你扛不扛得住这个旧世界随时甩过来的一记闷棍。**

客户的遗留环境，是混乱的。这不是形容词，是客观描述。你会遇到乱序的 webhook、说断就断的网络、对不上账的字段、被别人覆盖掉的脏数据。你的系统如果按“理想条件”来设计，上线第一天就会被这些混乱按在地上摩擦。

所以我做集成，脑子里始终绷着一句从实战里捡来的话：

**你的系统必须容忍客户环境的混乱——乱序、重试、断网、脏数据，而不是假设理想条件。**

以及它的姊妹句，也是这一节我想让你带走的那句：

为失败而设计——上游模型挂了就降级，再挂就用缓存，用户体验不中断。

这两句话，是集成大战里真正的护身符。

落到工程上，"为失败而设计"是几件很具体的事。外面那个模型 API 抽风了，别让页面一直转圈转到用户骂娘，自动切到备用的、稍弱一点的模型，先保证"有答案"；备用的也挂了，就吐缓存里上一次的结果，标注一下时间戳；网络断了，就本地先攒着，等恢复了一起重放；上游推了重复的消息，就用幂等去重，别让同一笔账记两遍。

你有没有发现，这些动作，没有一个是"让系统更快"，全是"让系统别死在意外上"。但它们恰恰是用户敢持续用你的底气。他用了几十次，没有一次被卡死、没有一次丢数据，他才会放心地把更重要的活儿，一件件交给你。

集成大战的胜利，从来不是"把所有系统都接上"，而是"用最小的代价，保证核心价值能跑通，并且跑不垮"。把这几个字记牢，往后你遇到的每一场集成，都能少走一半弯路。

## 5.5 一次真实的三周激活复盘

讲了这么多道理，光说不练假把式。我用一个真实感很强的三周实施记录，看看"激活"这件事，落到现场到底是怎么做的。

场景是这样的：一个 AI 产业园（我姑且叫它"人工智能小镇"）里，一批关键用户要上手用 AI 工具。任务是：让这些平时忙业务、不太懂技术的员工，真的把 AI 用起来。

如果按"常规"思路做，流程会是：做需求调研，上一个大平台，培训员工，老板压着大家用。结果通常是：员工抵触、用不动、项目黄掉。

这个团队，换了个思路，反过来了。他们做了一件看似"偷懒"、实则极聪明的事：**不替员工做，让员工自己做。**

为什么说这个思路厉害？因为它把"激活"的发动机，从"老板自上而下的命令"，换成了"员工自下而上的意愿"。命令只能让人动起来，意愿才能让人留下来。

具体怎么做的？分三周。

## 第一周：先把家伙事儿备齐

他们没急着上什么复杂系统。第一周，就干了一件很基础的事：给关键用户配上能跑得起来的电脑——内存加到 32G，因为要在本机跑多个 Agent，一台机器能同时挂五个以上。

别小看"换电脑"这一步，它其实是在扫清门槛的物理部分。你让一个员工用一台跑起来都卡的老机器去试 Agent，他第一次就卡死了，从此再也不会碰第二次；你先把机器给他配得顺顺当当，打开就用，他"这玩意儿好像有点意思"的前提才成立。门槛这东西，一半在软件，一半在硬件，很多人只顾前面那一半。然后，用现成的专家库工具，帮他们做了最基础的几件事：平面设计、课程设计、知识库。

这里有个细节特别有意思。他们做的知识库，没用高大上的 RAG，而是用了一个"带索引的 Grep"——说白了就是高效的全文检索。为什么？因为对这批用户来说，检索快、够用，就是最合适的。而它"猛得飞起"，图片抓取的适配率直接做到了 100%。

这暗合了前面讲的原则：**技术选型不是越新越高大上越好，而是越贴合用户当下需求越好。**

第一周的门道，还有一层：不教概念，只给工具。没有人去给员工讲"什么是 Agent 的架构""什么是语义检索"。大家就是领到一台能跑的电脑，用起来，感觉"这事儿不费劲"。信任，是这么一点一点攒出来的。

还有一层更深的：这三周，几乎没做传统意义上的"需求调研"。没有发一大堆问卷，问员工"你需要什么"。为什么不问？因为问也问不出来——员工对 AI 的想象力，受限于他没见过的东西。你问他"要什么"，他只能说"帮我省点事"；但你拉着他亲手搓出第一个 Agent，他瞬间就懂了"原来还能这么用"，然后自己会冒出十来个需求来。需求不是问出来的，是用出来的。

## 第二周：带着员工自己搓 Agent

到了第二周，关键动作来了。他们不自己埋头开发，而是带着员工，一起动手做 Agent。

先做两个"打样"的：一个待办 Agent，一个咨询师 Agent。待办 Agent，帮员工管理琐事；咨询师 Agent，则用来发掘痛点——它会引导员工去发现"我工作里有什么特别烦、特别重复的事"。

为什么先打样这两个？因为它们门槛极低、见效极快。待办 Agent 当天就能用上，员工第一天就尝到了甜头；咨询师 Agent 则在“顺手管理琐事”的壳子里，悄悄开始做最要紧的那件事——挖痛点。先给甜头，再挖痛点，这个顺序不能反。

这里我要停下来，点破咨询师 Agent 为什么是个妙招。它干的事，本质上是把第四章讲的“找痛点”，交到了员工自己手里。员工最清楚自己哪里累，但平时没人问他。咨询师 Agent 这一问，痛点就浮上来了——而且这个痛点，是员工自己说出来的，是他“想要解决”的，不是别人“布置给他”的。意愿，就在这一刻产生了。

员工的痛点被挖出来之后，直接转成一句提示词，员工把它扔给本机的 Coding Agent，让 AI 帮自己搭一个专属的小工具，搭好之后，就地部署、马上用。

这个链条很妙：员工自己找痛点，自己转成提示词，AI 帮他把工具搭出来，他自己用起来。全程，员工是主角，AI 是工具，团队只在旁边陪跑。

把这个画面，和本章开头那个“老板压着大家用”的画面摆在一起，差别就出来了。压着用，员工把系统当成负担，抗拒是必然的；自己搓，员工把系统当成自己的作品，维护是主动的。同一批人，同一个工具，位置的差别，就是天上地下。

有人会问：员工自己做的东西，质量能行吗？会不会很糙？我的回答是：糙，但它被用了。一个糙的、被天天用的工具，会自己长好；一个精致的、没人碰的工具，只会发霉。激活这台机器，先转起来，比先打磨完美，重要十倍。

### 第三周：喝茶，验收

到了第三周，让这个团队最有成就感的一幕出现了。

他们发现，自己“只需要喝茶，然后验收就可以了”。因为员工已经自己上手、自己迭代、自己把 AI 用进了日常，根本不需要团队在后面推着走。

有个细节最能说明问题：到了第三周，员工之间开始互相教。谁先搓出一个好用的 Agent，转头就拉着隔壁工位的同事过来看——“你看我这个，扫码登记的活儿，以前得手工抄一遍，现在一句提示词就完了”。这种炫耀“我自己做的工具”的画面，在任何一个强推的项目里都见不到。因为强推的系统是别人的孩子，没人会去炫耀别人的孩子。

这个三周的实施，金句是那句自述：“我只需要喝茶，然后验收就可以了。”

它完美地回答了"激活"这个难题：当使用 AI 变成员工自己的事，而不是老板强压的任务；当员工亲手把 AI 做成了自己想用的工具，而不是被迫接受一个陌生的系统——激活就不再是问题，而是自然而然的结果。

换个角度看，这三周还暗合了前面说的峰终定律。员工从第一周领到能跑的电脑，到第二周搓出第一个自己做的 Agent，再到第三周发现"这活儿真轻松了"，其实是一条不断踩在正反馈点上的链。第一次正反馈来得越早，他对这支团队、对 AI 这件事的信任，就扎得越深。等他开始主动向同事炫耀的时候，激活这件事，其实已经完成了——剩下的，只是时间问题。

这个故事里，藏着激活部署最深的一条规律：**最好的激活，不是让用户"接受"你的系统，而是让用户"拥有"这个系统。**

一旦它变成了"我的工具"，使用就内化了，阻力就消解了，续约也就有了最坚实的根基。

不过，这个三周能跑得这么顺，是有个前提的：它没怎么碰那些最难啃的旧系统。真实的项目，十有八九没这么干净。接下来这最后一节，我完整复盘一场真正的集成大战，看看"绕、啃、妥协"三条路，在一个具体的项目里，是怎么一步一步走下来的。

## 5.6 一次集成大战的完整复盘

前面三周那个案例，顺得有点不真实。它几乎没跟遗留系统正面交手。但你要知道，那是个例外。大多数项目，从进场第二天起，就撞上那些跑了十几二十年的老家伙。

这一节，我把一场集成大战拆开给你看，场景做脱敏，过程不掺水。

为什么偏要在讲"激活"的一章里，花一整节复盘集成？因为集成烂，激活就无从谈起。用户第一次用就卡在老系统对接的半道上，数据不对、答非所问，你门槛降得再低，也是把人请进了一间漏水的屋子。集成，是激活的地基。

挡在最前面的，是客户一个本地的 SQL 数据库，已经跑了快二十年。

这场的客户，是家传统行业的老厂，规模不小，系统却是一代代攒下来的，新旧混在一起，像一座住了二十几年的老房子，水管电线早就分不清哪根通哪。进场前画系统地图的时候，我们其实就有了心理准备——这一仗，躲不过去。

注意，是本地，不是云上的，也不是什么现代化架构。二十年里，它被一茬又一茬的员工修修补补，字段口径早就乱了——同一个东西，能在三个地方有三种叫法。更要命的，是它没

有一个像样的 API 给你接。你拿着新系统的接口规范，站在它面前，像对着一个不说话的老头。

我们一开始想绕。

绕的方案是：让客户定期把数据导出来，我们这边写个导入脚本。绕的好处是快，几天就能跑通一个能演示的版本，先让老板看到点东西。

但很快发现，绕不动。

因为客户要的核心价值，是"实时"。新数据一进来，AI 得马上知道、马上处理、马上把结果还回去。你让他手动导出、再由我们导入，中间这个时间差，就把"实时"这条命根子掐断了。绕掉的，不是集成，是整个价值主张。

那就只能啃。

啃的第一件事，是摸清那个老库的脾气。权限有限，我们只申请到部分表的只读权限。于是就在合法范围内，翻结构、看字段，再抓着客户公司里一个在那儿干了十五年的老员工，一个问题一个问题地问：这个字段现在还在用吗？这个表是不是废弃了？那个数字为什么时不时对不上？问出来的东西，比任何文档都值钱。很多"真相"，就藏在这种一问一答里。

摸底路上，还撞上一个权限的坎。要实时同步，我们就得往老库里写数据。可客户的 DBA——一个管了这库十年的老人——对"外部系统写库"这件事，天然抵触。他的顾虑也合理：万一你写坏了，二十年攒的家底就砸了。这一关，纯技术没用，是典型要"妥协"的局面。我们没有硬闯，退了一步：先做只读接入，把"读"这条最值钱的链路先跑通，用结果证明我们靠得住；"写"的那部分，等和 DBA、和他的领导，把权限边界、回滚方案、责任划分一条条谈妥了，再开写权限。先赚信任，再要权限。

啃的第二件事，是搭一层"适配层"，把新老两个系统隔开。我们写了一个双向同步引擎：老库里变了，同步到我们这边；我们这边产生了结果，再写回老库。

而老库那边最折磨人的，是"乱"。消息会乱序，会重试，会断网，会有脏数据。写到这，就又绕回了上一节那两句话。

这个同步引擎里，我们没有假设任何"理想条件"。相反，每一行代码都是按"最坏情况"写的：对方重复推送了，就幂等去重；对方断网了，就本地排队、恢复后重放；两边字段打架了，就给一条明确的冲突解决规则，而不是让系统自己拍脑袋。

同步跑起来之后，我们还专门加了一道“对账”：每晚拿两边的记录过一遍，条数对不上、金额对不上，第二天早上自动报警。这条命脉不是接上就完事的，得日日夜夜有人盯着它对齐。前一段讲的数据管道那场暗战，在这儿变成了例行的功夫。

还有一处，是“为失败而设计”真正落地的地方。我们的 AI 能力，依赖上游的模型服务。有一阵子，上游模型说挂就挂，一挂，整条链路就像被人掐了脖子，用户那边转圈转到发火。

我们做了三层退路。主模型挂了，自动降级到备用的、稍弱一点的模型，先保证“有结果”；备用也挂了，就看缓存——上一个小时已经算好的结果，先顶上去，标注时间；缓存都没有，就给用户一个明明白白的“稍后再试”，而不是一片空白地转圈。

你有没有发现，整场复盘里，最花力气的，不是“让 AI 更聪明”，而是“让这套东西在客户的烂环境里，别散架”。

这场的结局，不是“一次把所有系统都打通了”。那个卡在一个死活不肯配合的部门手里的接口，最后选择了妥协——先旁路，等能拍板的人点头之后，再回头接。而真正决定这场仗输赢的，是那条被硬啃下来的双向同步，和那套“挂了也不中断用户”的容错设计。

你细看这一场，绕、啃、妥协，三条路其实各走了一遍：先想绕，绕不动所以啃，啃不动的地方妥协。顺序也不是固定的，全看那一步对“核心价值”的重要性。价值小、代价大的，就绕；价值大、绕不掉的，就啃；啃不动的，就妥协借力。把这把尺子握在手里，集成这场仗，你就有了地图。

把这一场，和前面那三周放在一起看，你就抓住了这一章的全部：

激活，是让用户愿意用；质量，是让用户用了不失望；集成，是让这套东西在客户的旧世界里站得住。三件事，没有一件靠“上线”本身能解决，全都得靠你，在真实使用里，一点一点磨出来。

## 章末工具箱

这章的两个模板，一个在上线前用，一个在上线后用。前者帮你把“激活”这件事安排明白，后者帮你持续盯着它还活没活着。别把它们当成一次性填完的表格，当成你的值班本，翻起来才值钱。

工具一：激活清单

上线前后，逐条过一遍。任何一条没做到，激活都可能功亏一篑。

降门槛

- [ ] 系统是否出现在用户已经在用的界面里（企微 / 钉钉 / Excel / OA），而不是一个独立新平台？
- [ ] 用户是否需要额外的学习、配置，才能开始用？
- [ ] "什么都不做"是不是就能得到正确结果（默认值设计）？
- [ ] 有没有引入用户本来不用懂的新概念（RAG / Agent / 提示词）？
- [ ] 用户第一次用，能不能在几分钟内尝到"更省事 / 更快 / 更准"的甜头？

保质量

- [ ] 有没有一套可量化的质量指标（准确率、采纳率、节省时间）？
- [ ] 指标是不是能自动采集，而不是靠用户填问卷？
- [ ] 有没有机制，第一时间发现真实使用中被打标记的"烂输出"？
- [ ] 发现问题后，能不能快速修复（改提示词 / 补知识 / 调规则）？

过集成

- [ ] 哪些系统是价值必需、必须硬啃的，哪些可以暂时绕开？
- [ ] 卡在某个敌对部门或老系统时，是绕、是啃、是借力，有没有提前想好？
- [ ] 系统是不是按"为失败而设计"来搭的——模型挂了会降级、会走缓存、用户不会被中断？

工具二：健康度评估表

上路后，用它定期给"激活"做个体检。不是上线时填一次，而是每周扫一眼，看见危险信号就动手。

|    |      |      |
|----|------|------|
| 维度 | 健康信号 | 危险信号 |
|----|------|------|

|      |                |                 |
|------|----------------|-----------------|
| 使用频次 | 日活稳定或上升        | 日活一周内掉到个位数      |
| 使用深度 | 用户开始依赖、主动提需求   | 只点开看看，从不真正使用    |
| 价值感知 | 用户能说清"比以前省了什么" | 用户说不出到底有什么用     |
| 主动传播 | 新用户来自老用户推荐     | 全靠行政命令才有人用      |
| 反馈   | 用户会主动报错、提改进    | 错了也没人吭声（可能是不用了） |
| 质量水位 | 被标记的烂输出在减少     | 同一类错误反复出现、无人修复  |

怎么用这张表：每周挑一天，五分钟，瞄一眼每一行落在哪一列。只要有一行滑进右边那一列，就说明有个地方正在失血，别等月底。激活不是一锤子买卖，是得有人一直值班的岗。

## 本章要点回顾

首日魔咒的根子：用户没有任何理由，为了"你上线了系统"而改变行为。

激活的本质，不是功能上线，而是用户行为的改变。

降门槛三招：寄生现有界面、默认值设计、减少认知负担（不逼用户学新概念）。

上线才是真正开发的开始：建指标、定位劣质、快速修复，热修复文化。

AI 可以非常准确地处理一份错误知识——质量之患，常在数据不在模型。

集成大战要分清绕、啃、妥协三路，并按"为失败而设计"搭容错。

最好的激活，是让用户"拥有"系统，而不是"接受"系统。

## 常见误区速查

**把上线当终点：**部署完就庆祝收尾，真实用户一用就露馅。→ 上线只是真正开发的开始。

**做一个漂亮的新平台，逼用户来登录：**入口离用户的工作流太远。→ 寄生到企微 / OA / Excel 里。

**功能堆得越多越好：**按钮和参数在用户眼里全是认知负担。→ 减少决策点，别引入新概念。

**指望一次培训搞定激活：**培训解决不了"为什么用"，只解决"怎么用"。→ 靠门槛低和第一口甜头。

**一出错就怪模型：**很多时候错在数据和知识本身。→先追数据、追知识，四步走。

**集成上要么全接、要么全绕：**真实是三者混用，按"价值必需"来定。→绕可绕的，啃必啃的，妥协卡政治的。

## 本章行动清单

读完这一章，别急着翻走，先做三件事。

第一件，翻开你手上那个已经上线、或者马上要上线的系统，问一句：它出现在用户已经在用的哪个界面里？如果答案是一个独立的网址或者 App，回去补一个寄生入口。

第二件，把系统里所有需要用户"配置、选择、学习"的地方，列一张清单。每一条都问一句：这能不能由系统默认掉、替用户决定掉？能拿掉多少，激活就顺多少。

第三件，给你这个系统的"失败"列三档退路：上游挂了怎么办、备用挂了怎么办、缓存也没了怎么办。写不出来，说明它还没准备好进入客户的真实世界。

这三个动作，加起来花不了你半天，但能帮你少死一个项目。

激活做好了，系统才算真正"活"了。但活下去，不代表能一直活下去。接下来，你会面对一个更实际的问题：客户凭什么在合同到期时，还愿意续约？下一章，我们讲守住续约、扩大收入。

<div style="page-break-after: always"></div>

## 第 6 章 守住续约与扩大收入

续约，不是合同到期那一周的事，而是从部署那天就开始输或赢的。

我最早明白这个道理，是在一家做财税外包的公司的会议室里。

那天的验收会开得很顺利。我们给他们做的智能合同初审，当场就上了线。老板握着我的手说"不错，以后有需求再联系"。我那天挺高兴，觉得这单圆满了，可以收工了。

结果后来再没联系。

一年后我偶然听说，那套系统在他们那儿，早就没人打开了。负责对接的那个运营经理，早就跳槽去了别家。

那一刻我才反应过来：验收会上的那声"不错"，不是终点，而是一个新的开始——只是我们谁都没去经营它。

很多做企业项目的人，都有这个致命的错觉：以为"交付"是终点，合同签了、系统上了、验收过了，这事儿就结束了。

不对。对企业服务来说，**真正的生意，是从续约和扩大开始的。**

一次性的项目费，只能让你活下来。今天接一单，明天找下一单，你永远是个手停口停的作坊，客户的账是一次结清、以后不再相干。

只有客户愿意一续再续、越买越多，你才能规模化地赚到钱。第一单赚的是血汗钱，第二单赚的是信任钱，到了第三单第四单，才开始有"复利"的味道——客户已经信了你，磨合成本归零，交付速度翻倍，利润自然就上来了。

所以这一章讲的，是两件看起来"商业"、实则和每一步交付都深度绑定的事：**怎么让客户不流失，怎么让客户越买越多。**

你可能觉得这是销售的事，跟一个写代码、做交付的 FDE 没关系。错了。续约和扩大，恰恰是最能检验一个 FDE 成色的地方——因为客户续不续、扩不扩，看的从来不是你的谈判话术，而是你之前每一次交付，到底在他心里累积成了什么东西。

## 6.1 客户为什么流失：五类原因

要守住续约，先得搞清楚客户为什么会走。

很多团队是稀里糊涂失去客户的。客户说"今年先不续了"，他们一脸懵：之前不是合作得挺好吗？合同不是刚验收完吗？

问题就出在这个"懵"上。如果客户流失对你来说是个意外，说明你从头到尾没有一个框架，去理解"客户为什么会走"这件事。

我把客户流失的原因，归结成五类。它们往往不是单一发生，而是叠加在一起，最后在某个时间点一起爆发。五类里头，前两类你还能抢救，后三类基本是慢性病，等你发现的时候，病已经入了骨髓。

下面一类一类说，每一类，我都配一个我见过、或者我亲身经历过的场景。

## 第一类：价值蒸发

这是最普遍、也最隐蔽的一类。

客户当初签单，是因为相信这套 AI 能给他省下真金白银。可系统用着用着，价值却一点一点地"蒸发"了——注意，不是因为系统变差了，而是因为**系统周围的环境变了**。

我说个具体的。

有一年我给一家零售品牌做了个客服知识库，上线头三个月，客服人工咨询量降了将近四成，客户特别满意，续约是板上钉钉的事。结果到了续约前，对方突然说，客服这块业务要整体外包出去，由外包商统一提供客服系统，我们的东西就冗余了。

你看，系统没变差，知识库还在，问答还是准的。但"用这套系统的人"整体没了，价值就跟着蒸发了。

价值蒸发的路径，我数得来的就有这么几条：

业务模式调整了，当初解决的痛点不疼了——就像上面那个客服外包的案子，痛点本身跟人一起搬走了。

领导换了，新领导根本不 care 这个系统。新官上任三把火，第一把往往烧的就是前任留下的"外来户"。你做的系统，在旧领导那儿是政绩，在新领导那儿就是个"要花钱维护的遗留包袱"。

还有一条最憋屈的：竞争对手也用上了同样的东西，你这套不再稀缺了。你费劲做的差异化，三个月就被别人抄了个七七八八，客户心里的"值"，也就被摊薄了。

价值一旦蒸发，续约就成了无源之水。客户说不出哪里不好——系统没坏、人也没得罪他——但就是觉得"不值得续了"。

这种流失最要命的地方在于：它没有任何冲突，客客气气地就没了。你连吵架的机会都没有。

## 第二类：支持者离场

企业里，能推动一个项目落地的人，往往是少数几个"内部支持者"。就是那个力排众议、顶着别的部门压力、拍板说"这个 AI 我们上"的人。

他可能是那个懂业务的运营负责人，也可能是那个愿意背 KPI 的部门老总，甚至就是老板本人。

但这个人，是会流动的。

升职了，调岗了，跳槽了。他一走，你这个项目在企业里，就失去了"保护伞"。当初帮你说话的人没了，剩下的，要么是看热闹的，要么是当初被 project 得罪过的、现在等着看你笑话的。

我见过最典型的一个场景：一个制造企业的项目，靠的是他们供应链总监一个人力挺。项目挺好的，用了快两年，节节高。然后那个总监被调去管海外市场了，接手的是个空降的新人。新人对这套系统一无所知，也不觉得是自己的事。半年后，续约谈判桌上，对面换了一张陌生的脸，第一句话是："这个系统，我不太了解，你跟我讲讲，我们为什么还要续？"

那一刻你就知道，持续了两年的信任，归零了，全得从头再来一遍。而从头再来的成本，往往比当初第一次成交还要高。

支持者离场，往往是续约危机最直接的触发器。它不像价值蒸发那么慢，它是一下子的事——人一走，保护的壳就碎了，项目直接暴露在各部门的审视下。

### 第三类：质量漂移

系统上线时质量不错，但时间一长，慢慢就"漂"了。

知识过时了没人更新；模型的效果随着业务变化越来越差；当初承诺的准确率，现在悄悄往下掉；系统里累积了一堆没人管的小 bug。

这类流失最阴的地方在于：它是一点一点发生的，你今天看没事，明天看也没事，等真出了问题，客户心里那根刺已经种下了。

而质量漂移的根源，很多时候压根不在模型，而在"知识本身错了，且没人接管纠正"。

这里有个我印象特别深的真实遭遇，我在前面讲 MVD 的时候也提过，这里再从续约的角度说一遍。

2023 年，我给一家企业做知识库。是个熟人客户，之前没合作过，但彼此认识。上线测试那天，AI 回答错了一个产品问题。

我们第一反应是模型胡说，又怀疑知识库召回错了，再怀疑扫描件解析错了，一路追下去，发现技术链路全部是对的。

真正错的，是很多年前打印出来的一张产品设计文档——上面有一处印错了。这家公司的老员工都知道这处错，平时在脑子里自动改过来，所以十几年没出过事。但这个"正确的改法"，从来没有进入任何线上系统。

于是 AI 忠实读取了那份错误资料，非常准确地，给出了一个错误答案。

这件事的教训，我当时记了一句，到现在还贴在脑子里：**AI 可以非常准确地处理一份错误知识。**

放到续约的语境里，这句话更重了。因为客户不会记得你上线那天的漂亮，他只会记得最近一次用它时的糟糕体验。一个客服，用了你的系统，给了客户一个错答案，被客户投诉，还被扣了钱——这一次，就足以抹掉你之前九十九次对答如流的功劳。

质量漂移，是温水煮青蛙式的流失。青蛙直到水开了才跳，客户直到被坑了才想起，原来这系统已经开始不靠谱了。

而更麻烦的还在后面：质量一旦漂移，修复它的成本，是逐年递增的。知识错一处不管，错的人会把它当"对的"往下传；接口坏一个不修，用的人会绕开它另起炉灶。等你攒到年底想统一收拾，发现窟窿已经多到填不完了。

#### 第四类：更便宜的选择

这是最赤裸裸的一类。竞争对手开出一个更低的价格，承诺了"差不多的效果"，客户一纸通知，你出局了。

我先说一个荒诞的对比，帮大家理解这行的价格到底能水到什么程度。

同样一个需求，你按真实的交付周期和人力成本，报价十万。客户扭头拿着一张截图回来，问你："为什么网上有人只报两千？"

两张报价单，背后是截然不同的东西：交付范围不同、验收标准不同、质量保障不同、出了事谁兜底不同。可到了甲方眼里，它们都叫"AI 解决方案"，长得一样，那就比谁便宜。

很多时候，你就是这样被一个成本价、甚至亏本价抢单的小作坊，硬生生挤出去的。

这说的是获客端的比价。到了续约端，也是一样的逻辑：客户用着你的系统，突然有个厂商过来，说“你那个我也能做，价格打五折”。客户一听，动心了，回头就来跟你压价，压不拢，就走人。

对这类流失，唯一的防御是：让客户觉得“换掉的成本”远高于“忍受你的成本”。

“换掉的成本”从哪来？两条路。要么你做出了别人做不出的价值——那套东西里有你的行业 know-how，抄不走。要么你和他的业务已经深度绑定、拆不开了——数据在你这边跑顺了，流程在你的系统里固化了，换一家等于把已经谈顺的流程全部推翻重来一次。

只要这两条你一条都不占，光靠“关系好”和“服务态度好”，是挡不住一个更便宜的选择的。因为你便宜不过一个不要命的对手。

这里有个很实用的应对，我多说一句。

当客户拿着一个更低的报价来找你，你第一个要问的，不是“他们报多少”，而是“他们报价单上，没写进去的那部分是什么”。

十万的方案和两千的方案，差距从来不在“AI”这两个字上，而在那些看不见的东西：数据谁清洗、权限谁打通、上线后谁维护、出了错谁在几小时内响应、验收不过谁兜底。你把这份“差异清单”摊开来，让客户自己比，他多半会自己回到你这边来。

因为会算总账的客户都明白一个道理：便宜的那一半，往往在三个月后，以返工、扯皮、停摆的形式，加倍地找回来。

你要做的，从来不是跟他比谁更便宜——你便宜不过一个不要命的对手。你要做的，是帮他把这个“总账”，提前算给他看。

## **第五类：客户自身经营恶化**

这是最无奈、也最容易被忽视的一类。

客户自己不行了——业绩下滑、现金流紧张、开始砍预算。这时候，你的 AI 系统，无论做得多好，都可能是第一个被砍掉的东西。

为什么是“第一个”？因为在客户的预算表上，你的系统，往往是那个“看起来最像锦上添花”的支出。核心生产设备不能停，工资不能不发，供应商货款要结……轮到 AI 订阅费的时候，它就被最先划掉的那一行。

我在做项目的时候，就真实遇到过这种情况：辛辛苦苦做完项目，客户经营状况恶化，尾款都收不回来。

到了那一步，什么合同、什么追款流程，都未必能从一家已经失去支付能力的公司手里，变出钱来。合同写得再严，追款流程做得再规范，对面没钱，就是没钱。你能做的，只有认账。

这里有一句我经常提醒自己的话，来自一次亲身经历后的总结：

**一家公司若业绩持续下滑，没有 AI 也会裁员，有了 AI，AI 只是一个更好听的理由。**

你以为你的系统帮他提效了，能保住他的命。可在裁员的现实面前，你的系统本身就是被裁的对象。AI 不是救命稻草，它只是那场风暴里，一个更好听的、用来解释"为什么我们也要收缩"的理由。

所以后来我养成一个习惯：**选客户，本身就是守续约的第一步。**一个正在衰退的行业、一个连续亏损的客户，你从签单那天起，健康分就已经比别人低了一截。这跟你的交付水平无关，跟你的运气有关——而聪明人，是不赌运气的。

这五类流失，说穿了，指向同一个结论。

**续约不是靠"到期前那次续约谈判"赢来的，而是靠"部署之后持续创造价值"赢来的。**

价值蒸发，是价值停止创造的结果；支持者离场，是价值没人再替你背书的结果；质量漂移，是价值在悄悄缩水的表现；更便宜的选择，是你的价值没能形成壁垒；经营恶化，是价值再好也扛不住的大势。

五种死法，根子都长在"有没有持续创造价值"这一件事上。

所以，等客户开口说"不续"的时候，一切都已经太晚了。真正的续约拉锯，早在你部署的那天就已经开始，只不过大多数人，一直没意识到自己在拉锯而已。

## 6.2 守住续约：用健康分在流失前预警

既然续约是条长线，光靠到期前那一哆嗦是救不回来的，那你就需要一个"仪表盘"，能在客户流失之前，提前看到危险信号。

这个仪表盘，就是**客户健康分**。

在讲它之前，我先把衡量续约质量的那个核心数字，摆出来。因为后面所有动作，都是围着它转的。

## 一个关键指标：NRR

这个指标叫 **NRR, Net Revenue Retention, 净收入留存率**。

它的含义很简单，就一句话：**去年这一批老客户，今年还给你贡献了多少钱。**

打个具体的比方。你去年有十个老客户，一共给了你一百万。今年还是这十个人，中间有人走了，有人少买了，但也有人续了、还加购了。到最后，这十个老客户今年一共给你贡献了一百二十万。那你的 NRR，就是 120%。

如果 NRR 大于 100%，说明老客户不仅续了，还越买越多、或者你涨价了他也不走——这是健康的企业服务生意的标志。

如果 NRR 低于 100%，说明老客户在缩水。你只能靠拼命拉新客户来填补老客户流失留下的窟窿。这是一种不可持续的、累死人的增长——新客的获取成本越来越高，你有多少命去填这个无底洞？

顶尖的企业服务公司，NRR 能做到 110%、120%，甚至更高。这里面的道理很简单：他们不是靠“卖一次”，而是靠“一个客户越买越多”。

而大多数“接一单算一单”的团队，NRR 可能连 50% 都不到。今天这个客户做完了，明天就去找下一个，一年下来，十个客户走了七个，账面上看着忙，实际全是给别人做嫁衣。

NRR 这个数字，就是检验你“守续约”能力的照妖镜。你吹得再天花乱坠，NRR 一摆出来，是人是鬼，一目了然。

所以我建议，不管你团队多大，都养成一个习惯：每个月，把老客户的续费和增购单独拉出来，算一次 NRR。这个数开始往下掉，就是你整个续约体系亮起的第一盏红灯。

## 为什么要有“健康分”这回事

搞清楚了 NRR 这个结果指标，你会问：结果掉了，我当然知道；问题是，怎么在结果掉之前，就提前知道谁会掉？

这就是健康分的意义。

先说一个残酷的现实：客户很少会突然说"我不续了"。

在"不续"这句话说出口之前，往往已经有一连串的信号，早就暴露在明处了——使用数据在掉、对接人态度在冷淡、回款在变慢。这些信号，一个两个看，都不起眼；可它们组合起来，就是一份清晰的"流失前预测报告"。

问题是，大多数团队没有盯着看。

他们不看使用数据，不看组织变动，不看客户经营，只盯着合同到期日。于是客户流失这件事，在他们那儿，永远是一个"突发状况"，永远打得措手不及。

健康分预警，就是把这些信号量化的过程：给每个客户装一个仪表盘，盯着几类关键信号，定期打分，分数跌破线就拉警报，逼你提前行动。

## 四类信号，怎么盯

具体盯哪几类信号？我把它归成四类。这四类，正好和前面 6.1 讲的五类流失一一对应——你看，流失原因是果，信号是因，盯住了信号，就等于在果还没发生之前，先看见了苗头。

### 第一类，使用信号。

日活、周活在掉吗？掉得多快？

这个最直观，也最骗不了人。一个客户嘴上说"很满意"，结果后台数据显示，最近三十天的活跃用户从一百二掉到了四十，你就知道，满意是假的，流失是真的。

光看日活还不够，还要看"使用深度"。用户从"每天用"，变成"每周用"，再变成"想起来才用"，这个滑坡的每一次台阶，都是红旗。更要命的是"核心用户弃用"——原来天天打开的那个业务骨干，忽然半个月不登录了，这背后往往藏着具体的原因：某个接口没接上、某个报错一直没人修、某个新需求被晾在那里。这种信号，比一千句"挺好的"都真实。

### 第二类，组织信号。

你的内部支持者还在吗？客户那边的关键对接人换了吗？决策层对你这个项目还关注吗？

这一条对应的是"支持者离场"。组织信号最硬的一个指标，就是你在客户那边，还有没有一个"肯替你说话、也说得上话"的人。这个人变了，或者没了，你的健康分要立刻往下调。

怎么发现？靠日常的接触。你多久没和那个支持者吃过饭、通过电话了？最近一次找你的人，是不是从"愿意拍板的人"，变成了"只能传话的人"？对接人从业务负责人换成了刚入职的新人，意味着什么，不用我多说了。

### **第三类，价值信号。**

客户还能不能说清，这套系统给他省了多少钱、解决了什么麻烦？

这一条对应的是"价值蒸发"和"质量漂移"。

你去做一个简单的测试：把那个当初力挺项目的客户拉过来，让他不看 PPT，直接讲——"这套系统，去年帮你们省了多少钱？"如果他能不假思索地报出一个数，说明价值还在他心里，很鲜活。如果他支支吾吾、讲不出个所以然，说明价值已经在蒸发了——连受益者自己都说不清好处在哪，续约谈判的时候，他拿什么去说服他的领导？

再补一个质量向的信号：客户最近提过问题吗？报过错吗？他报的错，你们多久修好的？一个"报错没人理"的客户，和一个"从没报过错"的客户，完全是两种健康状态。

### **第四类，财务信号。**

客户自己的经营状况怎么样？回款是不是开始变慢？

这一条对应的是第五类流失"经营恶化"，也是最容易被人当"外部因素"忽略的一条。

回款变慢，是经营恶化的第一声咳嗽。原来三十天账期准时付，现在拖到六十天、九十天，你就该警觉了：不是他不想付，是他手上真没钱了。

再往外看一层：这个客户所在的行业，最近在扩张还是在收缩？他自己公司，是还在招人，还是开始裁员？这些信息，你在饭局上、在朋友圈里、在招聘网站上，都能侧面看到。真等到他连尾款都付不出，你才想起来查，就晚了。

## **分数怎么定，预警怎么触发**

把这几类信号放在一起，每个客户打一个 0 到 100 的健康分。

权重怎么分？我给个我常用的参考：使用信号占大头，三十分；组织信号和价值信号次之，各二十五分；财务信号二十分。具体到细节，后面章末工具箱里有完整的《客户健康分评估表》，你照着改就行。

分数跌破某个线（比如 70），就触发预警。触发之后不是让你去"催续约"，而是让你先回到现场，查清楚分数为什么掉，把掉下去的那块，补回来。

这里有个关键，我必须强调：**健康分预警的意义，不在于那个分数本身，而在于逼你持续地、结构化地去盯客户的真实状态。**

很多人一听"健康分"，第一反应是"又多了个要填的表格，烦"。可你要想清楚：没有这个分数，你对客户的认知，是散的、是拍脑袋的、是"感觉最近联系少了"。有了这个分数，一切变成了可比较、可追踪、可提前动作的东西。

你盯得越紧、干预得越早，客户流失的概率就越低。把"被动续约"变成"主动守护"，这一步，是很多团队半年都迈不过去的坎，也是守住续约的核心分水岭。

### 健康分，多久打一次，谁来打

方法讲完了，还有个落地问题绕不开：这个分，多久打一次？谁来打？不落实，健康分就是一张废纸。

我的习惯是四个字：**按月打分，遇事随时更新。**

为什么按月？因为大多数流失信号，是以"月"为单位变化的。使用数据要按月才看得清趋势，组织变动和回款周期，也都不是天天变的事。打得太勤，会变成负担，坚持不了三个月就黄了；打得太疏，又会错过窗口——你三个月才打一次，客户刚好在第二个月流失了，你连预警的机会都没有。

"遇事随时更新"指的是：客户那边一有风吹草动——支持者跳槽了、对接人换了、公司大规模裁员的新闻上了热搜——当天就把他那个分，手动往下拉一档。这种硬信号，等不到月底。

谁来打？**必须让离这个客户最近的人打，不能让一个跟客户八竿子打不着的运营来填表。**

打分的人得看得见现场、闻得到味道。他知道客户上个月骂了谁、知道哪个接口三个月没修好、知道对接人最近是热情还是敷衍。这些信息，表格填不出来，只有泡在现场的人心里有。让一个对客户一无所知的人打分，打出来的就是个废数。

还有一条最关键的：分数打出来，要能触发动作，而不是躺在表格里吃灰。跌破 70 的那一周，就得有人站出来负责干预，把"捡使用、捡支持者、捡价值"的动作，排进接下来两周的日程。没有这一步，前面的打分、预警，全是白干。

我可以给你举个反例。我见过有团队，健康分表格做得特别漂亮，每月准时填、准时审、准时在管理会上展示，四色仪表盘红黄蓝绿，看着像那么回事。结果一整年，一个流失客户都没救回来。为什么？因为他们的分数，只用来"展示"，从来没用"行动"——黄了的客户，没人去管，等到红了，就是宣布死刑。

健康分不是用来汇报的，是用来救人的。这个分，你打了而不行动，跟没打，没区别。

## 6.3 扩大收入：从守到攻

守住了续约，下一步是扩大。

我把话先说在前面：扩大，不是销售去客户那儿"多卖点东西"。FDE 的扩大收入，是建立在交付之上的——你先把一个场景做扎实、做出可验证的价值，扩大才是水到渠成的事。反过来，价值没立住就急着扩张，那不叫扩张，叫把一个还不牢靠的基础，摊到更薄的面积上，早晚塌。

这节我分三块讲：怎么用"免费"打开局面而不自干（三本账）、怎么把收费方式一步步从"卖工时"进化到"卖结果"（四档进化链）、以及最被低估的增量来源——深耕老客户（横、纵、纵深）。

### 免费验证的"三本账"

很多 FDE 觉得，跟客户收费是件难事。客户还没信你呢，一上来谈钱，容易把人吓跑。于是宁可先"免费做个小样"来破冰。

免费验证本身没错。很多单子，就是靠一个免费的小样撬开的。但错在"免费了就真的免费了"——辛辛苦苦做个样，做完客户说了声"不错"，扭头就没了下文，你还不好意思去要钱。

这种白干，干多了，团队士气先崩。

正确的做法，是算清楚免费验证的"三本账"。免费不可怕，可怕的是稀里糊涂地免费。三本账算清楚，免费才是策略，而不是纯粹的亏损。

#### 第一本账，获客账。

免费小样，是拿来换信任、换正式订单的"饵"。它值不值，取决于能不能把客户从"观望"推进到"愿意真金白银下单"。

所以做免费样之前，你先问自己一句：这个样，做完之后，客户能立刻看见什么？能不能看见一个他愿意掏钱买的"结果"？

如果答案是"能看见"，那这个免费就是投资，花的每一分成本，都是在买成交的概率。如果答案模糊，那你干脆别做——做了也是白发，客户看了只会觉得"哦，AI 也就这样"。

我见过把获客账算得最透的人，是那些代运营出身、再转来做 FDE 的。他们手里攥着多年积累的老客户和信任关系，回头跟老客户讲一句"我帮你省两个人、提效三成"，先用个小样证明自己没吹牛，单子就签了。免费样在他们手里，是撬动存量信任的杠杆，一撬一个准。

### **第二本账，风险定价账。**

这一本，很多人从没算过。

免费，本质是你替客户承担了"这东西到底行不行"的风险。客户不买单，不是嫌贵，是信不过——他不知道你是个真能交付的，还是个 PPT 选手。他用"不付钱"，把风险挡在了门外。

你用免费，是替他把这个风险扛了下来。换句话说，你在卖一个"确定性"。

这笔账算清了，你就明白两件事。一是免费的额度要有分寸：不是无限免费，而是"免费到足以让他相信，又不至于让你伤筋动骨"。二是免费的成本，其实是可以折算进后面正式报价里的——你承担的风险，本身就是价值的一部分，凭什么不能算钱？

很多老实人吃亏就吃在这儿：免费做了，正式报价还不好意思往上加，等于把风险白扛了。

### **第三本账，失败价值账。**

就算这次免费验证失败了、客户没买单，你也得到了"真实项目里踩坑的经验"——这本身就是资产。

前提是，你要真的去复盘、去沉淀，而不是白忙一场。

一个免费样，客户没看上，你至少知道了三件事：这类客户的需求长什么样、这套方案在真实数据下哪里会崩、以及"下次这种客户，一开始就更不该免费"。这三条，是拿真金白银的时间换来的教训，比市面上任何一门课都值钱。

失败价值账的本质，是把每一次免费，都当成一次"带薪调研"。这样想，你就不会觉得白干了——因为你总能从里面拿走点东西。

免费验证能做，但别稀里糊涂地做。三本账算清楚，你心里就有数了：什么时候该免费？免费到什么程度？以后怎么靠这个样翻本？

## 按结果收费的"四档进化链"

比免费验证更进阶的，是直接改变收费方式——从"按开发收费"，进化到"按结果收费"。

这是收入结构的质变。卖工时，你的收入天花板是"你能干多少活"；卖结果，你的收入跟客户创造的价值绑定，天花板一下子打开了。

我把这条进化链，分成四档。绝大多数团队，一辈子都停在第一档。

### 第一档，按项目收。

一单一结，做多少算多少。这是最原始的形态，也是最被动的形态。

为什么被动？因为你收完钱，价值就跟你没关系了。你交付完了，客户用得再好、省了再多的钱，一分都不会再分给你。反过来，客户用得不好，锅也是你背。

按项目收的本质是"卖劳动"，不是"卖价值"。它的天花板，就是你的工时单价乘以你的总工时，低得可怜，累得要死。

### 第二档，按使用收。

按调用量、按席位数、按人头收。客户用得越多，你收得越多。

这一档的妙处在于激励对齐：它逼着你去把系统做得让人爱用。因为客户不用，你就没收入。你的利益和"客户真的在用"绑在了一起，而不是绑在"客户签了合同"上。

按使用收，是很多 SaaS 公司的基本盘。它比按项目收进了一步，因为它承认了一个道理：系统不是做出来交差就完了，系统是要被用起来才值钱。

### 第三档，按节省收。

再进一步：和客户约定，这套系统给你省了多少钱，我按比例分。比如省下来的人力成本里，你拿走二十个点。

这一档的关键，是你得有办法"证明"省了多少。说不出具体数字，这钱就没法分。这又回到了第4章讲的经济性检验——账算不清，这一档就玩不转。

按节省收，对客户其实挺友好：他不是"先掏钱买一个不确定的东西"，而是"省了钱再分你一点"。说服成本低，但他要求你，必须把价值的账，摆到台面上来算。

#### **第四档，按结果分成。**

最高的形态。系统直接帮你创造了多少增量收入，我按比例抽成。

这一档，你的利益和客户的利益，完全绑死。他要多赚，你才多赚。所以你会不遗余力地把系统做出真价值，你会比他更关心他的业务。

四种收费方式，本质是四种不同的信任深度和信任结构。你越敢往后退一档（按结果收），说明你越对自己的交付有底气；而你能按结果收费的前提，是你能把价值和账算清楚。

有人会问：那我是不是应该一开始就冲第四档？

不是，千万别。

四档不是随便跳的。它是一个"信任爬坡"的过程，每一档，都在给下一档攒本钱。

你一个没打过交道的客户，上来就要"按结果分成"，人家只会觉得你疯了——他凭什么把自己收入的抽成，分给一个他还没验证过的人？他连你的交付成色都不知道，凭什么把命门交给你？

所以现实的爬法，是这么走的：先用一个"按项目收"的小单，把信任种下去；做得让客户喜欢用、离不开，自然就升级到"按使用收"；用到某个节点，账能算得清清楚楚了，再去谈"按节省收"；最后的"按结果分成"，往往要等你们深度绑定、他把你当半个自己人之后，才谈得下来。

这个顺序不能倒。反过来，如果你的交付撑不起"按结果"的信任，就老实待在低档，把活先干扎实。

**档位不是越高越光荣，而是越往后越靠真本事。**硬往上跳，跳得越高，摔得越惨——因为你拿到的每一分按结果的钱，都得靠真金白银的价值去兑现。兑现不了，客户会连着前面攒下的信任，一起收走。

所以别羡慕那些"敢签对赌"的同行。敢签，是因为人家交付真的硬；你交付没到那份上，硬学这个，学的是自杀。

这里有一个绕不开的标杆案例，我得花点笔墨讲。

Palantir 这家公司，做企业 AI 交付的方式，一直是我反复研究的对象。他们有个"发票争议"的项目，公开材料是这么描述的：帮客户接入 SAP、Salesforce、合同、订舱和一堆旧系统，项目做了一到六个月。做完之后，客户**每年多收回 5000 万美元以上**，客户问询量减少了约 10%。

（这几个数字，是 Palantir 官方公布的口径，我这里照录，金额你可以理解为"以官方口径为准"。）

我想请你看的不是数字本身，而是数字背后的那个判断。Palantir 内部有句话，大意是：**客户掏钱买的，是那些不再漏走的钱，Ontology 这个词本身一分钱不值。**

这句话的分量，你现在品品。

他们那一整套复杂的技术、那个被吹上天的本体论（Ontology），在客户眼里，只是一个"工具"，不值钱。真正让客户掏钱的，是结果：每年多收回来的那五千万美元。客户不是为你的技术付费，是为实实在在多出来的钱付费。

这就是按结果收费的精髓。

Palantir 的公开材料里，还有两条"速度样本"，正好能说明"怎么把账算清楚、把价值证明出来"。

一条是 **Panasonic Energy North America**（松下能源北美）：他们首个用例，约一个月就做出来了，把原本一个 4 小时的流程，压到了 15 分钟。

另一条更耐人寻味。一家**财富 100 强的消费品公司**，内部至少跑着 7 套 ERP。换成一般人，看到 7 套 ERP，第一反应是"我要先统一它们"，于是掉进一个无底洞。但 Palantir 的团队没上这个当。他们只盯了一个决定：**在物料清单里，有没有更便宜、又不影响生产的替代原料。**就这一个决定，5 天跑出了一个能用的东西，一周之内，采购人员就已经开始用了。

这两条样本背后，是同一门手艺：**盯住"一个能被改变的决定"，比"接全公司系统"更值钱。**

这也和 6.3 开头呼应了——扩大收入的根基，仍然是那一套“找对问题、做出价值、证明结果”的方法。收费方式再高级，也高级不过“你帮客户多赚了多少钱”这个最朴素的真相。

## 存量深耕：横、纵、纵深

讲完了怎么收费，再讲一个更实在的问题：增量从哪来？

大多数人一提起“扩大收入”，就想着“多拉新客户”。但我得说：扩大收入，最被低估的路径，其实是**深耕老客户**。

因为老客户已经有信任、有数据、有磨合，扩大的成本远低于拉新。拉一个新客户，你要重新进场、重新尽调、重新建立信任、重新过一遍法务安全审查——每一步都是钱。而老客户那里，这些早就做完了，你只需要在已有的地基上加一层。

深耕有三个方向，我一个个说。

### 横向，新场景。

你在一个部门做成了，往隔壁部门复制。比如先做了客服的问答，再往销售、市场、人力扩展。

横向的杠杆是“可复制”：你解决的，是一个足够普适的问题，换一个部门、换一批人，痛点还在，方案稍微改改就能接着用。你前面攒下的实施经验、组件、模板，让第二个场景的交付速度，比第一个快上一大截。

### 纵向，深链条。

你在一个环节做成了，往上下游延伸。比如先做了流失预警，再往获客、转化、留存前移，把整条客户生命周期都吃下来。

纵向的杠杆是“深扎”：你在这个客户的一条业务链上，越扎越深，到后面，这条链上的每一个环节都离不开你。那时候，客户换掉你的成本，高到他根本不敢想。

### 纵深，多部门。

从一个业务点，渗透进整个组织的多个层级、多个业务板块。你做的不再是“某个部门的一个工具”，而是“**全公司都在用的一套东西**”。

纵深靠的是"组织信任"：你在 A 部门做成了，B 部门的负责人是 A 部门负责人的老熟人，一句话就把你推荐过去了。这种内部口碑的传播，比任何销售话术都管用。

三条路，说得再漂亮，落到根上是一句：

**横向靠"可复制"的普适价值，纵向靠"深扎"的场景壁垒，纵深靠"深耕"的组织信任。**

三条路，殊途同归，都指向同一个目标：让一个客户的价值，从"一单"，变成"一窟"。

说到存量深耕，我特别想提一下那些"代运营出身"的 FDE。

这个观察，可能跟你想象的不一樣。真正靠 FDE 这行活得滋润的，很多不是算法工程师，也不是架构师，而是过去做企业代运营、做获客、做出海陪跑的那批人。

他们没有正规的技术团队，这反而成了优势：天然避开了深度定制那种无底洞的承诺。他们极度懂业务场景，靠 Codex、靠 Agent，自己 vibe coding，把智能体一个一个搓出来。卖的东西也很轻——紧贴场景的轻量智能体，加上落地的陪跑服务。

他们最牛的地方，恰恰在"存量深耕"上：手握多年积累的存量客户和信任关系，回头跟老客户讲一句"我能帮你省两个人、提效三成"，小样一摆，单子就签了。深度绑定着 WorkBuddy、阿里、360 这些平台生态，厂商还源源不断地给导流订单。

这批人看不懂 NR R 的公式，不懂什么四档进化链。但他们凭本能，把"深耕老客户"这件事做到了极致——因为他们清楚，一个信了你好几年的老客户，比十个陌生询盘的性价比都高。

方法论是帮你想清楚"为什么"，本能是帮你省掉"犹豫"。真正的高手，两样都有。

## 6.4 案例：Anthropic 与 FIS 的扩张路径

讲完了方法论，我拿一个海外的扩张案例，把"从一个场景切入、再横向多点开花"这条路，走一遍给你看。

这个案例是 Anthropic 和 FIS 的合作。FIS 你可能不熟，说一句你就懂了：它是全球金融科技里的巨无霸，处于"成千上万家金融机构的钱是怎么流动的"这个中心位置——全球的银行、支付机构，很多底层系统跑的就是 FIS 的东西。

FIS 和 Anthropic 的合作，公开口径里有一句很关键的话，是 FIS 的高管说的，我翻译个大意：

"当我们开始构建 AI 智能体的时候，我们知道自己需要一个能信任的提供商。Anthropic 是明确的选择。"

下面这句话才是重点，我尽量照原意说：

"我们正在一起构建一个能把反洗钱调查，从'数天'压缩到'数分钟'的智能体，接下来还有信用决策、欺诈预防、存款留存这些智能体会跟进。FIS 的客户不需要自己搭这套基础设施，它已经在这里了。"

（这段是官方发布的口径，数字和表述请以官方为准，但方向和顺序是明确的。）

我把这个案例拆开，里面藏着的，正是 FDE 扩大收入最标准的打法节奏。

**第一步，从单一场景切入，而且是一个"结果可量化"的场景。**

FIS 选的第一个场景，是反洗钱调查（AML investigations）。这个场景选得极其讲究：它足够疼（金融机构做反洗钱调查，动辄耗上几天，人力成本极高）、结果可量化（从几天压到几分钟，省了多少人力，一算就出来）、而且风险可控（AML 是强监管场景，做砸了是大事，但正因为是强监管，也说明它够重要、预算够足）。

这跟 Palantir 那家财富 100 强消费品公司"只盯替代原料一个决定"，是同一套逻辑：第一个用例，一定是一个能用一句话说清"以前多久、现在多久、省了多少"的场景。

**第二步，用第一个场景的胜利，打开横向的信任。**

FIS 明确说了，AML 只是第一个，后面跟着的是信用决策、欺诈预防、存款留存。

注意这个顺序里的策略：它不是一开始就铺开做四个，是先做一个，做成，然后才谈后面的。第一个场景的成功，把"这家提供商值不值得信"这个最贵的问题，一次性回答了。后面三个场景，都是在消费第一个场景攒下的信用。

**第三步，把"你的能力"变成"客户生态里的基础设施"。**

最后那句话最关键：FIS 的客户不需要自己搭这套基础设施，它已经在这里了。

这是什么意思？意思是，Anthropic 的价值，不是卖给 FIS 一个工具，而是借由 FIS，嵌进了 FIS 背后那成千上万家金融机构的业务里。FIS 的客户，天然就用上了这套能力，而不需要任何一家去单独谈判、单独部署。

这就是纵深扩张的终极形态：你不再是一个“供应商”，而是变成了客户业务底座里的一层。客户不是“买你的产品”，而是“生活在你提供的底层能力之上”。

这个案例对我们的启发，有三点。

一是**场景的选择，决定了扩张的难易**。选一个有量、可量化、强监管的场景切入，是为了让它天然具备“向邻居辐射”的能力。A 场景证明了你，B、C、D 场景就都是顺水推舟。

二是**扩张要一步步来，让每一仗都为下一仗铺路**。FIS 不是四个智能体同时立项，是一个先落地，四个在排队。每一仗打赢，都在为下一仗降低成交成本。

三是**最高的扩张，是把自己变成对方生态的地基**，而不是对方采购清单上的一项。当客户说“我的客户不需要单独买你，因为你已经在这里了”，你的收入结构，就彻底从“卖一次”变成了“跟着客户的整个盘子在长大”。

当然，达到 FIS 这个量级，靠的是平台、产品和生态的合力，不是单个 FDE 在客户现场能复现的。但这条路的方向，是一模一样的：先打一个可量化的点，再横向复制，最后把自己焊进客户的业务底座。你在本土做一个几十万的项目，用的，也是同一套节奏。

## 6.5 案例：一个差点流失的客户，是怎么救回来的

道理讲了这么多，回到一个更贴近日常的场景：一个客户，其实已经半只脚踏出门口了，你怎么把他拉回来？

我不想编一个戏剧化的“力挽狂澜”故事。因为我见的真实救回，从来不是靠最后一刻的那一通电话，也不是靠临门一脚的降价，而是靠平时积累下来的健康分预警。

下面这个案例，是我按自己经历过的几次“预警—干预—救回”的真实过程，合并抽象出来的。情节是拼接的，但每一个动作，我都在真实的项目里干过。

先看背景。手上有一个客户，合作了一年半，做的是供应链对账的自动化。最初那段蜜月期，效果很好，健康分一度打到 85 以上。

然后，健康分在三个月里，从 85 一路滑到了 62。

滑落的原因，拆开看是三个。

**使用频次在掉。** 原来每天用这套系统的业务骨干，最近两周只打开了两三次。后台一看，活跃用户从顶峰的四十多人，掉到了十几个。

**内部支持者调走了。** 那个一路力推项目的中层——就是当初顶着别的部门反对、把 AI 引进来的人——上个月升职去了别的部门。接手的人，是个一问三不知的新人。

**价值感知在模糊。** 新对接人在一次闲聊里，私下说了一句："这个系统到底有什么用，我也想问。"

三件事叠在一起，健康分直接击穿了 70 那根线。

如果你没有健康分这个东西，你会怎么样？

你会等到续约前一个月，才从别人嘴里听说"客户好像冷淡了"。那时候你慌慌张张冲过去，又是打折又是加功能，试图亡羊补牢——大概率，来不及。客户的心已经凉了，凉了的心，不是打折能焐热的。

但如果你盯着分数，看到它跌破 70 的那一刻，你就有了三个月的时间窗口。三个月，足够你做很多事了。

介入的动作，不是去"谈判续约"。谁也谈不动的，因为问题不在续约本身，在于价值已经被掏空了。你要做的，是回到现场，把当初丢掉的三个东西，一个一个捡回来。

**捡使用。**

你先回到现场，找到那几个还在用、但开始用得少的老员工，一个一个问清楚："为什么不怎么用了？"

不要指望他们主动说，你要去挖。答案往往细小得像根针：某个新需求的接口没接上；某个报错一直没修，绕了三个月；换了个新电脑，装不上了，不好意思问。

这三个小疙瘩，个个都好解决，但放在那儿，就能一点点磨掉使用意愿。你把它们扫干净，使用频次，自己就回来了。

**捡支持者。**

支持者走了，是捡不回来的。但你得重新物色一个。

两条路。要么，跟新接手的那个中层建立关系，帮他搞懂这套系统，让他在他的新位置上，也意识到"这东西是我这边的资产，值得维护"。

要么，往上找。找到那个比原支持者更高一层、真正从这个项目里受益的老板，重新把"这件事为什么值得做"的共识建立起来。老板一出面，很多阻力自动消解。

这一步最慢，也最关键。没有一个新的支持者，你做的所有修复，都可能在下一次人事变动里，再次归零。

### 捡价值。

这一条，是根。

你和新对接人，重新算一遍账：这套系统上线以来，到底省了多少人、多了多少单、少出了多少错。

注意，不是你去"宣讲"价值，是带着他一起，把账算出来，算到他自己能复述出来的程度。能把价值用一句话说清的人，才可能成为你下一个支持者。

这一步，我那次是这么做的。我没带 PPT 去，就带了一张 A4 纸，上面列了三行数：上一年，光对账这一个环节，他们手工核对的工时是多少；漏单、错单造成的损失是多少；系统上线之后，这两项分别降到了多少。两个数一减，省下的钱就摆在眼前。

新对接人看完那张纸，愣了几秒，说了一句："这么说，这系统去年就省了几个人的工资？"

我说，对。他接着说："那我得跟领导说说，这个还得续，而且别的环节也可以试试。"

你看，不是我讲明白了，是他自己算明白了。这句话，比我说一百句"我们系统很好"都值钱。

三个月后，健康分回到了 80。

客户续约了。不但续了，还加了一个新场景——因为那个重新算出来的账，让新对接人看清了这套东西的底子，他主动提出，想把对账自动化，延伸到下游的另一个环节。

这个故事平淡无奇。没有绝地反击，没有生死一瞬。

但恰恰是它平淡，才真实。

守护续约，靠的不是什么惊天动地的绝招，而是**盯着健康分，在客户自己都还没意识到要流失的时候，先一步把他拉回来**。预警那一下的价值，比你事后做的十件补救，都大。

## 6.6 扩大收入的另一面：别让客户觉得被绑定

扩大收入，我还想泼一盆冷水，说一个容易被忽略的反面。

你在一个客户身上越做越大、越绑越深，客户反而可能会越来越警惕——他会琢磨：我是不是被你"绑架"了？

这不是杞人忧天。

有机构预测（我记得是高德纳的一个预测），到 2028 年，将有相当比例的企业——大概七成——会因为供应商成本过高、加上内部技能不足，而放弃由外部供应商主导的 AI 方案。

换句话说，客户被绑得越死，反弹就可能越猛。你越是不可替代，他就越觉得"成本失控、命门被你捏着"，最后反而下定决心，走人自建。

这是个反直觉的地方：扩张做得越成功，反而越要警惕"被依赖"演变成"被反噬"。

所以，健康的扩大，不是把客户绑死在你身上，而是让客户在"离不开你的价值"的同时，仍然觉得"我有选择权、我没被人拿捏"。

具体怎么做？三条，都很朴素。

**不要垄断数据。**

数据是客户的，不是你的。你要让客户随时能拿回自己的数据，哪怕他有一天不用你了，数据也要能干干净净地带走。

这既是职业道德，也是一种聪明的长期主义。你越不设套，客户越信任你，越愿意长期跟你合作。反过来，你在数据上留一手、卡一道，客户迟早会察觉，察觉的那天，就是信任崩塌的那天。

**不要制造黑箱。**

让客户看得懂你做了什么、价值从哪来。一个完全不可解释的黑箱系统，客户心里永远是虚的。虚到一定程度，他就会想摆脱你——不是因为你不好，是因为他不知道你在里面干了什么，这种"未知"，本身就是恐惧。

### **不要垄断唯一的通道。**

你的价值，应该建立在"结果更好"上，而不是"客户别无选择"上。

因为靠"别无选择"维持的关系，本质是胁迫。胁迫换来的忠诚是假的——一旦出现替代品，哪怕贵一点、差一点，客户也会跑。只有"结果更好"换来的关系，才是真的、能扛得住竞争的。

守住续约和扩大收入，说到底，是一次长期主义的修行。

你既要让客户持续觉得"值"，又要让他一直觉得"自由"。

"值"，靠的是持续创造价值——用健康分盯着，用四档进化链把价值兑现成收入，用横纵纵深把一单做成一窟。

"自由"，靠的是克制——不垄断数据、不做黑箱、不卡唯一通道，让客户随时都有说"不"的底气，而他有了这个底气，反而不会轻易说"不"。

这两者平衡好了，你的续约率、NRR、扩大收入，才会是可持续地往上的。失衡了，要么客户走，要么客户反噬，都是白忙。

## **章末工具箱**

### **工具一：客户健康分评估表**

给每个客户定期打分（0-100），四大类信号。跌破 70 分，触发预警，主动介入。

#### **使用信号（30 分）**

[ ] 日活 / 周活是稳定上升，还是持续下滑？（10 分）

[ ] 使用深度是在加深，还是变浅？用户从"每天用"退到"想起来才用"了吗？（10 分）

[ ] 有没有出现"核心用户弃用"的红旗？原来天天用的骨干，是不是半个月不登录了？（10 分）

### 组织信号（25 分）

☐ 内部支持者是否还在位？最近有没有升职、调岗、跳槽？（10 分）

☐ 关键对接人换了吗？新对接人对项目是什么态度？（8 分）

☐ 决策层对这个项目还关注吗？还是已经无人过问？（7 分）

### 价值信号（25 分）

☐ 客户能否脱离 PPT，说清"省了多少钱、解决了什么麻烦"？（13 分）

☐ 有没有新的价值点，被持续创造出来？还是价值已经一年没更新？（7 分）

☐ 客户最近报的错、提的问题，得到了及时响应吗？（5 分）

### 财务信号（20 分）

☐ 客户自身经营是否稳定？行业是在扩张还是收缩？（10 分）

☐ 回款是否准时？有没有从 30 天拖到 60、90 天的苗头？（10 分）

### 打分说明

80 分以上：健康，按正常节奏维护即可。

70-80 分：观察区，找出失分项，主动补漏。

70 分以下：预警区，立刻回到现场，按"捡使用、捡支持者、捡价值"三件事逐一排查干预。

## 工具二：续约风险排查清单

续约前三个月，逐条过一遍。任何一条回答为"是"，都当成一个待处理的红色信号。

☐ 合同到期日是几号？我提前了多久开始准备续约？

☐ 客户内部支持者是谁？他最近有没有流动的迹象？

☐ 最近三个月，系统的使用数据，是升还是降？掉得有多快？

☐ 客户最近一次主动说"这系统有用"，是什么时候？

[ ] 竞争对手有没有在接触这个客户？有没有人报过更低的价？

[ ] 客户能在一分钟内，说清这套系统帮他省了多少钱吗？

[ ] 客户自己的经营状况、付款能力，最近有没有恶化？回款变慢了吗？

[ ] 我手上，还有没有客户自主可控的数据出口？（防"绑死"反弹）

[ ] 如果客户明天说"不续"，我知道真正的原因是什么吗？

### 三个自问（每次续约前问自己）

**这个客户，我盯的是合同到期日，还是健康分？** 只盯到期日的，是赌运气；盯着健康分的，是把续约攥在自己手里。

**我是在谈"价格"，还是在补"价值"？** 客户说贵，往往是价值没兑现的借口。先补价值，再谈价格。

**我帮客户"变值了"，还是让客户"变怕了"？** 前者带来复购，后者埋下反噬。健康的扩大，永远让客户觉得值，也让他觉得自由。

守住了续约、扩大了收入，你会越来越忙，接的单也越来越多。这时，你会撞上 FDE 模式最根本的困境：你是要靠不断接新项目来"堆人"，还是找到一条能"复制"的路？下一章，我们讲规模化复制。

<div style="page-break-after: always"></div>

## 第 7 章 规模化复制：从定制项目到平台产品

定制项目不是终点。沉淀下来的知识、组件、产品，才是真正的资产。

做到这一章，你已经走过了进场、定义、部署、激活、续费，能独立把一个企业 AI 项目做成、做活了。

先别急着高兴。这时候，一个大问题会浮出水面，也是绝大多数 FDE 和交付团队的死穴：

**项目做成了，然后呢？**

我见过太多人卡在这个问题上，而且卡得几乎一模一样。

去年年底，有个同行约我吃饭，桌上跟我算了一笔账。他这一年做了七个企业项目，单子都不小，团队也扩到了十几个人。听着挺风光。可他把账本一摊开，眉头就皱起来了：今年的收入，减去人天、差旅、外包、返工，剩下的利润薄得可怜。更要命的是，明年要还想赚这么多，他还得再接七个一模一样体量的项目，一天都不能停。

他说了一句让我记到现在的话："我感觉我不是在开公司，我是在做钟点工。还是一个带了十几个伙计的钟点工。"

这就是"卖时间"的真相。

如果你每做一单，都是从零开始、重新定制、重新踩坑，那你本质上就是在卖自己的时间。你的收入，和你投入的人天严格线性绑定，你要想多赚一倍的钱，就得再找一倍的活儿。这是一种看似忙碌、实则天花板极低的模式——而且这模式还有个更阴险的地方：它不止有天花板，它还会塌。

问出"然后呢"的人，面前大概就三条路。

第一条，认了，永远做项目公司，赚点辛苦钱，哪天遇到一个暴雷项目，赔进去半年的积累。第二条，把自己挂靠到一个大平台下，接它的派单，从此看平台脸色过日子。第三条，就是这一章要说的：把一个个项目里攒下的东西，提炼、抽离、固化，让第二十一个项目的成本，只有第一个项目的十分之一。

走第一条的人最多，走第二条的人最舒服，走第三条的人最少。但只有走第三条的人，能从"匠人"变成"经营者"。

这一章，讲的就是怎么跳出这个模式：从"一个项目一个项目地做"，进化到"把项目沉淀成资产，让资产去复制"。

## 7.1 定制化陷阱与三级复制杠杆

先说清楚问题所在，我把它叫作"定制化陷阱"。

做企业服务的人，很容易陷入一种惯性：每个客户的需求都不一样，于是每个项目都从头定制。客户的特殊性，成了一种无法抗拒的诱惑和借口——"这个客户特殊在……"，于是你又一次放弃了标准化的努力。

这句话我太熟了。每次我提醒一个团队“你们这个是不是可以抽成组件复用”，对面十有八九会回一句：“这个客户跟上一个不一样，他家的流程是……”然后叽里呱啦讲一堆特殊之处。

讲得对不对？多半是对的。客户确实特殊。但问题是——你为这份特殊付出的代价，从来没人在账上算过。

结果就是：你做的永远是定制，永远无法复用，永远在重复劳动。你的团队越大，越要堆人，越堆人越忙，越忙越接不了几个项目。忙到一定程度，你会发现自己掉进了一个死循环：不接单没钱活着，接了单没命干。

为什么道理这么明白，人还是陷进去？我观察下来，就一个字：忙。项目一个压着一个，交期紧得喘不过气，客户天天在群里催。在这种节奏里，“做完这个就复盘”永远是下一句，而下一个项目永远来得比复盘快。

所以定制化陷阱，本质上是一个时间陷阱——它用无穷无尽的当下，偷走了你所有用来沉淀的“以后”。你想跳出来，光靠自觉没用，得靠制度，把沉淀这件事钉进流程里，让它像交付一样不可跳过。这一层，到 7.3 再细说。

这里有个残酷的教训，我见过真实发生。说的是“总包”这种模式。

总包的打法，听起来特别聪明：自己不写代码，自己负责商务谈判、资源整合，把开发和数据清洗分包给下游的小团队，中间抽一层管理费，吃差价。单子接得越多，差价吃得越多，看着像是躺着放大。

漂亮。但它的文本里藏着一个致命的漏洞——**活可以分包，责任分不出去。**

为什么分不出去？因为甲方认的是你的章。合同是你签的，钱打到你的对公账户，验收节点挂的是你的名字。下游团队掉链子、撂挑子、跑路，在甲方眼里，那就是“你出了问题”。他不会去找那个你分包出去的小作坊，他只认你。

我见过有人真就栽在这上面。他用这套总包的逻辑，一口气拉来了四五个项目，每个几十万，全分发给下游。账面上看，这一年的流水漂亮极了。然后就是教科书式的失控：三个项目同时延期，两个下游团队直接撂挑子跑路，剩下一堆烂摊子。救火、赔付、安抚甲方、重新找人填坑——所有的压力，一瞬间全砸回他一个人头上。

钱还没赚到手多少，人先垮了。

这个教训指向一条铁律，我建议你把它刻在脑子里：

**你的业务规模上限，等于你最坏情况下能同时兜底的能力。**

堆人、分包、接更多单，这些动作都叫"放大"，不叫"扩展"。放大是线性的，堆到一定程度还会失控；真正的扩展，走的是另一条路。

那么，正确的放大方式是什么？

答案是**复制**。复制你的知识、你的组件、你的产品——让每一份沉淀，都能在下一个客户身上，重复产生价值。你今天花一个下午写出来的东西，明天、明年、给十家客户，都能接着用。这才是把"卖时间"变成"卖资产"。

我把它概括成**三级复制杠杆**，从低到高，价值依次放大。

## 第一级：知识复制

最低一级的复制，是复制"知识"。

上一个项目里，你总结出了一套踩坑经验、一份流程、一个 check 清单。下一个项目，你不用重新摸索，直接套用。这就是知识复制。

听着不起眼，但它是一切复制的起点。我给你举两个最常见的知识复制载体。

一个是复盘文档。项目做完，趁热写一篇：这个客户的数据长什么样、他家接口有哪些坑、哪个关键人最好提前搞定、验收怎么过。下一篇，你拿到同类客户，翻出来照着走，能避开一半的雷。

另一个是清单和话术。比如进场尽调的那张"五张地图"问卷，比如给客户讲"为什么 AI 不能一次性做完"的那套说辞。这些都不是什么高科技，但它们是踩了坑之后凝出来的，值钱就值钱在——它们已经替你踩过一遍了。

那怎么写出一篇真正能复用的复盘？我踩出来的经验是三条。第一条，不写情绪，只写事实和结论。"客户太难缠了"是情绪，"客户对验收口径的理解和合同不一致，应该在签约时用一份确认单钉死"是事实。第二条，不写大道理，只写"下次遇到该怎么做"的具体动作。第三条，不写孤立的一次性经验，写清楚它依赖什么前提——同样一个技巧，在制造业管用，在金融业可能失效，前提不写清，抄的人就要翻车。

很多复盘废掉，不是因为深度不够，是因为写成了流水账，某月某日遇到了某问题、大家讨论了一下午。这种记录，别人读不进去，自己回头也读不懂。真正能复用的知识，长得像一条规则：当什么条件出现，就做什么动作。规则能被复制，感受不能。

但知识复制的杠杆很小。原因很残酷——知识藏在人的脑子里、藏在文档里，换一个人、换一个团队，往往就传不下去了。文档躺在那里没人看，是知识最常见的死法。这也是为什么，光靠知识复制，团队永远长不大：它复制的是“经验”，而经验是不可见、不可验证、不可交接的。

所以知识复制，只能当起点，不能当终点。它解决的是“下一个我能不能少踩坑”，解决不了“下一个项目能不能少干活”。

## 第二级：组件复制

高一级的复制，是复制“组件”。

什么叫组件？就是你把某个项目里反复出现的一个功能——比如“文档解析”“权限隔离”“日志审计”“对账比对”——抽出来，做成一个独立、可拆装、能复用的模块。下一个项目要用，直接拿来改一改配置就能用，不用重新写。

这里的差别，不是量的差别，是质的差别。知识复制的产出是“一份文档”，组件复制的产出是“一段已经跑通过、验证过的代码或配置”。文档要人来读懂、人来执行；组件直接就能跑。你把“经验”，变成了“已经做出来的东西”。

一个成熟的组件，能让你在下一个项目里，省下成倍的开发时间。我见过最典型的一个：有团队给第一个客户做“PDF/扫描件解析”这套东西，从解析、纠偏、切块到结构化，硬啃了一个多月，坑踩了个遍。做完之后他们学乖了，把这套抽成了组件。第二个客户再要类似功能，三天配完。第三个客户，半天。

哪些东西最值得抽成组件？有个很实用的判断角度：看它是不是“客户不感知、但每次都要做”的底层活。文档解析、权限隔离、日志审计、消息推送、数据脱敏——这些活儿，客户根本看不见，但每个项目都得来一遍，而且特别耗工时。这种“暗功夫”，是组件化的第一优先级。反过来，那些客户天天盯着看的业务界面，往往各家都不一样，硬抽反而别扭。

那什么时候该把一个东西抽成组件？我给你一个很土、但有用的判断标准：**当你第三次做同一件事，就停下来，把它抽成组件。**第一次做，先做通，别想着抽象；第二次做，顺手把坑记下来；第三次你还得重写，那就是在浪费生命了，必须抽。

很多人卡在这一步，卡在一个心理障碍上：总觉得“下一个客户不一样，抽了也没用”。其实不是没用到，是你还没遇到第二个、第三个客户。抽组件的时机，永远在你“预见”到复用的那一刻，而不是等复用已经发生的第二天。

多说一句组件和产品的分界线，因为这俩最容易混。组件是“给你的下一个项目用”的，产品是“给不特定的很多客户用”的。区别不在代码量，在谁买单、谁服务。组件是你私有的弹药库，产品是要摆上货架、接受陌生客户检验的。很多团队一辈子满足于攒组件——反正自己人用着顺手，省了工时，够了。这个选择没有错。但你要清楚，组件攒得再多，它的价值也锁死在你自己的项目数里；只有产品，才可能跳出你的项目数，去吃你没接触过的客户。攒组件是节流，做产品才是开源。

### 第三级：产品复制

最高一级的复制，是复制“产品”。

当你的组件、流程、方法论积累到一定程度，你就可能发现：很多客户的需求，其实有巨大的共性。把这些共性抽象成一个标准化的产品，你就不再是“给每个客户定制”，而是“卖一个标准产品 + 少量现场适配”。

产品复制，杠杆最大。因为它可以边际成本趋近于零地卖给无数客户，而不用再靠堆人。你做第一份的时候成本是十万，做到第一百份的时候，成本可能只有几千——甚至趋近于零。这才是“业务规模上限”能被真正抬高的方式：你的上限不再由“多少人能兜底”决定，而由“这个产品能吃多少客户”决定。

但我要泼一盆冷水：产品化不是升级打怪，不是组件攒够了就自动变成产品。它是另一门完全不同的手艺，要求你从“交付思维”跳出来，换成“经营思维”。中间这道坎，很多人一辈子没跨过去，一辈子在“组件”和“产品”之间反复横跳——东西攒了一堆，就是成不了产品。

最后说一句心态。这三级的顺序，不是让你做完知识就冲组件、做完组件就冲产品。很多很好的团队，就停在组件这一级，日子过得也很好。三级杠杆的真正含义是：你要知道自己在哪一级，别自欺。正在做定制，就说自己在做定制，别骗股东说自己在做产品；停在组件，就享受组件的省心，别眼红产品的天花板。位置清楚，才谈得上往哪走。

从知识到组件到产品，是一条清晰的进化路径。绝大多数团队，死在了"知识复制"和"组件复制"之间的某个地方——永远在消耗，从不沉淀。他们不是不努力，恰恰是太忙了，忙到腾不出一个下午，去写那篇复盘、去抽那个组件。

而这个下午你不肯花，你就得花几十个下午，在下一个客户身上，把同样的坑再踩一遍。

如果你现在还是一个纯定制的团队，别想着一步跳到产品。就做一件事：这个月，挑一个你手上重复了两次以上的动作，把它抽出来。抽成一个组件也好，写成一页能给别人看的沉淀也好。先让"沉淀"这个动作发生。复制是一条慢慢走的路，可第一步迈不出去，后面的三级杠杆都跟你没关系。

## 7.2 产品化四问：这个能力，值不值得做成产品？

那么，怎么判断你手里的某个能力、某个组件，值不值得升级成产品？

我知道，很多人是拍脑袋的：客户多了，需求像了，感觉"是不是该做个产品了"，就一头扎进去搭平台。结果做出来，卖不出去，又灰溜溜退回定制。这种例子我见得太多了。

所以我用四个问题来筛。而且这四个问题，有严格的顺序：先看有没有共性，再算泛化成本，再验好不好用，最后看现场。很多人一上来就纠结"现场怎么办"，却连最基本的共性都没验证——顺序错了，后面全是白忙。

这四个问题，没有一个是"你的技术够不够强"——因为产品化失败，几乎从来不败在技术上。

### 第一问：有没有共性？

这个能力，是只有这一家客户用得上，还是这个行业里成百上千家客户都有同样的需求？共性是产品化的前提。没有共性，就只能继续当定制做。

怎么判断有没有共性？三个土办法。

第一，看同行。你做完这个客户，能不能再报出三家同类型、同规模、同痛点的企业？报不出来，多半是"这一家的问题"，不是"这一行的问题"。

第二，看问询。你有没有被不止一个客户，问过几乎相同的话？"你们能不能也给我们做个对账的""你们那个知识库能不能卖我们一套"。客户主动来问，是共性最硬的信号，比你在屋里推演的十页 PPT 都管用。

第三，看复盘的频率。你复盘里反复跳出来的那个痛点，如果出现了三次以上，那它就是行业性的，不是个体性的。

共性不够怎么办？不怎么办，继续当定制做，别硬产品化。定制不丢人，硬把定制包装成产品、结果卖不动，才丢人。

我讲一个反面典型。有个朋友，花了大半年，把他给第一个客户做的某个管理工具，硬做成了一个"产品"，还上了架。结果一年卖出去两套，还都是老客户介绍、半卖半送。复盘下来，他那个"共性"是假的——他以为"我客户有的需求，同行业都有"，其实那只是他家客户的组织结构特殊，造成的一个偶发需求，整个行业里别人根本不这么干。

所以判断共性，最怕的就是拿一个样本脑补出整个行业。有没有共性，一定要走出办公室去问，去问第二家、第三家、第四家，而不是坐在屋里想。

## 第二问：泛化成本多高？

把它从"这一个客户的特殊实现"抽象成"通用产品"，要花多少力气？

这一问，技术出身的人最容易栽。他们会本能地低估泛化成本——"这代码我写的，抽象一下就完了吗"。真抽象起来才发现，那个实现里到处写死了这个客户的字段名、账号、组织架构、业务规则。要把它抠干净、变成可配置，工作量可能接近重写一遍。

泛化成本至少包括四块：代码重构、配置化改造（把写死的东西变成可配置项）、说明文档（让别人不看源码也能用）、以及最难估的——将来每个新客户的适配成本。前三块是一次性的，第四块是持续的，而且很容易被漏掉。

判断方法很简单：估一下泛化的成本，再估一下它未来三年能帮你省下或赚到的钱。泛化的成本，比它将来省下的钱还高，那就不是产品化的好时机。这个账不难算，难的是愿意去算。

除了明面上的重构，泛化还有一笔隐形税，很多人到后来才察觉。产品化之后，你要写文档、要做培训材料、要接新客户的支持工单、要修各种你没遇到过的新环境的 bug。这些成本，在"要不要产品化"的那个下午，你是算不到的，但它们会跟着产品的销量一起涨。所以看

泛化成本，别只算"抽出来"那一下，要算"养着它"的长期账。很多做出来卖不动的产品，不是抽失败了，是养不起。

### 第三问：好不好用？

做成产品之后，客户能不能开箱即用，而不是还得你手把手教上三个月？

这一问，戳的是产品化的本质。产品化的价值，恰恰在于把"服务"变成"自助"。你卖一个产品，客户自己部署、自己配置、自己用起来，你才是真正地把人从交付里抽出来了。如果做成产品之后，你还是得派两个人驻场三个月，那它就不是真产品，只是换了包装的定制。

怎么判断好不好用？看一个指标：**一个新客户，在你完全不介入的情况下，能不能自己把它跑起来、用出第一个结果。**能，是真产品；不能，是伪产品。

有人会辩解："我们客户的 IT 水平不行，必须得有人带。"听我一句——客户 IT 水平差，恰恰是你产品没做好的证明，不是你不做产品的理由。真正好的产品，是把复杂藏在自己肚子里，把简单露给客户。

我一般用两条线去量"开箱即用"。一条是时间线：一个新客户从拿到产品，到自己跑出第一个有效结果，中间要多久。超过一周还搞不定，说明你的门槛太陡。另一条是求助线：这过程中，他要找你们多少次。三天两头来问"这个按钮什么意思""这个配置填什么"，说明你把复杂度漏给了用户。两条线都短，才叫真的好用。

### 第四问：还需不需要现场？

产品交付后，还需要多少"现场空间"？

这一问，很多人会忽略。企业 AI 有个特殊的地方：它不是买个软件装上就完事，它嵌在企业的流程、权限、组织里，天生的现场感很重。有些需求，比如陪跑、培训、帮员工上手，缺了现场的人，产品再好也落不了地。

所以这一问，不是在问"要不要现场"，而是在问"需要多少现场、能不能把现场也标准化"。答案决定了你的形态：现场极少的，适合做纯软件产品；现场很重的，适合做"产品 + 服务"的混合形态，把服务也打包成可复制的。

这里有个很有说服力的例子，我亲眼见过。

有人办"三天、两万元、一场一百人"的 AI 运营工具培训，供不应求。他交付的是什么呢？不是底层代码，是"把现成 AI 工具深度嵌入运营业务"的方法论，加一套带教体系。

你看，这种形态，边际成本极低——同样是那套课件、那套带教流程，讲给一百人和讲给两百人，成本几乎一样。但它又离不开"现场"：老师本人得在场，手把手带着做。

这说明一个反直觉、但对 FDE 特别重要的事：**标准化未必要是一段代码，服务也可以是产品**。一场培训、一套陪跑流程、一套内训体系，只要它能被标准化、被反复售卖，它就是产品，只不过它的载体是人，不是软件。

很多技术出身的 FDE，死活想不明白这一点，非要把所有东西都做成软件。其实大可不必。判断你是不是在"产品化"，不看交付物是代码是人还是纸，看的是它能不能脱离"一次性投入"、被反复售卖。

那"现场"怎么标准化？我有一个很朴素的做法：把服务里的每一类动作，都写成"剧本"。培训，就写一套标准课件加一个固定的话术库，谁上都能讲；陪跑，就写一份分阶段的《陪跑 SOP》，第几周做什么、交付什么、怎么验收。然后把这些人天密集环节，能教的教给客户自己，不能教的才留人。

留人不是问题。问题是同样的现场服务，你做了十遍，有没有沉淀成第十一遍能直接抄的剧本。没有，你就是在用今天的现场，买明天的负债。

这四个问题，本质是在逼你想清楚一件事：**你沉淀下来的东西，到底是"资产"，还是"负债"**。资产，是能复利放大、越攒越值钱的；负债，是每次都要重新维护、越攒越累赘的。产品化，就是把资产从负债里分离出来的过程。

最后补一句：这四个问题，不是你在某一天问一次就一劳永逸的。市场和客户在变，去年没共性的，今年可能就有了共性。所以这套问题，最好每半年，把你手上攒的那些组件和沉淀过一遍。有的能力，正好升级；有的，继续压着；还有的，赶紧砍掉——因为它已经不是资产，是纯负债了。定期清理，跟定期复盘一样，都是把组织往轻了养。

### 7.3 从定制到平台：用数据验证，用复盘沉淀

观念上认同了"产品化"，还不够。认同是一秒钟的事，做是很多年的事。

真要做，靠两样东西：数据，和复盘。一个是往外看，验证自己是不是真的在变；一个是往内看，把项目变成组织自己的肌肉。

## 用数据验证模式

怎么知道你正在从"卖时间"走向"复制"？靠感觉不行。感觉最容易骗人——你忙了一年，新签了很多项目，很容易自我感动，"我在扩张"。可扩张和复制，是两个东西。扩张是收入多了，复制是"同样的收入，成本变少了"。

有两个硬指标值得盯，我建议你每个季度看一次。

### 第一，交付毛利率。

一个纯定制的项目，毛利率往往很低。为什么？因为它的绝大部分成本，都花在了不可复用的人天上。每个客户都是新的，每个需求都要重写，你收了一百万，可能九十万都喂给了工资和外包。

当你开始组件化、产品化，同样的收入，成本会显著下降。组件是抄自己的作业，产品是卖同一份东西。第三次交付一个"对账组件"，你的成本可能是第一次的十分之一。毛利率就是这么上来的。

所以，毛利率的趋势，是你"复制能力"最直观的体检报告。毛利率趴在原地不动，说明你其实还在堆人，只是单子变多了；毛利率一路往上，才说明你的沉淀真的开始复利了。

我见过一个很直观的演变。一个团队，刚起步，纯定制，毛利率 20% 出头，还得捏着鼻子接单。攒了两年组件，毛利率爬到 40%。后来把一个需求做成标准产品，毛利率直接上了 60%。这三个数字之间隔着的，不是运气，是他们一笔一笔攒下来的可复用资产。

毛利率这个数怎么算，我给你个土办法，别去套会计那套复杂口径。你就记：一个项目，收进来的钱，减去这个项目直接花掉的钱——人天（按人头时薪折算，老板和自己团队的人天也要算进去）、外包、差旅、为它单独买的软件设备——剩下的，除以收进来的钱。很多人算出来的毛利虚高，就是漏了给自己团队算人天。算准之后，把它按季度拉成一条线，看趋势，比看任何一个单季的数字都有用。

### 第二，人均营收。

如果你堆一个人头，只能对应一份固定的营收，你就是在卖时间。人多营收多，人少营收少，两者绑得死死的。

但如果你的复利开始奏效，同样的人在单位时间里能创造出更多价值——因为这个季度，他们交出东西的速度，靠的是上个季度攒下的组件。人均营收往上走，才说明复制真的在起作用，而不是人海战术的幻觉。

人均营收还有个妙用：它能帮你发现团队的隐性冗余。同样七八个人，人均营收三个月没动，但你明显感觉大家更忙了，那问题多半是时间花在了不产生营收的返工和救火上——也就是你的沉淀没跟上，团队在替你重复踩坑。这种时候，别急着加人，先去做沉淀。加人解决的是"人不够"，解决不了"活太多是因为你没复用"。

这两个指标，不用精确到小数点，但趋势一定要看。它们是防止你"自欺欺人地说自己在产品化、其实还在堆人"的照妖镜。算账不丢人，不敢算账才丢人。

顺带说一句，看这两个指标的时候，记得把"客单价"和"单量"分开看。有人收入大涨，是因为单价涨了，不代表你更会复制了；有人收入没涨，人均营收涨了，那才是真正的变强。别被总额骗了。

## 用复盘沉淀知识

数据是照镜子，复盘是往里挖。没有复盘，数据就算暴露了"你还在堆人"，你也说不出为什么，更不知道从哪改。所以这两样得一起用。

复制的前提，是"知识"能沉淀下来。而知识沉淀，靠的不是灵感，不是某个天才拍脑袋，是制度化的复盘。

没有制度，复盘就是一句空话。项目一结束，大家松了一口气，下一个项目又来了，谁还记得复盘这回事？所以复盘必须是一种"不讨论、直接做"的动作，像交付节点一样硬。

复盘怎么开？我习惯的节奏是这样的：

项目结束或阶段结束后，把核心交付的人拉到一起，两个小时，一块白板。不做庆功，也不做批斗，就干一件事：把这个项目解剖了。

复盘的内容，至少包括四个问题：

这个项目踩了哪些坑？根因是什么？是数据、流程、组织、系统，还是我们自己的成功标准没定清楚？

哪些做法是有效的，值得固化成流程或组件？

哪些东西是"这一个客户特有"的，哪些是"整个行业通用"的？这一条最关键，它是后面"该不该产品化"的原始素材。

下一步，我能把什么抽出来、复用起来？这句话，逼着复盘不变成闲聊，变成一次"资产入库"。

四个问题问完，必须产出一个东西：一份写下来的、能给别人看的沉淀。哪怕就一页。写不下来的洞见，等于没有洞见。

这里我想强调一个反常规的点：**失败项目的复盘，价值往往远大于成功项目。**

为什么？因为成功的项目里，你很难分清"是我做对了，还是运气好"。客户配合、数据干净、决策人给力，这些好运会被你会错当成自己的能力。而失败的项目，坑是真金白银踩出来的，每一个都是别人想绕也绕不开的教训。

所以我的团队里有一条不成文的规矩：失败的项目，复盘优先，更要掰开了揉碎了讲。哪个同事在失败项目里栽了跟头，我不会让他藏着掖着，我会让他把这个坑，讲给全组听。一个人掉进去过，全组就都不要再掉进去，这笔账太划算了。

这里还有个关键细节，很多人栽在复盘会的氛围上。复盘最怕两样东西：一样是开成批斗会，一样是开成表扬会。批斗会里，没人敢说实话，坑就藏起来了；表扬会里，大家只挑好的讲，坑还是藏起来了。我的办法是：开会前先定一条规则——复盘只对事、不追人，谁踩的坑谁的锅不重要，重要的是这个坑的机制是什么、怎么防。管理者尤其要带头讲自己踩过的坑，你把自己最丢人的失败先摆上桌，下面的人才敢跟着摆。

再往深一步说，"复盘"这件事，最大的敌人不是没人写，是写了没人用。很多人复盘做得漂亮，文档整整齐齐，然后锁进文件夹吃灰，下一个项目照样重新踩坑。

所以除了"写"，还得解决"用"。我的办法是两条：一条，把复盘里的"可复用项"，直接转成一桩任务——抽组件的就去抽组件，改流程的就去改流程，不给它留落地的缝；另一条，让复盘文档跟着项目走，下一个同类项目的进场清单里，第一项就是"读上一家的复盘"。

可以毫不夸张地说，一个团队真正值钱的，不是它做了多少个项目，而是它把多少个项目的失败和成功，都转化成了可以复用的知识。前者是流水，是过手就走的钱；后者才是资产，是越攒越值钱的家底。让每个项目都变成组织的知识，靠的就是这套制度化的复盘——它把你一个人的血泪，熬成整个组的铠甲。

再补一个落地的小建议：把这些复盘，统一放进一个大家都能随手搜到的地方，而不是散在各人的微信收藏和本地文档里。一个团队的知识，一旦散落到个人手里，就等于没有知识。它得有一个家，一个新人来了能自己翻、老手想不起来能搜的家。这个家不用 fancy，一个共享笔记、一个私有知识库都行，关键是，要真的有人在里面放东西，也真的有人翻。

## 7.4 生态借势与组织政治

复制这条路，光靠自己埋头干，是不够的。你技术再强、组件再全，也改变不了一个事实：一个人、一个小团队的能力，是有物理边界的。

你还得学会"借势"。

借势，有两个层面。一个往外借，借平台的势；一个往里借，借组织的势。

### 第一层，借平台的势

不要什么都自己从零造。现在有大大小小的 AI 平台、工具生态，它们已经把很多基础能力做成熟了——模型调用、Agent 编排、数据接入、安全合规。这些底层的東西，你重复造，既没有竞争力，也耗不起。你要做的，是站在这些平台上，聚焦于你最擅长的"业务翻译 + 现场交付"，而不是重复造轮子。

这里有个现象，特别值得 FDE 琢磨，因为它颠覆了很多人的认知。

我见过一类活得很滋润的团队。你去看他们的背景，会有点意外——他们多数不是算法工程师，不是架构师，而是过去做企业代运营、做获客、做出海陪跑的。他们没有正规的技术团队，按说这是劣势。结果呢，这反而成了优势：因为没有技术包袱，他们天然就避开了"深度定制"这个承诺，从不给自己挖"我要给你重建一套系统"的坑。

他们怎么干活？极度懂业务场景，然后用 Codex、Agent 这类工具，自己"vibe coding"搓出贴着场景的轻量智能体。他们卖的，是紧贴场景的轻量智能体，加上落地的陪跑服务。

更重要的是，他们手里有多年积累下来的存量客户和信任关系。回头跟老客户说一句"这个能帮你省两个人、提效 30%"，单子就签了。这比你冷启动去拓新客，容易一百倍。

还有一条，是最让我开眼的：他们深度绑定在 WorkBuddy、阿里、360 这些平台生态里，靠厂商导流拿订单。平台有货架、有流量、有现成的工具链，他们站在货架上，做那个"把货翻译成生意"的人。

这背后是一个关键认知，我建议你反复咀嚼：

**在 AI 生态里，你不一定要当那个造平台的人，你可以当那个把平台能力翻译成业务价值的人。**

这，恰恰是 FDE 的本分。平台造出来了，得有千万个懂业务的 FDE，把它翻译成一个个具体的、能落地的、能算账的业务场景。你不需要去跟大厂比模型，你只需要比隔壁团队更懂你家客户的那摊事。

当然，这不是让你放弃技术。恰恰相反，你越懂平台能干什么、不能干什么，你翻译得就越准。所谓"站在平台上"，前提是你对平台的能力边界一清二楚——哪些它已经替你扛了，哪些它扛不了、必须你来补。把两者分清楚，你的价值就显出来了。

但借平台的势，有一条要警惕：别从"借势"滑成了"被绑定"。你全部订单都依赖某一两个平台的导流，一旦平台调整政策、换个方向，你的命就悬了。所以聪明的借势，是"站在平台上，但不把命交给平台"——平台给你流量和工具，你同时还要靠自己的存量客户和复购，养出第二条腿。两条腿走路，才站得稳。

## 第二层，借组织的势

说完往外借，再说往里借。

交付从来不只是技术。你做得再好，如果在客户组织内部推不动，还是白搭。能不能推动一件跨部门的事，能不能让决策层为你的项目站台，能不能在组织政治里合纵连横——这些"软"能力，决定了你的复制能不能在一个客户内部就先铺开。

这一层，其实和第三章的"政治地图"一脉相承：你越懂组织的暗流，越能把一个项目的成功，放大成多个部门的生意。

一个灯塔部门做出了成绩，你不应该止步于此。你要借这个部门的成功，去敲隔壁部门的门。让第一个部门的人，帮你在内部说好话——这比你自己去讲，可信一百倍。

具体怎么借？我给你一条很实际的路子：把一个项目的成果，主动做成"能给别人看的东西"。一张省下来的工时、一笔少漏的钱、一段前后的对比，包装成一页纸，递到能拍板的人桌上。组织内部的信息是极其稀缺的资源，你帮一个部门把它的成绩"翻译"给组织看，这个部门就会成为你最铁的同盟，帮你把复制铺开。

讲到找同盟，我再给两条好操作。一条是：进场的时候，除了找“能拍板的人”，还得专门找“能帮你说话的人”。拍板的人给你授权，说话的人给你铺路，两者常常不是同一个。另一条是：对那位隐性的抵制者，别硬碰。你越逼他，他越会用安全、合规、预算的借口拖死你。正确做法是，绕开正面对抗，把项目做成他无法公开反对的东西——先出一个他挑不出毛病的窄切片，让事实替他松口。

记住，组织的势，从来不是抢来的，是攒出来的。

很多人做技术做久了，看不起这些“经营关系”的活，觉得不务正业。但现实是，企业里的复制，从来不纯粹是产品问题，它有一半是政治问题。你不碰政治，政治就会来碰你。

说到底，很多 FDE 抵触“组织政治”这四个字，觉得脏。但你想通了就明白，所谓组织政治，无非是企业里“谁有权、谁受益、谁害怕”的规则。你做的每一单，都踩在这些规则之上。你可以假装看不见，但规则不会因为你看不见就不起作用。与其被规则绊倒，不如主动把它摸清楚——这跟手段脏不脏没关系，这是进场的基本功。

## 7.5 一个组件，是怎么复制到五个客户的

前面几节讲的都是道理。最后，用一个具体的演化，看看“知识 → 组件 → 产品”这条路径，到底是怎么一步步走出来的。

我拿“供应商对账自动化”这个场景来讲。它是我做的一个推演，你把它套进任何一个你自己做过的项目，都能对上号。

### 第一次，只有知识。

你做了第一个项目，给一家制造企业做“供应商对账自动化”。做的时候你很痛苦，因为这个客户的需求极其琐碎：每家供应商的对账单格式都不一样，字段对不上，还要处理各种表格里的脏数据。你硬着头皮做完，客户也验收了。

还记得那个采购经理跟你说的原话吗？“我这每个月底，光对供应商的账就要对三天，对得眼睛都花了，还老出错，一出错财务就在群里骂我。”你当时心里咯噔一下，记住了这句话。就是这句话，后来成了你判断“共性”的一块压舱石——它让你意识到，你解决的不是“这一家的 ERP 对接”，而是“采购员月底对账”这个普遍存在的痛。

这时候，你手里最值钱的东西，其实不是那段对账代码，而是一个你慢慢咂摸出来的认知：很多制造企业的采购，都在被“每周手工对账”这种又琐碎又容易出错的事折磨。这个认知，你连同踩过的坑——哪家供应商账单有什么花样、哪些字段最容易对错、客户验收最在意什么——写进了复盘文档。

这时候你只有“知识”，一行能复用的代码都没有。但你比没做过这个项目的人，多了一块敲门砖。

有人会问：那这个“认知”能卖钱吗？现在还不能。它还只是一个想法、几行复盘笔记。但它是一切的种子——没有这个认知，你永远不会想到“对账这件事能复用”。很多团队做完第一个项目就把它彻底忘了，认知烂在了脑子里，那后面三级杠杆，一级都起不来。

### **第二三次，抽出了组件。**

接着你接了第二个、第三个项目，碰巧都是制造企业，又都撞上了对账的需求。第二次，你翻出第一次的代码，改吧改吧还能用，但你发现改得很别扭；第三次，你不干了——你把上一个项目里写的那套“对账比对引擎”抽了出来，做成了一个可配置的组件。

不同客户，只需要配置一下字段映射——这家供应商的账单列名叫“A”，那家叫“B”，配一下就行——就能复用那套比对逻辑。开发时间，从第一次的几周，压缩到了几天。

这时候，你的“知识”变成了“组件”。它是一段实实在在、跑通过、验证过的代码，不再是一个躺在文档里没人看的心得。

### **第四五次，熬成了产品。**

做完第三家，你咂摸出味了：“采购对账”这个需求，在整个制造行业太普遍了，几乎家家都有，家家都头疼。

于是你把组件进一步产品化，做成了一个标准化的“智能对账工具”：客户自己导入数据，系统自动比对、自动出差异、自动把差异推给复核的人。你不再需要为每个客户重写代码，只需要做少量适配和部署，甚至配着文档，客户自己都能上手。

当然，产品化也不是顺理成章的。你中间也犹豫过：做成产品，是不是意味着以后新客户都按“标准”走了，那个愿意付大价钱的深度定制客户，会不会跑了？这个纠结很真实，答案是：别盲目砍掉定制。定制和产品可以并行——愿意付高价的，继续给定制；买不起定制的，卖产品。两条线互为掩护。而且产品卖掉一百套，还会反过来给你带来源源不断的定制线索。

当然，五个客户也不是一路顺风。第四家接进来的时候，你就发现，它的采购流程里多了一道"三方签核"，你的标准工具推出来的差异，得先过它内部的审批才能落账。你本可以为了这一家，把工具改出一坨特殊逻辑，但你忍住了——你只给它加了一个可配置的"审签环节开关"，别的客户默认关。这个开关进的是产品，不是死代码。第五家再遇到类似的，两个开关一配，又过去了。

于是，一个最初只活在一个项目里的组件，被复制到了五个客户。你的边际成本，随着每一个新客户的加入而下降；你的毛利率，随着每一个新客户的加入而上升。第五个客户交付完，你回头看一眼第一个客户——那段当初让你头疼的代码，已经长成了一个能自己卖钱的东西。

这就是复制的力量。它不炫酷，甚至每一步都很朴素——说到底就是"做完别扔、抽出来、再抽出来"。但它把 FDE 从"卖时间的匠人"，变成了"造资产的经营"。

而这一切的起点，可能只是你在第一个项目结束后，认真写下的一篇复盘。

故事讲完了。你有没有发现，从知识到组件的关键一跳，是你第一次"受不了重复"；从组件到产品的关键一跳，是你第一次"看到了共性"。两跳都不需要什么天才，只需要你在第一个项目做完的那个晚上，多问了自己一句：

这件事，下次还要不要重来一遍？

如果你读到这里，还记得开头那句"总包暴雷"的教训，这一章说到底就一句话：不要靠堆人去放大，要靠沉淀去复制。前者是一条越走越窄的巷子，后者是一条越走越宽的路。

## 章末工具箱

### 工具一：产品化评审清单

决定要不要把某个能力产品化之前，过一遍。任何一条答不上来、或者答案是反的，先别急着投钱投入。

[ ] 有没有跨客户的共性？是"这一家"的问题，还是"这一行"的问题？

[ ] 能不能报出三家以上有同样需求的目标客户？

[ ] 把它泛化成通用产品的成本，高不高？和未来三年它能省下的钱比，值不值？

- [ ] 做成产品后，客户能否开箱即用（或接近开箱即用），而不是还依赖大量人工驻场？
- [ ] 交付后还需要多少"现场空间"？适合"纯产品"还是"产品 + 服务"？
- [ ] 现有客户的反馈，是否已经印证了"这类需求反复出现"？
- [ ] 这件产品，会不会反过来吃掉你现有定制业务的利润和口碑？（产品化要算总账，不是只看新产品能卖多少）

## 工具二：项目复盘模板

每个项目结束或阶段结束，开两个小时复盘会，回答下面四组问题，产出一页纸的沉淀。

### 坑与根因

这个项目踩了哪些坑？

每个坑的根本原因是什么？（数据？流程？组织？系统？还是我们自己的成功标准没定清？）

### 有效做法

哪些动作是有效的、值得复用的？

哪些可以固化成流程、清单、或组件？

### 通用 vs 特有

哪些问题 / 做法是"这个客户特有"的？

哪些是"整个行业通用"的？（这一条，是产品化的原始素材）

### 下一步沉淀

我能把什么抽出来，做一个可复用的组件？

有没有潜力升级成一个产品？泛化成本大概多少？

这次沉淀，交给谁、什么时候做完？（不给任务、不定时间，复盘就是闲聊）

这两个工具怎么配合用？项目结束后，先填《项目复盘模板》，把"通用 vs 特有"那一栏填扎实；过阵子，当你觉得某个能力可能值得产品化，再拿起《产品化评审清单》过一遍。前

者负责把项目变成资产，后者负责判断这块资产值不值得放大。一个往里收，一个往外投，顺序别反。

## 本章要点回顾

定制化陷阱的本质：永远从头做，收入与人天死绑，规模上限等于兜底能力。

三级复制杠杆：知识复制（起步）→ 组件复制（省工时）→ 产品复制（抬天花板）。

产品化四问：有没有共性、泛化成本多高、好不好用、还需不需要现场。

标准化未必是代码，服务也可以是产品——培训、陪跑、内训，照样品复制。

两个照妖镜指标：交付毛利率、人均营收。趋势比绝对值重要。

失败的复盘，价值高于成功的复盘；知识不转成任务，等于没沉淀。

复制一半靠产品，一半靠借势：往外借平台，往里借组织。

复制讲完了，它解决了“怎么放大”的问题。但走到这里，你可能也意识到了：所有的方法，最终都要落在一个具体的“技术底座”上。下一章，我们把视角切回到技术——不是讲怎么追最新的模型，而是讲怎么让技术，真正服务于“可计算的问题”。

<div style="page-break-after: always"></div>

## 第 8 章 技术选型与本体论

技术选型不是追新，是让技术服务于“可计算的问题”。

有个工程师朋友，今年已经第三次找我抱怨了。

他说客户那边没法干活。这周客户看新闻，说行业里都在上大模型；下周客户看同行直播，说人家都上 Agent 了；再下周开会，老板拍板：我们也搞个智能体平台。朋友攥着一摞需求，从 RAG 改到 Agent，从 Agent 改成 Multi-Agent，架构图前后画了十几版，一版都没落地。

我听完问他一句：这个客户，到底要算清楚什么问题？

他愣了半天。

这就是这一章想聊的东西。技术选型四个字，在 FDE 手里，和在一个普通工程师手里，压根不是同一件事。

普通工程师选技术，看的是新不新、强不强、社区火不火、跑分高不高。FDE 选技术，只看一件事：这玩意儿，能不能把客户那个具体的问题，翻译成一道能算的题。

能算，再土的技术也是对的。不能算，再新的技术也是摆设。

这一章分三块讲。第一块，把当下最热的三样东西——RAG、Agent、评测——在真实企业里的落地要点讲透，重点讲它们各自的坑。第二块，讲一个我越来越信的观念，Palantir 的本体论，也就是怎么把一家公司的数据、规则、权力关系，翻译成机器能算的模型。第三块，用一个医院的例子，把“动词建模”这件最抽象的事，一步步做给你看。

章末照例给工具：《技术选型决策树》和《本体建模检查清单》。

先声明一句。这一章会提到不少技术名词，RAG、召回、embedding、Ontology。但你别把它当成技术手册读。我一直觉得，FDE 不是靠记住名词吃饭的，是靠知道“哪个名词在哪个坑里会翻车”吃饭的。所以这一章真正重要的，是那些坑，和每一种坑怎么躲。

## 8.1 RAG、Agent、评测的落地要点

带团队做企业 AI 这几年，我逐渐摸出一个规律：技术演示里最漂亮的东西，往往是生产环境里最先翻车的东西。RAG 是，Agent 更是，评测是最容易被整个忽略掉的那个。

下面一个一个说。先讲 RAG，因为它最普遍，坑也最密。

### 8.1.1 RAG 的坑，从来没在模型身上

先讲一个我亲历的事。这个故事，我这几年跟同行讲过很多遍，因为它太典型了。

2023 年，我给一家企业做知识库。熟人介绍的客户，之前没合作过，但彼此认识。需求不复杂：把公司内部的产品资料喂进去，让员工用自然语言问问题，AI 给出答案。

上线测试那天，AI 回答错了一个产品问题。

一个关于产品参数的答案，错得很离谱，但语气斩钉截铁。客户那边的人脸色就有点不好看了。做知识库，第一天上测试就答错，这是最伤信任的事。

我们当时的第一反应，跟所有人一样：是不是模型胡说了？

于是开始查。查的过程，我后来把它总结成四步，这也是我排查 RAG 出错的固定套路。

第一步，查模型。是不是幻觉，是不是没按我们给的料来答，是不是 prompt 写岔了，是不是温度设太高了。查下来，"不是"两个字。prompt 正常，模型老实地按检索到的资料在答，没有任何胡编的迹象。

第二步，查召回。是不是知识库检索错了，把不相关的文档喂给了模型。查下来，也没有。召回的那份文档，正是讲这个产品参数的，主题完全对得上。

第三步，查解析。文档是扫描件，是不是 OCR 或者解析的时候把数字认错了。查下来，还是不是。扫描件解析出来的内容，和纸面上印的，一字不差。

到这里，我们仨都傻了。模型对，召回对，解析对，那到底错在哪？

第四步，查知识本身。我们把那份扫描件的原始纸质文档调出来，一个字一个字对。对到某一处的时候，停住了。

真正的错误，在这份多年前打印出来的产品设计文档上——上面印错了一个参数，错了二十年。

更要命的是后面这一层：这家公司的老员工，全都知道这一处印错了。平时要用到这个参数，他们脑子里会自动改过来，从来不会照着错的填，所以二十年没出过任何事。但这个"正确的改法"，从来就没有进入过任何线上系统。它活在老员工的脑子里，不在文档里。

于是 AI 忠实读取了那份印错的资料，非常准确地，给出了一个错误答案。

这件事给我的教训深到今天都硌着疼：**AI 可以非常准确地处理一份错误知识。**

这句话我把它当成做企业 AI 的座右铭之一。它背后藏着一条很多人不愿意承认的真相：RAG 这套东西，出问题的环节，绝大多数不在模型，而在这条链的其余三个地方——知识本身、文档解析、召回。

模型这十年进化得飞快，幻觉问题一代比一代轻。但企业的那堆文档，二十年没动过。你以为你拼的是最先进的模型，其实你拼的是二十年前那台打印机里吐出来的一张纸。

所以我把 RAG 的坑，拆成三个环节，一个一个讲清楚，每个环节给你一套排查思路。你现在用不上，将来出问题，翻回来照着查。

## 第一个环节：知识本身是错的。

这就是上面那个故事的落点。它的可怕之处在于，它不显山不露水。

脏的数据，你一眼能看出来不对。缺失的字段，你一查就能查到空。但"印错了二十年的参数"这种，数据看起来完全正常，格式工整，数字齐全，逻辑自洽，谁都不会怀疑它。它唯一的破绽，藏在某个早就不从事具体业务的老师傅的脑子里。

更麻烦的是，这种错误往往是成套的、相关的。一个参数印错了，下游引用了它的十几个文档，可能全都被带着错。你修了其中一份，另外十几份还在继续错。

怎么办？老实说，没有银弹。我能给的，是三个土办法。

土办法一，上线前让最懂业务的人坐在一起，把系统答的那批高频问题，人肉过一遍。别让工程师自查，工程师不懂业务，他只会看"格式对不对""逻辑顺不顺"。要的是那个干了十年、脑子里有正确值的老师傅。花一个下午，他能揪出你三个月都发现不了的错。

土办法二，在系统里留一个"纠错"的口子。线上谁发现 AI 答错了，能点一下，把错误和正确值都留下来。这个口子在后面讲评测的时候还会再提，它是我认为性价比最高的一个设计。

土办法三，也是最重要的一条：认清楚，知识治理是个持续的事，不是上线前一次性洗一遍就完事。企业知识会过期、会改版、会出新版本，只要你这个系统还活着，就得有人长期为"知识对不对"这件事负责。这个人是谁，是进场那天就要问清楚的问题，不是上线以后才想起来问。

## 第二个环节：文档解析出错。

这个环节，技术含量最高，翻车也最隐蔽。客户给你的料，从来不是规规矩矩的 Word。

它可能是扫描件，斜的、模糊的、带水印的。可能是 PDF，两栏混排，你按左到右读，它把两栏的文字搅在一起。可能是 Excel，合并单元格满天飞，你要的一格数字藏在三个表头的下面。可能是图片里的表格，OCR 认出来一半对一半错。还有那种一代代复制粘贴过来的文档，格式烂到打开都要卡半天。

这些，随便哪一个，都能让你的系统在"看起来解析得挺好"的表象下，把关键数字吞掉半个。

解析出错的后果，比你想的严重。解析错了，召回抓到的就是错的内容，模型再聪明，也只能基于错的内容答错。你在模型层怎么调都调不回来，因为错在更下游。

我的经验是：解析这关，必须在项目早期单独拿一批真实样本压测。别等上线了才第一次碰真实文档。拿客户最典型的那几十份文档，扫进来的、复印的、两栏的、带表格的，各挑几份，先把“能不能忠实还原出来”这条底线验证了，再谈后面的检索和生成。

说句得罪人的实话：现在市面上那些“开箱即用的知识库产品”，最经不起推敲的，往往就是这一层。演示的时候喂进去的都是干净文档，扫得正、格式齐，自然漂亮。真到客户环境里，一堆二十年的扫描件怼进来，出来的东西能错得亲妈都不认识。

### 第三个环节：召回出错。

这是被讨论得最多的一环，也是最“技术味儿”的一环。

召回的逻辑，是把文档切成块，向量化，用户问一句，就去向量库里找最相关的几块，喂给模型。这套东西，问题出在三个地方：切块、向量、阈值。

切块切得不对，一句话被拦腰截断，上下文丢了，模型拿到的是半截话，答非所问。向量化的模型不对，判断“相关”的标准就偏了，你问 A，它给你召回 B。阈值设得不对，要么召回一大坨不相关的噪声，要么把该召回的漏掉，两种都难受。

这三个地方，全是要一层层试出来的手艺活，没有标准答案。我见过有人为了一个召回效果，调了一个礼拜的 chunk size。

但我想提醒的是另一件事，比技术参数更重要。

很多人做知识库，把力气全花在“调召回”上，幻想调出一个完美参数，系统就稳了。这是天真的。真实企业里，召回的问题很多时候不在参数，而在料本身——资料组织得烂，互相矛盾，新老版本混在一起，你自己都不知道哪份是有效的，让算法去猜，它当然猜不准。

所以我总跟团队说一句话：**检索是放大器，不是垃圾处理器。**你喂进去的是垃圾，它帮你更快地找到垃圾；你喂进去的是整理过的、口径统一的知识，它才帮你更快地找到答案。

先治理知识，再调召回。顺序反了，白费力气。

这三个环节排下来，你会发现一个挺残酷的结论：RAG 落地，真正花时间的，模型占不到十分之一，剩下的九成，全在知识、解析、召回这些“不性感”的脏活上。

那些天天在群里讨论"该用哪个 embedding 模型"的人，大概率还没真正下场做过企业项目。真下过场的都知道，卡住你的从来不是模型选择，是那台二十年前的打印机，和那批早该退休的扫描件。

说到这，我想把一个更大的背景摊开。RAG 的坑，表面是数据问题，骨子里是个更大的前提：你面对的不是一个整洁的实验室，是一整个"脏旧世界"。

这几年我带过的项目，翻来覆去撞墙，撞来撞去，其实就是撞这五类东西：跟遗留系统集成、安全合规、部署可靠性、证明商业价值、可传承性。RAG 的坑，主要落前两类：Agent 的坑，落在可靠性上；评测，落在可传承性上——一套别人接管不了、出问题修不好的系统，是没有"可传承"可言的。这五类，就是我给企业 AI 项目分类的骨架。

想在脏旧世界里把东西交付出去，有一条我刻在脑门上的原则：**为失败而设计**。

大多数人设计系统，是按"一切正常"来假设的。网络不会断，接口不会超时，上游不会挂，数据不会乱。然后上线第一天，现实就来教他做人。

为失败而设计，是反过来的假设：一切会坏的东西，都会在最不该坏的时候坏。你的系统，要在这些"坏"发生的时候还能扛住，至少别把用户晾在那干瞪眼。

我见过一个把这事做到极致的项目，就属于"跟遗留系统集成"这一类。它的目标很硬：把一个现代 SaaS，跟客户跑了二十年的那套本地 SQL 数据库，做成实时双向同步，还要带冲突解决。

这活儿难在哪？就说一层。老库那边不是规规矩矩地吐数据：消息会乱序，重试会重复，网络说断就断，数据里塞满各种历史遗留。你要在这上面做实时同步加冲突解决，等于在一辆正高速开着、还缺俩轮子的车上，装一套新导航。

这种项目，靠"假设一切正常"是做不出来的。它做得成，原因就在于从一开始就在为失败设计：乱序到的，按序重排；重复的，用幂等去重；断网的，本地缓存顶住、恢复后再补齐；脏数据，过一层清洗兜底。

这个项目我反复用来给团队立规矩，因为它的每一条应对，都是先承认"坏事情一定会发生"，再去想"发生了怎么办"。我把这个念头，浓缩成了两句挂在嘴边的话。

第一句：**你的系统必须容忍客户环境的混乱——乱序、重试、断网、脏数据，而不是假设理想条件。**

第二句：为失败而设计——上游模型挂了就降级，再挂就用缓存，用户体验不中断。

这两句，我现在写进每一份交付方案的第一页。它们回答了一个 FDE 最该想清楚的问题：当客户环境里那些必然会发生的坏事情发生时，你交出去的东西，是变成一摊废铁，还是安静地扛过去，让客户压根没察觉出过事。

这个差别，就是"能跑"和"能活下去"之间的差别。

### 8.1.2 Agent 与工具调用的边界

说完 RAG，说说 Agent。

如果说 RAG 的坑在"数据这条链上"，那 Agent 的坑，就全在"边界"这两个字上。

Agent 和 RAG 最大的区别在于：RAG 只是回答，Agent 会动手。它能调工具、能查系统、能写数据库、能发消息、能下单。这个"能动手"，既是 Agent 的价值，也是它的危险。

我见过最多的 Agent 翻车，不是技术不行，是动手动得太欢了。

举一个我反复用来吓唬团队的场景。你给客服做了一个 Agent，让它自动回复客户消息。半夜两点，一个客户来问"我要退款，把订单取消掉"。Agent 理解得很好，也很热情，它调了工具，直接把订单状态改成了"已取消"，还顺手给客户发了一条"已为您取消订单，退款将在 3 个工作日内到账"。

你看，这条链上，每一步技术上都是对的。但它错在——没有一个人在深夜，为"这笔订单该不该取消"这件事签过字。

客户可能是随口一问，可能明天就反悔了，可能那笔订单牵着一笔大额合同。Agent 不知道这些，它只知道"取消订单"这个工具调用在技术上成立。

这里就要引出我这一节最想讲的一个概念：**可逆操作和不可逆操作**。

判断一个工具调用该不该让 Agent 自动执行，标准就一条——这个操作错了以后，能不能低成本地撤销。

能撤销的，叫可逆操作。查一个数据、检索一份资料、生成一版草稿、算一个数，错了就错了，重来一遍就行，损失可以忽略。这类操作，放心交给 Agent 自动跑，跑错了也没大事。

不能撤销，或者撤销代价极高的，叫不可逆操作。给客户发消息、改数据库里的真实数据、下订单、转账、删记录、发布内容到对外的渠道。这些操作一旦做出去，就收不回来了。发出去的消息删不掉，改过的库存对不上账，取消的订单伤的是客户关系。

我的铁律是：**不可逆操作，一律收口，做之前必须有人确认。**

这句话看着简单，做起来会一直被挑战。因为"收口"意味着慢，意味着多一道人工，意味着不再是全自动。客户那边、甚至你自己老板那边，都会有人问：能不能让它全自动一点？

这时候你要顶住。你可以这么想：自动化省下来的那点人工，和一次"半夜误取消了客户大额订单"造成的损失，完全不是一个量级。前者是优化，后者是事故。FDE 的职责，是用技术和流程，把事故发生概率压到最低，而不是为了演示好看，把事故概率垫高。

具体怎么收口？我常用的做法，是给 Agent 的每一个工具调用，打一个标签：可逆、半可逆、不可逆。

可逆的，绿灯，自动执行，日志留档就行。

不可逆的，红灯，Agent 只负责"草拟"，走下面的流程：它生成一份完整的执行方案——要动哪个库、改什么值、发什么消息给谁——然后停下来，把这个方案推给一个活人。人点确认，才真正执行。

半可逆的，就是介于两者之间的那种。比如"修改数据但留了审计日志""发送消息但只发给内部人员"。这类我建议也收口，宁可多一道人工，别赌。

你会发现，这个"草拟—审核—执行"的裁剪，恰恰是把 Agent 用在刀刃上：让 Agent 干它最擅长的——快速、不知疲倦地把方案做出来，把人留下来干他最擅长的——为结果签字负责。

这里我想多说一句关于"人"的话。很多人设计 Agent 系统，本能地想把所有环节都自动化，把人从流程里挤出去。这是方向性的错误。

FDE 脑子里必须始终清楚：企业要的，从来不是"无人化"，是"该自动化的地方自动化，该人负责的地方人负责"。人的价值不是被挤掉的那部分，恰恰是被留下来的那部分——判断、签字、兜底、承担责任。

一个把所有环节都自动化的 Agent 系统，看着性感，实则危险，因为它把一个企业最值钱的东西——人对结果的责任——给悄悄抹掉了。真出了事，你找不到一个该为此负责的人，项目也就活到头了。

所以，Agent 的边界，本质上画的不是"技术能做什么"，是"责任该落到谁头上"。

### 8.1.3 评测的双环

最后说说评测。这是三个里面最不起眼、却最要命的一个。

不起眼，是因为它不产生任何可见的功能，客户看不到"评测系统"，它只是后台的一堆数。要命，是因为没有评测，你就永远不知道你交付的东西到底好不好，也不知道它什么时候开始变坏了。

我常遇到这样的对话。客户的 IT 负责人问："你们的这个 AI，准确率到底是多少？"

新人往往会张口报一个数：90%，95%。报完自己心里也虚，因为那个数是从哪来的，说不清。

老手会反问他一句：你说的准确率，是指什么准确率？在什么数据集上测的？这个数据集哪来的？覆盖了我们多少真实问题？

这三问，基本会把"准确率"这个虚词，戳回它的真面目。

因为脱离"证据"谈指标，全是要流氓。这就牵出我这几年反复用的一件工具——证据分级。

这套东西的来源，是一次深圳的路演分享，我把它的精髓提炼了过来。核心思想一句话：一个说法能不能写进你的交付方案，取决于它背后有什么证据，以及这个证据能不能被重新检查。

具体怎么分？六类：

**REQUIREMENT**——客户明确要求过的。这类证据最硬，因为是客户自己要的。

**CALCULATED**——按公开的模型假设计算出来的。比如"按官方公布的吞吐量，这个配置一天能跑多少请求"。它是算的，不是测的。

**SIMULATED**——在记录了输入条件的模拟环境里跑出来的。比 CALCULATED 硬，因为它真跑了，但环境是模拟的，不是生产。

**MEASURED**——在可重现的条件下，真实测出来的。这类最值钱，因为别人能照着你的条件，再测一遍，验证你说的是不是真的。

**SUPPLIER\_DECLARED**——供应商口头或书面声明的。"支持私有化部署""通过 ISO 认证"。注意，这跟真的通过认证，是两码事。

**CERTIFIED**——有认证记录背书的。有证书、有编号、可查。

这六类，本身没有高低贵贱。一个需求也可以是很硬的约束。真正要防的，是**互相冒充**。

供应商说"我们支持私有化"，这是 SUPPLIER\_DECLARED。可它不等于"这个部署已经通过了你客户自己的安全审查"——后者是 REQUIREMENT 和 MEASURED 的结合。很多项目翻车，就是栽在把"供应商声明"，当成"已经验证过"来使。

你把这句话记牢，能过滤掉一半的坑：**不是每一句漂亮话，背后都有能复查的证据。**

说回评测体系。证据分级是"说法"层面的纪律，评测体系是"系统"层面的纪律。我把一个健康的评测体系，概括成两个环。

第一个环，离线评测。

上线前，你得有一套评估集和一堆指标，在离线环境里反复测。评估集怎么来？从客户真实的问题里挖，洗掉敏感信息，标上标准答案，变成一个能反复跑的"考卷"。每改一版模型、换一次 prompt、调一次召回，就把这张卷子重跑一遍，看分数是涨了还是跌了。

这里有个新手常犯的错：评估集是一次性做出来、做完就不动了。错。评估集会老化。业务在变，客户的问题在变，你这张卷子要跟着更新。更关键的，是你上线后发现的每一个真实错误，都应该被吸收成评估集里的新题——这叫"回归用例"，专门用来防止同一个坑踩第二次。

第二个环，线上反馈。

离线评测做得再好，也替代不了线上。因为真实用户问的问题、反馈的方式，永远比你的评估集野。

线上要抓的信号，我总结成几类：

用户点了"没帮助"，或者把 AI 的答复原样转给了人工——这是最直白的负面信号。同一类问题被反复提问，说明第一次没答好。答复被人工修改后才发出，说明 AI 这版不行。这些信号，散落在系统日志里，你要做的就是建一条管道，把它们收拢、分类、定期回流到那个离线评估集里。

两个环，一静一动，合起来才是一个闭环：离线用它测，线上用它收，收到的再喂回离线。

这就是我要强调的"双环"。缺了离线，你拍脑袋改系统，越改越乱；缺了线上，你闭门造车，上线才发现真实世界根本不是你想的那样。

讲完评测这套纪律，还想补六条原则。它们同样来自那批深圳路演的分享，我把它们当成做企业 AI 的六根准绳，挂在项目墙上，谁都能看见。

一，数据优先。能拿真实数据说话，就别用假设说话。

二，证据分级。每个说法，标清楚它是哪一类证据，别让它冒充更硬的。

三，可重现。你的实验、你的指标，要让别人照着条件，能再跑一遍、得到差不多的结果。

四，可恢复。系统挂掉、数据出错、误操作，要有办法回到出错之前的状态。配合前面说的不可逆操作收口，一起用。

五，人类负责。再自动化的系统，最终负责的是一个具体的、有名有姓的人。别让"反正 AI 做的"成为没人背锅的借口。

六，隐私边界。评测要用真实数据，但真实数据里有客户的人和机密。脱敏、最小化、限定用途，这根线踩不得。

这六条，单看都朴素，但合起来，就是一道防止你把项目做成"漂亮但不可信"的护栏。

RAG、Agent、评测，这三块讲完，落点其实就一句话：技术本身的难度，早就不是 FDE 的门槛了。真正的门槛，是把这三样东西，稳稳地焊进一个"可信、可查、可恢复"的企业环境里。

而要做到这一点，你就不能只把客户当成一堆数据和接口。你得真正理解，这家公司的业务，是怎么一回事。这就是下一节要讲的，本体论。

## 8.2 Palantir 本体论 (Ontology)

先讲一个关于钱的场景。

Palantir 的客户，大多是那种体量吓人的企业。他们跟客户签的合同里，会有一个词反复出现：Ontology，本体论，或者说本体。有一次，客户方的管理层拿到发票，问了一个特别直接的问题：

你们账单上这个 Ontology，到底值多少钱？

这个问题，换个场景就是“你卖给我的这个名词，凭什么值这么多”。

Palantir 的回答，我印象极深，它后来几乎成了我对本体论的理解锚点：

**客户掏钱买的，是那些不再漏走的钱。Ontology 这个词本身，一分钱不值。**

这句话，锤在点子上。

客户不在乎你是不是用了什么高级的技术名词。他在乎的是，之前每个季度都有一笔钱，因为信息不通、口径不一、部门扯皮，静悄悄地漏走了。你把这笔钱堵住了，他就认账。你跟他讲十天 Ontology 的哲学，他转头就把你划进“又一个卖概念的”名单里。

所以，别再纠结 Ontology 这个词念起来有多玄。它翻译成大白话，就一句话：**把一家公司的数据、规则、权力关系，重新整理成机器能算的模型。**

关键词是“重新整理”。因为企业的现状是，这些东西本来就散着。数据散在七个系统里，规则活在制度文件和老员工的脑子里，权力关系体现在“谁有权改这个字段、谁能批这个流程”这些没人写下来的惯例上。

本体论要干的，就是把这摊散着的东西，重新翻译一遍。翻译完了，机器才能在上面做计算，你的 AI 才有地方落地。

我把这个“翻译”，拆成三层讲。

**第一层，数据层。**

最底下这层，是把源数据接进来。企业的 SAP、Salesforce、合同库、订舱系统、那些跑了二十年的本地 SQL 数据库，全都算。这层干的是体力活：接接口、对齐格式、打通管道。

大部分 FDE 进场以后，最先撞上的就是这层，也最容易陷在这层出不来。因为它脏、它累、它看不到尽头。前面几章讲过的代工厂故事，就是陷在这层的一个极端。

但要清楚，数据层只是底座，不是终点。很多人做了一辈子，都在数据层打转，管这管那，最后交付的东西，客户还是用不起来。因为你只是把数据挪了个位置，没回答一个更关键的问题：这些数据，到底是干什么用的？

## 第二层，业务对象层。

这是本体论的心脏。

在这一层，你要把企业里流转的那些"东西"，从数据里抽象出来，变成一个个看得见、叫得出名字、能互相指认的对象。订单是一个对象，客户是一个对象，物料是一个对象，仓库里的库存是一个对象，一个工单是一个对象。

这里马上有人要跳起来说：这不就是建数据库表吗？建个订单表、客户表、物料表。

不是。这就是本体论和传统数据库最根本的分歧，也是我这一章最想掰开揉碎讲的地方。

传统数据库，建的是**静态的数据表**。表里装的是字段：订单编号、金额、日期、状态。它是"东西长什么样"的档案。

本体论，建的是**对象加动作加权限**。它不光回答"订单是什么"，还要回答"订单能发生什么变化""谁能让它发生这个变化"。

差别在哪？差别在，静态表是一堆躺着的数字，本体论是一群活着的、会发生事情、会被改变的东西。

举个最小的例子。传统表里，订单有个字段叫 `status`，值有"待付款""已付款""已发货"三种。这看着已经很业务化了，对吧？但它没告诉你：谁有权把"待付款"改成"已付款"？这个改动要走什么流程？改错了能不能撤销？改了以后要通知谁？

本体论把这些全显式地建出来。"支付"不是一个字段值的变化，它是一个**动作**。这个动作，只有财务角色、走完审批、并且通过某个支付网关，才能执行。执行完，要自动联动库存、通知客户、记一条审计流水。

看出差别了吗？同样一个订单，在传统表里，它是死的数据；在本体论里，它成了一个有行为、有边界、有权限的活物。

### 第三层，应用层。

最上面这层，是面向人的。不同岗位的人，看到的界面不一样，能做的动作也不一样。采购看到的，是物料和供应商；财务看到的，是账和审批；销售看到的，是客户和商机。

有意思的地方在于，应用层可以千变万化，可以今天给销售做一个界面，明天给仓库挂一个大屏，但它们底下，用的是同一套业务对象。应用是这个换的，对象是不变的。

这就是本体论真正的价值所在：**一次建模，处处复用**。你把"订单""物料""供应商"这几个对象，连同它们的动作和权限，一次性建清楚了，后面无论上面长出多少个应用，都能共用这一套。反过来，如果你每个应用都自己单建一套数据、单写一套逻辑，那就是我们现在到处能看到的"系统烟囱"——数据各存各的，口径各说各话，改一处要动十处。

说到这，就绕不开那个关于动词的问题了。上面反复在强调"动作"，我要专门把它讲透，因为它是最反直觉、也最值钱的一点。

### 动词建模：动作是一等公民

传统的数据建模，是名词的世界。你进任何一家公司，问 IT 要数据模型，他们给你的是一堆名词：客户、订单、产品、库存、合同。全是名词。

但业务是动词的世界。业务不是靠名词跑起来的，是靠动作跑起来的：下单、发货、收款、退货、补货、审批、结算。企业真正在乎的，从来不是"订单"这个名词存在不存在，而是"下单""改单""取消单"这些动作，做得顺不顺、有没有出错、谁来负责。

这个错位，是很多系统失败的根源。

你建了一堆漂亮的表，把名词都摆好了，可业务真正在流动的那些动词，你一个都没建模。于是动作的执行逻辑散落在各处——有的写在老系统的存储过程里，有的写在制度文件里，有的只活在某个老员工的操作习惯里。机器算不明白这些动词，你也就没法在它上面做任何智能。

所以本体论做了一个大胆的翻转：**把动词，也就是动作、函数，提到和名词同等的地位，甚至更高。动作是一等公民。**

这不是文字游戏，它直接决定了你能做什么。

每个对象，除了它的属性，要显式地建模出它能做的动作。订单有"创建、修改、取消、支付、退款"这些动作。物料有"入库、出库、盘点、调价"。每个动作，都要回答三件事：

第一，谁有权限执行它。不是谁都能取消订单，也不是谁都能改价格。

第二，执行它有什么前置条件和副作用。取消订单，前置是"还没发货"，副作用是"要触发退款流程"。

第三，它能不能撤销，撤销会怎么样。这就是上一节讲的"可逆不可逆"，在这里落到了建模层面。

把这三件事建清楚，你的系统就从一个"存数据的仓库"，变成了一个"能算业务怎么流转的模型"。AI 在这个模型上工作，才能真正理解：现在这个订单处在什么状态，接下来可能发生什么，谁有资格让它发生。

我见过太多团队，一上来就急着接模型、写 prompt，想靠大模型"理解业务"。这是把力气用错了地方。大模型再聪明，它面对七套彼此矛盾的 ERP，面对一堆没有口径的数据，它也没法替你理解业务。**理解业务这件事，恰恰是不能外包给模型的，它是建模的工作，是人的工作。**

本体论，就是把你对业务的理解，掰开、揉碎、显式地写下来，写成机器能算的样子。

这里有个特别能说明问题的真实样本，来自 Palantir 的一个客户，一家财富 100 强的消费品公司。

这家公司，至少有七套 ERP 系统。注意，是至少七套。要按一般的思路，接这样的客户，第一反应就是：得先统一 ERP，把七套撮合成一套，不然数据对不上。这是个无底洞，多少咨询公司陷进去过，三年五年出不来，钱烧完了，ERP 还在打架。

但 Palantir 的团队没碰这个无底洞。他们只盯住了一个决定——

物料清单里，有没有更便宜、又不影响生产的替代原料。

就这一个决定。

他们用五天，跑出了一个能用的东西。一周之内，采购人员就开始用它了。

这个案例，我在很多场合讲过，每次讲都有人不信：七套 ERP 的巨无霸公司，五天能出活？凭什么？

就凭他们没去解决"七套 ERP 不统一"这个问题。他们把力气，全压在了"替代原料"这一个决定上——把这一个动作，连同它需要的对象（物料、价格、供应商），连同它的权限（谁能建议替代、谁能批准），建模清楚。七套 ERP 不用统一，只要能从中把物料和价格的数据抽出来，对齐到这一个决定上，就够了。

你看，这里又回到了本章开篇那句话：技术选型不是追新，是让技术服务于"可计算的问题"。

"统一七套 ERP"不是一个可计算的问题，它是一个组织政治的泥潭。"找出更便宜的替代原料"，是一个干干净净、能算、能立刻产生价值的问题。Palantir 选了后者，五天生值。

这也是为什么我越来越信本体论这个东西。不是因为这个词高级，是因为它逼着你，把注意力放在真正的刀刃上——那个能被改变、被计算、被验证的决定上，而不是浮在面上的那堆庞然大物上。

那么，本体论到底怎么建模？讲道理不如看一个完整的例子。下一节，我用一个医院，把"动词建模"从头做一遍。

### 8.3 案例：一个医院的动词建模实战

这个例子，我想写细一点。因为"动词建模"这四个字，讲再多的道理，也不如亲眼看着一个真实的业务，被一步步拆成对象和动作来得实在。

场景是一家中型医院。规模不大，但有门诊、有药房、有住院、有结算窗口，该有的都有。他们找到你，说的需求还是一贯的宏大："我们想上 AI，把医院的流程智能化。"

你去现场转了一圈，看见的是这样的画面。

门诊大厅里，患者排队挂号。挂完号，去诊室门口等。医生看诊，开检查单或者开药。患者拿着单子去缴费，再去药房取药。住院那边，还有入院、出院、医保结算一套流程。每个环节都有排队的、催的、跑错窗口再折回来的。

如果你带着"上 AI"的心理预期来，很容易就乱了：这么多环节，从哪下手？做导诊机器人？做智能问诊？做影像识别？每个都有人做过，每个听起来都对，但没有一个是这家医院"此刻最该被解决"的那个问题。

所以第一步，还是老规矩——不是挑技术，是挑问题。你得先把这家医院，用"对象 + 动作"的眼光，重新看一遍。

传统上，一个做软件的团队进来，会先画一堆名词。他们会说：医院里有患者、医生、科室、挂号单、处方、药品、床位，我们把这些建表。名建全了，系统看着挺全，实际上医院该怎么转，一个字都没回答。

动词建模的路子，是反过来的。你先别管名词，先问一个最简单的问题：**这家医院，每天在发生哪些动作？**

你顺着患者的路径走一遍，就能列出一条动作链：

**预约 → 分诊 → 看诊 → 开药 → 缴费 → 取药 → （可能的）住院 → 结算出院。**

就这么一条链，你要把它当主线，一路拆下去。下面我拆几个关键动作给你看，看是怎么把"动作"建出来的。

先看"预约"这个动作。

表面上看，预约就是患者选个时间、点个确认。但你要是把它当成一个动词来建模，要回答的问题就不止这些了：

谁有权限发起预约？患者自己，还是必须挂号窗口的人代办？（答案不同，系统设计完全不同。）

预约的前置条件是什么？这个科室这个医生，这个时间段还有号吗？患者有没有爽约的黑历史？

预约的副作用是什么？号被占住了，别人不能再约；到点患者没来，这个号要不要自动释放，还是扣他信用？释放了要不要通知候补的人？

你看，一个"预约"，拆出来是一串牵扯权限、前置、副作用的动作逻辑。这些逻辑，今天活在哪？活在挂号室那个大姐的脑子里，活在她口头跟我约定的规矩里。你把它显式地建出来，机器才第一次"懂"了预约这件事。

再看"开药"这个动作。

医生开出一张处方。这里的关键，不在处方这个名词，在"开药"这个动词里藏着的规则。

开药的权限：只有有处方权的医生能开，实习生开的必须带教老师签。

开药的前置：这个药，患者的医保能不能报销？医院药房有没有库存？会不会和患者正在吃的别的药冲突？剂量超没超安全线？

开药的副作用：处方一出，库存要预扣，费用要进账单，如果药房没货要提示药师给患者换替代药。

这些规则，现在活在哪？活在药典里、活在医院的管理制度里、活在那套老 HIS 系统的存储过程里，还有一部分，活在有经验的药师的警惕心里——他会多看一眼，拦下那个剂量开错的单子。

你把"开药"这个动作建出来，把它嵌在这些规则里的权限、前置、副作用全显式化，等于把老药师脑子里那根"多看一眼"的弦，也搬进了系统。

接着看"结算"这个动作，这是钱的口子，最敏感。

结算要回答的问题更狠：谁有权限结算、退款？费用项目和医保目录对不对得上？会不会重复收费、漏收费？一笔退款打出去，谁签字、能不能撤销？

还记得上一节的铁律吗？不可逆操作，必须收口。在医院里，退款、销账、退药，全是不可逆操作。这些动作，建模的时候就要打上"红灯"标签——系统可以算、可以草拟，但最后那一锤子，必须落在一个活人手里。

现在回头看，你会不会觉得，这一套动词建模，其实跟"上 AI"没半毛钱关系？

对，这就是我想让你体会的点。动词建模，本质上不是 AI 的事，是**业务翻译**的事。你把医院的业务翻译成对象加动作加权限，这个翻译本身，就是本体论。

而 AI 在里面的位置，是等你把这个模型建好之后，再入场。它在这个模型上，可以干这些事：根据患者的描述和症状，自动判断该挂哪个科，也就是帮"分诊"这个动作跑得更快；根据患者的既往用药和过敏史，在"开药"这个动作上做一道安全校验；根据库存和替代关系，在"取药"时自动推荐替代品。

这些才是 AI 真正能发光的地方。但它们全都依赖一件事——你先把动作和对象建清楚了。模型只是在一个已经能算的业务模型上，加了一层聪明。没有底下那层业务模型，模型是悬空的，说得再好听，也落不了地。

所以，医院这个例子，我给你的结论就一句话：

别一进医院就想着导诊机器人。先把"预约到结算"这条动作链，一个动词一个动词地建清楚。建到一半，你会发现，你其实已经能回答"AI 该在哪几处使劲"这个问题了——而且这个答案，比任何技术大厂的方案，都更贴这家医院的实情。

动词建模，说穿了，就是逼你先把一家公司到底在"做什么"，彻底搞明白。搞明白了"做什么"，"用什么技术做"，是个顺理成章的事，根本不用纠结。

到这儿，这一章想讲的三件事——RAG/Agent/评测的坑、本体论、动词建模实战——都讲完了。它们串起来，其实是同一条线：

技术上，知道每样技术会栽在哪；理解上，用本体论把客户翻译成能算的模型；执行上，用动词建模把动作、权限、边界钉死。

这条线走通，你就不是一个"会调模型的工程师"，而是一个能让技术真正在客户业务里生根的 FDE。

章末照例，给你两个工具。一个是在一堆技术名词面前，帮你做决定的决策树；一个是建本体时，逼你自己回答对问题的检查清单。

## 章末工具箱

### 工具一：技术选型决策树

拿到客户需求，先别打开技术选型的文档，先走这棵树。每一步答不上来，就退回去一步。

#### 第 0 步：这个问题可计算吗？

客户要的，能翻译成一个具体的、可验证的"决定"或"结果"吗？

答不上来 → 先回第 4 章做问题定义，别谈技术。

答得上 → 往下走。

### 第 1 步：它需要"动手"吗？

只是回答 / 检索 / 生成草稿（答、查、写）→ 考虑 RAG 或纯生成，走第 2 步。

需要调工具、改数据、执行流程（动手）→ 考虑 Agent，走第 3 步。

只是把业务翻译成能算的模型（长期底座）→ 直接做本体论建模，走第 4 步。

### 第 2 步：RAG 场景，先问数据三连：

知识本身是干净、可信、有人负责更新的吗？

不是 → 先做知识治理，别急着上检索。

客户给的是标准文档，还是一堆扫描件和烂格式？

后者 → 先拿真实样本压测"解析"这一层。

召回不准，是参数没调好，还是资料本来就乱？

先治理，再调参。

### 第 3 步：Agent 场景，逐个工具打标签：

每个工具调用，问一句：错了能不能低成本撤销？

能（可逆）→ 自动执行，留日志。

不能（不可逆：发消息、改库、下单、转账、删数据）→ 收口，草拟—人工确认—执行。

说不清（半可逆）→ 一律按不可逆处理。

### 第 4 步：建模，动词优先：

先列这条业务链上"每天在发生的动作"，再列名词。

每个动作，回答三件事：谁有权执行？前置和副作用是什么？能不能撤销？

答完三件事，再问那句最关键的："客户此刻最该被解决、且能被算清楚的那一个决定，是什么？"

## 第5步：评测，双环都齐吗？

有没有一套来自真实问题的、会持续更新的离线评估集？

有没有一条管道，把线上反馈（没帮助、转人工、被修改）回流到评估集？

两个都没有 → 交付的准确率就是拍脑袋，先补评测再谈上线。

决策树走到头，你手上那个“该用什么技术”的答案，自然就浮出来了。它不是从名词里挑出来的，是从问题里长出来的。

## 工具二：本体建模检查清单

建模是一个动作一个动作抠出来的。每建一个对象，或者每建一个动作，回答下面这些问题。答不全，说明你对这个业务的翻译还没到位。

### 对象层

- ☐ 这个对象，是业务实际流转的“东西”，还是我们拍脑袋造出来的名词？
- ☐ 它的属性，来自哪套源系统？口径统一了吗？冲突的时候以谁为准？
- ☐ 它和别的对象是什么关系？（一个订单属于一个客户，一张处方对应一个患者）
- ☐ 这个对象的“生命周期”是什么？它会经历哪些状态变化？

### 动作层

- ☐ 这个对象能发生哪些动作？（别只列属性，把动作列出来）
- ☐ 每个动作：谁有权限执行？（角色、部门、审批链）
- ☐ 每个动作：前置条件是什么？（什么状态下才能做）
- ☐ 每个动作：副作用是什么？（做了之后会联动改变什么、通知谁）
- ☐ 每个动作：可逆还是不可逆？不可逆的，收口给人工确认了吗？

### 权限层

- ☐ 谁有权改这个动作的规则本身？（业务规则谁说了算，得显式定下来）

[ ] 有没有审计记录，能查出"谁在什么时候动了什么"？

[ ] 出了错，负责追责的那个活人是谁？找得到吗？

## 复用层

[ ] 这套对象，能不能被上面多个应用共同使用，而不是每个应用单建一套？

[ ] 换一个界面、换一个岗位，还能不能复用同一套对象和动作？

检查清单的目的，不是让你对着它打勾交差，是逼你在建模的时候，把那些"本该问但常常漏问"的问题问出来。多数项目翻车，不是败在大决策上，是败在某个动作的"谁能执行、错了谁负责"没问清楚上。

## 本章要点回顾

技术选型看的是"能不能把问题算出来"，不是新不新、火不火。

RAG 出错的三个环节：知识本身、文档解析、召回——模型是最不容易出错的那一环。

C12 的教训：AI 可以非常准确地处理一份错误知识，真值常活在老员工脑子里，不活在系统里。

Agent 的边界 = 责任落到谁头上。可逆操作自动跑，不可逆操作（发消息、改库、下单、转账）必须人工确认。

评测要双环：离线评估集 + 线上反馈回流，合起来才闭环。

证据分级六类，要防互相冒充：REQUIREMENT / CALCULATED / SIMULATED / MEASURED / SUPPLIER\_DECLARED / CERTIFIED。

六条原则：数据优先、证据分级、可重现、可恢复、人类负责、隐私边界。

本体论是把数据、规则、权力关系翻译成机器能算的模型，分数据层、业务对象层、应用层三层。

动词是一等公民：建模的是动作、函数、权限，不是静态数据表。

客户掏钱买的是"不再漏走的钱"，Ontology 这个词本身一分钱不值。盯住一个能改变的决定，比接全公司系统更值钱。

## 常见误区速查

**追最热的模型：**听说哪个模型又刷新跑分了，赶紧换上。→ 你的瓶颈在二十年前的打印纸，不在模型。

**把知识库当一次性工程：**上线前洗一遍数据就以为万事大吉。→ 知识会过期，得长期有人为"知识对不对"负责。

**Agent 全自动化：**把所有环节都自动化，把人挤出去。→ 不可逆操作必须收口，人的价值是签字兜底，不是被挤掉。

**拍脑袋报准确率：**客户一问准确率，张口就报 90%。→ 先问"在什么证据、什么数据集上"，没有评测的准确率都是耍流氓。

**把供应商声明当真验证：**供应商说"支持私有化"，自动当成"过了安全审查"。→ 证据分级，防互相冒充。

**建一堆名词表就当建模：**订单表客户表物料表摆得整整齐齐，业务怎么转一点没答。→ 动词优先，动作、函数、权限才是本体的核心。

## 本章行动清单

读完这一章，做三件事。

第一件，把你手头正在做、或者准备做的那个项目，用"技术选型决策树"从头走一遍。重点看第 0 步——这个问题到底可不可计算。我敢说，走完你会发现，一半的技术纠结，其实都该回到"问题定义"上去解决，而不是在技术里打转。

第二件，挑你客户业务里最核心的一条动作链（像医院的"预约到结算"那样），把它的一个动作，用"谁有权执行、前置是什么、副作用是什么、能不能撤销"四个问题，写清楚。写完你就会发现，自己对这个业务的理解，比之前"看着都懂"深了一层。

第三件，翻出你现在的知识库或者 Agent，查一件小事：它有没有一条把"线上反馈"收回来、变成"回归用例"的管道。大概率没有。没有就开始搭，哪怕先做最简陋的一版——让"没帮助"和"转人工"两个信号，能定期流回一张表里。

这三件事，都不难，但都很容易被漏掉。而它们，恰恰是区分"会调模型的人"和"能落地的人"的那道缝。

<div style="page-break-after: always"></div>

## 第 9 章 底线与道路

技术能力决定你能走多快，职业底线决定你能走多远。

有个时刻，我一直记得。

那是一个项目快要收尾的晚上。客户的一个部门负责人，把我拉到楼道口，递了根烟，说了一句话。

"系统跑得挺好。"他吸了一口，"就是那个失败率，报表里你能不能别写那么难看？写漂亮点，底下报上去，脸上都好看。"

我愣了一下。这活儿我太熟了——不就是把真实数字，往脖子上抬一抬，让曲线温柔一点嘛。改个口径的事，五分钟。

还有一次，更早，另一家客户。进场做完尽调那天，对方 IT 直接把一个数据库账号甩了过来："你们自己进去看看吧，全都在里面。"我点进去，客户名单、销售流水、员工薪资，一张表接一张表，像一扇没上锁的门。

那一刻我才意识到，这活儿真正难的地方，从来不在代码里。

前面八章，我讲的全是"怎么把事情做成"：怎么画地图、怎么选问题、怎么切 MVD、怎么让用户真用、怎么守续约、怎么规模化、怎么选技术。那些都是把事做成的本事。

这一章不一样。这一章讲两件事：哪些事不能做，以及干完这些活之后，你这个人要往哪去。

前半句是底线，后半句是道路。

### 9.1 六条职业道德底线

先讲底线。

不是那种贴在墙上的员工守则，也不是什么高大上的宣言。就是几条，我在这些年里，看着同行一个个栽进去、也逼着自己一次次刹住车之后，总结出来的硬线。

六条。每一条背后，都有真人真事，都有具体的场景。你可以当故事听，但我建议你当戒律背。

## 第一条：数据主权

数据是客户最值钱的东西，也是 FDE 最容易弄脏手的地方。

为什么容易弄脏手？因为你要干活，就绕不开数据。客户为了让项目快点跑起来，往往会把数据权限敞口给你——就像我开头说的，一个数据库账号甩过来，“你随便进”。采购记录、销售流水、员工名单、客户手机号，全在那里面。

这时候，你手里握着的，是别人全副身家。这也是诱惑最大的时候。

诱惑不是一个“要不要偷数据”这么低级的问题。大多数 FDE 没那么坏。真正的诱惑要隐蔽得多，它长这样：

“这个数据顺手拉出来，跟下一个项目能对上，一起做了得了。”

“你们另一个子公司的数据，反正都是自家，一起读了省事。”

“这份名单拍照存个档，以后别的项目还能用。”

这些念头，每一个听起来都合情合理，每一个起心动念都“不是为了害人”。但它们踩的是同一条线：你分不清“客户给你用”和“数据属于你”这两件事。

客户给你数据权限，是让你交付，不是把主权移交给你。

守住这条线，靠的不是道德自觉，是几个做死的动作。我自己的习惯，进场第一天就跟客户把三件事钉死：

第一，我只要哪些数据，其他的我一概不碰，哪怕账号能进。进得去，和应该进，是两码事。

第二，数据放哪儿、谁能看、多久删。拷出来的样本，项目一结，当面清空，不留备份。

第三，数据出不出域、出到哪。这条最要命，很多客户的数据是绝对不能出局域网的，你方案里哪怕只提了一句“上传到云端模型”，安全审查就直接把你拦下来。

这三件事，白纸黑字写进交付附件里。不是给客户看的，是给你自己设的闸。

我见过反面教材。有个同行，接了 A 客户的活，把 B 客户的数据当"内部经验"搬了半份过去，被 A 的 IT 发现，一个电话打到老板那儿，项目黄了不说，人在行业里也臭了。数据这种东西，一次越界，十年洗不白。

数据主权这条底线，本质上是两句话：别人的数据，不是你的原料；你能进的地方，不等于你能去的地方。

## 第二条：结果诚实报告

这条，是最难守的，也是最值钱的。

因为 FDE 干的活，天然处在一个"可以往好了说"的位置上。模型效果、失败率、省了多少人力，这些数字怎么报，弹性大得惊人。你把这个不算漂亮的结果，说成一个漂亮的结果，客户短期根本发现不了。

开头楼道口递烟那位，要的就是这个。改个口径，皆大欢喜。

可这个"皆大欢喜"，代价是下一次。你夸一次，客户信你一次，也把信任的底子烧掉一层。等真出了事，没人再信你的数字，你再说什么，都是狼来了。

诚实报告这件事，光靠"不说谎"的决心还不够，你得有工具。没有工具的人，心里想诚实，手上也不知道该怎么把话讲准。

我用的工具，是一套"证据分级"。这东西我在深圳一次项目路演上见过，后来一直用到现在。

一个数字、一句话，它背后站着的证据，分六种：

REQUIREMENT，客户明确要求的；

CALCULATED，按公开模型假设计算出来的；

SIMULATED，记录了输入条件的模拟结果；

MEASURED，可重现条件下实测出来的；

SUPPLIER\_DECLARED，供应商自己声明的；

CERTIFIED，有认证记录支持的。

这里有个特别容易踩的坑：六种证据，本身不分高下。供应商声明的就比实测的低一等吗？不一定。真正要防的，是它们互相冒充。

我举个最典型的。你问某个模型供应商："这个功能支持私有化部署吗？"对方拍胸脯："支持，白纸黑字写在我们官网上的。"

这是 SUPPLIER\_DECLARED。供应商声明的。

但你要交付给客户的，是这个"支持私有化"，能不能通过客户自己的安全审查。那是另一回事，那需要 CERTIFIED 或者客户实测过。两件事，长得像，是两码事。你把供应商的声明，当成了客户会认的证据，写进交付方案，等审查那天被卡下来，锅就是你的。

所以我自己定了个死规矩：一句话能不能写进交付方案，看两件事——它有什么证据，以及这个证据能不能被重新检查。能重新检查的，说出去；不能的，就说"还没验证"。

再说说"掩盖失败"。

这是比夸大更危险的一项。夸大了，至少你还在报告。掩盖失败，是把雷直接埋进地里，等它在生产环境里炸。

我见过最值钱的一个教训，跟一份印错的产品文档有关。有次给客户做知识库，上线测试，AI 回答错了一个产品问题。我们第一时间查模型是不是胡说，没有；查知识库是不是召回错了，也没有；查扫描件是不是解析错了，还是没错。技术链路全部正确。

最后查出来，错在很多年前打印出来的一张产品设计文档，上面有一处印错了。全公司的老员工都知道这处错，平时在脑子里自动改过来，从没出过事。但这个"正确的改法"，从没进过任何线上系统。于是 AI 忠实读取了那份错误资料，非常准确地，给了一个错误答案。

这个故事我在前面讲过，这里再拿出来，是因为它藏着一条职业戒律：**AI 可以非常准确地处理一份错误知识**。而那处错误之所以能藏到现在，就是因为一直有人知道、一直没人报告。

做 FDE，你会无数次撞上这样的时刻：你发现了一个失败、一处错误、一个丑数字。你报告，就是给自己添麻烦；你咽下去，就是给未来埋雷。

我的选择是报告。不是因为我是多高尚，是因为我算过一笔账：添麻烦是短期的、能补救的；埋的雷是长期的，会在你最没防备的时候炸，而且炸的是你在这个行业里攒下的全部信用。

### 第三条：避免制造依赖

这条说出来，很多人会愣一下：制造依赖，不是大家都梦寐以求的吗？客户离不开我，续约才稳啊。

没错，很多同行是这么干的。把系统做成黑盒，把接口藏起来，把文档扣着不给，让客户离了自己就转不动。客户一有需求，只能回头找你，续约的水到渠成。

短期看，这招真管用。长期看，这是在给自己掘墓。

因为有一组数字，我建议每个 FDE 都记住。外部研究机构高德纳的预测：到 2028 年，70% 的企业，可能因为供应商成本过高、加上内部技能不足，放弃由 FDE 主导的 Agent 方案。

70%。这个数你怎么理解？

我理解的意思是：你把客户绑得太死，客户不会老老实实付你一辈子钱，客户会算账。算来算去，发现“被你绑着”这件事的成本，高到还不如干脆不要这个方案。

你以为锁住的是客户的续约，其实锁出的是客户的戒备。一个天天琢磨怎么被“解绑”的客户，比一个正在用你的客户，难伺候一百倍。

真正的持续关系，靠的从来不是“我得依赖你”，而是“我希望继续用你”。这两句话，差着十万八千里。

怎么做到后者？就一条：主动让客户有一天可以不用你。

好理解吗？你交付的时候，顺手把方法交给对方——这个提示词怎么调、这个流程为什么这么设计、这份数据从哪来、坏了怎么修。你把人教会，把文档留下，把资产沉淀给客户自己。

有人会问：那客户学会了，不就把我炒了吗？

会吗？现实里不会。因为客户就算学会了你现在这套，你的价值从来不是“这套别人学不会”，而是“下一个更难的问题，我先他一步想明白了”。你交付的每一代价值，才是你续约的真正理由。

反过来，你藏着掖着，客户一旦动了换人的念头，第一件事就是翻旧账：你这两年到底给了我什么、离了你我是不是真就玩不转。这一翻，你藏的那些私心，全现了原形。

别把"客户离不开我"当成护城河。护城河是你能持续交付新价值，不是你在对方脖子上拴了根绳。

#### 第四条：关注被替代人员

AI 落地，是要动别人饭碗的。

这是这行最扎心、也最没人愿意摊开讲的一条。你做一个自动化，本来三个人的活，变成一个半人。那两个半人，怎么办？

很多方案书里，这一块是空白的。PPT 上写满了"提效 300%"、"节省人力"，没有一个字提到：被省下来的那个人，是采购部每天对账的张姐，是车间里誊数据的李师傅，是客服里背下了全部产品参数的王姨。

他们的饭碗，就是你的提效数据。

你说这不归你管。可这事儿，恰恰是 FDE 该管的一环。

为什么？两个原因。

第一个原因，是良心。你眼睁睁看着自己的方案，把一个具体的人的岗位拆没了，你不能假装没看见。至少，方案设计的时候，你该问一句：这个岗位被自动化之后，人往哪去？是转岗学新东西，还是原地被裁？

第二个原因，更实在——**被替代的这些人，往往是最懂业务的人。**张姐对了两年的账，哪家供应商老爱对错、哪个字段最坑，她全在脑子里。李师傅手里的那些工艺参数，跟第 4 章代工厂里的老师傅一样，是几十年攒下的隐性知识，系统里一个字都没有。

你要是不在乎他们，你的系统就永远学不会他们脑袋里那套东西。你今天用"提效"把他们扫地出门，明天就得花三倍的力气，去把那些没进系统的隐性知识重新挖回来。

所以我给自己定了个习惯：凡是会动到具体岗位的方案，先跟客户把"人怎么办"聊清楚，再签。

这个人，转岗去做监督和复核，还是去学怎么用这个新系统？

系统上线后，他的经验能不能变成训练数据、变成规则、变成他教会别人的东西？

哪些节点，恰恰必须留一个人来兜底？

这不是替你客户老板操心，这是替你的交付质量操心。一个把老员工扫地出门、没人接得住隐性知识的方案，上得了线，活不过第二年。

还有一个更坏的做法，我要单独拎出来骂一句：**拿“优化人力”当卖点去忽悠老板**。有些销售为了签单，张口就是“这套系统能砍掉你们一半人”。这种话，短期能让老板眼睛发亮，长期是把你往火坑里推。老板真按“砍一半人”来验收，你交不出来；老板真砍了，剩下的人恨你入骨，系统上不了场。

关注被替代人员，不是让你做圣母，是让你做长线。人安置不好，机器就落不了地。

## 第五条：合规边界

这条，看起来最枯燥，其实是很多项目真正的生死线。

安全审查、数据出域、私有化部署，这些词，技术人听了打哈欠。但真到了你手里，就是一道道过不去的门。

我讲个更具体的。客户提了硬要求：数据绝对不能出局域网。你回去一查，你手上最好用的那个模型，API 要走公网。怎么办？

摆在面前三条路。

一条是老老实实换方案：上私有化部署的模型，或者做本地的脱敏代理，让数据不出域。慢，贵，麻烦。

一条是动点小心思：数据脱敏一下、样本化一下，名义上“没出域”，实际上半条腿迈出去了。快，省事。

一条是更野的：直接绕过审查，先干了再说，等客户发现再说。赌客户不查。

第一条是正道。第二条是踩线。第三条是找死。

我见过太多人在第二条和第三条上要小聪明。仗着客户 IT 人数不够、审查不严，糊弄着把项目过了。结果呢？一次数据泄露、一次上级检查、一次审计，前面所有的糊弄，全变成了把柄。

合规这件事，有个铁一样的规律：出问题之前，它一文不值；出问题之后，它价值连城。你在审查上偷的懒，都会在出事那天，连本带利找回来。

所以我的态度很硬：安全审查不是流程，是硬线。过不去，就停下来改方案，改到过得去为止。别在审查上玩"变通"，那点变通省下的时间，还不够日后擦屁股用的。

更别说，这条线跟你自己的招牌绑在一起。客户把数据交给你，交的是命根子。你在合规上栽一次，以后没有哪个有安全要求的客户，敢把数据交给你第二次。

再往深了说一句：合规的底层，是"谁能看见什么"。多租户的隔离做得干不干净、PII 脱敏代理有没有漏、SSO 单点登录和权限有没有和客户现有的体系对上、每个动作有没有审计留痕——这些细节，第 8 章讲技术的时候我提过，这里再强调一遍，因为它们是"数据主权"这条底线在工程上的落地。

你嘴上说尊重客户数据，代码里却没有隔离、没有脱敏、没有审计，那你的尊重，值几个钱？

## 第六条：行业责任

最后这条，是给这个行业松土浇水的。

我得说点不好听的。

今天市场上打着"FDE"旗号的人里，有相当一部分，是换了个名字上岗的。原来的售前、实施、驻场外包，一看这词热，摇身一变都成了"前线部署工程师"。

他们进客户公司的第一句话，往往是"数据库有几个？接口什么时候能开？"然后拿几张脱敏表，赶一个大屏，接一个模型，开会演示一遍，散会之后，那套东西再也没人打开过第二次。

业内有人骂得很直白：国内 99% 的所谓 FDE，都是扯淡。这话我也觉得狠，但它戳中了一件事——太多人在玩"FDE cosplay"。

Cosplay 是什么意思？就是披了一张 FDE 的皮，里子还是老一套。

最典型的是，国内很多团队，只抄了 FDE 最容易抄的那一样东西——驻场。把成熟平台删掉，把真实数据删掉，把业务负责人和一线用户删掉，把写回权限删掉，把产品回灌删掉，最后只剩一个工程师，坐在客户办公室里。

这叫什么？这不叫前线部署，这叫把外包又打包卖了一遍。

更离谱的是，没做过一天交付的人，也出来开班培训 FDE 了。为什么敢？因为不谈真实数据、不谈生产动作、不谈结果和复用，剩下的全是名词。名词堆一堆，谁都能讲三个小时。

这就是为什么我说，每个 FDE 都背着"行业责任"。

你做得好，客户会觉得"这群人靠谱"；你搞一次 cosplay，客户会觉得"FDE 也不过如此"。你每一次交付，都在给这三个字投票。这票投多了，整个行业的口碑，要么被你抬起来，要么被你砸下去。

技术能力决定你能走多快，但这条路能走多远，看你给身后的人留了什么。

一个被 cosplay 坏掉的行业，你个人再能打，也没人信你的买家会越来越多。反过来，你把交付做扎实了，客户口口相传，全行业跟着你沾光。

行业责任，说到底是一句很朴素的话：**别砸了后来人的饭碗。**

## 9.2 FDE 的职业成长与出路

底线讲完了。现在讲道路。

先泼一盆冷水：在中国，FDE 最危险的结局，不是找不到岗位。而是兴冲冲冲进去，最后把自己，干成了一个懂 AI 的驻场外包。

这句话，是我最想塞给每一个想做这行的人的警告。

### 别把自己干成驻场外包

什么叫干成驻场外包？我先给你看一个反例，你就懂了。

这是一个教育转化的场景。客户要"提高转化"，你就入场了。

先看一组真实数字。近三十天，新增线索 4090 个；有支付时间的，92 个；支付率大概 2.25%。再往下，D1 到课 1346 人，D1 交作业只有 322 人；D2 到课 1006 人，D2 完课 506 人；最后进入支付页的，50 人。

漏斗一路漏，漏得触目惊心。

现在问题来了：如果你是那个被叫去救火的人，你会怎么干？

驻场外包的思路是这样的：补话术、多追单、让机器人更像真人。D3 到 D6 的广播追单，写得更狠一点；AI 回复的语气，调得更像人一点；能自动挡的问题，多挡一点，好减轻人工压力。

你越干越忙，客户觉得你天天在干活。但漏斗的形态，纹丝不动。

因为根上的问题不在这。根上的问题是什么？

D1 到课 1346 人，交作业的只有 322 人，掉了一大半。为什么？因为那个"作业"，从头到尾就只是个作业。它没有完成一次启动诊断——没有搞清楚：这个用户每天能投入多久？他卡的是时间、预算、信任，还是"怕学不会"？

再看 D2 的人工接管率。外包第一反应是"接管率高=AI 不行，得让 AI 多挡点"。但事情没那么简单。

D2 的高人工接管，恰恰可能是离成交最近的信号。用户在这个节点，问的是分期、付款路径、权益确认、"我到底适不适合"。这些哪是噪音，这是成交信号。你把它们全交给 AI 自动回复，看似高效，其实是把最值钱的一批客户，亲手送走了。

所以真 FDE 的正解，不是"AI 全自动成交"，而是：AI 识别问题类型，人工收口关键点。D1 的作业改成启动诊断，D2 的高人工接管拆成成交信号，D3 到 D6 把广播式追单改成分层触达。

看出差别了吗？

驻场外包盯着的是"活儿"，是模型输出、是话术、是机器人像不像人。

FDE 盯着的是"状态、边界、责任、指标"，是用户有没有被推进到"问题被识别"的状态，是这个状态要不要人工收口。

这中间，差着一整层。所以有句话，我把它当成这章的题眼：

**驻场外包卖时间，FDE 卖把问题拆清以后，企业不再人肉扛。**

你把这话嚼透了，就知道自己该往哪条路上走了。

### 三个权力

怎么判断你现在干的到底是 FDE，还是变相的外包？我给你三个词，你对照照。

我管它们叫"三个权力"。你手里有几样，你就离 FDE 有多近。

### **第一个，场景定义权。**

客户来找你，第一句话是"我要个数字人""我要个知识库""我要个智能体"。你能不能不被这句话牵着走，而是往下问一句：你原来的流程，到底哪里坏了？

这就是场景定义权。用"客户说的需求"和"真实要的东西"之间的落差，最说明问题。

我讲个车企新媒体的例子。客户开口就是：数字人、无人直播、短视频脚本、多平台账号整合。

听起来要的是"数字人"。但你要真追下去，会发现他要的是另一套东西：总部想看所有直播间和短视频的数据，经销商账号要纳进监测，脚本要按车型、热点、平台规则来生成，直播数据要能反哺运营调整。

这才是真相。所以那句话特别狠：**企业买的不是数字人，是一条内容生产和数据复盘链路。**

外包会被这单活活拖死——帮总部拉数据、帮经销商配账号、帮内容团队改脚本、帮老板做看板。人一撤，系统还是散的。

FDE 拿到这单，第一件事是重画链路，连问六个问题：账号数据从哪来？脚本谁生成？品牌口径谁审？投放结果怎么回流？复盘建议怎么进下一轮选题？经销商和总部看的是不是一个指标？

六个问题问完，场景才算被你定义下来。而不是客户说啥你做啥。

### **第二个，边界定义权。**

客户说"做个内部知识库问答"，外包一听，立刻干活：收文档、切知识库、接企业微信、调提示词、测试问答、上线培训。一气呵成，特别有效率。

真 FDE 会停下来，往下追八个问题：文档版本谁负责？旧文件能不能用？不同岗位能不能看同一批材料？回答必须带引用来源吗？错了谁来纠正？哪些问题必须拒答？哪些转人工？上线后到底看准确率、使用率，还是看重复咨询下降了没有？

这八个问题，客户一个都答不上来，你就别急着做——说明这个"知识库"，连边界都没人想过。

追到底，一个知识库项目，其实就四件事：知识资产怎么进、权限边界怎么管、错误反馈怎么回、上线验收怎么看。

把这四件事钉清楚了，你才配叫边界定义权。不钉清楚，你做出来的问答系统，纸糊的一样。

这里藏着一句特别扎心的话，我也给你：**最贵的不是模型费，是返工费，是上线后没人用的沉默成本。**边界不定，返工和沉默，就等着你。

### 第三个，方法沉淀权。

这是三个权力里，最能决定你走多远的一个。

一场救火救完，你能不能把它，沉淀成下一次能复用的资产？表单、任务卡、验收指标、异议矩阵、流程包——这些别人抄不走的东西，才是你真正的身家。

没有方法沉淀权的人，永远在救火。今天这家客户救一场，明天那家救一场，救到最后，除了累，什么都没攒下。

有方法沉淀权的人，每救一场火，手里多一套家伙。第十场火救完，他已经不是原来的他了。

三个权力说完，我还给你三个自问。每次接活之前，对着自己念一遍：

我现在离钱近吗？我现在离现场近吗？我有没有机会，去定义验收、去沉淀方法？

三问里有一问是"否"，这活你就得掂量掂量，是不是又要滑回外包的老路了。

对了，还有两句红线，是我从这行血水里捞出来的，你背下来：

**客户不是为 AI 付钱。客户为少漏线索、少丢订单、少返工、少扯皮、少依赖某个老员工付钱。**

**你越靠谱，越容易被所有人叫去擦屁股。**

第一句告诉你价值在哪，第二句提醒你边界在哪。靠谱是你的资本，但擦不完的屁股，会把这个资本一点点烧没。

### 三条路

明白了三个权力，再聊出路。

FDE 干到一定年头，面前大致摆着三条路。没有哪条天然更高贵，看你的性子。

#### 第一条，垂直深耕。

挑一个你干得最顺手的行业，扎下去。教育转化、车企新媒体、制造业、工程咨询，随便哪个，做深了，你就不再是“会 AI 的人”，而是“这行里能把 AI 落地的人”。

这两者，身价差了不止一档。前者客户一抓一大把，后者客户愿意点名等你。

垂直深耕的底气，就是场景定义权和边界定义权。你在一个行业里滚了十几个项目，客户一张嘴，你就知道他要不是数字人，是一条链路；他说知识库，你知道要追哪八个问题。

#### 第二条，转产品、转售前。

FDE 这活儿，天然往产品经理和售前这两个方向流的。

你天天泡在客户现场，知道什么功能是客户真需要的、什么验收是客户会认的。这种手感，是坐着画原型的产品经理，求都求不来的。

而且 FDE 那个“往下追边界”的本事，正好是产品和售前最缺的。一个能把“知识资产怎么进、权限边界怎么管、错误反馈怎么回、上线验收怎么看”这四件事说明白的 FDE，转售前，是降维打击。

#### 第三条，创业。

这条路最难，也最不该被浪漫化。但真有人走通了，而且走通的，往往不是你以为的那批人。

### 中国市场的那点名堂

聊到创业和中国市场的特殊性，我得说点反直觉的观察。

你以为做得好的 FDE，是算法工程师、是架构师？不是。

在真实市场里活得滋润的那批人，多数不是技术科班出身。他们过去是做企业代运营的、做获客的、做出海陪跑的。

这批人没有正规技术团队，反而成了优势——因为他们天然不会去接"深度定制"这种承诺，他们只接自己 hold 得住的活：紧贴业务场景的轻量智能体，加落地陪跑。

他们怎么做的？极度懂业务场景，然后用 Codex、Agent 这类工具，vibe coding 自己搓智能体。卖的不是底层代码，是"把这套工具嵌进你的业务里，帮你省两个人、提效 30%"。

更关键的是，他们手里攥着多年积累的存量客户和信任关系。回头跟老客户说一句"我能帮你省两个人"，单子就签了。

这批人，深度绑定着 WorkBuddy、阿里、360 这些平台生态，吃厂商的导流订单。

所以他们活得滋润的秘密，一句话就点透了：

**帮别人用上 AI、用好 AI，就是 FDE 的本土化本质。**

这是中国市场跟海外最不一样的地方，也是你判断自己路该怎么走的重要参照。

具体到市场特殊性，我拆成三样，你记着：

**其一，计价模式不一样。**海外 Palantir 那一套，是大单、长周期、订阅加结果分成。中国本土，更多是"轻量智能体+陪跑"的定价，按效果、按场景收。你报一个纯技术的价，客户网上随便一搜，就有人报 2000 块——不是你的活不好，是比价场上，纯技术的东西早就被打到地板了。真正值钱的，是你懂业务、能落地的那部分，那个没人能跟你比价。

**其二，平台底座不一样。**海外 FDE 背后站着的是 Palantir、OpenAI 这种庞然大物。中国本土，你背后站着的是 WorkBuddy、阿里、360 这些平台生态。你吃的是它们的导流，用的是它们的底座，卖的是你自己那层"懂业务+陪跑"的功夫。选哪个平台底座，很大程度上决定了你的客从哪里来。

**其三，人才定价不一样。**纯会调模型的人，供过于求，价越来越贱；懂业务、能落地、能对结果负责的人，供不应求，价越抬越高。这中间的剪刀差，就是 FDE 这个岗位在本土市场的溢价来源。

看明白这三点，你再看前面那三条路——垂直深耕、转产品售前、创业——就知道该怎么给自己定价了。

你值多少钱，不取决于你会多少模型，取决于你离钱多近、离现场多近、有没有自己的验收标准和沉淀下来的方法。

这又绕回了那三个权力。

### 9.3 选择雇主与组织的重要性

讲完个人，讲组织。

因为 FDE 这个工种，个人的天花板再高，也架不住你在一个烂组织里，被活活耗死。

怎么判断一个组织值不值得待？我给你几条硬标准，都是拿真金白银换来的教训。

**第一，看它给不给你配齐“背后的人”。**

一个真正的 FDE，背后必须站着平台、产品、数据、业务负责人。光靠你一个人背着所有结果往前冲，那不是 FDE，那是 cosplay。

前面讲行业乱象的时候说过，国内很多团队只抄了个“驻场”，把平台删了、把数据删了、把产品回灌删了，只剩一个工程师坐客户办公室。你要是进了这么个组织，再能打，也是一个人扛一座山。

**第二，看它敢不敢让你碰真实数据、真实业务负责人。**

一个组织要是从头到尾只让你看脱敏表、只让你跟 IT 对接、从不让你见业务负责人和一线用户，那你要警惕了。它要的不是 FDE，是一个能上大屏的摆设。

真 FDE 的命根子，是现场。没有现场，就没有场景定义权，没有边界定义权，你干到最后，还是个外包。

**第三，看它给不给你方法沉淀权。**

接完项目，它是让你把救火过程沉淀成资产——表单、任务卡、验收指标、异议矩阵，还是让你歇口气，接着扑向下一个火场？

只压榨、不沉淀的组织，干十年，你也不过是从年轻外包变成老外包。肯让你沉淀的组织，干三年，你手里就有一整套能复用的东西。这三年的差距，是十年都补不回来的。

**第四，看它验收的时候，是跟你一起扛结果，还是把锅全扣你头上。**

这个不用多解释。一个会在客户面前跟你抢功、会上把失败全甩给你的组织，技术再强，也不值得。FDE 卖的是"对结果负责"，结果既不是功劳簿，也不是甩锅墙，你总得跟它算清楚。

**第五，看它嘴上说的"结果导向"，会不会在季度末变成"工时导向"。**

有些组织，口口声声结果导向，一到考核，就开始数你的工时、数你的驻场天数。这说明它骨子里，还是把你当卖时间的。

真正尊重 FDE 结果的组织，考核你的是"客户续了没""价值证明了没""方法沉淀了没"，而不是你几点打卡。

五条标准，你对着照。照出来底色是暗的，趁早走。

因为 FDE 这个岗位特殊就特殊在：你的个人品牌，是写在每一次交付上的。你待在一个不尊重这份结果的组织里，它消耗的不仅是你的时间，是你在这个行业里的信誉。

## **收官：这门手艺，一句话说全**

书读到这，九个章节，走完了。

我很想把整本书，最后收成一段话。不是为了漂亮，是为了让你合上书之后，脑子里能留下一个完整的骨架。

一个 FDE，进了客户的门，先画清楚五张地图——数据、流程、组织、系统、政治，摸清底再动手。然后找一个真问题，用三关检验过一遍：疼不疼、值不值、做不做得动。问题对了，用一个窄而完整的 MVD 切进去，用真实数据，缩范围不缩价值，定死截止时间。

东西做出来不算完，你得让它真的被用起来，寄生在用户熟悉的界面里，让"什么都不做"就是正确结果。用起来了，再守续约、扩收入，盯住那些会流失的信号。

等一个项目稳了，别止步——把救火沉淀成资产，复用到下一个、再下一个客户，从定制走向平台。这一路，技术选型要服务于"可计算的问题"，别追新。

而托着这一切的，是最后这一章讲的：数据主权、结果诚实、不制造依赖、善待被替代的人、守住合规、给行业留口好名声。

这，就是一个 FDE 的完整方法论。

但我最后还想补一句，可能是全书最要紧的一句。

前面八章，教你怎么走快。这一章，教你怎么走远。

你会碰到无数个让你"快一点"的诱惑：数据敞开了，顺手拉一把；数字难看了，往上报一点；客户绑住了，续约稳一点；饭碗动了，装看不见一点；审查紧了，糊弄一点；cosplay了，热闹一点。

每一个"一点"，都让你当下快一点，也都让你以后短一点。

所以开篇那句话，我把它再放回这里，作为全书的落点：

**技术能力决定你能走多快，职业底线决定你能走多远。**

你这一路，靠技术敲门进去，靠诚实留下来，最后，靠底线，走出去。

## 章末工具箱

### 工具：职业道德自查清单

这是一份给你自己用的清单。不是交差的表，是隔一段时间，拿出来对着自己过一遍的镜子。每条没有标准答案，但你心里得有个数。

#### 一、数据主权

☐ 我只拿了本次交付必需的数据，没有"顺手"多拉别的吗？

☐ 数据存放位置、谁能访问、多久删除，有白纸黑字的约定吗？

☐ 样本结项后，有没有当面清空、不留备份？

☐ 数据要不要出域、出到哪，用户清楚并同意了吗？

#### 二、结果诚实报告

☐ 我报告的数字，证据是哪一类（要求/计算/模拟/实测/供应商声明/认证）？我说清楚了吗？

☐ 有没有把"供应商声明"当成"客户会认的证据"写进方案？

☐ 那些失败的、难看的、出错的数字，我报告了，还是咽下去了？

[ ] 一句话说出去之前，它经得起别人重新检查吗？

### 三、避免制造依赖

[ ] 我有没有故意把系统做成黑盒、扣着文档不给客户？

[ ] 我的价值，是"客户离不开我"，还是"客户还想继续用我"？

[ ] 我交付时，有没有把方法、文档、资产沉淀给客户自己？

[ ] 客户今天换掉我，它还能不能运转？不能的话，是因为我优秀，还是因为我拴了根绳？

### 四、关注被替代人员

[ ] 我的方案动到哪个具体岗位的人？我想过他的去路吗？

[ ] 方案里，有没有给他留转岗、学习新技能、做复核兜底的位置？

[ ] 那些只在他脑子里的隐性知识，有没有变成训练数据、规则、或他教别人的东西？

[ ] 我有没有拿"优化人力"当卖点去忽悠老板？

### 五、合规边界

[ ] 数据出域、私有化、多租户隔离、PII 脱敏、SSO 权限、审计留痕，这几样我过了一遍吗？

[ ] 有没有在安全审查上动过"变通"的小心思？

[ ] 过不去的审查，我是停下来改方案，还是绕过去硬上？

### 六、行业责任

[ ] 我这次交付，是让人更信 FDE 这三个字，还是更不信？

[ ] 我有没有在玩 cosplay——只驻场、没数据、没业务负责人、没产品回灌？

[ ] 我不做这件事，会不会只是因为"不谈真实数据、不谈生产动作、不谈结果，剩下的全是名词"？

### 临接活前的三个自问

[ ] 我现在离钱近吗？

[ ] 我现在离现场近吗？

[ ] 我有没有机会，定义验收、沉淀方法？

三个都答不上来，先别急着接。守住底线，也守住自己的路。

<div style="page-break-after: always"></div>

## 附录 A FDE 常用指标清单

指标不是用来交差的报表，是用来照妖的镜子。它照出你到底是"在交付价值"，还是在"自欺欺人地堆人干定制"。

### A.0 怎么用这份清单

先说三条使用纪律，不然这份清单会变成又一份躺在收藏夹里的"干货"。

**第一，指标要少而狠，不要多而全。** 一个 FDE 项目，真正值得盯的指标不会超过五个。多了，人就会挑好看的看，反而看不见危险。挑出和"客户是否变好、我们是否在复制"直接相关的那几个，盯死。

**第二，趋势比绝对值重要。** 你有多少客户、毛利率是 30% 还是 50%，单看一个数没意义。有意义的是方向：它在往上走，还是在往下掉。所以每个指标，我都会讲"什么时候该慌"。

**第三，指标要能落到责任人。** 一个没人负责、也没人因为它的变化而改变动作的指标，是废指标。下面每个指标，我都会顺带说一句"它变了，谁该动"。

### A.1 时间类指标

#### TTV (Time To Value, 达到首个价值的时间)

**定义：**从项目正式开始，到客户第一次真实地、可感知地"得到了价值"（比如某条流程第一次跑完、某份活第一次不用人工、某个结果第一次自动出来），中间隔了多少天。

**怎么算：**从签约/开工日，到"首值日"的自然天数（不是工作日）。

**采集方式：**别用"上线日"冒充"首值日"。上线不代表有价值。要在第一刀 MVD 定义单里就写死"什么结果算首值"，然后用那个结果第一次真实发生的日期。

**使用建议：**这是 FDE 项目管理里最该早盯、也最容易被跳过的指标。决定客户信不信任你的，往往就是"多快让他看到第一下变化"。TTV 越短，后面越顺。

**什么时候该慌：**开工一个月了，客户还没见过任何一个"以前要两小时、现在十分钟"的瞬间。那大概率是问题又选大了。

## 首日复活率 / 首周留存（激活期）

**定义：**系统上线后，第一天用过的用户，第七天还在用的比例；或者上线第一周活跃、第四周仍活跃的比例。

**怎么算：**周活/首周活。分母是"上线第一周真正打开用过的人"，分子是"这批人到第四周还在用的人"。

**采集方式：**必须在系统里埋使用埋点，哪怕是最简单的登录、点击、调用日志。没有埋点，这个指标就是空的。

**使用建议：**这是"激活"这一关的体温计。它直接回答第五章讲的"首日魔咒"有没有被破解。

**什么时候该慌：**首周复活率低于三成，说明用户没把它当工具，只是当天被拉着开了个会。这时候该做的不是加功能，是去现场看"为什么不用"。

## A.2 转化与商业化指标

### PoC 转化率（PoC → 生产/付费）

**定义：**做过的概念验证（PoC）里，有多少真正转成了生产部署或付费合同。

**怎么算：**转正的 PoC 数 ÷ 总的 PoC 数。分子要以"进了生产、有真用户在真用"为准，不是"签了个口头意向"。

**采集方式：**在 CRM 或你自己的项目表里，给每个 PoC 记录一个终态（转正/搁置/死亡），并注明死亡原因。

**使用建议：**这个指标能反过来检验你"选问题"的水平。PoC 转化率持续很低，问题通常不在演示能力，而在进场时就选错了题——见了太多"演示好看、上线即死"的客户。

**什么时候该慌：**连续几个 PoC 都停在演示、没人买单。停下来复盘"问题是不是没选对"，比继续接下一个 PoC 更有用。

## 续约率 / 流失率

**定义：**到期客户里，愿意继续付费的比例（续约率）；或者到期后不再续的比例（流失率）。

**怎么算：**续约客户数 ÷ 到期客户数。口径要区分"按客户数"还是"按金额"——按金额更实，因为一个大客户流失顶十个小的。

**采集方式：**到期前 90 天就要开始跟踪续约风险，别等合同到期那天才知道。

**使用建议：**续约是"部署当天就开始输赢"的指标。配合第六章的《客户健康分评估表》和《续约风险排查清单》一起用。

**什么时候该慌：**健康分掉到预警线、关键支持者离职、联系人开始不接电话，这些信号出来了，续约就危险了。

## NRR (Net Revenue Retention, 净收入留存率)

**定义：**同一批老客户，今年带来的收入，是去年的百分之多少。它把"续约 + 扩张 + 降级 + 流失"四个动作拧成一个数。

**怎么算：** $(\text{期初老客户的收入} + \text{扩张收入} - \text{降级/流失损失}) \div \text{期初老客户收入} \times 100\%$ 。  
超过 100% 说明老客户越做越值钱；低于 100% 说明在失血。

**采集方式：**按客户归集收入，追踪每个老客户今年 vs 去年的收入变化，标注是"扩张"还是"降级"。

**使用建议：**这是第六章"守住续约与扩大收入"的总账本。NRR 能不能稳定站上 100% 甚至 110%、120%，基本决定了一家做企业 AI 服务的团队是在增长，还是在永远替流失客户打工。

**什么时候该慌：**NRR 长期在 100% 以下，即便新签客户在涨，整体也危险。因为你在用一个漏水的桶打水。

## A.3 经营与复制指标

### 交付毛利率 (Gross Margin)

**定义：**单个项目（或一段时间所有项目）的收入，扣除直接交付成本后，剩下的比例。

**怎么算：** $(\text{收入} - \text{直接成本}) \div \text{收入}$ 。直接成本要算全：人力人天、差旅、外包、算力、给客户的返点。最容易漏的是“服务期的隐性人力”。

**采集方式：**每个项目单独记成本，不要让销售和交付共用一个大池子算不清。算不清，毛利率就是假的。

**使用建议：**这是第七章“从定制到平台”的核心体检指标。纯定制的毛利率天然低，因为成本都是不可复用的人天；开始组件化、产品化后，毛利率会随复用而上升。看趋势，不看单点。

**什么时候该慌：**项目越接越多、毛利率却一路下滑——说明你在“堆人放大”，不是在“复制放大”。这就是第七章 C3 总包暴雷的前兆。

### 人均营收 (Revenue per Head)

**定义：**一段时间内的总收入，除以交付团队的平均人数。

**怎么算：** $\text{营收} \div \text{团队 FTE (全职等效人数)}$ 。人数要算真正参与交付的人，不是把行政、销售也混进来。

**采集方式：**按月或按季度看，和毛利率放一起看。

**使用建议：**它回答一个核心问题——团队每多一个人，是线性地多一分钱，还是多出来的钱在变多？前者是卖时间，后者是复制起了作用。

**什么时候该慌：**人均营收停滞甚至下降，同时你还在不断扩人。这是最典型、也最容易被“忙”掩盖的死法。

### 客户健康分 (Health Score)

**定义：**一个综合分，用来提前预判客户会不会续约、会不会扩张、会不会流失。

**怎么算：**把几个信号加权合成——使用活跃度、价值达成度、关键人关系稳定度、客户自身经营状况、竞品/替代出现情况。权重按你所在行业调。

**采集方式：**与《客户健康分评估表》（第6章）对齐，每月或每季度评一次，记录趋势。

**使用建议：**健康分的价值不在“得分”，在“预警”——在客户说出“我们不续了”之前好几个月，你就该看得到信号，然后去干预。

**什么时候该慌：**健康分连续两个周期下滑，或者客户经营基本面恶化（参考第六章 C13 回款风险）。

## A.4 质量类指标（针对 AI 输出本身）

### 准确率 / 错误率

**定义：**AI 输出的结果里，正确（或错误）的比例。

**怎么算：** $\text{正确样本数} \div \text{总样本数}$ 。关键是“谁来判对错”——必须有一个可复核的标准或人工抽检，否则准确率是自己哄自己。

**采集方式：**抽检 + 用户反馈（用户点了“这条不对”）双路采集。

**使用建议：**别只盯总体准确率。真正伤人的是“高置信度的错误”——AI 用自己的语气说了一件错事，用户还信了。第六章 C12 那条“AI 非常准确地处理错误知识”就是最典型的坑。

**什么时候该慌：**错误率不高，但错的都是关键环节；或者用户开始私下吐槽“这玩意儿不靠谱”。

### 人工接管率 / 转人工率

**定义：**本可由 AI 处理、但最终转给人工的比例。

**怎么算：** $\text{转人工次数} \div \text{AI 处理总数}$ 。

**采集方式：**在系统里记录每一次“转人工”动作和原因分类。

**使用建议：**别一看到接管率高就想“让 AI 多挡一点”。有些接管是噪音，有些是金矿（比如成交前用户问付款、权益，其实是正信号）。要区分接管的原因，分级处理。第四章 4.5 教育转化案例讲的就是这个。

**什么时候该慌：**接管率突然飙升，且集中在某一个环节——说明那个环节的 AI 出了系统性错误，赶紧修。

证据分级（针对方案承诺）

**定义：**把交付方案里的每一条承诺，标上它属于哪一类证据（而非笼统的"我们支持"）。

**怎么算：**对照第八章 C9 六类——REQUIREMENT（客户明确要求）、CALCULATED（按公开假设计算）、SIMULATED（记录输入条件的模拟）、MEASURED（可重现实测）、SUPPLIER\_DECLARED（供应商声明）、CERTIFIED（有认证记录）。

**采集方式：**写方案时就给每条能力打标签，注明"这条的证据是什么、能不能被复查"。

**使用建议：**六类不分高低，真正要防的是互相冒充——供应商写"支持私有化"，不等于你通过了自己的安全审查。这条指标直接服务"结果诚实"的底线。

**什么时候该慌：**你发现自己或团队在演示里把"供应商声明"说成了"我们实测过"。

A.5 一份拿来就能用的指标看板

把上面最关键的几个，压成一张季度看的看板：

| 维度   | 核心指标       | 健康信号         | 危险信号        |
|------|------------|--------------|-------------|
| 交付首段 | TTV        | 越来越短         | 开工一个月没首值    |
| 激活   | 首周留存       | 逐步爬升         | 首周复活率 < 30% |
| 转化   | PoC 转化率    | 稳定且偏高        | 连续 PoC 停在演示 |
| 留存   | NRR / 续约率  | ≥ 100% / 高续约 | 长期 < 100%   |
| 质量   | 准确率 + 接管率  | 双路监控         | 高置信度错误增多    |
| 复制   | 毛利率 + 人均营收 | 同步上升         | 越接越多毛利率越低   |

看板不是给老板看的，是给你自己看的。照一次，你就能看清自己卡在哪一环。

下一份附录 B，把散落在九章"章末工具箱"里的全部模板，逐章全文收进一个文件，方便你随手翻找——它是本书的模板总目录。

<div style="page-break-after: always"></div>

## 附录 B 全流程交付 SOP 模板合集（逐章全录版）

这一版，把散布在九章“章末工具箱”里的模板，全部原样收进一个文件。目的只有一个：你翻这本书的时候，不用把手上的项目放下、再去第九章找一张表。

模板不是用来收藏的，是用来在正确的时刻拦住你的。所以每份模板前面，我都会用一句话告诉你：它该在哪一刻拿出来用。

### B.0 怎么用这本附录

**第一，按阶段翻，不按页码翻。** 你的项目处在哪一环（进场？定义问题？激活？续约？），就翻对应的模板。别把整本看完才动手，那又成了收藏。

**第二，模板上那些“问号”，比“勾”值钱。** 勾代表你自认为做了；那些答不上来的问号，才是指出你真正风险的东西。一副模板填完，全是问号的地方，就是你下一步该先去啃的硬骨头。

**第三，别把“填完”当“做完”。** 模板的意义，不是让你交差，是逼你在正确的时间，问出那个正确的问题。问出来了、并且带着答案去行动了，模板才算起了作用。

### B.1 模板总表（T01 - T14）

| 编号  | 模板名称           | 来源章节  | 一句话用途                   |
|-----|----------------|-------|-------------------------|
| T01 | FDE 岗位能力自评表    | 第 1 章 | 给自己“技术/翻译/主人翁”三维度打分     |
| T02 | FDE 能力模型五阶段对照表 | 第 2 章 | 定位自己在哪个成长阶段             |
| T03 | 交付物 vs 结果一句话自查 | 第 2 章 | 汇报前问一句“客户哪个动作变了”        |
| T04 | 进场尽调问卷（五张地图版）  | 第 3 章 | 进场前列清单，摸清数据/流程/组织/系统/政治 |
| T05 | 灯塔客户四问         | 第 3 章 | 签单前问清谁出钱/拍板/受益/抵制       |
| T06 | 三关检验清单         | 第 4 章 | 需求先过“痛点/经济性/可行性”三关再动手   |

|     |                     |       |                            |
|-----|---------------------|-------|----------------------------|
| T07 | MVD 定义单             | 第 4 章 | 把第一刀"最小可行部署"的范围钉死          |
| T08 | 缩小范围的三个提问           | 第 4 章 | 范围膨胀、不知道砍哪里时用它             |
| T09 | 激活清单 + 健康度评估表       | 第 5 章 | 上线前后让用户真正用起来、持续盯活          |
| T10 | 客户健康分评估表 + 续约风险排查清单 | 第 6 章 | 提前预判客户会不会流失、续不续约           |
| T11 | 产品化评审清单 + 项目复盘模板    | 第 7 章 | 判断能力值不值得做成产品、沉淀项目知识        |
| T12 | 技术选型决策树 + 本体建模检查清单  | 第 8 章 | 选 RAG/Agent/建模前先过一遍、建模时防漏问 |
| T13 | 职业道德自查清单            | 第 9 章 | 六条底线逐条自查                   |
| T14 | SOW（工作说明书）模板        | 附录 B  | 签合同前把范围、验收、边界写死            |

（B.1 之后的正文，T01–T13 就是各章工具箱的原样全文，T14 是全书正文没放下的完整模板。）

## B.2 逐章模板全录

### T01 FDE 岗位能力自评表（第 1 章）

对照每一项，给自己打 0–5 分。分数不代表水平高低，只代表你在这一项上的"被验证程度"——你做过什么，才算真的拥有它。别客气，也别吹牛，这张表只有你自己看。

#### 技术通才

- ☐ 能在一个不熟悉的技术栈里，快速跑通一个最小可用的东西（0–5）
- ☐ 遇到没文档的老系统，能靠翻代码、问人、试错把接口摸出来
- ☐ 能独立完成从数据接入到部署上线的一条完整链路
- ☐ 对 RAG、Agent、工具调用等概念，不止"听过"，真实做过
- ☐ 面对烂数据（纸质、对不上账、口径不一），有动手清洗的路子，而不是只会抱怨

#### 业务翻译能力

- [ ] 能把客户一句含糊的需求，反推成具体要解决的业务问题
- [ ] 能说清"客户以前怎么做、现在怎么做"之间的差别
- [ ] 能和业务负责人用业务语言对话，而不是一上来谈技术方案
- [ ] 能听懂、也能表达"这个方案能为客户省多少钱 / 赚多少钱"
- [ ] 客户说一个炫酷的词（数字人、中台、智能体平台），你能往下追出他真正的痛点

### 主人翁意识

- [ ] 项目上线后没人用，你会觉得"这是我的责任"，而不是"客户的问题"
- [ ] 你衡量自己成败的标准，是客户是否真正用起来、产生了价值
- [ ] 你愿意为最终结果负责到底，而不是做完自己的模块就撤
- [ ] 遇到阻力时，你倾向于去现场解决，而不是发个邮件了事
- [ ] 系统静默死亡了，你会睡不着，而不是觉得"至少交付完成了"

### 怎么用这张表

如果"技术通才"高、后两项低，你目前是个不错的工程师，距离 FDE 还差"翻译"和"负责"两步。

如果"业务翻译"和"主人翁意识"都高、技术偏弱，你其实已经具备了 FDE 最稀缺的内核，技术可以补。

三项都不高也没关系——每一章都会把每一项拆成你能练的动作。

## T02 FDE 能力模型五阶段对照表（第 2 章）

| 阶段       | 要练的能力               | 最典型的标志          | 对应本书章节  |
|----------|---------------------|-----------------|---------|
| 1. 技术基础  | 编码、模型 API、RAG、Agent | 能独立做出一个能跑的 demo | 全书前提    |
| 2. 企业门槛  | 遗留系统、接口、安全合规、私有化    | 能在真实企业环境里把东西跑起来 | 第 3、4 章 |
| 3. 运营成熟度 | 稳定性、监控、故障恢复、热修复     | 上线后系统能稳定运营      | 第 5 章   |

|           |                |                  |       |
|-----------|----------------|------------------|-------|
| 4. 软技能与商业 | 客户沟通、信任、ROI、计价 | 能算清账、谈得下、续得上     | 第 6 章 |
| 5. 对外证明   | 可传承、组件化、产品化、复制 | 项目能沉淀成资产、复制到多个客户 | 第 7 章 |

### 用这张表给自己定位

如果你在 1、2 层之间：先把企业门槛补上，别再满足于"跑通 demo"。去找一个脏、乱、旧的真实环境练手，而不是再做一只漂亮的玩具。

如果你卡在 3 层：说明你缺的不是技术，是对"上线之后"的关注——往下读第 5 章。

如果你 1-4 层都不错、唯独 5 层乏力：你的天花板在"复制"，第 7 章是给你的。

如果四层都很强，却总觉得自己在当高档外包：你多半不是能力不够，是没拿到"定义"和"沉淀"的那点权力。

## T03 交付物 vs 结果一句话自查（第 2 章）

每当你做完一件事、准备汇报之前，先问自己一句：

**这个月，客户的哪一个具体动作，变了？**

答得上来，还能用一个数字撑住（"客服找资料从两小时到秒回""漏单从上月 17 单掉到 3 单"），你交付的是结果。

答不上来，只说得出一堆"我建了什么""我上线了什么""我优化了什么"，那你交付的，多半还只是交付物。

（第 2 章还给了"四个岗位一句话记住边界"，作为 FDE 定位的速记：算法工程师对模型负责，销售工程师对签单负责，交付顾问对合同负责，FDE 对结果负责。）

## T04 进场尽调问卷（五张地图版）（第 3 章）

以下问题，进场前逐条过。答不上来的，先别签合同，先回去摸底。宁可在签单前多花三天，别在进场后填三个月的坑。

### 数据地图

[ ] 关键数据存在哪？是数据库、Excel、纸质单据，还是只在某个人脑子里？

[ ] 数据的形态，AI 能不能直接读？要不要先做录入、识别、结构化？

[ ] 谁拥有数据权限？拿到数据要过谁那一关？那关好不好过？

[ ] 数据质量如何？字段全吗？口径统一吗？

[ ] 要不要清洗？清洗的工作量和成本，估过吗？

[ ] 有没有"纸面上对、实际上在人脑子里改"的隐性数据？

### 流程地图

[ ] 核心业务线从起点到终点，经过哪些步骤、哪些人？

[ ] 从头到尾走一遍，最快多久？最慢多久？差异在哪？

[ ] 哪些环节是高度重复、可标准化的？

[ ] 最慢、最易出错、最依赖某个人的环节，是哪一个？

[ ] 名义流程和实际流程，是一套还是两套？

[ ] 有没有"嘴上走系统、实际靠微信"的环节？

### 组织地图

[ ] 这件事，谁最终拍板？谁出钱？

[ ] 做成了谁受益、谁升职？做砸了谁背锅？

[ ] 谁可能因为这件事丢饭碗、丢权威，从而成为抵制者？

[ ] 一线的真实用户是谁？他们现在是怎么干活的？

### 系统地图

[ ] 现在在跑的有哪些系统？用的什么技术栈？

[ ] 哪些系统有接口、哪些是黑盒？接口要接，找谁开？

[ ] 关键系统的文档还在吗？当年的开发商还在吗？

[ ] 有多少技术债？接一个老系统，大概要花多少成本？

## 政治地图

[ ] 预算从哪个部门出？这个部门图的是什么？

[ ] 涉及部门之间有没有过节？谁和谁不对付？

[ ] 这个项目，是否被某个人当作政绩或政治工具在用？

[ ] 有没有谁，表面支持、暗地里会拖后腿？

## T05 灯塔客户四问自查（第 3 章）

签单前，把下面四件事问清楚。问不清的，这个客户就不值得你花大力气。

**谁出钱？**——钱从哪个预算走，账期多久。

**谁拍板？**——真正有决策权的那个人，我见过了吗。

**谁受益？**——做成了，谁得利、谁升职，他愿不愿意推。

**谁抵制？**——做成了，谁失权、谁丢饭碗，他会怎么拦。

## 五张地图一张表速查

| 地图   | 回答的问题          | 画不出来会怎么样          |
|------|----------------|-------------------|
| 数据地图 | 数据在哪、归谁、质量如何   | 一头扎进数据治理无底洞，洗到超预算 |
| 流程地图 | 业务怎么流、瓶颈在哪     | AI 落不进真实工作流，做了没人用 |
| 组织地图 | 谁拍板、谁受益、谁抵制    | 找不到真甲方，被隐形抵制者拖死   |
| 系统地图 | 遗留系统、接口、技术债    | 低估集成成本，一半预算拿去修旧账  |
| 政治地图 | 部门利益、预算归属、隐性阻力 | 站错队，把做成的事做成政治牺牲品  |

**绘制顺序：**先画数据地图和系统地图（偏硬，靠查、靠访、靠要清单就能画）；再画流程地图（靠现场跟单）；最后画组织地图和政治地图（靠吃饭、私下聊、观察，急不得）。记住一条：硬的地图先画，软的地图后画。

## T06 三关检验清单（第 4 章）

拿到需求后，逐条过一遍。任何一条回答不出来，先回去问，别急着动手。

## 痛点检验

- ☐ 这个问题"现在"正在让谁付出什么样的真实代价？
- ☐ 这个代价能不能被量化（时间、人工、错误率、钱）？
- ☐ 如果不解决，到年底会有人因此被扣钱 / 多加班 / 丢客户吗？
- ☐ 客户说的是具体现象，还是一场宏大想象？

## 经济性检验

- ☐ 客户愿意为这个问题单独掏钱吗（不是打包买单）？
- ☐ 解决问题省下的钱或赚到的钱，够不够覆盖投入？
- ☐ 前置成本（数据治理、接口打通、采买）是不是已经超过合同额本身？
- ☐ 如果不做 AI，这个问题还能撑多久、代价多大？

## 可行性检验

- ☐ 需要的数据真的拿得到吗？在谁手里？谁有权限？
- ☐ 业务负责人愿意为结果负责吗？
- ☐ 依赖链有多长？最关键的那一环，卡在谁身上？
- ☐ 在客户给定的时间、安全、权限约束里，做得出来吗？

## T07 MVD 定义单（第 4 章）

| 项目        | 怎么填                   |
|-----------|-----------------------|
| 要解决的那一个价值 | 用一句话说清"以前怎么做，现在怎么做"   |
| 明确不做的范围   | 哪些环节、系统、用户，这次坚决不碰     |
| 真实数据来源    | 数据在哪、谁提供、质量如何、有没有隐藏的坑 |
| 截止时间      | 定死一个时间，用时间倒逼取舍        |
| 完成标志      | 什么结果，算"这一刀真的切进去了"     |

## T08 缩小范围的三个提问（第 4 章）

当范围膨胀、不知道砍哪里的时候，逐个问自己：

砍掉它，客户还能不能完整解决那个核心麻烦？能，就砍。

留下它，是为了客户的价值，还是为了我们自己的面子？为了面子，就砍。

它能不能放到下一轮再做，而不会让这一轮的价值打折？能，就放到下一轮。

（第4章还给了两种帮你想清问题的句式：问题三层——表层诉求 / 直接障碍 / 根本原因；"谁、被什么、困扰到什么程度"的问题陈述句式。）

## T09 激活清单 + 健康度评估表（第5章）

这章的两个模板，一个在上线前用，一个在上线后用。前者把"激活"安排明白，后者持续盯着它还活没活着。把它们当成值班本，翻起来才值钱。

**激活清单（上线前后逐条过，任何一条没做到，激活都可能功亏一篑）**

降门槛：

☐ 系统是否出现在用户已经在用的界面里（企微 / 钉钉 / Excel / OA），而不是一个独立新平台？

☐ 用户是否需要额外的学习、配置，才能开始用？

☐ "什么都不做"是不是就能得到正确结果（默认值设计）？

☐ 有没有引入用户本来不用懂的新概念（RAG / Agent / 提示词）？

☐ 用户第一次用，能不能在几分钟内尝到"更省事 / 更快 / 更准"的甜头？

保质量：

☐ 有没有一套可量化的质量指标（准确率、采纳率、节省时间）？

☐ 指标是不是能自动采集，而不是靠用户填问卷？

☐ 有没有机制，第一时间发现真实使用中被打标记的"烂输出"？

☐ 发现问题后，能不能快速修复（改提示词 / 补知识 / 调规则）？

过集成：

☐ 哪些系统是价值必需、必须硬啃的，哪些可以暂时绕开？

[ ] 卡在某个敌对部门或老系统时，是绕、是啃、是借力，有没有提前想好？

[ ] 系统是不是按"为失败而设计"来搭的——模型挂了会降级、会走缓存、用户不会被中断？

**健康度评估表（上线后每周扫一眼，看见危险信号就动手）**

| 维度   | 健康信号           | 危险信号            |
|------|----------------|-----------------|
| 使用频次 | 日活稳定或上升        | 日活一周内掉到个位数      |
| 使用深度 | 用户开始依赖、主动提需求   | 只点开看看，从不真正使用    |
| 价值感知 | 用户能说清"比以前省了什么" | 用户说不出到底有什么用     |
| 主动传播 | 新用户来自老用户推荐     | 全靠行政命令才有人用      |
| 反馈   | 用户会主动报错、提改进    | 错了也没人吭声（可能是不用了） |
| 质量水位 | 被标记的烂输出在减少     | 同一类错误反复出现、无人修复  |

用法：每周挑一天，五分钟，瞄一眼每一行落在哪一列。只要有一行滑进右边那一列，就说明有个地方正在失血，别等月底。

## T10 客户健康分评估表 + 续约风险排查清单（第 6 章）

**客户健康分评估表**——给每个客户定期打分（0-100），四大类信号。跌破 70 分，触发预警，主动介入。

使用信号（30 分）：

[ ] 日活 / 周活是稳定上升，还是持续下滑？（10 分）

[ ] 使用深度是在加深，还是变浅？用户从"每天用"退到"想起来才用"了吗？（10 分）

[ ] 有没有出现"核心用户弃用"的红旗？原来天天用的骨干，是不是半个月不登录了？（10 分）

组织信号（25 分）：

[ ] 内部支持者是否还在位？最近有没有升职、调岗、跳槽？（10 分）

[ ] 关键对接人换了吗？新对接人对项目是什么态度？（8 分）

[ ] 决策层对这个项目还关注吗？还是已经无人过问？（7 分）

价值信号（25 分）：

- [ ] 客户能否脱离 PPT，说清"省了多少钱、解决了什么麻烦"？（13 分）
- [ ] 有没有新的价值点，被持续创造出来？还是价值已经一年没更新？（7 分）
- [ ] 客户最近报的错、提的问题，得到了及时响应吗？（5 分）

财务信号（20 分）：

- [ ] 客户自身经营是否稳定？行业是在扩张还是收缩？（10 分）
- [ ] 回款是否准时？有没有从 30 天拖到 60、90 天的苗头？（10 分）

打分说明：80 分以上健康，正常维护；70-80 观察区，找出失分项主动补漏；70 分以下预警区，立刻回现场，按"捡使用、捡支持者、捡价值"三件事逐一排查干预。

**续约风险排查清单**——续约前三个月逐条过，任何一条回答为"是"，都当成待处理的红色信号。

- [ ] 合同到期日是几号？我提前了多久开始准备续约？
- [ ] 客户内部支持者是谁？他最近有没有流动的迹象？
- [ ] 最近三个月，系统的使用数据，是升还是降？掉得有多快？
- [ ] 客户最近一次主动说"这系统有用"，是什么时候？
- [ ] 竞争对手有没有在接触这个客户？有没有人报过更低的价？
- [ ] 客户能在一分钟内，说清这套系统帮他省了多少钱吗？
- [ ] 客户自己的经营状况、付款能力，最近有没有恶化？回款变慢了吗？
- [ ] 我手上，还有没有客户自主可控的数据出口？（防"绑死"反弹）
- [ ] 如果客户明天说"不续"，我知道真正的原因是什么吗？

三个自问（每次续约前问自己）

这个客户，我盯的是合同到期日，还是健康分？只盯到期日的，是赌运气；盯着健康分的，是把续约攥在自己手里。

我是在谈“价格”，还是在补“价值”？客户说贵，往往是价值没兑现的借口。先补价值，再谈价格。

我帮客户“变值了”，还是让客户“变怕了”？前者带来复购，后者埋下反噬。健康的扩大，永远让客户觉得值，也让他觉得自由。

## T11 产品化评审清单 + 项目复盘模板（第 7 章）

**产品化评审清单**——决定要不要把某个能力产品化之前过一遍。任何一条答不上来、或者答案是反的，先别急着投钱投入。

[ ] 有没有跨客户的共性？是“这一家”的问题，还是“这一行”的问题？

[ ] 能不能报出三家以上有同样需求的目标客户？

[ ] 把它泛化成通用产品的成本，高不高？和未来三年它能省下的钱比，值不值？

[ ] 做成产品后，客户能否开箱即用（或接近开箱即用），而不是还依赖大量人工驻场？

[ ] 交付后还需要多少“现场空间”？适合“纯产品”还是“产品 + 服务”？

[ ] 现有客户的反馈，是否已经印证了“这类需求反复出现”？

[ ] 这件产品，会不会反过来吃掉你现有定制业务的利润和口碑？（产品化要算总账，不是只看新产品能卖多少）

**项目复盘模板**——每个项目结束或阶段结束，开两个小时复盘会，回答下面四组问题，产出一页纸的沉淀。

坑与根因：

这个项目踩了哪些坑？

每个坑的根本原因是什么？（数据？流程？组织？系统？还是我们自己的成功标准没定清？）

有效做法：

哪些动作是有效的、值得复用的？

哪些可以固化成流程、清单、或组件？

通用 vs 特有：

哪些问题 / 做法是"这个客户特有"的？

哪些是"整个行业通用"的？（这一条，是产品化的原始素材）

下一步沉淀：

我能把什么抽出来，做一个可复用的组件？

有没有潜力升级成一个产品？泛化成本大概多少？

这次沉淀，交给谁、什么时候做完？（不给任务、不定时间，复盘就是闲聊）

两个工具怎么配合用？项目结束后先填《项目复盘模板》；过阵子觉得某个能力可能值得产品化，再拿起《产品化评审清单》过一遍。前者把项目变成资产，后者判断这块资产值不值得放大。一个往里收，一个往外投，顺序别反。

## T12 技术选型决策树 + 本体建模检查清单（第 8 章）

**技术选型决策树**——拿到客户需求，先别打开技术选型的文档，先走这棵树。每一步答不上来，就退回去一步。

第 0 步：这个问题可计算吗？

客户要的，能翻译成一个具体的、可验证的"决定"或"结果"吗？

答不上来 → 先回第 4 章做问题定义，别谈技术。

答得上 → 往下走。

第 1 步：它需要"动手"吗？

只是回答 / 检索 / 生成草稿（答、查、写）→ 考虑 RAG 或纯生成，走第 2 步。

需要调工具、改数据、执行流程（动手）→ 考虑 Agent，走第 3 步。

只是把业务翻译成能算的模型（长期底座）→ 直接做本体论建模，走第 4 步。

第2步：RAG 场景，先问数据三连：

知识本身是干净、可信、有人负责更新的吗？不是 → 先做知识治理，别急着上检索。

客户给的是标准文档，还是一堆扫描件和烂格式？后者 → 先拿真实样本压测"解析"这一层。

召回不准，是参数没调好，还是资料本来就乱？先治理，再调参。

第3步：Agent 场景，逐个工具打标签：

错了能不能低成本撤销？能（可逆）→ 自动执行，留日志。

不能（不可逆：发消息、改库、下单、转账、删数据）→ 收口，草拟—人工确认—执行。

说不清（半可逆）→ 一律按不可逆处理。

第4步：建模，动词优先：

先列这条业务链上"每天在发生的动作"，再列名词。

每个动作，回答三件事：谁有权执行？前置和副作用是什么？能不能撤销？

答完三件事，再问那句最关键的："客户此刻最该被解决、且能被算清楚的那一个决定，是什么？"

第5步：评测，双环都齐吗？

有没有一套来自真实问题的、会持续更新的离线评估集？

有没有一条管道，把线上反馈（没帮助、转人工、被修改）回流到评估集？

两个都没有 → 交付的准确率就是拍脑袋，先补评测再谈上线。

**本体建模检查清单**——建模是一个动作一个动作抠出来的。每建一个对象或动作，回答下面这些问题。答不全，说明你对这个业务的翻译还没到位。

对象层：

[ ] 这个对象，是业务实际流转的"东西"，还是我们拍脑袋造出来的名词？

[ ] 它的属性，来自哪套源系统？口径统一了吗？冲突的时候以谁为准？

[ ] 它和别的对象是什么关系？（一个订单属于一个客户，一张处方对应一个患者）

[ ] 这个对象的"生命周期"是什么？它会经历哪些状态变化？

动作层：

[ ] 这个对象能发生哪些动作？（别只列属性，把动作列出来）

[ ] 每个动作：谁有权限执行？（角色、部门、审批链）

[ ] 每个动作：前置条件是什么？（什么状态下才能做）

[ ] 每个动作：副作用是什么？（做了之后会联动改变什么、通知谁）

[ ] 每个动作：可逆还是不可逆？不可逆的，收口给人工确认了吗？

权限层：

[ ] 谁有权改这个动作的规则本身？（业务规则谁说了算，得显式定下来）

[ ] 有没有审计记录，能查出"谁在什么时候动了什么"？

[ ] 出了错，负责追责的那个活人是谁？找得到吗？

复用层：

[ ] 这套对象，能不能被上面多个应用共同使用，而不是每个应用单建一套？

[ ] 换一个界面、换一个岗位，还能不能复用同一套对象和动作？

检查清单的目的，不是让你对着它打勾交差，是逼你在建模时，把那些"本该问但常常漏问"的问题问出来。

### T13 职业道德自查清单（第 9 章）

这是一份给你自己用的清单。不是交差的表，是隔一段时间，拿出来对着自己过一遍的镜子。每条没有标准答案，但你心里得有个数。

一、数据主权：

☐ 我只拿了本次交付必需的数据，没有"顺手"多拉别的吗？

☐ 数据存放位置、谁能访问、多久删除，有白纸黑字的约定吗？

☐ 样本结项后，有没有当面清空、不留备份？

☐ 数据要不要出域、出到哪，用户清楚并同意了吗？

## 二、结果诚实报告：

☐ 我报告的数字，证据是哪一类（要求/计算/模拟/实测/供应商声明/认证）？我说清楚了吗？

☐ 有没有把"供应商声明"当成"客户会认的证据"写进方案？

☐ 那些失败的、难看的、出错的数字，我报告了，还是咽下去了？

☐ 一句话说出去之前，它经得起别人重新检查吗？

## 三、避免制造依赖：

☐ 我有没有故意把系统做成黑盒、扣着文档不给客户？

☐ 我的价值，是"客户离不开我"，还是"客户还想继续用我"？

☐ 我交付时，有没有把方法、文档、资产沉淀给客户自己？

☐ 客户今天换掉我，它还能不能运转？不能的话，是因为我优秀，还是因为我拴了根绳？

## 四、关注被替代人员：

☐ 我的方案动到哪个具体岗位的人？我想过他的去路吗？

☐ 方案里，有没有给他留转岗、学习新技能、做复核兜底的位置？

☐ 那些只在他脑子里的隐性知识，有没有变成训练数据、规则、或他教别人的东西？

☐ 我有没有拿"优化人力"当卖点去忽悠老板？

## 五、合规边界：

[ ] 数据出域、私有化、多租户隔离、PII 脱敏、SSO 权限、审计留痕，这几样我过了一遍吗？

[ ] 有没有在安全审查上动过"变通"的小心思？

[ ] 过不去的审查，我是停下来改方案，还是绕过去硬上？

六、行业责任：

[ ] 我这次交付，是让人更信 FDE 这三个字，还是更不信？

[ ] 我有没有在玩 cosplay——只驻场、没数据、没业务负责人、没产品回灌？

[ ] 我不做这件事，会不会只是因为"不谈真实数据、不谈生产动作、不谈结果，剩下的全是名词"？

**临接活前的三个自问**

[ ] 我现在离钱近吗？

[ ] 我现在离现场近吗？

[ ] 我有没有机会，定义验收、沉淀方法？

三个都答不上来，先别急着接。守住底线，也守住自己的路。

## B.3 T14 SOW（工作说明书）模板

SOW 是项目失控的第一道防线。很多 FDE 的麻烦，都源于这份东西写得像散文——范围模糊、验收含糊、边界不清。下面这份模板，逼你把该说死的地方都说死。

### 一、项目目标（用一句话）

这一句不是愿景，是靶子。写"谁，在什么场景下，被什么问题困扰，我们要把它变成什么样"，而非"提升效率、全面智能化"。

要解决的问题（用第 4 章的"问题陈述"句式）：

解决之后，客户"以前怎么做、现在怎么做"：

## 二、交付范围（明确做什么）

按条目列清楚，每一条都对应一个可验证的结果：

- 1.
- 2.
- 3.

## 三、明确不做的范围（比"做什么"更重要）

这一段往往被略过，结果就是范围膨胀、需求反复。把它写足：

本次**不包含**的环节/系统/用户：

本次**不包含**的数据治理/接口打通/老系统改造（若有，单独说明由谁负责）：

明确排除的"未来可能想要但这次不做"的东西：

## 四、真实数据与前置条件

这是最容易在签完后爆雷的地方，务必写清"客户要提供什么"：

| 前置条件         | 由谁提供 | 截止时间 | 若无法提供的影响 |
|--------------|------|------|----------|
| （如：某系统的接口权限） |      |      |          |
| （如：历史数据的导出）  |      |      |          |
| （如：业务负责人的配合） |      |      |          |

若前置数据存在缺失/脏乱，后续的治理成本由谁承担、是否影响验收，需在此约定。

## 五、里程碑与验收标准

每个里程碑，都要有一个"可验证的完成标志"，而不是"开发完成"这种模糊话：

| 里程碑 | 交付内容 | 验收标准（可验证） | 时间 |
|-----|------|-----------|----|
|     |      |           |    |

验收标准要能回答：达到什么结果，算"这一刀真的切进去了"。参考第 4 章 MVD 定义单里"完成标志"那一栏。

## 六、计价与付款

计价方式：（固定总价 / 人天 / 按结果分成 / 混合）

付款节点：签约、里程碑、上线、验收，分别付多少

若采用"按结果收费"，结果的口径、测量方式、分成比例，写清楚（参考第 6 章四档进化链）

## 七、变更与范围管理

需求变更的流程：谁提、谁批、是否加钱加时

超范围的默认态度：本轮不做、放到下一轮（用第 4 章"定死截止时间"来正当拒绝）

## 八、边界与责任（谁对什么负责）

出错/失败时，责任的边界（尤其不可逆操作必须有客户方人工确认）

数据主权与隐私边界（参考第 9 章职业道德底线）

## B.4 一份项目的"模板使用顺序"

模板不是孤立用的，它们有先后。一个完整项目，大致按这个顺序往下走：

选客户（T04 进场尽调 + T05 灯塔四问）

→ 定义问题（T06 三关 + T08 缩小范围 + T07 MVD 定义单）

→ 签 SOW（B.3 T14）

→ 部署与激活（T09 激活清单 + 健康度）

→ 守住续约（T10 健康分 + 续约排查）

→ 复盘与复制（T11 产品化评审 + 项目复盘）

→ 技术选型（T12 决策树 + 本体建模清单）

→ 守住底线（T13 职业道德自查）

全程用它校准自己 → T01 自评 / T02 五阶段 / T03 结果自查

每一个箭头，都是一个"该停下、对照模板问自己一句"的节点。模板的价值，不在于你照着填，而在于它在正确的时刻，逼你问出那个正确的问题。

这份附录，和附录 A（指标清单）、附录 C（案例索引）一起，构成了本书的"工具箱层"。它不是读一遍就放下的，是你在项目里一遍一遍回来翻的。希望你用到的次数，越多越好。

<div style="page-break-after: always"></div>

## 附录 C 全书案例与项目索引

本书所有案例，均来自自己采集的公开一手分享文章，可溯源。写作时坚持一个原则：细节可脱敏，情节不编造。

### C.0 说明

这本书不是"成功学案例集"。它有意识地让**失败案例 > 成功案例**，因为能救命的，从来不是别人怎么赢，而是别人怎么死、又怎么从死里爬出来。所以这份索引，我会把每个案例的"障碍类型"和"结论"标清楚，方便你拿自己的项目来对照。

案例按"五类障碍"归类——这是全书的案例分类主线，也是 FDE 项目最容易翻车的五个地方：

- 与遗留系统集成（技术债、接口、权限）
- 安全与合规（数据主权、权限、审计）
- 部署与可靠性（稳定性、激活、用户行为）
- 商业价值证明（付费意愿、经济性）
- 可传承性（知识沉淀、组件化、产品化）

### C.1 案例总表

| 编号 | 案例名称                 | 行业/场景   | 障碍类型        | 出现章节      | 结论                      |
|----|----------------------|---------|-------------|-----------|-------------------------|
| C1 | 代工厂 40 万 AI 转型项目     | 制造业（代工） | 遗留系统 / 数据治理 | 第 1、3、4 章 | 反面：问题选错，45 天洗数据，成本烧穿合同额 |
| C2 | 30 万单 vs 100 万前置数据治理 | 通用      | 商业价值 / 数据   | 第 4 章     | 反面：签完约才发现前置成本是合同额三倍     |
| C3 | 总包暴雷：三               | 企业服务    | 组织 / 规模化    | 第 7 章     | 反面：不能靠                  |

|     |                    |        |            |               |                        |
|-----|--------------------|--------|------------|---------------|------------------------|
|     | 延期两跑路              |        |            |               | 堆人放大，规模上限=兜底能力         |
| C4  | 线上比价：10万 vs 2000 元 | 通用获客   | 获客 / 商业    | 第 3 章         | 反面：公域询价是比价修罗场，须线下成交    |
| C5  | 三天两万 AI 培训         | 培训/服务  | 规模化 / 标准化  | 第 7 章         | 正面：标准化的交付物可以是服务，不只是代码  |
| C6  | 代运营团队转型 FDE        | 企服/代运营 | 定位 / 商业    | 第 2、3、6、7、9 章 | 正面：业务理解 > 技术，绑平台生态闷声赚钱 |
| C7  | 人工智能小镇三周实施         | 培训/陪跑  | 激活 / 陪跑    | 第 5 章         | 正面：让用户自己上手，激活靠行为改变而非强推 |
| C8  | 五类障碍框架 + Project 1 | 通用技术   | 技术 / 全类别   | 第 3、5、8 章、附录  | 框架：全书案例分类主线；双向遗留同步引擎   |
| C9  | 深圳路演"证据分级"         | 技术纪律   | 可靠性 / 诚实   | 第 8、9 章       | 方法论：六类证据分级，防承诺互相冒充     |
| C10 | 垂直切片与八小时价值         | 通用交付   | 范围控制       | 第 4、5 章       | 方法论：MVD 第一刀，缩小范围不缩小价值  |
| C11 | Palantir 发票争议与价值证明 | 大企业/海外 | 商业价值证明     | 第 2、6、8 章     | 正面：按结果收费，客户买的是"不再漏走的钱" |
| C12 | 知识库"AI 正确处理错误知识"   | 通用     | 数据质量 / 可靠性 | 第 1、2、5、6、8 章 | 反面：质量漂移的根源常在知识本身，不在模型  |
| C13 | 客户经营恶化与尾款难收        | 通用     | 财务健康 / 回款  | 第 6 章         | 反面：健康分要前置到"选客户"，而非     |

|     |                       |      |           |           |                                       |
|-----|-----------------------|------|-----------|-----------|---------------------------------------|
|     |                       |      |           |           | 续费前一月                                 |
| C14 | 教育转化系统<br>"状态跃迁"      | 教育   | 定位 / 边界   | 第 2、4、9 章 | 正面：FDE<br>盯状态与边界，外包盯话术与追单             |
| C15 | 工程咨询问答<br>"四件事"       | 咨询   | 边界定义      | 第 9 章     | 方法论：把模糊需求拆成四件事，找回边界定义权                |
| C16 | 车企新媒体"<br>买的不是数字人"    | 汽车营销 | 场景定义      | 第 1、9 章   | 正面：不被客户第一句话牵着走，重画链路                   |
| C17 | 三个权力 + 红线表            | 职业道路 | 职业道路      | 第 9 章     | 方法论：别把自己干成驻场外包，守住三个权力                 |
| C18 | "FDE cosplay"<br>行业乱象 | 行业   | 职业道德 / 责任 | 第 1、9 章   | 反面：只抄了<br>"驻场"皮毛的<br>FDE 是<br>cosplay |

## C.2 按章节定位案例（快速跳转）

| 章节    | 主题                | 该章主案例                      | 佐证案例           |
|-------|-------------------|----------------------------|----------------|
| 第 1 章 | 95% 的失败率背后        | 一个项目的静默死亡（C1）              | C12、C16        |
| 第 2 章 | 你不是在卖模型，是交付结果     | 岗位边界对比（方法论）                | C6、C11、C12、C14 |
| 第 3 章 | 进场：先画出五张地图        | 进场尽调避开一个大坑                 | C1、C4、C6、C8    |
| 第 4 章 | 找到正确的问题：PSF 与 MVD | 砍掉八成需求，反而活了（C14）           | C1、C2、C10      |
| 第 5 章 | 从能跑到用户真用：激活       | 集成大战复盘（C8 / Project 1）     | C7、C8、C10、C12  |
| 第 6 章 | 守住续约与扩大收入         | 差点流失的客户救回来 + Anthropic-FIS | C6、C11、C12、C13 |
| 第 7 章 | 规模化复制：从定制到平台      | 一个组件复制到五个客户                | C3、C5、C6       |
| 第 8 章 | 技术选型与本体论          | 医院的动词建模实战（方法论）             | C8、C9、C11、C12  |

|       |       |                          |                               |
|-------|-------|--------------------------|-------------------------------|
| 第 9 章 | 底线与道路 | 全书回顾 + FDE<br>cosplay 反例 | C6、C9、C14、C15、<br>C16、C17、C18 |
|-------|-------|--------------------------|-------------------------------|

注：表中个别"主案例"（如第 4 章的 C14、第 5 章的 C8）在正文中是以"教育转化""双向遗留同步"的脱敏故事形式出现的，故在索引里直接用案例编号定位，方便你反查素材库。表格不再使用写作早期草案里的（A）-（G）占位代号，正文并未沿用那一套编号，避免产生找不到对应物的悬空引用。

### C.3 关键案例速查（写书用）

#### 你可以反复借用的三个"反面教材"

**C1 代工厂**——问题选错的极致案例。想接"全面 AI 化"，结果 45 天洗数据，成本烧穿合同额。用在任何"要警惕范围失控、警惕数据泥潭"的地方都成立。

**C2 前置数据治理推演**——"30 万单要额外批 100 万扫垃圾"，讲经济性检验的预期崩塌，双方都委屈、又都没错。

**C3 总包暴雷**——"活可以分包，责任分不出去"，讲堆人放大的死路。

#### 你可以反复借用的两个"正面样板"

**C11 Palantir**——按结果收费的标杆：客户掏钱买的不是 Ontology 这个词，是"每年不再漏走的那 5000 万美元"。

**C7 人工智能小镇**——激活的样板：让用户自己上手，实施者"只需要喝茶然后验收"。

#### 一条贯穿全书的金句

"你越靠谱，越容易被所有人叫去擦屁股。"（C17）

它提醒每一个 FDE：光会干活是不够的，你得守住场景定义权、边界定义权、方法沉淀权，否则越干越累、越累越像驻场外包。

### C.4 素材缺口说明（诚实交代）

这本书的案例，绝大多数来自真实的公开分享。但有几个地方，我要如实标注：

**第 6 章"健康分预警→干预→救回"的完整正向闭环**：目前素材里以"没救回的反面"（C13）为主，正向闭环暂缺，写作时用"方法论 + 模板"补位，未虚构一个不存在的救回故事。

**第8章本体论的医院动词建模：**案例为方法论讲解性质，非某个单一企业的完整纪实，写作时以"抽象的动词链"呈现，不冒充真实医院数据。

**海外数据**（如 C11 的金额、C18 高德纳预测）：以官方口径/公开报道为准，写书时标注"以官方口径为准"。

诚实，是这本书对所有读者（包括作者自己）的最后一条底线。

附录 A（指标）、附录 B（模板）、附录 C（案例索引）三份一起，构成了本书的"工具箱层"。希望它们能被你真正拿去用，而不是读完就合上。

公共案例 A 价值链